

EDADSL Documentation

Andreas G.

15th of April 2026

Contents

1	Introduction	10
1.1	Abstract	10
1.2	Reading this Document	10
I	Core Language	11
2	Design Principles	12
2.1	Design Goals	12
2.2	Applications and Application-Areas with special Priority during Design . . .	12
2.3	Design Philosophy	13
3	Lexical Structure	14
3.1	Character Set	14
3.2	Whitespace	14
3.3	Comments	14
3.4	Identifiers	15
3.5	Keywords	15
3.6	Literals	15
3.7	Operators	15
3.8	Delimiters	16
3.9	Tokenization Rules	16
3.10	Statement Separators	16
3.11	End of File	16
4	Formal Grammar	17
4.1	Primitives	17
4.2	General Grammar	17
4.3	Grammar relating to the electrical System	22
4.3.1	Make Statement	22
4.3.2	Electrical Statements	22
4.3.3	Physical Constraints	23
4.3.4	System Statements	24
4.3.5	Additional Grammar Rules	24
4.3.6	Common Rules	25

5	Types	26
5.1	Generic Types	26
5.1.1	Boolean	26
5.1.2	Null	26
5.1.3	Integer	27
5.1.4	Real	29
5.1.5	Complex	31
5.1.6	Set	32
5.1.7	String	34
5.1.8	List	34
5.2	Function	40
5.2.1	Function Data Type Definition	40
5.2.2	Function Data Type Creation	40
5.2.3	Function Data Type Calling	40
5.2.4	Function Data Type Annotations	41
5.2.5	Function Data Type Equality	41
5.2.6	Function Data Type Collections	41
5.2.7	Function Data Type Printing	41
5.3	Electrical Types	41
5.3.1	Part	41
5.3.2	Pin	42
5.3.3	Connection	43
5.3.4	Net	43
5.4	Type Casting with <code>as</code>	44
5.4.1	Syntax	44
5.4.2	Numeric Conversions	44
5.4.3	Complex Conversions	45
5.4.4	List/Tensor Conversions	45
5.4.5	Error Handling	46
5.4.6	Edge Cases	46
6	Variables	47
6.1	Variable Declaration and Assignment	47
6.1.1	Type-Inferred Assignment	47
6.1.2	Variable Redeclaration	47
6.2	Mutability	48
6.3	Scope	48
6.4	Examples	49
7	Expressions	50
7.1	Expression Grammar	50
7.2	Operator Precedence	52
7.3	Evaluation Order	52

8	Operators	53
8.1	Built-in Operators	53
8.1.1	Arithmetic	53
8.1.2	Hyperoperations	56
8.1.3	Increment / Decrement	57
8.1.4	Compound Assignment	57
8.1.5	Comparison	63
8.1.6	Boolean Logic	65
8.1.7	Bitwise Operations	66
8.1.8	Collection Operators	67
8.1.9	List Operators	68
8.1.10	Tensor Operators	69
8.1.11	Set Comparison	72
8.1.12	Conditional Expression	72
8.1.13	Null Operators	73
8.1.14	Lambda Expressions	75
8.1.15	Pipeline	77
8.1.16	Function Composition	78
8.2	Operator Precedence	79
8.3	User-defined Operators	80
8.3.1	Definition	80
8.3.2	Examples	80
8.3.3	Precedence	81
9	Constants	82
9.1	Built-in Constants	82
9.1.1	Proper Constants	82
9.1.2	Pseudo-Constants	87
9.2	User-defined Constants	88
10	Control Flow	89
10.1	Conditional Execution	89
10.2	Cases	90
10.3	Alias Statements	91
10.3.1	Function Alias	91
10.3.2	Namespace Alias	92
10.3.3	Edge Cases	92
10.4	For Loops	92
10.5	While Loops	92
10.5.1	Basic While Loop	93
10.5.2	While with Else Clause	93
10.5.3	Do-While Loops	94
10.5.4	Break and Continue	94
10.5.5	Nested While Loops	95
10.5.6	Safety: Maximum Iteration Limit	95
10.5.7	Infinite Loops	95
10.6	Goto Statements	96

10.7	End of Program Statements	96
10.8	Output and Logging	96
11	Functions	97
11.1	Built-in Functions	97
11.1.1	Mathematical Functions	97
11.1.2	String Functions	142
11.1.3	Vectorization	160
11.1.4	Tensor Indexing	161
11.1.5	List and Set Functions	165
11.1.6	Ndarray Functions	168
11.1.7	Integer Sequence Functions	196
11.1.8	Higher-Order Functions	202
11.1.9	Signal Processing Functions	213
11.1.10	Debug Functions	219
11.1.11	System Functions	222
11.1.12	System Variables	224
11.1.13	Time Functions	226
11.1.14	Random Number Functions	229
11.1.15	Hash Functions	233
11.1.16	Probability Distributions	239
11.1.17	Numerical Analysis	259
11.1.18	Shared Functions	269
11.1.19	Special Functions	269
11.1.20	Engineering Math Functions	271
11.2	User-defined Functions	326
11.2.1	Function Definition	326
11.2.2	Function Calls	326
11.2.3	Return Values	326
11.2.4	Local Scoping	327
II	Formal Semantics	329
12	Evaluation Model	330
12.1	Evaluation Order	330
12.2	Operator Precedence	330
12.3	Type Coercion	330
13	Execution Order	332
13.1	Sequential Execution	332
13.2	Expression Evaluation	332
13.3	Assertions	332
13.4	Error Handling	332
13.5	Function Value Calling	333
13.6	Safe Function Calling	333
13.6.1	Safe Call (?())	333

13.6.2	Null-Safe Call (??())	333
13.7	Function Composition	334
13.8	Declaration Order	334
14	Constraint Resolution	335
14.1	Conflict Resolution	335
15	Type System Rules	336
15.1	Type Categories	336
15.2	Type Safety	336
15.3	Error Types	336
III	Core Formal Model	338
16	Structure of the Program	339
16.1	Program Entry Point	339
16.2	Global Scope	339
16.3	Execution Model	339
17	Structure of the Electrical System	340
17.1	Electrical Graph	340
17.2	Object Lifecycle	340
18	Structure of the Solver	341
18.1	Constraint Representation	341
18.2	Resolution Algorithm	341
IV	Hardware Model	342
19	Circuit Model	343
19.1	Graph Structure	343
19.2	Hierarchy	343
20	Electrical Objects	344
20.1	Nets	344
20.1.1	Creation	344
20.1.2	Properties	344
20.2	Parts	344
20.2.1	Creation	345
20.2.2	PCB Text Visibility Flags	345
20.2.3	Properties	345
20.3	Pins	345
20.3.1	Creation	345
20.3.2	Reference Syntax	345
20.4	Connections	346
20.4.1	Creation	346

20.5	Models	346
20.5.1	Model Declaration Syntax	346
20.5.2	Built-in Models	347
20.5.3	Custom Model Examples	347
20.6	Electrical Modules	348
20.6.1	Definition	348
20.6.2	Instantiation	349
20.6.3	Example	349
20.7	Relationship between the Objects	349
21	Querying	350
21.1	Output and Input Quantifiers	350
21.2	Net based Querying	350
21.3	Tagged Net based Querying	350
21.4	Affiliation-based Querying	351
21.5	General Querying	351
21.5.1	Set Builder Notation	351
21.5.2	Set Query Notation	352
21.5.3	Nesting	352
V	Physical System	354
22	Physical Constraints	355
22.1	Units and SI Prefixes	355
22.2	Priority System	355
22.3	Proximity Constraint	356
22.4	Creepage Constraint	356
22.5	Layer Constraint	356
22.6	Board Definition	356
22.7	Footprint Unification	357
22.8	Net Shrink	357
22.9	Plane Creation	358
22.10	Text Annotation	358
22.11	Via Fill	358
22.11.1	Pattern Types	359
22.11.2	Examples	359
22.12	Connection Current	360
22.13	Alignment	360
22.14	Connection Matching	360
22.14.1	Properties	360
22.14.2	Value Calculation Methods	361
22.14.3	Matching Objective	363
22.14.4	Example	364

23 Solver Model	365
23.1 Overview	365
23.2 Algorithm Selection	365
23.2.1 Available Algorithms	365
23.2.2 Default Algorithm	373
23.3 Constraint Resolution	373
23.3.1 No Conflicts	373
23.3.2 Conflicts Between Different Priorities	373
23.3.3 Conflicts Between Same Priorities	374
23.4 Layout Generation Pipeline	374
23.4.1 Phase 1: Constraint Collection	374
23.4.2 Phase 2: Board Setup	374
23.4.3 Phase 3: Component Placement	374
23.4.4 Phase 4: Trace Routing	374
23.4.5 Phase 5: Via Placement	375
23.4.6 Phase 6: Plane Generation	375
23.4.7 Phase 7: Design Rule Check	375
23.4.8 Phase 8: File Export	375
23.5 Ordering of Global Set Constant *e	375
23.5.1 Canonical String Rules	375
23.5.2 Example	376
24 Flags	377
24.1 Available Flags	377
24.2 PCB Part Text Visibility	378
VI Toolchain	379
25 Compiler and Execution Pipeline	380
25.1 Compilation Phases	380
25.2 Entry Point	380
26 Code Generation	381
26.1 PCB Generation	381
26.2 Schematic Generation	381
26.3 Export	381
27 Standard Library	382
27.1 Component Models	382
27.2 Footprint Libraries	382
27.3 Material Library	382
28 External Library	383
28.1 Importing Libraries	383
28.2 Supported Formats	383

29 Debugging and Logging	384
29.1 Debug Mode	384
29.2 Quick Reference	384
29.3 Workflow	385
 VII Reference	 386
30 Exceptions	387
30.1 Type Errors	387
30.2 Value Errors	387
30.3 Reference Errors	387
30.4 Constraint Errors	387
 31 Error and Warning Model	 388
31.1 Errors	388
31.2 Warnings	388
 32 Future Improvements	 389
 33 Implementations	 390
33.1 Reference Implementation	390
33.2 Build and Installation	390
 34 References	 391
34.1 Language Specification	391
34.2 Related Standards	391
 A Standard Libraries	 392
A.1 Material Libraries	392
A.1.1 PCB Substrate Material Library	392
A.1.2 PCB Conductor Material Library	392
A.1.3 PCB Surface-finish Material Library	392
A.2 Basic Model Libraries	392
A.2.1 Passive Component Model Library	392
A.2.2 Active Component Model Library	393
A.2.3 Integrated Circuit Component Model Library	393
A.2.4 Board Hardware Component Model Library	393
A.3 Additional Language Libraries	393
A.4 Footprint Libraries	393
 B Examples	 394
B.1 LED Circuit	394
B.1.1 Intent	394
B.1.2 EDADSL Code	394
B.1.3 Explanation	394
B.2 Arithmetic and Type Coercion	395

B.2.1	Intent	395
B.2.2	EDADSL Code	395
B.2.3	Explanation	395
B.3	Conditionals and Classification	395
B.3.1	Intent	395
B.3.2	EDADSL Code	395
B.3.3	Explanation	396
B.4	Lambdas and Higher-Order Functions	396
B.4.1	Intent	396
B.4.2	EDADSL Code	396
B.4.3	Explanation	397
B.5	Complex Arithmetic and FFT	397
B.5.1	Intent	397
B.5.2	EDADSL Code	397
B.5.3	Explanation	398
B.6	Ndarray Matrix Operations	398
B.6.1	Intent	398
B.6.2	EDADSL Code	398
B.6.3	Explanation	399
B.7	Closures and Lambdas	399
B.7.1	Intent	399
B.7.2	EDADSL Code	399
B.7.3	Explanation	400
B.8	Design-Rule-Driven Component Placement	400
B.8.1	Intent	400
B.8.2	EDADSL Code	400
B.8.3	Explanation	401
B.9	Signal Processing Pipeline	401
B.9.1	Intent	401
B.9.2	EDADSL Code	401
B.9.3	Explanation	402
C	Diagrams	403
C.1	Execution-phase Diagram	403
C.2	Type Hierarchy Diagram	404
C.3	Operator Precedence Diagram	404
C.4	Example Circuit: LED Current-Limiting	405
C.5	FFT Pipeline Diagram	406
C.6	Ndarray Operations	407
C.7	Custom Model Definition and Usage	408
C.7.1	Intent	408
C.7.2	EDADSL Code	408
C.7.3	Explanation	409

Chapter 1

Introduction

1.1 Abstract

EDADSL (Electronic Design Automation Domain Specific Language) is a Domain-specific Language (DSL) specialized for programmatic Electronic Design Automation (EDA) and Hardware-Synthesis. It is based on an Electrical System and a constraint-driven optimisation engine. This Document is a full outline and as of now the authoritative Specification of the EDADSL Language as well as the systems underlying it.

1.2 Reading this Document

This document is organized into the following parts:

- **Part I: Core Language** – Defines the programming language constructs including lexical structure, types, variables, expressions, constants, control flow, and functions.
- **Part II: Formal Semantics** – Describes the evaluation model, execution order, constraint resolution, and type system rules.
- **Part III: Core Formal Model** – Defines the structure of programs, the electrical system, and the solver.
- **Part IV: Hardware Model** – Describes the circuit model, electrical objects, and querying system.
- **Part V: Physical System** – Defines physical constraints, the solver model, and system flags.
- **Part VI: Toolchain** – Describes the compiler pipeline, code generation, standard library, and debugging.
- **Part VII: Reference** – Contains exceptions, error model, future improvements, and references.
- **Appendices** – Contains standard libraries, examples, and diagrams.

Readers new to EDADSL should start with Parts I and IV, which define the language and electrical system. Those interested in physical layout should read Part V. Implementers should consult Parts II and III for formal specifications.

Part I

Core Language

Chapter 2

Design Principles

2.1 Design Goals

EDADSL was designed with the following primary goals:

1. **Programmatic PCB Design** – Enable engineers to describe circuits and layouts programmatically, enabling version control, automation, and reproducibility.
2. **Constraint-Driven Layout** – Allow physical constraints (clearance, creepage, alignment) to be specified declaratively, with the solver handling the complex optimization.
3. **Electrical Awareness** – Maintain a rich electrical model (parts, pins, connections, nets) that understands circuit topology, not just geometry.
4. **KiCad Integration** – Generate industry-standard KiCad files (schematics and PCBs) for further editing and manufacturing.
5. **Readability** – Use clear, prefixed syntax (elec.*, phys.*, syst.*) to make code self-documenting and easy to understand.
6. **Simplicity** – Provide a minimal set of powerful constructs rather than a large number of specialized features.
7. **Accessibility** – Serve both professional engineers and hobbyists with appropriate defaults while supporting advanced features for complex designs.

2.2 Applications and Application-Areas with special Priority during Design

EDADSL is particularly well-suited for:

- **Power Electronics** – Where creepage distances, high-current traces, and thermal management are critical. The constraint system supports isolation requirements between high-voltage and low-voltage sections.
- **High-Speed Digital Design** – Where trace length matching, impedance control, and signal integrity are important. The `phys.connmatch` constraint supports length and impedance matching.

- **Mixed-Signal Designs** – Where analog and digital sections must be carefully isolated. The constraint system supports separation of noisy and sensitive circuits.
- **Prototyping** – Where rapid iteration and parametric design enable quick exploration of design alternatives.
- **Automated Manufacturing** – Where programmatic generation enables batch production and design automation pipelines.

2.3 Design Philosophy

EDADSL follows a declarative-imperative hybrid philosophy:

- **Imperative Circuit Definition** – Components and connections are defined sequentially, allowing natural expression of circuit topology.
- **Declarative Physical Constraints** – Physical requirements are declared as constraints, letting the solver determine optimal placement and routing.
- **Separation of Concerns** – The electrical description (what the circuit does) is separated from the physical description (how it is laid out).
- **Constraint Hierarchy** – A priority system allows designers to express preferences while ensuring critical requirements are met.
- **Block Scoping** – Variables declared inside code blocks (`if`, `for`, `while`, `do-while`) are local to that block. Functions create their own local scope.

Chapter 3

Lexical Structure

3.1 Character Set

EDADSL uses ASCII for identifiers, keywords, operators, and grammar symbols. Unicode characters are permitted within string literals and comments.

3.2 Whitespace

Whitespace is defined using the following Regular Expression:

```
RegEx:
Whitespace          = [ \t\n\r]+
```

3.3 Comments

Comments are defined using the following Regular Expressions:

```
RegEx:
Line_Comments       = "#c#" [^\n]*
Multiline_Comments  = "#c#", "{", [any characters], "}#c#"
```

Multiline comments start with `#c#{` and end with `}#c#`. Nested multiline comments are not supported.

Examples:

```
#c# This is a line comment
$x [int]= 5  #c# Inline comment

#c#{
This is a
multiline comment
}#c#

$x [int]= 5 #c#{ inline }#c# + 3
```

3.4 Identifiers

Identifiers are defined using the following Regular Expressions:

```
Regex:
id      = [_a-zA-Z][_a-zA-Z0-9]*
```

3.5 Keywords

The language uses the following Keywords defined using the following Regular Expressions:

```
Generic_Keywords:  start halt if else otherwise for while do break continue
                   gotomarker gotojump
                   writetolog make.pcb make.schematic export
                   echo return let letop skip function L::
                   null assert
```

```
Physical_Keywords: phys\[a-z]+
                   Known Physical Keywords:
                   phys.prox phys.creep phys.layer
                   phys.shrinknet phys.mkplane
                   phys.viafill phys.connectioncurrent
                   phys.align phys.connmatch phys.board
                   phys.mktext
```

```
Electrical_Keywords: elec\[a-z]+
                   Known Electrical Keywords:
                   elec.part elec.connect elec.net
                   elec.tag elec.untag
```

```
System_Keywords:  syst\[a-z]+
                   Known System Keywords:
                   syst.flag\[a-z]+
```

Keywords are reserved identifiers and may not be used as variable names, function names, or constant names. Keywords are case-sensitive: `if` is a keyword, but `If` or `IF` is an identifier.

3.6 Literals

For the Lexical Structure of Literals see: Section 5.1 (Types).

3.7 Operators

For the Lexical Structure of built in Operators see: Section 8 (Operators).
User-defined Operators are defined using the following Regular Expressions:

```
User_Operators    = o\.u\[a-zA-Z0-9]+
```


The `o.u.` prefix is distinct from dot-access chains (`obj.property`): `o.u.` is always followed by an operator name and used in infix position, while dot-access is always `identifier.identifier` and denotes property access.

3.8 Delimiters

EDADSL uses the following Delimiters:

`() [] {} . , : ; @ ~ | %N`

The `%N` token is a statement separator, not a comment. It allows multiple statements on a single line. The `::` token denotes a code block opener. The semicolon `;` appears in modifier syntax (`;M=, ;P=, ;T=`) and property separators.

3.9 Tokenization Rules

The Tokenization follows the following Rules:

The Lexer uses the Longest Match Rule: in Cases where multiple valid tokenizations are possible the longest token is chosen.

If Lexeme matches both a Keyword and an Identifier it is classified as a Keyword.

Whitespace and Comments are discarded.

3.10 Statement Separators

Statements within a code block are separated by implicit newlines. The `%N` token acts as an explicit statement separator, allowing multiple statements on a single line:

```
#c# These are equivalent:
::{
    f.print("a")
    f.print("b")
}

::{
    f.print("a") %N f.print("b")
}
```

The `%N` token is primarily useful inside inline code blocks where writing multiple lines is impractical.

3.11 End of File

Source files must end with a newline character. A line comment at the end of the file without a trailing newline is permitted but discouraged.

Chapter 4

Formal Grammar

The Formal Grammar of EDADSL is defined using Extended Backus–Naur form, while Primitives are given using Regular Expressions.

4.1 Primitives

The Primitives are defined using Regular Expressions. For more Information on Lexical structure see Part I Chapter 2.

```
idchar  = [_a-zA-Z0-9]
id       = [_a-zA-Z] [_a-zA-Z0-9]*
letter  = [a-zA-Z]
```

4.2 General Grammar

```
program                                = "start", statement,
                                       { ("%N" | newline), statement }

statement                             = assignment
                                       | flow_control
                                       | function
                                       | electrical_system_statement
                                       | compound_assignment
                                       | postfix_expression
                                       | echo_statement
                                       | writetolog
                                       | letop
                                       | assertion
                                       | try_except
                                       | alias_stmt

assignment                           = variable_assignment
                                       | constant_assignment
```

function	= function_call_built_in function_call_ud function_call_var function_definition
flow_control	= conditional cases iterator while_loop do_while break_stmt continue_stmt jump halt noop
conditional	= "if", boolean_expression, "::", code_block, { "otherwise", boolean_expression, "::", code_block }, ["else", "::", code_block]
cases	= "cases", "(", variable, typed_assign, data, ")", "::", "{", { case_clause }, [otherwise_clause], "}"
case_clause	= "case", "(", boolean_expression, ")", "::", code_block
otherwise_clause	= "otherwise", "::", code_block
iterator	= "for", variable, "E", (list set), "::", code_block # ## Note: "E" in the iterator is disambiguated from the set comparison "E" # ## by parser context after "for \$var" it is the iterator keyword.
while_loop	= "while", boolean_expression, "::", code_block, ["else", "::", code_block]
do_while	= "do", "::", code_block, "while", boolean_data_type
break_stmt	= "break"
continue_stmt	= "continue"
jump	= ("gotomarker" "gotojump"), id
echo_statement	= "echo", variable

```

writetolog                = "writetolog", string_expression

letop                      = "letop", "o.u.", id, assigner, data

function_definition = "function", "f.u.", id,
                      "(", [ { variable, "," }, variable],
                      ")", ":", "{", f_statement,
                      {f_statement}, "}"

f_statement = statement | "return", [data]

function_call_ud           = "f.u.", id, "(",
                          [{ data, "," }, data] ,")"

function_call_built_in    = "f", ".", namespace, ".", id, "(",
                          [{ data, "," }, data] ,")"

function_call_var         = variable, "(",
                          [{ data, "," }, data] ,")"

safe_function_call        = variable, "?", "(",
                          [{ data, "," }, data] ,")"

null_safe_function_call   = variable, "??", "(",
                          [{ data, "," }, data] ,")"

namespace                 = "math" | "trig" | "hyp" | "cplx" | "log"
                          | "special" | "str" | "list" | "tnsr"
                          | "stat" | "linalg" | "ee" | "int" | "util"
                          | "seq" | "dbg" | "sys" | "time" | "rng" | "hash"

variable_assignment       = variable,
                          (typed_assign | inferred_assign),
                          [";", "M", "=", boolean_data_type]

typed_assign              = assigner
inferred_assign           = "t", "=", data

safe_typed_assign         = variable, "?[", type, "]==", data

safe_inferred_assign      = variable, "?t?=", data

lazy_binding              = variable, ( typed_lazy_binding | inferred_lazy_binding )
typed_lazy_binding        = ":L", "[", type, "]==", data
inferred_lazy_binding     = ":Lt=", data

```

```

compound_assignment      = variable, compound_op, data
| variable, compound_unary_op
compound_op              = "+=" | "-=" | "*=" | "/"= | "%=" | "^="
| "<<=" | ">>=" | "&&=" | "||=" | "^="
| "u=" | "n=" | "."= | "x=" | ".*=" | ".^="
| "+r=" | "-r=" | "/r=" | "%r=" | "^r="
| "xr=" | ".*r=" | ".^r="
| "***=" | "****=" | "*****="
| "**r=" | "***r=" | "****r="
| "<<r=" | ">>r="
| "<<<" , "[", int, "]", "="
| ">>>", "[", int, "]", "="
| "<<<" , "[", int, "]", "r="
| ">>>", "[", int, "]", "r="
| "->=" | "!->=" | "->r=" | "!->r="
| "b&&=" | "b||=" | "b^=" | "b!&=" | "b!|=" | "b!^="
| "b->=" | "b!->=" | "b->r=" | "b!->r="
| "<o=" | ">o="
compound_unary_op        = "~.T" | "~.H" | "~.E" | "~.E+" | "~b!"

assigner                  = "[bool]=", boolean_type
| "[int]=", int_type
| "[real]=", real_type
| "[list]=", list_type
| "[string]=", string_type
| "[set]=", set_type
| "[func]=", func_type
| electrical_system_assigner

constant_assignment = "let", "c.u.", id, assigner,
[";", "P", "=", boolean_data_type]

constant_ref           = "c", ".", const_namespace, ".", id

const_namespace        = "math" | "phys"

halt                    = "halt"
noop                    = "skip"

list                    = "[", [{ data, "," }, data], "]"

variable                = "$", id

code_block              = "{", { ("%N" | newline), statement }, "}"

```

<code>boolean_data_type</code>	<code>= expression (any expression evaluating to bool)</code>
<code>data</code>	<code>= variable</code> <code> literal</code> <code> constant</code> <code> constant_ref</code> <code> function_call_built_in</code> <code> function_call_ud</code> <code> function_call_var</code> <code> expression (see Chapter 4 for expression grammar)</code>
<code>pipeline_expression</code>	<code>= expression, " =>", function_ref</code> <code> expression, " =>", lambda</code>
<code>composition_expression</code>	<code>= expression, "<o", expression</code>
<code>type_cast</code>	<code>= expression, "as", type_name</code>
<code>type_name</code>	<code>= "int" "real" "bool" "complex"</code> <code> "list" "tensor"</code>
<code>function_ref</code>	<code>= "f", ".", namespace, ".", id</code> <code> "f.u.", id</code>
<code>func_type</code>	<code>= function_ref lambda "null"</code>
<code>assertion</code>	<code>= "assert", boolean_expression, "::"</code> <code>, string_type, [";", error_type]</code>
<code>error_type</code>	<code>= "Assertion Error" "aerr"</code> <code> "Type Error" "terr"</code> <code> "Value Error" "verr"</code> <code> "Division by Zero Error" "Odiv"</code> <code> "Runtime Error" "rerr"</code> <code> "Permission Error" "perm"</code>
<code>try_except</code>	<code>= "try", "::", code_block</code> <code>, { "otherwise", [error_type], "::", code_block }</code> <code>, ["except", "::", code_block]</code>
<code>alias_stmt</code>	<code>= "alias", function_ref, "as", id</code> <code> "alias", namespace, "as", id</code>

4.3 Grammar relating to the electrical System

```
electrical_system_statement  = make_statement
                              | elec_statement
                              | phys_statement
                              | syst_statement

electrical_system_assigner    = "[part]=", part_type
                              | "[connection]=", connection_type
                              | "[pin]=", pin_type
                              | "[net]=", net_type
```

4.3.1 Make Statement

```
make_statement = "make.", ("pcb" | "schematic")
```

4.3.2 Electrical Statements

```
elec_statement = "elec.",
( "part", part_reference, ":", model_name,
  "(", [model_params], ")",
  [";", "des", "=", string_type],
  [";", "fp", "=", string_type]
| "connect", (net_type | pin_type),
  pin_type, { pin_type }
| "net", string_type, { string_type }
| "tag", net_type, string_type,
  { string_type }, ["-remove"]
| "untag", net_type, string_type,
  { string_type }
| "makemodel", model_name, ":",
  "{", pin_entries, [metadata], "}"
| "makemodule", id, "(", [variable, {"", variable}], ")",
  "{", module_body, "}"
)

part_reference = string_type
                | "[", { string_type, ",", string_type, "]"

model_name      = id, ["/", id]
model_params    = [{ id, "=", data, ",", id, "=", data}]

pin_entries     = { pin_entry, ";" }
pin_entry       = id, "->", integer, ";", "T", "=", pin_type
pin_type        = "input" | "power" | "output"
metadata        = des_entry | dts_entry | fp_entry
des_entry       = "des", "=", string_type
```

```

dts_entry      = "dts", "=", string_type
fp_entry       = "fp", "=", string_type

module_body    = { statement }

```

4.3.3 Physical Constraints

```

phys_statement = phys_prox | phys_creep | phys_layer
                | phys_board | phys_shrinknet
                | phys_mkplane | phys_viafill
                | phys_connectioncurrent | phys_align
                | phys_connmatch | phys_mktext
                | phys_unite

phys_prox      = "phys.prox", set,
                ( set, "min", "=", real, "max", "=", real
                  | "minim" ), priority

phys_creep     = "phys.creep", set, set,
                "d", "=", real, priority

phys_layer     = "phys.layer", set, layer_ref, priority

phys_board     = "phys.board",
                "N=", int,
                "T=", list,
                "W=", list,
                ["M=", list]

phys_shrinknet = "phys.shrinknet", net, priority

phys_mkplane   = "phys.mkplane", layer_ref, net

phys_viafill   = "phys.viafill", pad,
                "v", "=", real, "d", "=", real,
                ["p", "=", pattern],
                ["l", "=", int]

phys_connectioncurrent = "phys.connectioncurrent",
                (connection | set), real, priority

phys_align     = "phys.align", list,
                "axis", "=", ("x" | "y"),
                "d", "=", real, priority

phys_connmatch = "phys.connmatch",
                match_property, set, priority

```



```

phys_mktext = "phys.mktext",
              "txt", string,
              ["font", string],
              ["size", real],
              ["material", string],
              ["layer", layer_ref]

phys_unite   = "phys.unite", "{", part, {"",",", part}, "}"
part         = id

```

4.3.4 System Statements

```

syst_statement = syst_flag | syst_cm | syst_alg

syst_flag = "syst.flag.", flag_name, (real | string_type)

flag_name = id

syst_cm    = "syst.cm.calc",
            ("approx" | "calculus" | "hybrid")

syst_alg   = "syst.alg", id

sys_var    = "$$", sys_var_name

sys_var_name = "sys_mem_SHA512"
              | "sys_temp_", id
              | "sys_temp_available_sensors"

```

4.3.5 Additional Grammar Rules

```

set_builder   = "{", variable, "E", ( set | list ),
              ":", expression, [ "~", expression ], "}"

set_query     = "{", set_expr, "@~", "[",
              expression, "]",
              [ "~[", expression, "]" ],
              "~>", [ variable ], "}"

lambda        = "L::", "(", [variable, {"",",", variable}], ")",
              "->", "{", { statement }, expression, "}"

conditional_expr = expression, "?->",
                  "(", expression, ")",
                  "!", "(", expression, ")"

```

```
pipe_syntax    = elec_connect, { "->", elec_connect }
```

4.3.6 Common Rules

```
layer_ref      = int | "T" | "B"  
priority       = ["-d" | "-nth" | "-imp" | "-vimp"  
                  | "-oeo" | "-eo" | "-ne" | "-nonneg"]  
pattern        = "grid" | "hex" | "stagger"  
                | "radial" | "random"  
match_property = "l" | "res" | "L" | "cap"  
                | "imp" | "t"  
net            = id  
pad            = id | int
```

Chapter 5

Types

This Section defines Type Literals using EBNF and RegEx.

5.1 Generic Types

5.1.1 Boolean

Boolean Data Type Definition

The Boolean data type represents logical truth values. It has exactly two possible values: **True** and **False**. Booleans are used in conditional expressions, logical operations, and control flow decisions.

Boolean Data Type Literals

```
boolean_literal = "True" | "False"
```

Boolean literals are case-sensitive: only **True** and **False** (capitalized) are valid. Lowercase **true/false** are not recognized as boolean literals.

Boolean Data Type Casting and Coercion

Boolean values can be coerced from other types:

- Int and real values coerce to **False** if zero, **True** otherwise
- Strings coerce to **False** if empty, **True** otherwise
- Lists and Sets coerce to **False** if empty, **True** otherwise
- **None** coerces to **False**

Explicit conversion is not supported; boolean evaluation is implicit in conditional contexts.

5.1.2 Null

Null Data Type Definition

The Null data type represents the absence of a value. Null is a special singleton value that indicates “no value”. Variables can be explicitly assigned null. Functions may return null to indicate failure or missing data.

Null Data Type Literals

```
null_literal = "null"
```

- `null` is case-sensitive: only lowercase `null` is valid.
- `null` is distinct from `None`. `None` is the implicit return value of functions that do not explicitly return; `null` explicitly represents the absence of a value.
- Accessing a member or index of `null` raises a `Runtime Error` unless the safe access operator (`?.`) or safe indexing operator (`?[]`) is used.

Null Coercion

- `null` coerces to `False` in boolean contexts.
- `null` coerces to 0 in numeric contexts (raises `Type Error` for explicit casting).

Null Coalescing

The null coalescing operator `??` returns the left operand if it is not `null`, otherwise the right operand:

```
a ?? b    Returns a if a is not null, else b
```

Safe Access

The safe member access operator `?.` returns `null` if the left operand is `null`, instead of raising a `Runtime Error`:

```
obj?.member    Returns obj.member if obj is not null, else null
```

The safe indexing operator `?[]` returns `null` if the left operand is `null`:

```
array?[index]    Returns array[index] if array is not null, else null
```

Null Examples

<code>\$x [int]= null</code>	<code>#c# \$x is null</code>
<code>\$y = \$x ?? 5</code>	<code>#c# \$y = 5 (null coalescing)</code>
<code>\$obj?.property</code>	<code>#c# returns null if \$obj is null</code>
<code>\$arr?[0]</code>	<code>#c# returns null if \$arr is null</code>

5.1.3 Integer

Integer Data Type Definition

The Integer data type represents whole numbers (elements of $\mathbb{Z}$). Integers support standard arithmetic operations and can express values from very large negative to very large positive using SI decimal prefixes (kilo and above).

Integer Data Type Literals

Integer literals can be expressed in multiple formats:

EBNF:

```
int_literal = decimal_int | binary_literal
             | octal_literal | hex_literal | bin_exp_int
```

```
decimal_int = ["+" | "-"], digit, {digit}, [si_prefix | bin_prefix]
```

```
binary_literal = "0b", bindigit, {bindigit}, [si_prefix | bin_prefix]
```

```
octal_literal = "0o", octdigit, {octdigit}, [si_prefix | bin_prefix]
```

```
hex_literal = "0x", hexdigit, {hexdigit}, [si_prefix | bin_prefix]
```

```
bin_exp_int = ["+" | "-"], digit, {digit}, ("w" | "W"), "+", digit, {digit}
```

```
si_prefix = "q" | "r" | "y" | "z" | "a" | "f" | "p" | "n" | "u" | "m"
           | "c" | "d" | "h" | "D"
           | "k" | "M" | "G" | "T" | "P" | "E" | "Z" | "Y" | "R" | "Q"
```

```
bin_prefix = "ki" | "Mi" | "Gi" | "Ti" | "Pi" | "Ei" | "Zi" | "Yi" | "Ri" | "Qi"
```

RegEx:

```
digit = [0-9]
```

```
bindigit = [01]
```

```
octdigit = [0-7]
```

```
hexdigit = [0-9a-fA-F]
```

SI prefixes $\geq k$ and scientific notation produce integers when the result is numerically equal to an integer:

```
6.2k    ## = 6200 (int)
1M      ## = 1000000 (int)
-5e9    ## = -5000000000 (int, exact)
5e-1    ## = 0.5 (real, not exact)
4Q      ## = 4e30 (int)
```

Binary (IEC 60027-2) prefixes produce integers by multiplying with powers of 2:

```
1ki     ## = 1024 (210)
4Mi     ## = 4194304 (220)
2Gi     ## = 2147483648 (230)
1Ti     ## = 1099511627776 (240)
0x10ki  ## = 16384 (0x10 * 1024)
```

Binary, octal, and hex literals always produce integers:

```
0x10    ## = 16
0b1010  ## = 10
0o17    ## = 15
```

Binary exponent notation uses **w**/**W** (mnemonic: **w**ide) to multiply a mantissa by 2^{exponent} . When the mantissa has no decimal point and the exponent is ≥ 0 , the result is an integer:

```
4w10    #c# = 4096 (4 * 2^10)
5W1     #c# = 10  (5 * 2^1)
1w0     #c# = 1   (1 * 2^0)
```

Integer Data Type Casting and Coercion

Integers can be coerced from other types:

- Booleans coerce to 0 (False) or 1 (True)
- Numeric strings with no decimal point are parsed as integers (e.g., "42" becomes 42)
- real values are truncated (e.g., 3.7 cast to `int` becomes 3)

5.1.4 Real

Real Data Type Definition

The Real data type represents floating-point numbers (elements of $\mathbb{R}$). Reals support standard arithmetic operations and can express values from very small (quecto-scale, 10^{-30}) to very large (quetta-scale, 10^{30}) using SI decimal prefixes.

Real Data Type Literals

Real literals can be expressed in multiple formats:

EBNF:

```
real_literal = decimal_real | scientific_real | bin_exp_literal
```

```
decimal_real  = ["+" | "-"], digit, {digit}, ".", {digit}, [si_prefix]
               | ["+" | "-"], ".", digit, {digit}, [si_prefix]
```

```
scientific_real = ["+" | "-"], digit, {digit},
                  [ "." , {digit} ], ("e" | "E"),
                  ["+" | "-"], digit, {digit}
               | ["+" | "-"], ".", digit, {digit},
                  ("e" | "E"),
                  ["+" | "-"], digit, {digit}
```

```
bin_exp_literal = ["+" | "-"], digit, {digit}, [ "." , {digit} ],
                  ("w" | "W"), ["+" | "-"], digit, {digit}
               | ["+" | "-"], ".", digit, {digit},
                  ("w" | "W"), ["+" | "-"], digit, {digit}
```

A literal with a decimal point or scientific notation produces a real, even if the fractional part is zero:

```

3.14    #c# = 3.14 (real)
42.0    #c# = 42.0 (real)
42e0    #c# = 42.0 (real)
.4      #c# = 0.4 (real)
.5e2    #c# = 50.0 (real, leading-dot scientific)
2e-6    #c# = 0.000002 (real)

```

Binary exponent notation produces a real when the mantissa has a decimal point or the exponent is negative:

```

1w-3    #c# = 0.125 (1 * 2^-3)
3.5w2   #c# = 14.0 (3.5 * 2^2, real because decimal mantissa)
.5w-1   #c# = 0.25 (0.5 * 2^-1)

```

SI prefixes below k produce reals:

```

100n    #c# = 0.0000001 (real)
1m      #c# = 0.001 (real)
5c      #c# = 0.05 (real)

```

The full SI prefix table:

Prefix	Value	Name	Prefix	Value	Name
<i>Sub-unit (< 1)</i>			<i>Multiple (≥ 1)</i>		
q	10 ⁻³⁰	quecto	k	10 ³	kilo
r	10 ⁻²⁷	ronto	M	10 ⁶	mega
y	10 ⁻²⁴	yocto	G	10 ⁹	giga
z	10 ⁻²¹	zepto	T	10 ¹²	tera
a	10 ⁻¹⁸	atto	P	10 ¹⁵	peta
f	10 ⁻¹⁵	femto	E	10 ¹⁸	exa
p	10 ⁻¹²	pico	Z	10 ²¹	zetta
n	10 ⁻⁹	nano	Y	10 ²⁴	yotta
u	10 ⁻⁶	micro	R	10 ²⁷	ronna
m	10 ⁻³	milli	Q	10 ³⁰	quetta
c	10 ⁻²	centi	h	10 ²	hecto
d	10 ⁻¹	deci	D	10 ¹	deca

Prefixes ≥ k produce integers. Prefixes < k produce reals.

The full binary (IEC 60027-2) prefix table:

Prefix	Value	Decimal	Name
ki	2 ¹⁰	1 024	kibi
Mi	2 ²⁰	1 048 576	mebi
Gi	2 ³⁰	1 073 741 824	gibi
Ti	2 ⁴⁰	1 099 511 627 776	tebi
Pi	2 ⁵⁰	1.1259 × 10 ¹⁵	pebi
Ei	2 ⁶⁰	1.1529 × 10 ¹⁸	exbi
Zi	2 ⁷⁰	1.1806 × 10 ²¹	zebi
Yi	2 ⁸⁰	1.2089 × 10 ²⁴	yobi
Ri	2 ⁹⁰	1.2379 × 10 ²⁷	robi
Qi	2 ¹⁰⁰	1.2677 × 10 ³⁰	quebi

Binary prefixes always produce integers.

Real Data Type Casting and Coercion

Reals can be coerced from other types:

- Integers coerce to real (e.g., `real(5)` becomes `5.0`)
- Booleans coerce to `0.0` (False) or `1.0` (True)
- Numeric strings with a decimal point are parsed as reals (e.g., `"3.14"` becomes `3.14`)
- Non-numeric strings raise a type error

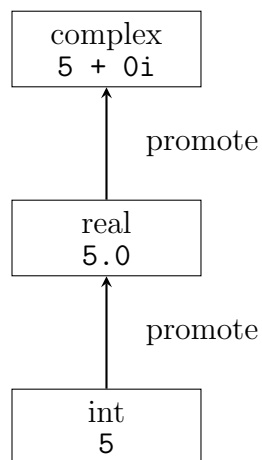
Lists of ints or reals can be passed to mathematical functions, which operate element-wise.

5.1.5 Complex

Complex Data Type Definition

The Complex data type represents complex numbers (elements of $\mathbb{C}$). A complex number has a real part and an imaginary part, and can be expressed in rectangular or polar form.

Type Hierarchy



Auto-promotion is implicit and lossless: an `int` can be used wherever a `real` is expected, and both can be used wherever a `complex` is expected.

Complex Data Type Literals

EBNF:

```
complex_literal = rectangular_literal | polar_literal
```

```
rectangular_literal = ["+" | "-"], number, ["+", number, "i"]  
                    | ["+" | "-"], number, "-", number, "i"  
                    | ["+" | "-"], number, "i"
```

```
polar_literal = number, "<|", number
```

The `i` suffix on a number creates a complex literal. The `<|` operator creates a polar literal (magnitude $\angle$ — angle in radians).

Rectangular Form

```
3 + 4i      #c# real=3, imag=4
-2 - 7i     #c# real=-2, imag=-7
5i          #c# real=0, imag=5
3           #c# real=3, imag=0 (int promoted to complex)
3.14 + 2.71i #c# real=3.14, imag=2.71
```

Note: `3 + 4i` is a single complex literal in rectangular form. The `+` is part of the literal, not the addition operator.

Polar Form

```
2 <| 1.5708  #c# magnitude=2, angle=pi/2 (radians)
1 <| 0       #c# magnitude=1, angle=0 (real number 1)
1 <| 3.14159 #c# magnitude=1, angle=pi (real number -1)
```

Complex Data Type Casting and Coercion

- `int` promotes to `complex`: `5` becomes `5 + 0i`
- `real` promotes to `complex`: `3.14` becomes `3.14 + 0i`
- Mixed operations (`int + complex`, `real + complex`) produce `complex`
- `complex` cannot be coerced to `real` or `int` (use `real()` or `int()` to extract parts)
- Booleans coerce to `complex`: `True` becomes `1 + 0i`
- Lists of `complex` can be passed to mathematical functions, which operate element-wise

Complex Components

The real and imaginary parts of a complex number are accessed via functions:

```
real($z)  #c# Returns the real part (real)
imag($z)  #c# Returns the imaginary part (real)
```

See Section 11.1.1 for full documentation.

5.1.6 Set

Set Data Type Definition

A Set is an unordered collection of unique elements. Sets can contain any data type:

- Integers and reals (numbers)
- Strings
- Booleans
- Lists
- Parts

- Connections
- Pins
- Nets

Sets containing mixed types are allowed. Duplicate elements are automatically removed. Sets support standard set algebra operations (union, intersection, difference, subset, element-of).

Set Data Type Literals

EBNF:

```
set_literal = "{" , [ { data , "," } , data ] , "}"
             | "{" , range_spec , "}"
```

Examples:

```
$numbers [set]= { 1, 2, 3, 4, 5 }
$mixed [set]= { 1, "hello", True, Q1 }
$parts [set]= { Q1, Q2, R3 }
```

Sets also support range generation using the same syntax as lists:

```
{start:increment:end}      #c# explicit increment
{start:end}                #c# increment defaults to 1
{start|end|count}          #c# evenly spaced, count elements
{start >: increment :< end} #c# exclusive start and end (colon)
{start >|< end | count}    #c# exclusive start and end (pipe)
```

Examples:

```
$evens [set]= {0:2:10}      #c# {0, 2, 4, 6, 8, 10}
$range [set]= {0:5}        #c# {0, 1, 2, 3, 4, 5}
$lin [set]= {0|1|5}        #c# {0, 0.25, 0.5, 0.75, 1}
$excl1 [set]= {0>:4}        #c# {1, 2, 3, 4}
$excl2 [set]= {0:<5}        #c# {0, 1, 2, 3, 4}
$excl3 [set]= {0>:<5}       #c# {1, 2, 3, 4}
$excl4 [set]= {0>:2:<10}    #c# {2, 4, 6, 8}
$excl5 [set]= {0>|<1|5}    #c# {0.1667, 0.3333, 0.5, 0.6667, 0.8333}
```

Note: since sets are unordered and unique, duplicates within a range are silently ignored. For example, {0:0.3:1} produces {0, 0.3, 0.6, 0.9} with no duplicate 0 entries.

Set Data Type Casting and Coercion

Sets can be created from Lists using the `f.set()` function. When converting a List to a Set, duplicate elements are removed and order is lost.

Example:

```
$my_list [list]= [ 1, 2, 2, 3, 3, 3 ]
$my_set [set]= f.set( $my_list ) #c# { 1, 2, 3 }
```

Sets can be converted to Lists using the `f.list()` function. The order of elements in the resulting List is determined by the order they appear in the global set `*e`.

5.1.7 String

String Data Type Definition

The String data type represents textual data. Strings can be delimited by single or double quotes, and support escape sequences. Long strings (multi-line) are delimited by triple quotes.

String Data Type Literals

EBNF:

```
string_literal = long_string | short_string
short_string  =  "'", {short_string_item_I}, "'" |
                '"', {short_string_item_II}, '"'
long_string   =  '"""', {long_string_item_I}, '"""' |
                '"""', {long_string_item_II}, '"""'
```

RegEx:

```
short_string_item_I = [^"\\n)]|(\\. )
short_string_item_II = [^'\\n)]|(\\. )
long_string_item_I = [^"\\]|(\\. )
long_string_item_II = [^'\\]|(\\. )
```

String Data Type Casting and Coercion

Strings can be created from other types using the `f.str()` function, which concatenates all arguments into a single string.

Example:

```
$msg [string]= f.str( "The value is ", 42,
    " and color is ", "red" )
## $msg = "The value is 42 and color is red"
```

Additional string functions include `f.upperc()` (uppercase), `f.lowerc()` (lowercase), and `f.format()` (formatted output).

5.1.8 List

List Data Type Definition

The List data type represents an ordered, indexed collection of objects. Lists support 0-based indexing, slicing, and can contain elements of different types. Lists are mutable and can be dynamically resized. Assignment creates a copy (not a reference).

List Data Type Literals

EBNF:

```
list_literal = "[", [{ data, "," }, data], "]"
              | "[", range_spec, "]"
```

```

set_literal = "{", [ { data, "," }, data ], "}"
              | "{", range_spec, "}"

range_spec  = expression, [ ">" ], ":", expression, [ ":", [ "<" ], expression ]
              | expression, [ ">" ], "|", [ "<" ], expression, "|", expression

slice       = [ expression ], ":", [ expression ], [ ":", expression ]

boolean_indexing = expression, "[", boolean_expression, "]"

tensor_indexing  = expression, "[", indexing_expr, "]"

indexing_expr    = expression                # single index or boolean
                  | expression, ",", expression # multi-dimensional
                  | slice_spec                # slice
                  | list_literal               # index list
                  | "...", "[", indexing_expr # ellipsis

slice_spec       = [ expression ], ":", [ expression ], [ ":", [ "-", ] expression ]

safe_tensor_indexing = expression, "?[", indexing_expr, "]"

reshape          = ".", "reshape", "(", int, { ",", int }, ")"

```

Tensor Data Type Literals

EBNF:

```

tensor_literal = "|", "[", [{ data, "," }, data], "]"
                | "|", "[", range_spec, "]"

```

The pipe prefix `|` distinguishes tensor literals from list literals. Tensors are explicitly typed and support the same range syntax as lists.

Warning Incorrect syntax: `| []` — this will cause immediate program termination. **Correct syntax:** `| []` — the pipe prefix is only at the opening bracket.

- `| []` — valid empty tensor
- `| [1, 2, 3]` — valid 1D tensor
- `| [[1,2], [3,4]]` — valid 2D tensor
- `| []|` — **SYNTAX ERROR** — program terminates

Exclusive Ranges The colon and pipe range syntax supports optional exclusive boundary modifiers. The `>` modifier (placed before `:` or `|`) excludes the start value. The `<` modifier (placed after `:` or `|`) excludes the end value. Both modifiers can be combined.

For colon ranges, exclusive boundaries shift the first or last element by one step:

- `[start >: end]` — exclusive start: elements begin at `start + increment` (default increment is 1)
- `[start :< end]` — exclusive end: elements stop at `end - increment`
- `[start >:< end]` — exclusive both
- `[start >: increment :< end]` — exclusive both with explicit increment

For pipe ranges, exclusive boundaries change the step calculation:

- `[start | end | count]` — inclusive: $\text{step} = \frac{\text{end} - \text{start}}{\text{count} - 1}$, requires `count` ≥ 2
- `[start >| end | count]` — exclusive start: $\text{step} = \frac{\text{end} - \text{start}}{\text{count}}$, first element is `start + step`
- `[start |< end | count]` — exclusive end: $\text{step} = \frac{\text{end} - \text{start}}{\text{count}}$, last element is `end - step`
- `[start >|< end | count]` — exclusive both: $\text{step} = \frac{\text{end} - \text{start}}{\text{count} + 1}$, first element is `start + step`

List Data Type Casting and Coercion

Lists can be created from Sets using the `f.list()` function. The order is determined by the element order in `*e`.

Lists support range generation using the syntax:

```
[start:increment:end]      #c# explicit increment
[start:end]                #c# increment defaults to 1
[start|end|count]          #c# evenly spaced, count elements
[start >: increment :< end] #c# exclusive start and end (colon)
[start >|< end | count]    #c# exclusive start and end (pipe)
```

Examples:

```
$r1 [list]= [0:2:10]      #c# [0, 2, 4, 6, 8, 10]
$r2 [list]= [0:5]         #c# [0, 1, 2, 3, 4, 5]
$r3 [list]= [0|1|5]       #c# [0, 0.25, 0.5, 0.75, 1]
$r4 [list]= [0>:4]        #c# [1, 2, 3, 4]
$r5 [list]= [0:<5]         #c# [0, 1, 2, 3, 4]
$r6 [list]= [0>:<5]        #c# [1, 2, 3, 4]
$r7 [list]= [0>:2:<10]     #c# [2, 4, 6, 8]
$r8 [list]= [0>|<1|5]     #c# [0.1667, 0.3333, 0.5, 0.6667, 0.8333]
```

Tensor

Definition A Tensor is a subtype of List that is **rectangular** (all sublists have the same length) and contains only **numeric** values (`bool`, `int`, `real`, or `complex`). Tensors are explicitly typed using the `[tensor]` annotation or the pipe-prefixed literal syntax.

Historical Note Tensors were previously called “ndarrays” (N-dimensional arrays) in earlier versions of this specification. The name has been changed to “tensor” to better reflect the mathematical concept and to distinguish the explicit type from the old auto-detected behavior.

Type Hierarchy Tensor is a subtype of List:

- A Tensor IS-A List: all list operations work on tensors
- A List IS NOT necessarily a Tensor: lists may have mixed types or non-rectangular shapes
- Tensor elements must be numeric (`bool`, `int`, `real`, or `complex`)
- Tensor shape is fixed at creation time

Creation Tensors are created using:

- Explicit type annotation: `$var [tensor]= value`
- Pipe-prefixed literal: `| [1, 2, 3]` or `| [[1,2],[3,4]]`
- Function return values (e.g., `f.linalg.inv`, `f.tnsr.zeros`)

Shape The shape of a Tensor is described by its dimensions. Tensors support **arbitrary dimensionality**:

- 1D: `| [1, 2, 3]` has shape (3)
- 2D: `| [[1,2], [3,4], [5,6]]` has shape (3, 2) (3 rows, 2 columns)
- 3D: `| [[[1,2],[3,4]],[[5,6],[7,8]]]` has shape (2, 2, 2) (2 layers, 2 rows, 2 columns)
- 4D: `| [[[[1,2],[3,4]],[[5,6],[7,8]]],[[9,10],[11,12]],[[13,14],[15,16]]]` has shape (2, 2, 2, 2)
- nD: arbitrary nesting depth, all dimensions must be rectangular
- Shape is immutable after creation

Validation Rules A list literal is a valid Tensor if and only if:

1. All elements are `bool`, `int`, `real`, or `complex` (no strings, functions, or other types)
2. All sublists (if any) have identical length (rectangular)
3. Nesting is consistent: if element i is a list, all elements $j \neq i$ must also be lists of the same length

Examples

- `| [1, 2, 3]` — valid tensor (1D, shape (3))
- `| [1+2i, 3+4i]` — valid tensor (1D, shape (2), complex)
- `| [[1,2], [3,4], [5,6]]` — valid tensor (2D, shape (3,2))
- `| [[1+2i, 3], [4, 5+6i]]` — valid tensor (2D, shape (2,2), complex)
- `| [[[1,2], [3,4]], [[5,6], [7,8]]]` — valid tensor (3D, shape (2,2,2))
- `| [[1,2], [3,4,5]]` — INVALID (different lengths)
- `| [1, "hello", True]` — INVALID (non-numeric)
- `| []` — valid tensor (1D, shape (0))

Broadcasting When a Tensor is combined with a scalar, the scalar is broadcast to match the Tensor's shape:

```
$t [tensor]= |[1, 2, 3]
$t + 5          #c# = |[6, 7, 8]
$t * 2          #c# = |[2, 4, 6]
$m [tensor]= |[ [1,2], [3,4]]
$m + 10         #c# = |[ [11,12], [13,14]]
```

Element-wise Operations When two Tensors of the same shape are combined with a standard operator (+, -, *, /, %, ^), the operation is applied element-wise:

```
$a [tensor]= |[1, 2, 3]
$b [tensor]= |[4, 5, 6]
$a + $b      #c# = |[5, 7, 9]
$a * $b      #c# = |[4, 10, 18]
```

Slicing Tensors (and all lists) support Python-style slicing with the syntax `list[start:stop:step]:`

- `list[start:stop]` – elements from `start` up to (not including) `stop`
- `list[start:]` – from `start` to end
- `list[:stop]` – from beginning to `stop`
- `list[:]` – full copy
- `list[start:stop:step]` – with step size
- Negative indices count from the end: `list[-1]` is the last element
- Negative zero is equivalent to zero: `list[-0]` is the same as `list[0]`

```

$a [tensor]= |[0, 1, 2, 3, 4, 5]
$a[1:4]      #c# = |[1, 2, 3]
$a[2:]       #c# = |[2, 3, 4, 5]
$a[:3]       #c# = |[0, 1, 2]
$a[:,2]      #c# = |[0, 2, 4] (every 2nd element)
$a[-2:]      #c# = |[4, 5]    (last 2 elements)
$a[::-1]     #c# = |[5, 4, 3, 2, 1, 0] (reversed)

```

Reshape Tensors support reshaping via the `.reshape()` method. The total number of elements must remain constant.

```

$b [tensor]= |[[1,2,3],[4,5,6]]
$b.reshape(3, 2) #c# = |[[1,2],[3,4],[5,6]]
$b.reshape(6)    #c# = |[1, 2, 3, 4, 5, 6] (flatten)
$b.reshape(1, 6) #c# = |[[1,2,3,4,5,6]]    (row vector)

```

Tensor Operators Tensors support special operators prefixed with `..`. Without the prefix, operations are element-wise.

- `.*` — matrix multiply (dot product for 1D, matrix multiply for 2D)
- `.T` — transpose (postfix operator)
- `.T` — transpose in place
- `.H` — conjugate transpose (postfix operator)
- `.H` — conjugate transpose in place
- `.E` — erase (sets variable to null)
- `.^n` — matrix power (n must be int $\neq 0$)

See Section 8 for detailed operator descriptions and edge cases.

Tensor Functions Tensors have dedicated functions in the `f.arr`, `f.stat`, and `f.linalg` namespaces:

- Creation: `f.tnsr.zeros`, `f.tnsr.ones`, `f.tnsr.identity`
- Inspection: `f.tnsr.flatten`, `f.tnsr.norm`, `f.tnsr.normalize`
- Statistics: `f.stat.sum`, `f.stat.prod`, `f.stat.mean`, `f.stat.median`, `f.stat.mode`, `f.stat.std`, `f.stat.var`, `f.stat.min`, `f.stat.max`, `f.stat.argmin`, `f.stat.argmax`, `f.stat.peak`, `f.stat.rms`, `f.stat.percentile`, `f.stat.quantile`, `f.stat.histogram`, `f.stat.correlation`, `f.stat.covariance`, `f.stat.linearRegression`
- Linear algebra: `f.linalg.det`, `f.linalg.inv`, `f.linalg.pinv`, `f.linalg.rank`, `f.linalg.trace`, `f.linalg.solve`, `f.linalg.eig`, `f.linalg.eigvec`, `f.linalg.lu`, `f.linalg.qr`, `f.linalg.chol`, `f.linalg.svd`, `f.linalg.cond`

See Section ?? for detailed function descriptions.

5.2 Function

5.2.1 Function Data Type Definition

The Function data type represents a callable value. A function value encapsulates either a built-in function (referenced by name, e.g., `f.math.sin`), a user-defined function (referenced by name, e.g., `f.u.myfunc`), or a lambda expression. Function values can be stored in variables, passed as arguments, returned from functions, and stored in collections.

5.2.2 Function Data Type Creation

Function values are created in two ways:

Function Reference

A function reference names a function without calling it. The reference is created by writing the function name without parentheses:

```
$f t= f.math.sin      ## $f holds a reference to sin
$g t= f.u.myfunc      ## $g holds a reference to user-defined function
```

- Built-in references use the `f.namespace.name` form (no parentheses)
- User-defined references use the `f.u.name` form (no parentheses)
- A `Type Error` is raised if the name does not resolve to a function

Lambda Expression

A lambda expression creates an anonymous function value:

```
$fn t= L::($x [int])->{ $x * 2 }
```

See Section 8.1.13 for lambda syntax details.

5.2.3 Function Data Type Calling

Function values are called using the variable-call syntax `$var(args)`:

```
$f t= f.math.sin
$g t= L::($x [int])->{ $x ^ 2 }
```

```
$f(0)          ## Returns 0.0  (calls f.math.sin)
$g(5)          ## Returns 25   (calls the lambda)
```

- The variable must hold a `func` value; a `Type Error` is raised otherwise
- A `Runtime Error` is raised if the variable holds `null`
- Arguments are evaluated left-to-right before the function is called
- The function must accept the number and types of arguments provided; a `Type Error` is raised otherwise

5.2.4 Function Data Type Annotations

Variables can be annotated with the `func` type:

```
$fn [func]= f.math.cos
$fn [func]= L::($x [int])->{ $x + 1 }
$fn [func|= null]= null
```

5.2.5 Function Data Type Equality

Function equality is determined by reference identity:

- Two function references to the same function are equal: `f.math.sin == f.math.sin` $\rightarrow$ `True`
- Two distinct built-in references are not equal: `f.math.sin == f.math.cos` $\rightarrow$ `False`
- A lambda is always unique: two separate lambda expressions are never equal, even with identical bodies
- A lambda is not equal to a built-in reference to the same logical function
- `==` and `!=` work on function values
- Ordered comparisons (`<`, `<=`, `>=`, `>`) raise a `Type Error`

5.2.6 Function Data Type Collections

- Lists can contain function values: `[f.math.sin, f.math.cos]` is a valid list
- Sets can contain function values but this is impractical (lambda equality is always unique), so set membership checks on function values are unreliable
- Tensors cannot contain function values (tensor elements must be numeric)

5.2.7 Function Data Type Printing

Function values print as their reference form:

```
f.util.print(f.math.sin)          #c# Prints: f.math.sin
f.util.print(L::($x [int])->{ $x }) #c# Prints: <lambda>
```

5.3 Electrical Types

5.3.1 Part

Part Data Type Definition

A `Part` represents a physical electronic component in the circuit. Each `Part` has a reference designator, a model type, a description, and a footprint. `Parts` contain `Pins` which can be connected to `Nets` or other `Pins`.

Part Data Type Literals

Parts are created using the `elec.part` statement:

EBNF:

```
part_statement = "elec.part", part_reference,  
    ":", model_name, "(", [model_params], ")",  
    [";", "des", "=", string_type],  
    [";", "fp", "=", string_type]
```

```
part_reference = string_type |  
    "[", { string_type, "," }, string_type, "]"  
model_name      = id  
model_params    = [ { id, "=", data, "," },  
                    id, "=", data ]
```

Examples:

```
elec.part "R1": r( R = 100, P = 0.25,  
    type = "metal film" )  
    ; des = "100 ohm resistor"  
    ; fp = "Resistor_SMD:R_0805_2012Metric"  
  
elec.part ["R2", "R3"]: r( R = 220, P = 0.25 )  
    ; des = "220 ohm resistors"; fp = "Resistor_SMD:R_0805_2012Metric"
```

Part Data Type Casting and Coercion

Parts cannot be cast from other types. Parts are created via `elec.part` statements and accessed by their reference designator. Parts support the following properties:

- Model parameters (accessed via model-specific syntax)
- Pin references (using `part_ref[pin_ref]` syntax)
- Description (set via `des` parameter)
- Footprint (set via `fp` parameter)

5.3.2 Pin

Pin Data Type Definition

A Pin represents a connection point on a Part. Pins are referenced using the syntax `part_ref[pin_ref]`, where `pin_ref` is typically a numeric index or a named terminal (e.g., `R1[1]`, `Q1[d]`, `U1[v+]`).

Pin Data Type Literals

Pins are created implicitly when Parts are instantiated. Pin references are formed using the bracket notation:

```
pin_literal = part_ref, "[", pin_ref, "]"
```

Examples:

```
R1[1]      #c# Pin 1 of resistor R1
Q1[d]      #c# Drain pin of MOSFET Q1
U1[v+]     #c# Non-inverting input of op-amp U1
J1[3]      #c# Pin 3 of connector J1
```

The available pin references depend on the Part's footprint and model definition.

Pin Data Type Casting and Coercion

Pins cannot be cast from other types. Pins are accessed via Part references and support the following operations:

- Connection to other Pins or Nets via `elec.connect`
- Membership testing in Sets
- Iteration over Pins of a Part

5.3.3 Connection

Connection Data Type Definition

A Connection represents an electrical connection between two Pins. Connections are binary (exactly two Pins) and are created using the pipe syntax or the `elec.connect` statement.

Connection Data Type Literals

Connections can be created using the pipe syntax:

```
connection_literal = "(", pin_literal, "|", pin_literal, ")"
```

Example:

```
( R2[1] | J4[7] ) #c# R2 pin 1 to J4 pin 7
```

Connections are also created implicitly by `elec.connect` statements.

Connection Data Type Casting and Coercion

Connections cannot be cast from other types. Connections support:

- Membership testing in Sets
- Property queries (length, resistance, inductance, capacitance, impedance)

5.3.4 Net

Net Data Type Definition

A Net represents a named electrical node that connects multiple Pins and Connections. Nets are the fundamental grouping mechanism in the electrical system, allowing multiple components to share a common electrical connection.

Net Data Type Literals

Nets are created using the `elec.net` statement:

EBNF:

```
net_statement = "elec.net", string_type, { string_type }
```

Example:

```
elec.net "Gnd"
```

```
elec.net "hvbus" "hv" "high_i" "noisy"
```

The first argument is the net name; subsequent arguments are optional tags.

Net Data Type Casting and Coercion

Nets cannot be cast from other types. Nets support:

- Tagging/untagging via `elec.tag` and `elec.untag`
- Membership testing in Sets
- Connection to Pins via `elec.connect`
- Querying connected objects using quantifier syntax (e.g., `I@Gnd`, `P@Vin`)

5.4 Type Casting with `as`

The `as` keyword performs explicit type conversion. It is an expression-level operator that converts a value from one type to another.

5.4.1 Syntax

EBNF:

```
type_cast = expression, "as", type_name
```

```
type_name = "int" | "real" | "bool" | "complex"  
           | "list" | "tensor"
```

5.4.2 Numeric Conversions

`int as real` Promotes an integer to a real value.

```
5 as real          #c# Returns 5.0  
0 as real          #c# Returns 0.0  
-42 as real        #c# Returns -42.0
```

`real as int` Truncates a real to an integer (towards zero).

```
3.7 as int         #c# Returns 3  
-2.9 as int        #c# Returns -2  
0.5 as int         #c# Returns 0
```

bool as int Converts boolean to integer.

```
True as int      #c# Returns 1
False as int     #c# Returns 0
```

int as bool Converts integer to boolean.

```
0 as bool        #c# Returns False
1 as bool        #c# Returns True
-5 as bool       #c# Returns True
```

real as bool Converts real to boolean.

```
0.0 as bool      #c# Returns False
0.5 as bool      #c# Returns True
```

5.4.3 Complex Conversions

int as complex Promotes an integer to complex with zero imaginary part.

```
5 as complex     #c# Returns 5 + 0i
-3 as complex    #c# Returns -3 + 0i
```

real as complex Promotes a real to complex with zero imaginary part.

```
3.14 as complex  #c# Returns 3.14 + 0i
```

complex as real Returns the real part if the imaginary part is zero. A warning is issued if the imaginary part is non-zero (returns the magnitude).

```
(3 + 0i) as real  #c# Returns 3.0
(3 + 4i) as real  #c# Warning: imaginary part lost; returns 5.0
```

complex as int Truncates the real part to integer if the imaginary part is zero. A warning is issued if the imaginary part is non-zero (truncates the magnitude).

```
(5 + 0i) as int   #c# Returns 5
(3.7 + 0i) as int #c# Returns 3
(3 + 4i) as int   #c# Warning: imaginary part lost; returns 3
```

5.4.4 List/Tensor Conversions

list as tensor Converts a list to a tensor. All elements must be the same type (bool, int, real, or complex). A `Type Error` is raised if the list contains mixed types.

```
[1, 2, 3] as tensor      #c# Returns tensor [1, 2, 3]
[1.0, 2.0, 3.0] as tensor #c# Returns tensor [1.0, 2.0, 3.0]
[1, 2.0, 3] as tensor    #c# Type Error (mixed int/real)
```

`tensor as list` Flattens a tensor to a 1D list.

```
| [1, 2, 3] as list          ## Returns [1, 2, 3]
| [[1,2],[3,4]] as list    ## Returns [1, 2, 3, 4]
```

5.4.5 Error Handling

A `Type Error` is raised for unsupported conversions (e.g., `string as int`, `list as real`).

5.4.6 Edge Cases

- A `Type Error` is raised if the conversion is not supported between the given types.
- A warning is issued for `complex as real` or `complex as int` when the imaginary part is non-zero.
- `list as tensor` requires all elements to be the same scalar type; mixed types raise a `Type Error`.
- `tensor as list` flattens all dimensions to a single list.

Chapter 6

Variables

Variables in EDADSL are prefixed with the `$` character and are assigned values using type-specific assignment operators.

6.1 Variable Declaration and Assignment

Variables are declared and assigned simultaneously using the following syntax:

EBNF:

```
variable          = "$", id
variable_assignment = variable, (typed_assign | inferred_assign),
                        [";", "M", "=", boolean_data_type]

typed_assign      = "[", type, "]", "=", data
inferred_assign   = "t", "=", data

type              = "bool" | "int" | "real" | "complex"
                  | "list" | "tensor" | "string" | "set"
                  | "func"
                  | "part" | "connection" | "pin"
                  | "net"
```

6.1.1 Type-Inferred Assignment

As syntax sugar, `$var t= val` is equivalent to `$var [type(val)]= val`. The type is inferred from the value at assignment time.

```
$a t= 3          #c# Equivalent to $a [int]= 3
$name t= "EDADSL" #c# Equivalent to $name [string]= "EDADSL"
$flag t= True    #c# Equivalent to $flag [bool]= True
$colors t= [1, 2, 3] #c# Equivalent to $colors [list]= [1, 2, 3]
```

6.1.2 Variable Redeclaration

Variables can be redeclared with a different type. The new declaration overwrites the previous value and type:


```
$x [int]= 5          ## $x is int
$x [real]= 3.14      ## $x is now real
$x [string]= "hi"    ## $x is now string
```

The type determines the variable's data type:

- `bool` – Boolean
- `int` – Integer
- `real` – Real
- `complex` – Complex number
- `list` – List
- `string` – String
- `set` – Set
- `func` – Function
- `part` – Part
- `connection` – Connection
- `pin` – Pin
- `net` – Net

6.2 Mutability

By default, variables are mutable (can be reassigned). To make a variable immutable, append `; M = False` after the assignment:

```
$immutable [int]= 42; M = False
$mutable [int]= 100    ## Can be reassigned
```

6.3 Scope

Variables declared at the top level of the program have **global scope** and are visible from their declaration to the end of the program. Variables declared inside a code block (`if`, `for`, `while`, `do-while`) are **local to that block** and are destroyed when the block exits. Variables declared inside a function are local to that function (see Section 11.2.4). A variable may be re-declared at the same scope level; the new declaration silently replaces the previous binding. Variables must be declared before they are accessed; reading an uninitialized variable raises a **Runtime Error**.

6.4 Examples

```
$a [int]= 3
$b [int]= 5
$result [int]= $a + $b          #c# Scalar assignment

$name [string]= "EDADSL"        #c# String assignment
$flag [bool]= True              #c# Boolean assignment

$colors [list]= ["red", "green", "blue"] #c# List
$hwset [set]= { Q1, Q2, R5 }      #c# Set
```

Chapter 7

Expressions

Expressions in EDADSL evaluate to values and can be composed of literals, variables, function calls, and operators.

7.1 Expression Grammar

EBNF:

```
expression          = arithmetic_expression
                    | boolean_expression
                    | comparison_expression
                    | set_expression
                    | postfix_expression
                    | pipeline_expression
                    | composition_expression
                    | function_call
                    | function_call_var
                    | lambda
                    | "(" expression ")"

pipeline_expression = expression, "|=>", function_ref
                    | expression, "|=>", lambda

composition_expression = expression, "<o", expression

function_ref         = "f", ".", namespace, ".", id
                    | "f.u.", id

function_call_var    = variable, "(", [{ data, ", " }], data, ")"

arithmetic_expression = expression,
    ("+" | "-" | "*" | "/" | "%" | "^" | "." | "x"
     | ".*" | ".^" | "**" | "***" | "****"),
    expression
    | "-" expression
```

```

| expression, (".T" | ".H")

boolean_expression = expression,
  ("&&" | "||" | "^"), expression
| "!", expression

comparison_expression = expression,
  ("<" | "<=" | "==" | "!=" | "=n=" | "><" | ">=" | ">"
  | "~" | "~*"), expression
| expression, "~+", "(", expression, "~", expression
| expression, "~*", "(", expression, "~", expression

set_expression = expression, ("u" | "n" | "/"), expression
| "||", expression
| expression, ("c" | "E" | "!E"), expression

null_expression = expression, "??", expression
| expression, "?.", id
| expression, "?[", expression, "]"

postfix_expression = variable, ("++" | "--")

compound_assignment = variable,
  ## arithmetic
  ("+=" | "-=" | "*=" | "/=" | "%=" | "^="
  | "+r=" | "-r=" | "/r=" | "%r=" | "^r=")
  ## set
| variable, ("u=" | "n=")
  ## vector
| variable, (".=" | "x=" | "xr=")
  ## tensor & secure erasure
| variable, (".*=" | ".^=" | ".*r=" | ".^r=")
| variable, ("**=" | "***=" | "*****=" | "**r=" | "***r=" | "*****r=")
| variable, ("~.T" | "~.H" | "~.E" | "~.E+")
  ## bitwise / shift
| variable, ("<<=" | ">>=" | "<<r=" | ">>r=")
| variable, ("<<<[" int_literal "]= "
| ">>>[" int_literal "]= "
| "<<<[" int_literal "]r="
| ">>>[" int_literal "]r=")
  ## boolean
| variable, ("&&=" | "||=" | "^=" | "!&=" | "!= " | "!^=")
  ## imply / n imply
| variable, ("->=" | "!-->=" | "->r=" | "!-->r="), expression
  ## bitwise logic
| variable, ("b&&=" | "b||=" | "b^=" | "b!&=" | "b!|= " | "b!^=" | "b->=" | "b!-->=" |

```

```

    #c# function composition
| variable, ("<o=" | "o>=")
| variable, "~b!"

```

7.2 Operator Precedence

Operators are evaluated in the following order of precedence (highest to lowest):

1. Hexation (****), Pentation (***), Tetration (**) – right-associative
2. Exponentiation (^) – right-associative
3. Boolean negation (!), Bitwise NOT (b!)
4. Postfix increment/decrement (\$var++, \$var--), transpose (.T), Hermitian transpose (.H), erase (.E), cryptographic shred (.E+)
5. Set cardinality (||a)
6. Multiplication (*), Division (/), Floor Division (//), Modulo (%), Dot Product (.), Cross Product (x), Matrix Multiply (.*), Matrix Power (.^)
7. Left Shift (<<), Right Shift (>>), Left Rotate (<<<[w]), Right Rotate (>>>[w])
8. Addition (+), Subtraction (-)
9. Comparison operators (<, <=, ==, !=, =n=, ><, >=, >, ~, ~*, ~+()~, ~*()~)
10. Type casting (as) – e.g., \$x as real
11. Set comparison (c, E, !E)
12. Logical AND (&&), NAND (!&), Bitwise AND (b&&), Bitwise NAND (b!&)
13. Logical OR (||), NOR (!|), Bitwise OR (b||), Bitwise NOR (b!|)
14. Logical XOR (^), XNOR (!^), Bitwise XOR (b^), Bitwise XNOR (b!^)
15. IMPLY (->), NIMPLY (!->), Bitwise IMPLY (b->), Bitwise NIMPLY (b!->)
16. Conditional expression (?->...!...)
17. Pipeline (|=>) – left-associative, applies RHS function to LHS value
18. Lambda expression (L::(...) -> {...})
19. User-defined operators (o.u.*)

Note: || has two distinct meanings depending on position. As a **prefix** operator (||*expr*), it denotes cardinality/length and has high precedence (level 4). As an **infix** operator (*expr* || *expr*), it denotes logical OR and has low precedence (level 10). Parentheses override precedence. See Section 8 for detailed operator descriptions.

7.3 Evaluation Order

Expressions are evaluated left-to-right for operators of equal precedence. Parentheses can be used to override precedence. Function arguments are evaluated before the function is called.

Chapter 8

Operators

8.1 Built-in Operators

8.1.1 Arithmetic

Arithmetic operators perform mathematical operations. Most operators support multiple type combinations.

```
n + m  Addition / Concatenation
n - m  Subtraction
n * m  Multiplication
n / m  Division (always returns real)
n // m Floor Division (returns int)
n % m  Modulo
n ^ m  Exponentiation
n ** m Tetration (n ↑↑ m)
n *** m Pentation (n ↑↑↑ m)
n **** m Hexation (n ↑ m)
n << m Left Shift (returns int)
n >> m Right Shift (returns int)
n <<<[w] m Left Rotate (returns int, w-bit word)
n >>>[w] m Right Rotate (returns int, w-bit word)
```

Type Overloading

- `+: int + int → int`
- `+: int + real → real`
- `+: real + int → real`
- `+: real + real → real`
- `+: string + string → string` (concatenation)
- `+: list + list → list` (append)
- `-: int - int → int`

- -: int - real $\rightarrow$ real
- -: real - int $\rightarrow$ real
- -: real - real $\rightarrow$ real
- *: int * int $\rightarrow$ int
- *: int * real $\rightarrow$ real
- *: real * int $\rightarrow$ real
- *: real * real $\rightarrow$ real
- *: int * string $\rightarrow$ string (repetition)
- *: int * list $\rightarrow$ list (replication)
- /: int / int $\rightarrow$ real (true division)
- /: real / int $\rightarrow$ real
- /: int / real $\rightarrow$ real
- /: real / real $\rightarrow$ real
- //: int // int $\rightarrow$ int (floor division)
- %: int % int $\rightarrow$ int
- %: real % int $\rightarrow$ real
- %: int % real $\rightarrow$ real
- %: real % real $\rightarrow$ real
- ^: int ^ int $\rightarrow$ int (non-negative exponents) or real (negative exponents)
- ^: int ^ real $\rightarrow$ real
- ^: real ^ int $\rightarrow$ real
- ^: real ^ real $\rightarrow$ real
- <<: int << int $\rightarrow$ int (left shift)
- >>: int >> int $\rightarrow$ int (right shift)
- <<<[w]: int <<<[w] int $\rightarrow$ int (left rotate)
- >>>[w]: int >>>[w] int $\rightarrow$ int (right rotate)
- +: int + complex $\rightarrow$ complex
- +: real + complex $\rightarrow$ complex
- +: complex + int $\rightarrow$ complex
- +: complex + real $\rightarrow$ complex

- $+$: $\text{complex} + \text{complex} \rightarrow \text{complex}$
- $-$: $\text{int} - \text{complex} \rightarrow \text{complex}$
- $-$: $\text{real} - \text{complex} \rightarrow \text{complex}$
- $-$: $\text{complex} - \text{int} \rightarrow \text{complex}$
- $-$: $\text{complex} - \text{real} \rightarrow \text{complex}$
- $-$: $\text{complex} - \text{complex} \rightarrow \text{complex}$
- $*$: $\text{int} * \text{complex} \rightarrow \text{complex}$
- $*$: $\text{real} * \text{complex} \rightarrow \text{complex}$
- $*$: $\text{complex} * \text{int} \rightarrow \text{complex}$
- $*$: $\text{complex} * \text{real} \rightarrow \text{complex}$
- $*$: $\text{complex} * \text{complex} \rightarrow \text{complex}$
- $/$: $\text{int} / \text{complex} \rightarrow \text{complex}$
- $/$: $\text{real} / \text{complex} \rightarrow \text{complex}$
- $/$: $\text{complex} / \text{int} \rightarrow \text{complex}$
- $/$: $\text{complex} / \text{real} \rightarrow \text{complex}$
- $/$: $\text{complex} / \text{complex} \rightarrow \text{complex}$
- $\wedge$: $\text{int} \wedge \text{complex} \rightarrow \text{complex}$
- $\wedge$: $\text{real} \wedge \text{complex} \rightarrow \text{complex}$
- $\wedge$: $\text{complex} \wedge \text{int} \rightarrow \text{complex}$
- $\wedge$: $\text{complex} \wedge \text{real} \rightarrow \text{complex}$
- $\wedge$: $\text{complex} \wedge \text{complex} \rightarrow \text{complex}$
- $**$: $\text{int} ** \text{int} \rightarrow \text{int}$ (arbitrary precision)
- $**$: $\text{int} ** \text{real} \rightarrow \text{real}$
- $**$: $\text{real} ** \text{int} \rightarrow \text{real}$
- $**$: $\text{real} ** \text{real} \rightarrow \text{real}$
- $**$: any complex operand $\rightarrow \text{complex}$
- $***$: same type rules as $**$
- $****$: same type rules as $**$
- For $**$, $***$, $****$: right operand must be `int`. A `Type Error` is raised otherwise.

Mixed-type operations promote to `real`. If any operand is `complex`, the result is `complex`. Floor division (`//`) truncates toward negative infinity. Modulo uses truncated division (`sign(result) = sign(dividend)`). Exponentiation is right-associative: $-2^3 = -8$.

Edge Cases

- Division by zero (`/`, `//`, `%`) raises a **Division by Zero Error**.
- Negative exponent on `int` base auto-promotes to `real`: `2 ^ -1` returns `0.5`.
- Shift operators (`<<`, `>>`) require `int` operands; a **Type Error** is raised otherwise. The right operand must be non-negative; a **Value Error** is raised for negative shift amounts.
- Rotation operators (`<<<[w]`, `>>>[w]`) require `int` operands; a **Type Error** is raised otherwise. The word width `w` must be a positive `int`; a **Value Error** is raised for non-positive widths. The rotation amount must be non-negative; a **Value Error** is raised otherwise.
- A **Type Error** is raised when operands are incompatible (e.g., `string + int`).

8.1.2 Hyperoperations

Hyperoperation operators compute iterated exponentiation and beyond. The right operand must always be `int`.

```
n ** m  Tetration      (n ↑↑ m)   iterated exponentiation
n *** m  Pentation     (n ↑↑↑ m)  iterated tetration
n **** m Hexation      (n ↑ m)   iterated pentation
```

Semantics

- `n ** m`: $n \uparrow\uparrow m = \underbrace{n^{n^{\cdot^{\cdot^{\cdot^n}}}}}_{m \text{ times}}$. Right-associative like $\wedge$.
- `n *** m`: $n \uparrow\uparrow\uparrow m = \text{iterated tetration}$.
- `n **** m`: $n \uparrow^6 m = \text{iterated pentation}$.
- The right operand must be `int` ≥ 0 . A **Type Error** is raised for non-integer right operand. A **Value Error** is raised for negative right operand.
- These operators are equivalent to `f.log.hyperop(4, n, m)`, `f.log.hyperop(5, n, m)`, `f.log.hyperop(6, n, m)` respectively.

Examples:

```
2 ** 4      #c# Returns 65536      (2 ↑↑ 4 = 2^2^2^2)
3 ** 3      #c# Returns 7625597484987 (3 ↑↑ 3 = 3^3^3)
2 ** 1      #c# Returns 2          (2 ↑↑ 1 = 2)
2 ** 0      #c# Returns 1          (2 ↑↑ 0 = 1 by convention)
2 *** 3     #c# Returns 65536      (2 ↑↑↑ 3 = 2 ↑↑ (2 ↑↑ 2))
2 **** 3    #c# Returns 65536      (2 ↑ 3)
```

Edge Cases

- Overflow: values grow astronomically fast. A **Value Error** is raised if the result exceeds 2^{20} bits.
- Complex operands: supported for `**` (tetration) with integer right operand. Returns `complex`.

8.1.3 Increment / Decrement

Postfix increment and decrement operators add 1 or subtract 1 from an int or real variable. The variable must already be declared.

```
$var++    Increment $var by 1 (returns old value)
$var--    Decrement $var by 1 (returns old value)
```

Semantics `$var++` increments `$var` by 1 and returns the original value before the increment. `$var--` decrements `$var` by 1 and returns the original value before the decrement.

Examples:

```
$counter [int]= 0
$counter++
## $counter = 1
```

```
$counter++
## $counter = 2
```

```
$counter--
## $counter = 1
```

```
## Can be used in expressions
$val [int]= $counter++    ## $val = 1, $counter = 2
```

8.1.4 Compound Assignment

Compound assignment operators combine an operation with assignment. They are available for all operators.

Arithmetic

```
## forward
$var += expr    $var = $var + expr
$var -= expr    $var = $var - expr
$var *= expr    $var = $var * expr
$var /= expr    $var = $var / expr
$var %= expr    $var = $var % expr
$var ^= expr    $var = $var ^ expr
```

```
## reverse (reversed operands)
$var +r= expr    $var = expr + $var
$var -r= expr    $var = expr - $var
$var /r= expr    $var = expr / $var
$var %r= expr    $var = expr % $var
$var ^r= expr    $var = expr ^ $var
```

Set

```
$var u= expr    $var = $var u expr
$var n= expr    $var = $var n expr
```

Vector

```
$var .= expr    $var = $var . expr    (dot product)
$var x= expr    $var = $var x expr    (cross product)
$var xr= expr   $var = expr x $var    (reverse cross product)
```

Tensor (Nddarray)

```
$var .*= expr   $var = $var .* expr   (matrix multiply)
$var .^= expr   $var = $var .^ expr   (matrix power)
$var .*r= expr  $var = expr .* $var   (reverse matrix multiply)
$var .^r= expr  $var = expr .^ $var   (reverse matrix power)
$var ~.T        $var = $var .T        (transpose in place)
$var ~.H        $var = $var .H        (hermitian transpose in place)
$var ~.E        $var = null           (erase, secure zeroize)
$var ~.E+       $var = null           (cryptographic shred, zeroize + overwrite)
```

Hyperoperations

```
$var **= expr   $var = $var ** expr   (tetration)
$var ***= expr  $var = $var *** expr  (pentation)
$var ****= expr $var = $var **** expr (hexation)
$var **r= expr  $var = expr ** $var   (reverse tetration)
$var ***r= expr $var = expr *** $var  (reverse pentation)
$var ****r= expr $var = expr **** $var (reverse hexation)
```

Bitwise / Shift

```
$var <=<= expr   $var = $var << expr
$var >>= expr   $var = $var >> expr
$var <<r= expr   $var = expr << $var
$var >>r= expr   $var = expr >> $var
$var <<<[w]= expr $var = $var <<<[w] expr
$var >>>[w]= expr $var = $var >>>[w] expr
$var <<<[w]r= expr $var = expr <<<[w] $var
$var >>>[w]r= expr $var = expr >>>[w] $var
```

Boolean

```
$var &&= expr   $var = $var && expr
$var ||= expr   $var = $var || expr
$var ^= expr    $var = $var ^^ expr
$var !=> expr   $var = !($var && expr)
$var !|= expr   $var = !($var || expr)
```

```
$var != expr  $var = $var == expr
```

IMPLY / NIMPLY

```
$var ->= expr  $var = !$var || expr  
$var !->= expr  $var = $var && !expr  
$var ->r= expr  $var = !expr || $var  
$var !->r= expr  $var = expr && !$var
```

Bitwise Logic

```
$var b&&= expr  $var = $var b&& expr  
$var b||= expr  $var = $var b|| expr  
$var b^^= expr  $var = $var b^^ expr  
$var b!&= expr  $var = $var b!& expr  
$var b!|= expr  $var = $var b!| expr  
$var b!^= expr  $var = $var b!^ expr  
$var b->= expr  $var = $var b-> expr  
$var b!->= expr  $var = $var b!-> expr  
$var b->r= expr  $var = expr b-> $var  
$var b!->r= expr  $var = expr b!-> $var  
$var ~b!        $var = b! $var
```

Function Composition

```
$var <o= expr  $var = $var <o expr  (compose-assign)  
$var o>= expr  $var = expr <o $var  (reverse compose-assign)
```

Examples:

```
$x [int]= 5  
$x += 3      #c# $x = 8  
$x *= 2      #c# $x = 16  
$x -= 4      #c# $x = 12  
$x /= 3      #c# $x = 4  
$x %= 3      #c# $x = 1  
$x ^= 3      #c# $x = 1
```

```
$str [string]= "hello"  
$str += " " + "world"  #c# $str = "hello world"  
$str +=r= "cd"         #c# $str = "cdhello"
```

```
$list [list]= [1, 2, 3]  
$list += [4, 5]      #c# $list = [1, 2, 3, 4, 5]
```

Reverse compound assignment operators reverse the operand order:

```
$x [int]= 5  
$x -r= 3      #c# $x = 3 - 5 = -2
```

```

$x /r= 2      #c# $x = 2 / 5 = 0.4
$x %r= 2      #c# $x = 2 % 5 = 2

$str [string]= "hello"
$str +r= "cd"  #c# $str = "cdhello"

$v [list]= [1, 2, 3]
$v xr= [0, 0, 1]  #c# $v = [0, 0, 1] x [1, 2, 3]

$x [int]= 5
$x .= 3      #c# $x = $x . 3 = 15 (in-place dot product)
$x x= [1, 0, 0]  #c# $x = $x x [1, 0, 0] (in-place cross product)

$b [bool]= True
$b != False  #c# $b = !(True && False) = True (NAND)
$b != False  #c# $b = !(True || False) = False (NOR)
$b != False  #c# $b = True == False = False (XNOR)
$b ->= False  #c# $b = !True || False = False (IMPLY)
$b !->= False  #c# $b = True && !False = True (NIMPLY)
$b ->r= True  #c# $b = !True || True = True (reverse IMPLY)
$b !->r= True  #c# $b = True && !True = False (reverse NIMPLY)

$x [int]= 5
$x <= 2      #c# $x = 5 << 2 = 20
$x >= 1      #c# $x = 20 >> 1 = 10
$x <<r= 3     #c# $x = 3 << $x = 3 << 10 = 3072
$x >>r= 2     #c# $x = 2 >> $x = 2 >> 3072 = 0

$x [int]= 0b1101
$x <<<[4]= 1  #c# $x = 0b1101 <<<[4] 1 = 0b1011 (left rotate by 1)
$x >>>[4]= 1  #c# $x = 0b1011 >>>[4] 1 = 0b1101 (right rotate by 1)
$x <<<[4]r= 2  #c# $x = 2 <<<[4] $x = 2 <<<[4]
                  #c# 1101 = 0111 (reverse left rotate)
$x >>>[4]r= 1  #c# $x = 1 >>>[4] $x = 1 >>>[4]
                  #c# 0111 = 1011 (reverse right rotate)

$m [list]= [[1,2],[3,4]]
$m .*= [[5,6],[7,8]]  #c# $m = [[19,22],[43,50]] (matrix multiply)
$m .^= 2              #c# $m = $m .^ 2 (matrix power)

$n [list]= [[1,2],[3,4]]
$n ~.T
      #c# $n = [[1,3],[2,4]] (transpose in place)

$p [list]= [[1,2],[3,4]]
$p .*r= [[5,7],[6,8]]

```

```

    #c# $p = [[5,7],[6,8]] .* [[1,2],[3,4]] (reverse)
$p .^r= 2
    #c# $p = 2 .^ $p (reverse matrix power)

```

Safe Null Assignment

Safe null assignment operators conditionally assign a value only when the variable is currently `null`. If the variable already holds a value, the assignment is skipped.

Safe Typed Assignment (?[type]?=)

`$var ?[type]?= expr` If `$var` is `null`, `$var = (type) expr`; else no-op

- The variable must already be declared
- If the variable is `null`: assigns the right-hand side with the specified type
- If the variable is not `null`: no assignment occurs, value is unchanged
- The right-hand side is only evaluated if the variable is `null` (short-circuit)

Safe Inferred Assignment (?t?=)

`$var ?t?= expr` If `$var` is `null`, `$var = expr (inferred type)`; else no-op

- The variable must already be declared
- If the variable is `null`: assigns the right-hand side with inferred type
- If the variable is not `null`: no assignment occurs, value is unchanged
- The right-hand side is only evaluated if the variable is `null` (short-circuit)

Examples:

```

$counter [int]= null
$counter ?[int]?= 10    #c# $counter = 10 (was null)
$counter ?[int]?= 20    #c# $counter = 10 (unchanged, not null)

```

```

$name [string]= null
$name ?t?= "Alice"    #c# $name = "Alice" (inferred string)

```

```

$config [string]= null
$config ?[string]?= f.str.toLowerCase("DEFAULT")    #c# only evaluated because null

```

Lazy Binding (:Lt= / :L[type]=)

Binds a variable to an expression that is **not evaluated** at declaration time. The expression is evaluated only on the first access of the variable. After evaluation, the result is cached and subsequent accesses return the cached value.

Inferred Type (:Lt=)

`$var :Lt= expr` Lazy binding: expr evaluated on first access, type inferred

Explicit Type (:L[type]=)

`$var :L[type]= expr` Lazy binding: expr evaluated on first access, type checked

- The variable must already be declared
- The right-hand side expression is **not evaluated** at binding time
- On first access: expression is evaluated, result is cached and returned
- With `:L[type]=`: if the evaluated result does not match the declared type, a **Type Error** is raised on first access
- On subsequent accesses: cached value is returned (no re-evaluation)
- The binding is immutable after first evaluation
- If the expression raises an error, it raises on first access (not at binding)

Examples:

```
$expensive [int]= null
```

```
$expensive :Lt= f.math.factorial(100)    ## not evaluated yet
```

```
## ... other code ...
```

```
$result [int]= $expensive    ## NOW evaluates factorial(100), caches result
```

```
$result2 [int]= $expensive    ## returns cached value (no re-evaluation)
```

```
$late [string]= null
```

```
$late :Lt= f.str.toLower("HELLO")    ## not evaluated yet
```

```
echo($late)                          ## NOW evaluates, prints "hello"
```

```
$config [int]= null
```

```
$config :L[int]= f.math.factorial(5)    ## type checked on first access
```

```
echo($config)                        ## 120
```

Examples:

```
## Scalar arithmetic
```

```
3 + 4      ## = 7
```

```
10 - 3     ## = 7
```

```
4 * 5      ## = 20
```

```
15 / 4     ## = 3.75
```

```
17 % 5     ## = 2
```

```
2 ^ 10     ## = 1024
```

```
5 << 3     ## = 40 (5 * 2^3)
```

```
40 >> 3    ## = 5 (40 / 2^3)
```

```
0b1101 <<<[4] 1  #c# = 0b1011 (left rotate 4-bit word by 1)
0b1101 >>>[4] 1  #c# = 0b1110 (right rotate 4-bit word by 1)
```

```
#c# String concatenation
"hello" + " " + "world"  #c# = "hello world"
```

```
#c# String repetition
"ab" * 3                #c# = "ababab"
```

```
#c# List operations
[1,2] + [3,4]           #c# = [1,2,3,4]
[0] * 5                 #c# = [0,0,0,0,0]
```

8.1.5 Comparison

Comparison operators evaluate to True or False.

```
n <  m Less Than
n <= m Less or Equal
b == c Equal
b != c Not Equal
a =n= b Numeric Equal (cross-type)
a >< b Three-way Numeric Compare
n >= m Greater or Equal
n >  m Greater Than
a ~  b Approximately Equal (uses syst.flag.approx_eps)
a ~* b Approximately Equal (uses syst.flag.approx_eps_mult)
a ~+( c )~ b Approximately Equal with additive tolerance c
a ~*( c )~ b Approximately Equal with multiplicative tolerance c
```

Type Overloading

- <, <=, >=, >: int or real → bool (ordered)
- <, <=, >=, >: string → bool (lexicographic)
- ==, !=: any → bool (strict equality)
- =n=: int or real → bool (numeric equality)
- ><: int or real → int (three-way numeric compare)
- ~, ~*, ~+()~, ~*()~: int or real → bool (approximate equality)

Ordered comparisons (<, <=, >=, >) only work on ints, reals, and strings. Using them on complex numbers raises a **Type Error**. Equality (==, !=) works on any type including lists, sets, parts, connections, and functions. =n= works only on numeric types.

Function Equality Equality (`==`, `!=`) works on function values using reference identity. Ordered comparisons (`<`, `<=`, `>=`, `>`) raise a **Type Error** when applied to function values.

- `f.math.sin == f.math.sin` $\rightarrow$ `True` (same reference)
- `f.math.sin == f.math.cos` $\rightarrow$ `False` (different references)
- `L::($x [int])->{$x} == L::($x [int])->{$x}` $\rightarrow$ `False` (lambdas are always unique)

Numeric Equality `=n=` compares values numerically across types. Unlike `==` which is type-strict, `=n=` returns `True` if the numeric values are equal regardless of whether the operands are `int` or `real`.

- `5 == 5.0` $\rightarrow$ `False` (strict: `int` vs `real`)
- `5 =n= 5.0` $\rightarrow$ `True` (numeric: same value)

Note: `==` is strict type equality. `5 == 5.0` is `False` because the left operand is `int` and the right is `real`. Use `=n=` when you want to compare values regardless of numeric type.

Three-way Numeric Compare `a >< b` performs a three-way numeric comparison using `=n=` semantics.

- Returns 1 if $a > b$
- Returns 0 if $a =n= b$
- Returns -1 if $a < b$

Raises a **Type Error** if either operand is not numeric.

Approximate Equality `a ~ b` returns `True` if $|a - b| \leq \text{sys.flag.approx_eps}$. `a ~* b` returns `True` if $|a - b| \leq \text{sys.flag.approx_eps_mult} \cdot \max(|a|, |b|)$. `a ~+(c) ~ b` returns `True` if $|a - b| \leq c$. `a ~*(c) ~ b` returns `True` if $|a - b| \leq c \cdot \max(|a|, |b|)$. The default value of `sys.flag.approx\_eps` is 1.

Examples:

```
3 < 5                #c# = True
"a" < "b"           #c# = True (lexicographic)
[1,2] == [1,2]       #c# = True (list equality)
Q1 == Q1             #c# = True (part identity)

#c# Numeric equality (cross-type)
5 == 5.0             #c# = False (strict: int vs real)
5 =n= 5.0            #c# = True  (numeric: same value)
5 =n= 5.1            #c# = False (different values)
3.14 =n= 3.14        #c# = True  (real vs real works too)

#c# Three-way numeric compare
5 >< 3                #c# = 1  (5 > 3)
5 >< 5                #c# = 0  (5 =n= 5)
```

```

5 >< 7          #c# = -1 (5 < 7)
5 >< 5.0        #c# = 0 (numeric equality)
3.14 >< 3.14    #c# = 0 (real vs real)

#c# Approximate equality
syst.flag.approx_eps 0.01
3.0 ~ 3.005     #c# = True (within 0.01)
3.0 ~ 3.1       #c# = False (not within 0.01)
3.0 ~+(0.2)~ 3.1 #c# = True (within 0.2)
3.0 ~+(0.05)~ 3.1 #c# = False (not within 0.05)

#c# Execution mode
syst.flag.exec_mode "strict" #c# warnings become errors

#c# Multiplicative approximate equality (relative tolerance)
100 ~*(0.01)~ 100.5 #c# = True (within 1%)
100 ~*(0.001)~ 100.5 #c# = False (not within 0.1%)

#c# Multiplicative approximate equality with system default
syst.flag.approx_eps_mult 0.01
100 ~* 100.5      #c# = True (within 1%)

```

8.1.6 Boolean Logic

Boolean operators combine or negate boolean values.

```

b && c Logical And
b || c Logical Or
b ^^ c Logical Xor
b !& c Nand (Not And)
b !| c Nor (Not Or)
b !^ c Xnor (Not Xor)
b -> c Imply (a implies b)
b !-> c Nimp (a does not imply b)
!b      Negation

```

- Operands: bool, bool
- Result: bool
- Short-circuit evaluation: `a && b` evaluates `b` only if `a` is True. `a || b` evaluates `b` only if `a` is False. However, if the unevaluated operand contains a function call that modifies global scope, both operands are always evaluated to preserve side effects.

Truth tables:

a	b	a!&b	a! b	a!^b	a->b	a!->b
T	T	F	F	T	T	F
T	F	T	F	F	F	T

F	T	T	F	F	T	F
F	F	T	T	T	T	F

Semantics:

- `!&` (NAND): `!(a && b)`
- `!|` (NOR): `!(a || b)`
- `!^` (XNOR): `a == b` (logical equality)
- `->` (IMPLY): `!a || b`. Not commutative: `a -> b ≠ b -> a`
- `!->` (NIMPLY): `a && !b`. Not commutative: `a !-> b ≠ b !-> a`

Examples:

```
True && False    #c# = False
True || False    #c# = True
True ^^ False    #c# = True
True ^^ True     #c# = False
True !& False    #c# = True  (NAND)
True !| False    #c# = False (NOR)
True !^ False    #c# = True  (XNOR)
True -> False    #c# = False (IMPLY)
True !-> False    #c# = True  (NIMPLY)
!True           #c# = False
```

8.1.7 Bitwise Operations

Bitwise operators apply boolean logic to individual bits of integers.

```
a b&& b  Bitwise AND
a b|| b  Bitwise OR
a b^^ b  Bitwise XOR
a b!& b  Bitwise NAND
a b!| b  Bitwise NOR
a b!^ b  Bitwise XNOR
a b-> b  Bitwise IMPLY
a b!-> b  Bitwise NIMP
b!a      Bitwise NOT (unary)
```

- Operands: `int`, `int`
- Result: `int`
- No short-circuit evaluation — both operands are always evaluated.

Semantics (bit-by-bit on integer representations):

- `b&&b`: `a & b` (bitwise AND)
- `b||b`: `a | b` (bitwise OR)

- $b^{\wedge}$: $a \wedge b$ (bitwise XOR)
- $b! \&$: $\sim(a \& b)$ (bitwise NAND)
- $b! |$: $\sim(a | b)$ (bitwise NOR)
- $b!^{\wedge}$: $\sim(a \wedge b)$ (bitwise XNOR)
- $b \rightarrow$: $(\sim a) | b$ (bitwise IMPLY)
- $b! \rightarrow$: $a \& (\sim b)$ (bitwise NIMP)
- $b!$: $\sim a$ (bitwise NOT, flips all bits)

Examples:

```
5 b&& 46      #c# = 4      (0b000101 & 0b101110 = 0b000100)
5 b|| 46      #c# = 47     (0b000101 | 0b101110 = 0b101111)
5 b^^ 46      #c# = 43     (0b000101 ^ 0b101110 = 0b101011)
5 b!& 46      #c# = -5     (\~{ }4)
5 b!| 46      #c# = -48    (\~{ }47)
5 b!^ 46      #c# = -44    (\~{ }43)
5 b-> 46      #c# = -6     ((\~{ }5) | 46)
5 b!-> 46     #c# = 4      (5 & (\~{ }46))
b! 0          #c# = -1     (all bits flipped)
```

8.1.8 Collection Operators

Collection operators manipulate sets, lists, and strings.

```
a u b  Union (sets only)
a n b  Intersection (sets only)
a / b  Difference (sets) / Division (numeric)
||a    Cardinality / Length
```

Type Overloading

- u : $\text{set} + \text{set} \rightarrow \text{set}$ (union)
- n : $\text{set} + \text{set} \rightarrow \text{set}$ (intersection)
- $/$: $\text{set} / \text{set} \rightarrow \text{set}$ (difference)
- $/$: $\text{int} / \text{int} \rightarrow \text{real}$ (division)
- $||$: $||\text{set} \rightarrow \text{int}$ (cardinality)
- $||$: $||\text{list} \rightarrow \text{int}$ (length)
- $||$: $||\text{string} \rightarrow \text{int}$ (length)

Set operations (u , n) only work on sets. The $/$ operator is overloaded: on sets it returns difference, on ints or reals it returns division. Cardinality ($||$) returns the element count for sets, lists, and strings.

Edge Cases

- A **Type Error** is raised for `u` or `n` if either operand is not a set.
- A **Type Error** is raised for `||` if the operand is not a set, list, or string.
- A **Type Error** is raised for `/` if the operands are a set and a non-set type (e.g., `set / int`).

Examples:

```
#c# Set operations
{1,2} u {2,3}      #c# = {1,2,3} (union)
{1,2} n {2,3}      #c# = {2} (intersection)
{1,2,3} / {2}      #c# = {1,3} (difference)

#c# Cardinality
||{1,2,3}          #c# = 3 (set size)
|[1,2,3,4]         #c# = 4 (list length)
|"hello"           #c# = 5 (string length)
```

8.1.9 List Operators

List operators perform vector and matrix operations on lists.

```
a . b  Dot product (real result)
a x b  Cross product (list result)
```

Type Overloading

- `.: list . list` $\rightarrow$ `real` (dot product, n-dimensional)
- `x: list x list` $\rightarrow$ `tensor` (cross product, 3D only)

The dot product computes $\sum_i a_i \cdot b_i$ for n-dimensional vectors (any length). The cross product computes the vector cross product (both lists must have length 3).

Edge Cases

- Dot product: A **Value Error** is raised if the vectors have different lengths.
- Dot product: A **Type Error** is raised if any element is not numeric (`int` or `real`).
- Dot product: Two empty vectors return 0.
- Cross product: A **Value Error** is raised if either vector does not have length 3.
- Cross product: A **Type Error** is raised if any element is not numeric.

Examples:

```
#c# Dot product (n-dimensional)
[1, 2, 3] . [4, 5, 6]      #c# = 32 (1*4 + 2*5 + 3*6)
[1, 2] . [3, 4]           #c# = 11 (2D)
[1, 2, 3, 4] . [5, 6, 7, 8] #c# = 70 (4D)
```

```
#c# Cross product (3D only)
[1, 0, 0] x [0, 1, 0]      #c# = [0, 0, 1]
[1, 2, 3] x [4, 5, 6]      #c# = [-3, 6, -3]
```

8.1.10 Tensor Operators

The `.` prefix on operators enables matrix/tensor operations on tensors. Without the prefix, operations are element-wise (see Section above).

Matrix Multiply

`a .* b` Matrix multiply (dot-product for 1D, matrix multiply for 2D)

- Operands: `tensor`, `tensor`
- Result: `tensor` or `real`
- Shape rules:
 - $1D \text{ } .* \text{ } 1D \rightarrow \text{scalar}$ (dot product, same as `.`)
 - $2D(m \times k) \text{ } .* \text{ } 2D(k \times n) \rightarrow 2D(m \times n)$
 - $2D(m \times k) \text{ } .* \text{ } 1D(k) \rightarrow 1D(m)$

Edge Cases

- A `Value Error` is raised if inner dimensions do not match (e.g., $2D(2 \times 3) \text{ } .* \text{ } 2D(2 \times 2)$).
- A `Value Error` is raised if $2D \text{ } .* \text{ } 1D$ dimension is incompatible (e.g., $2D(2 \times 3) \text{ } .* \text{ } 1D(2)$).
- A `Type Error` is raised if any element is not numeric.

Examples:

```
#c# 1D .* 1D = scalar (dot product)
[1, 2, 3] .* [4, 5, 6]      #c# = 32

#c# 2D .* 2D = matrix multiply
[[1,2],[3,4]] .* [[5,6],[7,8]] #c# = [[19,22],[43,50]]

#c# 2D .* 1D = matrix-vector multiply
[[1,2],[3,4]] .* [5, 6]      #c# = [17, 39]
```

Transpose / Hermitian

```
a .T   Transpose (postfix operator)
a ~.T  Transpose in place (no expression needed)
a .H   Hermitian transpose (conjugate transpose, postfix operator)
a ~.H  Hermitian transpose in place (no expression needed)
```

- Operands: tensor
- Result: tensor
- .T swaps rows and columns: $\text{shape } (m \times n) \rightarrow (n \times m)$
- .H returns the conjugate transpose: $(A.H)_{ij} = \overline{A_{ji}}$. For real matrices, .H is identical to .T.
- 1D arrays are treated as row vectors and transposed to column vectors.
- ~.T is shorthand for `[list]= $var.T` — transposes in place without repeating the variable name.
- ~.H is shorthand for `[list]= $var.H` — conjugate transposes in place.

Examples:

```
[1, 2, 3] .T          #c# = [[1],[2],[3]] (1D -> column)
[[1,2,3]] .T         #c# = [[1],[2],[3]] (row -> column)
[[1,2],[3,4]] .T     #c# = [[1,3],[2,4]]
```

```
$m [list]= [[1,2],[3,4]]
$m ~.T      #c# $m is now [[1,3],[2,4]]
```

```
# Hermitian transpose of complex matrix
[[1+2i, 3],[4, 5-1i]] .H  #c# = [[1-2i, 4],[3, 5+1i]]
```

Erase

```
$var ~.E  Erase (sets variable to null)
```

- Operands: any variable
- Result: none (statement, not expression)
- ~.E clears the variable's value, setting it to `null`. The variable binding remains and can be reassigned.
- Any subsequent access to the variable before reassignment raises a **Runtime Error**.
- Use case: secure zeroization of sensitive data (passwords, keys, tokens). The old value is released immediately and cannot be recovered. For cryptographic-grade destruction (provable irrecoverability), use `~.E+` instead.

Examples:

```

$password [string]= "s3cr3t"
## ... use $password ...
$password ~.E          ## $password is now null
## $password          ## Runtime Error (access before reassign)

$pin [string]= "1234"
$pin ~.E
$pin [string]= "5678"    ## reassign after erase is valid

```

Cryptographic Shred

`$var ~.E+ Cryptographic shred (overwrite + erase)`

- Operands: any variable
- Result: none (statement, not expression)
- `~.E+` performs a cryptographic shred of the variable's memory. Before releasing the value, the memory region is overwritten with pseudorandom data (8 passes alternating random bytes and complement bits), then the variable is set to `null`.
- The variable binding remains and can be reassigned.
- Any subsequent access to the variable before reassignment raises a **Runtime Error**.
- Use case: destruction of highly sensitive data (private keys, master secrets, cryptographic material) where a simple pointer nullification is insufficient to prevent forensic recovery.
- Overhead compared to `~.E`: 8x (8 overwrite passes). Recommended for secrets that must be provably irrecoverable.

Examples:

```

$secret [string]= "RSA-PRIVATE-KEY-..."
## ... use $secret ...
$secret ~.E+          ## memory overwritten 8x, then null
## $secret            ## Runtime Error (access before reassign)

$key [tensor]= |[0x4a, 0x9f, ...]
$key ~.E+             ## tensor memory shredded, then null
$key [tensor]= |[0x00] ## reassign after shred is valid

```

Matrix Power

`a .^ n Matrix power (repeated matrix multiply, n must be int >= 0)`

- Operands: tensor, int
- Result: tensor
- `A .^ 0` returns the identity matrix (square matrices only)
- `A .^ n = A .* A .* ... .* A` (n times)

Edge Cases

- A `Value Error` is raised if the matrix is not square.
- A `Value Error` is raised if the exponent is negative.
- A `Type Error` is raised if the exponent is not `int`.

Examples:

```
[[1,2],[3,4]] .^ 0      #c# = [[1,0],[0,1]] (identity)
[[1,2],[3,4]] .^ 1      #c# = [[1,2],[3,4]] (unchanged)
[[1,2],[3,4]] .^ 2      #c# = [[7,10],[15,22]]
```

8.1.11 Set Comparison

Set comparison operators test membership and subset relationships.

```
a c b  Subset of
a E b  Element of
a !E b Not element of
```

- Operands: `set`, `set` (subset)
- Operands: any, `set` (membership)
- Result: `bool`
- `a c b` is true if every element of `a` is in `b`
- `a E b` is true if `a` is an element of set `b`

Examples:

```
{1,2} c {1,2,3}      #c# = True (subset)
1 E {1,2,3}          #c# = True (membership)
Q1 E *p              #c# = True (part in all parts)
4 !E {1,2,3}         #c# = True (not element)
```

8.1.12 Conditional Expression

A ternary-like operator for inline conditional values.

```
(condition)?->(value_if_true)!(value_if_false)
```

- Operands: `bool`, any, any
- Result: type of the selected branch
- Evaluates condition, returns first value if `True`, second if `False`
- Both branches must be of the same type (or coercible per the type coercion hierarchy:
`int < real < complex`)

Examples:

```
$x [int]= 5
$result [int]= ($x > 3)?->("big")!("small")
#c# $result = "big"
```

```
$val [int]= 10
$abs [int]= ($val >= 0)?->($val)!($val * -1)
#c# $abs = 10
```

```
$a [int]= 1
$b [int]= 2
$max [int]= ($a > $b)?->($a)!($b)
#c# $max = 2
```

8.1.13 Null Operators

Null Coalescing (??)

Returns the left operand if it is not `null`, otherwise the right operand.

`a ?? b` Returns `a` if `a` is not `null`, else `b`

- Operands: any, any
- Result: type of the left operand if not `null`, else type of the right operand
- Right operand is only evaluated if left operand is `null` (short-circuit)

Examples:

```
$x [int]= null
$y [int]= $x ?? 5          #c# $y = 5
$name [string]= null
$display [string]= $name ?? "Unknown" #c# $display = "Unknown"
```

Safe Member Access (?.)

Returns the member access result if the left operand is not `null`, otherwise returns `null` without raising a `Runtime Error`.

`obj?.member` Returns `obj.member` if `obj` is not `null`, else `null`

- Operands: any, member name (identifier)
- Result: type of the member if left operand is not `null`, else `null`

Safe Indexing (?[])

Returns the index access result if the left operand is not `null`, otherwise returns `null` without raising a `Runtime Error`.

array?[index]	Returns array[index] if array is not null, else null
tensor?[i]	Returns tensor[i] if tensor is not null, else null
tensor?[i, j]	Returns tensor[i, j] if tensor is not null, else null
tensor?[start:end]	Returns tensor[start:end] if tensor is not null, else null
tensor?[[indices]]	Returns tensor[[indices]] if tensor is not null, else null
tensor?[condition]	Returns tensor[condition] if tensor is not null, else null
tensor?[..., i]	Returns tensor[..., i] if tensor is not null, else null

- Operands: any, any
- Result: element/sub-tensor of the collection if left operand is not null, else null
- Type Error / Value Error still raised if indexing is invalid (wrong type, out of bounds)

Examples:

```
$data = null
$val [int]= $data?.field      ## $val = null (no Runtime Error)
$arr = null
$item [int]= $arr?[0]        ## $item = null (no Runtime Error)
```

Safe tensor indexing

```
$mat = null
$mat?[0, 1]      ## Returns null (no error)
$mat?[0:2]       ## Returns null (no error)
$mat?[0:2, 0]    ## Returns null (no error)
$mat?[$mat > 0]  ## Returns null (no error)
$mat?[..., 0]    ## Returns null (no error)
```

Safe assignment

```
$mat?[0] = 99      ## No-op ($mat is null)
```

Safe Function Call (?())

Calls a function value if it is not `null`, otherwise returns `null` without raising a `Runtime Error`. Errors raised **within** the called function still propagate.

`fn?(args)` Returns `fn(args)` if `fn` is not `null`, else `null`

- Operands: a variable holding a `func` value or `null`, followed by optional arguments
- Result: return type of the function if the variable is not `null`, else `null`
- If the variable is `null`: returns `null` (no error)
- If the variable is not `func`: raises a `Type Error`
- Arguments are evaluated before the null check

```
$f [func]= L::($x [int]) -> { $x + 1 }
$val [int]= $f?(10)      ## $val = 11
```

```
$g = null
$val2 [int]= $g?(10)     ## $val2 = null (no Runtime Error)
```

Null-Safe Function Call (??())

Like ?(), but also suppresses any error raised **within** the called function, returning `null` instead. This is the safest calling convention—the expression can never raise an error.

`fn??(args)` Returns `fn(args)` if `fn` is not `null`; returns `null` on any error

- Operands: a variable holding a `func` value or `null`, followed by optional arguments
- Result: return type of the function, or `null`
- If the variable is `null`: returns `null` (no error)
- If the variable is not `func`: returns `null` (no error)
- If the function call raises any error (`Type`, `Value`, `Division by Zero`, `Runtime`, `Assertion`): returns `null` (error suppressed)
- Arguments are evaluated before the null check (errors in argument evaluation are **not** suppressed)

```
$f [func]= L::($x [int]) -> { 100 / $x }
$val [int]= $f??(5)           #c# $val = 20
$val2 [int]= $f??(0)          #c# $val2 = null (Division by Zero Error suppressed)

$g = null
$val3 [int]= $g??(5)          #c# $val3 = null (no Runtime Error)
```

8.1.14 Lambda Expressions

Lambda expressions create anonymous function values (of type `func`) that can be passed as arguments to higher-order functions, stored in variables, or called directly. They are defined with the `L::` prefix (consistent with the `f.u.` prefix for user-defined functions).

Syntax

EBNF:

```
lambda = "L::", "(", [ parameter, { ",", parameter } ], ")",
        "->", "{", { statement }, expression, "}"
parameter = "$", id, "[", type, "]"
```

Parameters must always have explicit types. The last expression in the body is the implicit return value. Multi-line bodies are allowed (multiple statements in `{}`).

Type Overloading

- Parameters may be any scalar type: `int`, `real`, `bool`, `string`
- Parameters may also be `list` when operating on lists of lists
- Return type is inferred from the body expression
- All parameter types must match the elements passed by the higher-order function

Scope Rules

- Lambdas have **read-only** access to variables from the enclosing scope (closures can read but not write)
- Lambdas **can** declare local variables inside their body (block-scoped to the lambda)
- Lambdas **cannot** modify variables from the enclosing scope
- Attempting to write to an enclosing variable raises a Compile Error

Return Value

The last expression in the body is the implicit return value. There is no explicit **return** keyword inside lambdas. If the body contains only statements with no trailing expression, the lambda returns nothing (useful for side effects only, e.g., with `f.print`).

Examples

```
#c# Basic lambda (passed to map)
map([1, 2, 3], L::($x [int])->{ $x * 2 })
#c# Returns [2, 4, 6]
```

```
#c# Lambda with multiple parameters
L::($x [int], $y [int])->{ $x + $y }
#c# Returns a function object (used inline)
```

```
#c# Multi-line lambda
map([1, 2, 3], L::($x [int])->{
    $y [int]= $x * 10
    $y + 1
})
#c# Returns [11, 21, 31]
```

```
#c# Closure (read-only access to outer variable)
$factor [int]= 10
map([1, 2, 3], L::($x [int])->{ $x * $factor })
#c# Returns [10, 20, 30]
```

```
#c# Lambda with real types
map([1.5, 2.5, 3.5], L::($x [real])->{ $x * $x })
#c# Returns [2.25, 6.25, 12.25]
```

```
#c# Lambda with nested list (2D tensor)
map([[1,2],[3,4]], L::($row [list])->{ $row })
#c# Returns [[1,2],[3,4]] (identity, demonstrates list parameter)
```

```
#c# Lambda with bool parameter
filter([True, False, True], L::($x [bool])->{ !$x })
```

#c# Returns [False]

Error Conditions

- Compile Error: parameter type is missing (e.g., `L :: ($x) -> { ... }` is invalid)
- Compile Error: attempting to write to an outer variable inside a lambda
- Runtime Error: type mismatch between parameter type and actual element type

8.1.15 Pipeline

The pipeline operator `|=>` passes the left operand as the argument to the function on the right. This enables readable left-to-right data flow without nesting.

```
value |=> function_ref
value |=> lambda
```

Semantics

- `a |=> f` evaluates to `f(a)`
- Left-associative: `a |=> f |=> g` evaluates to `g(f(a))`
- The left operand is fully evaluated before the function is called

Function Reference A function reference names a function without calling it. Built-in references omit parentheses; user-defined references use the `f.u.` prefix:

- Built-in: `f.math.sin` (not `f.math.sin()`)
- User-defined: `f.u.myfunc` (not `f.u.myfunc()`)
- Lambda: `L :: (x) -> { x + 1 }`

Examples

```
5 |=> f.math.sin           #c# = sin(5)
-3 |=> f.math.abs          #c# = 3
"hello" |=> f.str.upperc    #c# = "HELLO"
```

```
# Chained pipeline:
5 |=> f.math.sin |=> f.math.abs  #c# = |sin(5)|
```

```
# Lambda in pipeline:
5 |=> L :: (x) -> { x * x + 1 }  #c# = 26
```

```
# Pipeline with variables:
$f [list]= [1, 2, 3, 4, 5]
$f |=> f.list.sum |=> f.math.sqrt  #c# = sqrt(15)
```

Constraints

- The right-hand side must be a function reference or lambda – a function call (`f.math.sin()`) is not valid as a pipeline target
- The function must accept exactly one argument – a `Type Error` is raised otherwise
- For multi-argument functions, wrap in a lambda: `5 | => L::(x) -> { f.math.pow(x, 2) }`

8.1.16 Function Composition

The composition operator `<o` creates a new function that applies the right operand first, then the left operand. This is mathematical function composition: `$f <o $g` produces a function equivalent to `L::($x) -> { $f($g($x)) }`.

Syntax

```
$f <o $g          ## compose: returns new function ($x) -> $f($g($x))
$f <o= $g         ## compose-assign: $f = $f <o $g
$f o>= $g        ## reverse compose-assign: $f = $g <o $f
```

Semantics

- `$f <o $g` returns a new `func` value; neither `$f` nor `$g` is modified
- Right-associative: `$f <o $g <o $h = $f <o ($g <o $h)` $\rightarrow$ applies `$h`, then `$g`, then `$f`
- The composed function has the arity of the rightmost function (`$g`)

Type Overloading

- Both operands must be `func` – a `Type Error` is raised otherwise
- The return type of `$g` must be assignable to the parameter type of `$f` – a `Type Error` is raised otherwise
- Result is always `func`

Examples

```
$double t= L::($x [int])->{ $x * 2 }
$inc     t= L::($x [int])->{ $x + 1 }

# $f <o $g means ($x) -> $f($g($x))
$double_then_inc t= $double <o $inc
$double_then_inc(3)      ## Returns 8  (inc first: 3+1=4, then double: 4*2=8)

# Right-associative chaining
$add1 t= L::($x [int])->{ $x + 1 }
```

```

$mul2 t= L::($x [int])->{ $x * 2 }
$sqr t= L::($x [int])->{ $x ^ 2 }

$pipeline t= $sqr <o $mul2 <o $add1
# $sqr <o ($mul2 <o $add1) = ($x) -> $sqr($mul2($add1($x)))
$pipeline(3)          ## Returns 64  ((3+1)*2)^2 = 8^2 = 64

# Compose built-in functions
$to_upper_then_len t= f.util.len <o f.str.upperc
$to_upper_then_len("hello")  ## Returns 5

# Compose-assign: $f = $f <o $g
$f t= L::($x [int])->{ $x * 2 }
$f <o= L::($x [int])->{ $x + 10 }
$f(3)          ## Returns 26  ($f is now ($x)->($x+10)*2, so (3+10)*2=26)

# Reverse compose-assign: $g = $g <o $f  ($f goes on the LEFT of <o)
$double t= L::($x [int])->{ $x * 2 }
$add10 t= L::($x [int])->{ $x + 10 }
$double o>= $add10
# $double is now $add10 <o $double = ($x) -> $add10($double($x))
$double(3)          ## Returns 16  (3*2=6, then 6+10=16)

```

Error Conditions

- Type Error: either operand is not a func
- Type Error: return type of \$g is not assignable to parameter type of \$f
- Runtime Error: either operand is null

8.2 Operator Precedence

Operators are evaluated in the following order (highest to lowest):

1. $\wedge$ – Exponentiation (right-associative)
2. $!$ – Boolean negation, Bitwise NOT ($\mathbf{b}!$)
3. $\$var++$, $\$var--$, $\mathbf{.T}$, $\mathbf{.H}$, $\mathbf{\sim.E}$, $\mathbf{?.}$, $\mathbf{?[]}$, $\mathbf{?()}$, $\mathbf{??()}$, $\$var(args)$ – Postfix increment / decrement, transpose, Hermitian transpose, erase, safe member access, safe indexing, safe function call, null-safe function call, function call
4. $\mathbf{< o}$ – Function composition (right-associative)
5. $||a$ – Cardinality / length
6. $*$, $/$, $//$, $\%$, $\mathbf{.}$, $\mathbf{x}$, $\mathbf{.*}$, $\mathbf{.^}$ – Multiplication, division, floor division, modulo, dot product, cross product, matrix multiply, matrix power

7. <<, >>, <<<, >>> – Left shift, right shift, left rotate, right rotate
8. +, – – Addition, subtraction
9. <, <=, ==, !=, =n=, ><, >=, >, ~, ~*, ~+()~, ~*()~ – Comparison
10. as – Type casting (e.g., \$x as real)
11. c, E, !E – Set comparison
12. &&, !&, b&&, b!& – Logical AND, NAND, Bitwise AND, Bitwise NAND
13. ||, !|, b||, b!| – Logical OR, NOR, Bitwise OR, Bitwise NOR
14. ^^, !^, b^^, b!^ – Logical XOR, XNOR, Bitwise XOR, Bitwise XNOR
15. ->, !->, b->, b!-> – IMPLY, NIMPLY, Bitwise IMPLY, Bitwise NIMPLY
16. ?->...!... – Conditional expression
17. ?? – Null coalescing (right-associative)
18. |=> – Pipeline (left-associative)
19. L::(...) ->{...} – Lambda expression
20. User-defined operators (o.u.*)

Note: || has two distinct meanings depending on position. As a **prefix** operator (||*expr*), it denotes cardinality/length and has high precedence (level 3). As an **infix** operator (*expr* || *expr*), it denotes logical OR and has low precedence (level 9). Parentheses override precedence. Function arguments are evaluated before the function is called.

8.3 User-defined Operators

User-defined operators extend the language with custom infix notation.

8.3.1 Definition

EBNF:

"letop", "o.u.", id, assigner, data

The operator name must follow the pattern o.u. [_a-zA-Z0-9]+.

8.3.2 Examples

```
letop o.u.mod_minus_1 [int]=
    a % b - 1

f.print( 4 o.u.mod_minus_1 3 )  #c# = 0

letop o.u.parallel [int]=
    (a * b) / (a + b)

f.print( 100 o.u.parallel 100 )  #c# = 50
```

8.3.3 Precedence

User-defined operators have the lowest precedence among all operators. Use parentheses to ensure correct evaluation order.

Chapter 9

Constants

Constants are immutable values that cannot be reassigned after declaration.

9.1 Built-in Constants

9.1.1 Proper Constants

Proper constants are fixed values provided by the language.

Mathematical Constants

- `c.math.pi` – The mathematical constant $\pi \approx 3.14159265\dots$
- `c.math.tau` – $\tau = 2\pi \approx 6.28318530\dots$
- `c.math.e` – Euler’s number $e \approx 2.71828182\dots$
- `c.math.i` – Imaginary unit $\sqrt{-1}$ (complex type)
- `c.math.iinf` – Integer positive infinity
- `c.math.rinf` – Real positive infinity
- `c.math.inf` – Unsigned infinity (numeric)
- `c.math.NaN` – Not-a-Number (real type)
- `c.math.egamma` – Euler-Mascheroni constant $\gamma \approx 0.5772156649\dots$
- `c.math.sqrt2` – $\sqrt{2} \approx 1.4142135623\dots$ (RMS conversion: $V_{\text{rms}} = V_{\text{pk}}/\sqrt{2}$)
- `c.math.sqrt2_inv` – $1/\sqrt{2} \approx 0.7071067811\dots$ (the -3dB point)
- `c.math.sqrtpi` – $\sqrt{\pi} \approx 1.77245385\dots$
- `c.math.ln2` – $\ln(2) \approx 0.6931471805\dots$ (half-life, RC time constants)
- `c.math.ln10` – $\ln(10) \approx 2.30258509\dots$
- `c.math.log2e` – $\log_2(e) \approx 1.44269504\dots$
- `c.math.log10e` – $\log_{10}(e) \approx 0.43429448\dots$
- `c.math.expMinus1` – $e^{-1} \approx 0.3678794411\dots$

Physical Constants

Fundamental

- `c.phys.c` – Speed of light in vacuum $c = 299\,792\,458$ m/s (exact)
- `c.phys.h` – Planck constant $h = 6.626\,070\,15 \times 10^{-34}$ J·s (exact)
- `c.phys.hbar` – Reduced Planck constant $\hbar = 1.054\,571\,817 \times 10^{-34}$ J·s
- `c.phys.q` – Elementary charge $e = 1.602\,176\,634 \times 10^{-19}$ C (exact)
- `c.phys.kb` – Boltzmann constant $k_B = 1.380\,649 \times 10^{-23}$ J/K (exact)
- `c.phys.alpha` – Fine-structure constant $\alpha \approx 0.007\,297\,352\,564\,3$
- `c.phys.alphaInv` – Inverse fine-structure constant $1/\alpha \approx 137.035\,999\,177$
- `c.phys.G` – Newtonian gravitational constant $G \approx 6.674\,30 \times 10^{-11}$ m³ kg⁻¹ s⁻²

Electromagnetic

- `c.phys.mu0` – Vacuum permeability $\mu_0 \approx 1.256\,637\,061 \times 10^{-6}$ N/A²
- `c.phys.eps0` – Vacuum permittivity $\epsilon_0 \approx 8.854\,187\,819 \times 10^{-12}$ F/m
- `c.phys.eta0` – Characteristic impedance of vacuum $\eta_0 \approx 376.730\,313\,412$ Ω
- `c.phys.k` – Coulomb constant $k_e \approx 8.987\,551\,792 \times 10^9$ N·m²/C²
- `c.phys.G0` – Conductance quantum $G_0 = 2e^2/h \approx 7.748\,091\,730 \times 10^{-5}$ S
- `c.phys.G0Inv` – Inverse conductance quantum $h/(2e^2) \approx 12\,906.403\,730$ Ω
- `c.phys.phi0` – Magnetic flux quantum $\Phi_0 = h/(2e) \approx 2.067\,833\,848 \times 10^{-15}$ Wb
- `c.phys.RK` – von Klitzing constant $R_K = h/e^2 \approx 25\,812.807\,459$ Ω
- `c.phys.KJ` – Josephson constant $K_J = 2e/h \approx 483\,597.848\,4$ GHz/V

Atomic and Particle Masses

- `c.phys.me` – Electron mass $m_e \approx 9.109\,383\,714 \times 10^{-31}$ kg
- `c.phys.me_u` – Electron mass in u $\approx 0.000\,548\,579\,909$
- `c.phys.me_MeV` – Electron mass energy equivalent $\approx 0.510\,998\,951$ MeV
- `c.phys.mp` – Proton mass $m_p \approx 1.672\,621\,924 \times 10^{-27}$ kg
- `c.phys.mp_u` – Proton mass in u $\approx 1.007\,276\,467$
- `c.phys.mp_MeV` – Proton mass energy equivalent $\approx 938.272\,089$ MeV
- `c.phys.mn` – Neutron mass $m_n \approx 1.674\,927\,501 \times 10^{-27}$ kg
- `c.phys.mn_u` – Neutron mass in u $\approx 1.008\,664\,916$

- `c.phys.mn_MeV` – Neutron mass energy equivalent $\approx 939.565\,422$ MeV
- `c.phys.md` – Deuteron mass $m_d \approx 3.343\,583\,777 \times 10^{-27}$ kg
- `c.phys.md_u` – Deuteron mass in u $\approx 2.013\,553\,213$
- `c.phys.md_MeV` – Deuteron mass energy equivalent $\approx 1875.612\,945$ MeV
- `c.phys.mMu` – Muon mass $m_\mu \approx 1.883\,531\,627 \times 10^{-28}$ kg
- `c.phys.mMu_u` – Muon mass in u $\approx 0.113\,428\,925\,7$
- `c.phys.mMu_MeV` – Muon mass energy equivalent $\approx 105.658\,375\,5$ MeV
- `c.phys.mTau` – Tau mass $m_\tau \approx 3.167\,54 \times 10^{-27}$ kg
- `c.phys.mTau_u` – Tau mass in u $\approx 1.907\,54$
- `c.phys.mTau_MeV` – Tau mass energy equivalent ≈ 1776.86 MeV
- `c.phys.mAlpha` – Alpha particle mass $m_\alpha \approx 6.644\,657\,345 \times 10^{-27}$ kg
- `c.phys.mAlpha_u` – Alpha particle mass in u $\approx 4.001\,506\,179$
- `c.phys.mAlpha_MeV` – Alpha particle mass energy equivalent $\approx 3727.379\,412$ MeV
- `c.phys.amu` – Unified atomic mass unit $u = 1.660\,539\,069 \times 10^{-27}$ kg
- `c.phys.amu_MeV` – Atomic mass unit energy equivalent $\approx 931.494\,104$ MeV

Particle Magnetic Moments and Ratios

- `c.phys.meMu_e` – Electron magnetic moment $\mu_e \approx -9.284\,764\,692 \times 10^{-24}$ J/T
- `c.phys.meG_e` – Electron g-factor $g_e \approx -2.002\,319\,304\,362$
- `c.phys.meAnomaly` – Electron magnetic moment anomaly $a_e \approx 0.001\,159\,652\,180\,46$
- `c.phys.mpMu_p` – Proton magnetic moment $\mu_p \approx 1.410\,606\,795 \times 10^{-26}$ J/T
- `c.phys.mpG_p` – Proton g-factor $g_p \approx 5.585\,694\,689$
- `c.phys.mnMu_n` – Neutron magnetic moment $\mu_n \approx -9.662\,365\,3 \times 10^{-27}$ J/T
- `c.phys.mnG_n` – Neutron g-factor $g_n \approx -3.826\,085\,52$
- `c.phys.mMuMu_mu` – Muon magnetic moment $\mu_\mu \approx -4.490\,448\,3 \times 10^{-26}$ J/T
- `c.phys.mMuAnomaly` – Muon magnetic moment anomaly $a_\mu \approx 0.001\,165\,920\,62$
- `c.phys.bohrMag` – Bohr magneton $\mu_B \approx 9.274\,010\,066 \times 10^{-24}$ J/T
- `c.phys.nuclearMag` – Nuclear magneton $\mu_N \approx 5.050\,783\,740 \times 10^{-27}$ J/T

Atomic Structure

- `c.phys.bohrRadius` – Bohr radius $a_0 \approx 5.291\,772\,105\,44 \times 10^{-11}$ m
- `c.phys.rydberg` – Rydberg constant $R_\infty \approx 10\,973\,731.568$ m⁻¹
- `c.phys.rydberg_eV` – Rydberg constant $\times hc$ in eV $\approx 13.605\,693\,123$ eV
- `c.phys.rydberg_J` – Rydberg constant $\times hc$ in J $\approx 2.179\,872\,361 \times 10^{-18}$ J
- `c.phys.hartree_E` – Hartree energy $E_h \approx 4.359\,744\,722 \times 10^{-18}$ J
- `c.phys.hartree_eV` – Hartree energy in eV $\approx 27.211\,386\,246$ eV
- `c.phys.comptonWavelength` – Compton wavelength of electron $\lambda_C \approx 2.426\,310\,235 \times 10^{-12}$ m
- `c.phys.comptonWavelengthReduced` – Reduced Compton wavelength $\bar{\lambda}_C \approx 3.861\,592\,674 \times 10^{-13}$ m
- `c.phys.classicalElectronRadius` – Classical electron radius $r_e \approx 2.817\,940\,321 \times 10^{-15}$ m
- `c.phys.thomsonCrossSection` – Thomson cross-section $\sigma_e \approx 6.652\,458\,705 \times 10^{-29}$ m²

Compton Wavelengths

- `c.phys.comptonWavelengthProton` – Proton Compton wavelength $\approx 1.321\,410\,000 \times 10^{-15}$ m
- `c.phys.comptonWavelengthNeutron` – Neutron Compton wavelength $\approx 1.319\,591\,000 \times 10^{-15}$ m
- `c.phys.comptonWavelengthMuon` – Muon Compton wavelength $\approx 1.173\,444\,110 \times 10^{-14}$ m

Energy Conversions

- `c.phys.eV` – Electron volt $1\text{ eV} = 1.602\,176\,634 \times 10^{-19}$ J (exact)
- `c.phys.eV_Hz` – eV to Hz $\approx 2.417\,989\,242 \times 10^{14}$ Hz
- `c.phys.eV_invM` – eV to inverse meter $\approx 806\,554.394$ m⁻¹
- `c.phys.eV_K` – eV to Kelvin $\approx 11\,604.518$ K
- `c.phys.eV_kg` – eV to kg $\approx 1.782\,662 \times 10^{-36}$ kg
- `c.phys.Hz_eV` – Hz to eV $\approx 4.135\,668 \times 10^{-15}$ eV
- `c.phys.invM_eV` – Inverse meter to eV $\approx 1.239\,842 \times 10^{-6}$ eV
- `c.phys.K_eV` – Kelvin to eV $\approx 8.617\,333 \times 10^{-5}$ eV

Physical Parameters

- `c.phys.me_mp` – Electron-to-proton mass ratio $\approx 0.000\,446\,170\,214$
- `c.phys.mp_me` – Proton-to-electron mass ratio $\approx 1836.152\,673\,43$
- `c.phys.atm` – Standard atmosphere = 101 325 Pa (exact)
- `c.phys.Pa_atm` – Standard-state pressure = 100 000 Pa (exact)
- `c.phys.sigma` – Stefan-Boltzmann constant $\sigma \approx 5.670\,374\,419 \times 10^{-8} \text{ W}/(\text{m}^2 \cdot \text{K}^4)$
- `c.phys.F` – Faraday constant $F \approx 96\,485.332\,12 \text{ C/mol}$
- `c.phys.NA` – Avogadro constant $N_A \approx 6.022\,140\,76 \times 10^{23} \text{ mol}^{-1}$
- `c.phys.Rgas` – Molar gas constant $R \approx 8.314\,462\,618 \text{ J}/(\text{mol} \cdot \text{K})$
- `c.phys.molarVolumeSTP_100` – Molar volume of ideal gas (273.15 K, 100 kPa) $\approx 0.022\,711 \text{ m}^3/\text{mol}$
- `c.phys.molarVolumeSTP_101` – Molar volume of ideal gas (273.15 K, 101.325 kPa) $\approx 0.022\,414 \text{ m}^3/\text{mol}$
- `c.phys.molarMassC12` – Molar mass of carbon-12 = 0.012 kg/mol (exact)
- `c.phys.kT300` – Thermal voltage at 300K $kT/q \approx 0.025\,852 \text{ V}$
- `c.phys.Loschmidt_100` – Loschmidt constant (273.15 K, 100 kPa) $\approx 2.652 \times 10^{25} \text{ m}^{-3}$
- `c.phys.Loschmidt_101` – Loschmidt constant (273.15 K, 101.325 kPa) $\approx 2.687 \times 10^{25} \text{ m}^{-3}$
- `c.phys.luminousEfficacy` – Maximum luminous efficacy = 683 lm/W (exact)
- `c.phys.wienDisplacement` – Wien displacement constant $b \approx 2.897\,772 \times 10^{-3} \text{ m} \cdot \text{K}$
- `c.phys.wienFreqDisplacement` – Wien frequency displacement constant $\approx 5.878\,926 \times 10^{10} \text{ Hz/K}$
- `c.phys.secondRadiation` – Second radiation constant $c_2 \approx 0.014\,387\,769 \text{ m} \cdot \text{K}$
- `c.phys.firstRadiation` – First radiation constant $c_1 \approx 3.741\,772 \times 10^{-16} \text{ W} \cdot \text{m}^2$
- `c.phys.soundRefPressure` – Sound reference pressure $p_0 = 20 \text{ } \mu\text{Pa}$
- `c.phys.speedOfSoundAir` – Speed of sound in air at 20°C $\approx 343 \text{ m/s}$
- `c.phys.speedOfLightWater` – Speed of light in water $\approx 2.25 \times 10^8 \text{ m/s}$
- `c.phys.g` – Standard gravitational acceleration $g = 9.806\,65 \text{ m/s}^2$ (exact)

Silicon and Crystallography

- `c.phys.latticeSi` – Lattice parameter of silicon $a_{\text{Si}} \approx 5.431\,021 \times 10^{-10} \text{ m}$
- `c.phys.latticeSi220` – Lattice spacing of ideal Si(220) $d_{220} \approx 1.920\,156 \times 10^{-10} \text{ m}$
- `c.phys.molarVolumeSi` – Molar volume of silicon $V_{m,\text{Si}} \approx 1.205\,883 \times 10^{-5} \text{ m}^3/\text{mol}$

Planck Units

- `c.phys.planckLength` – Planck length $\ell_P \approx 1.616\,255 \times 10^{-35}$ m
- `c.phys.planckMass` – Planck mass $m_P \approx 2.176\,434 \times 10^{-8}$ kg
- `c.phys.planckMass_GeV` – Planck mass energy equivalent $\approx 1.220\,89 \times 10^{19}$ GeV
- `c.phys.planckTime` – Planck time $t_P \approx 5.391\,247 \times 10^{-44}$ s
- `c.phys.planckTemp` – Planck temperature $T_P \approx 1.416\,784 \times 10^{32}$ K

Miscellaneous

- `c.phys.TCMB` – Cosmic microwave background temperature $T_{\text{CMB}} \approx 2.725\,48$ K
- `c.phys.H0` – Hubble constant $H_0 \approx 2.242 \times 10^{-18}$ Hz (approx. 70 km/s/Mpc)
- `c.phys.SackurTetrode_100` – Sackur-Tetrode constant (1 K, 100 kPa) $\approx -1.151\,708$
- `c.phys.SackurTetrode_101` – Sackur-Tetrode constant (1 K, 101.325 kPa) $\approx -1.164\,871$
- `c.phys.neutronProtonMass` – Neutron-proton mass difference in u $\approx 0.001\,388\,449$
- `c.phys.weakMixing` – Weak mixing angle $\sin^2 \theta_W \approx 0.223\,05$
- `c.phys.fermiConst` – Fermi coupling constant $G_F \approx 1.166\,379 \times 10^{-5}$ GeV⁻²

9.1.2 Pseudo-Constants

Pseudo-constants are system-defined sets that provide access to electrical objects.

Set Pseudo-Constants

- `*n` – Empty set
- `*e` – All parts, connections, pins, holes, and nets
- `*i` – All pins and connections
- `*u` – All parts, connections, and pins on the upper side of the board
- `*b` – All parts, connections, and pins on the bottom side of the board
- `*h` – All holes
- `*c` – All connections
- `*p` – All parts
- `*t` – All pins
- `*v` – All nets
- `*fp` – All footprints
- `*pkg` – All packages

- `*layer` – All board layers

Example:

```
$my_set = *n    #c# Initialize with empty set
$my_set = $my_set u { R1, R2 }    #c# Add elements
```

9.2 User-defined Constants

User-defined constants are created using the `let` keyword with the `c.u.` prefix:

EBNF:

```
constant_assignment = "let", "c.u.", id, assigner,
                      [";", "P", "=", boolean_data_type]
```

Examples:

```
let c.u.$max_current [int]= 30
let c.u.$board_name [string]= "PowerSupply"
```

Constants are immutable by definition and cannot be reassigned.

Chapter 10

Control Flow

EDADSL provides several control flow constructs for conditional execution, iteration, and program flow control.

10.1 Conditional Execution

Conditional execution uses the `if` keyword followed by a boolean expression and a code block. Additional conditions can be tested with `otherwise`. A final `else` clause provides a fallback code block executed when no conditions match:

EBNF:

```
conditional = "if", boolean_expression,  
             ":", code_block,  
             { "otherwise", boolean_expression,  
               ":", code_block },  
             ["else", ":", code_block]
```

Example:

```
$flag [bool]= True  
if $flag ::{  
    f.util.print("Flag is set")  
} otherwise $other ::{  
    f.util.print("Other condition")  
} else ::{  
    f.util.print("Flag is not set")  
}
```

Basic if without else:

Example:

```
$flag [bool]= True  
if $flag ::{  
    f.util.print("Flag is set")  
}
```

10.2 Cases

The `cases` statement provides multi-branch conditional execution with a bound variable. It evaluates a list of boolean expressions in order and executes the first matching branch. This is a cleaner alternative to long `if/otherwise` chains when multiple conditions need to be tested against a common value.

EBNF:

```
cases = "cases", "(", variable, typed_assign, data, ")",
      ":", "{", { case_clause }, [ otherwise_clause ], "}"
```

```
case_clause = "case", "(", boolean_expression, ")",
            ":", code_block
```

```
otherwise_clause = "otherwise", ":", code_block
```

- The expression in the `cases(...)` header is evaluated once and bound to the variable
- Each `case` clause contains a boolean expression that can reference the bound variable
- Case clauses are evaluated in order; the first `True` wins
- The optional `otherwise` clause is a fallback when no case matches
- If no case matches and no `otherwise` is present, execution continues silently
- The bound variable is block-scoped to the `cases` block

Examples:

```
# Basic usage
$r [real]= f.tnsr.norm($v)
cases ($r [real]= f.tnsr.norm($v)) ::{
  case ($r < 0.1) ::{
    f.util.print("small")
  } case ($r < 1.0) ::{
    f.util.print("medium")
  } otherwise ::{
    f.util.print("large")
  }
}

# Without otherwise (silently continues if no match)
cases ($x [int]= f.rng.int(1, 10)) ::{
  case ($x == 1) ::{
    f.util.print("got one")
  } case ($x == 2) ::{
    f.util.print("got two")
  }
}
```

```
# Boolean expression doesn't need to reference bound variable
$mode [string]= "fast"
cases ($mode [string]= "fast") ::{
  case ($mode == "slow") ::{
    f.util.print("slow mode")
  } case ($mode == "fast") ::{
    f.util.print("fast mode")
  } otherwise ::{
    f.util.print("unknown mode")
  }
}
```

Equivalent if/otherwise chain The `cases` statement is equivalent to an `if/otherwise` chain:

```
# cases statement:
cases ($r [real]= f.tnsr.norm($v)) ::{
  case ($r < 0.1) ::{
    f.util.print("small")
  } case ($r < 1.0) ::{
    f.util.print("medium")
  } otherwise ::{
    f.util.print("large")
  }
}
```

```
# Equivalent if/otherwise:
$r [real]= f.tnsr.norm($v)
if $r < 0.1 ::{
  f.util.print("small")
} otherwise $r < 1.0 ::{
  f.util.print("medium")
} else ::{
  f.util.print("large")
}
```

10.3 Alias Statements

Alias statements create short names for built-in functions or namespaces. Aliases are file-scoped and cannot shadow existing variables or aliases.

10.3.1 Function Alias

EBNF:
`alias_stmt = "alias", function_ref, "as", id`

Example:

```
alias f.trig.sin as sin
alias f.trig.cos as cos
$y = sin($x)
```

10.3.2 Namespace Alias

EBNF:

```
alias_stmt = "alias", namespace, "as", id
```

Example:

```
alias f.trig as trig
$y = trig.sin($x)
```

10.3.3 Edge Cases

- A `Type Error` is raised if the alias name conflicts with an existing variable, alias, or built-in name.
- A `Type Error` is raised if the alias target does not exist (e.g., `alias f.trig.nonexistent as x`).
- Aliases are file-scoped: they only apply within the current file.
- Aliases cannot be reassigned after declaration.

10.4 For Loops

For loops iterate over a list or set:

EBNF:

```
iterator = "for", variable, "E",
          (list | set), "::", code_block
```

Example:

```
$colors [list]= ["red", "green", "blue"]
for $c E $colors ::{
    f.util.print($c)
}
```

The loop variable (`$c`) takes on each value in the list/set in sequence.

Variable Scoping The loop variable is block-scoped: it is only visible inside the loop body and is destroyed when the loop exits. If the loop iterates over an empty list, the loop body never executes and the variable is never initialized; accessing it after the loop raises a `Runtime Error`.

10.5 While Loops

While loops repeat a code block as long as a boolean condition evaluates to `True`.

10.5.1 Basic While Loop

EBNF:

```
while_loop = "while", boolean_expression,  
            ":", code_block
```

Example:

```
$i [int]= 0  
while $i < 5 ::{  
    f.UTIL.print($i)  
    $i = $i + 1  
}
```

Variable Scoping While loops do not introduce any implicit variables. Any variables declared inside the loop body are block-scoped and destroyed when the loop exits. Variables used in the loop condition must be declared before the loop.

10.5.2 While with Else Clause

An optional **else** clause executes when the loop condition becomes **False**. If the loop exits via **break**, the **else** clause is **not** executed:

EBNF:

```
while_loop = "while", boolean_expression,  
            ":", code_block,  
            ["else", ":", code_block]
```

Example:

```
$i [int]= 0  
while $i < 5 ::{  
    $i = $i + 1  
} else ::{  
    f.UTIL.print("Loop completed normally")  
}
```

Example with break:

```
$i [int]= 0  
while $i < 10 ::{  
    if $i == 3 ::{  
        break  
    }  
    $i = $i + 1  
} else ::{  
    f.UTIL.print("This will NOT be printed")  
}
```

10.5.3 Do-While Loops

Do-while loops execute the code block **at least once** before evaluating the condition:

EBNF:

```
do_while = "do", ":", code_block,  
          "while", boolean_expression
```

Example:

```
$count [int]= 0  
do ::{  
    $count = $count + 1  
    f.util.print($count)  
} while $count < 5
```

The code block executes first, then the condition is checked. If the condition is **True**, the loop repeats. The same block-scoping rules as while loops apply: variables declared inside the code block are destroyed when the block exits. If the code block raises an error, the loop terminates immediately and the condition is not evaluated. **break** and **continue** work as expected inside do-while loops.

10.5.4 Break and Continue

break exits the innermost loop immediately. **continue** skips the remainder of the current iteration and proceeds to the next evaluation of the loop condition:

EBNF:

```
break_stmt    = "break"  
continue_stmt = "continue"
```

Example:

```
for $x E $values ::{  
    if $x < 0 ::{  
        continue  
    }  
    if $x > 100 ::{  
        break  
    }  
    f.util.print($x)  
}
```

Example with while:

```
$i [int]= 0  
while $i < 10 ::{  
    $i = $i + 1  
    if $i % 2 == 0 ::{  
        continue  
    }  
    f.util.print($i)  
}
```

```
}
```

`break` and `continue` apply only to the innermost enclosing loop.

10.5.5 Nested While Loops

While loops may be nested to arbitrary depth:

Example:

```
$rows [int]= 3
$cols [int]= 3
$i [int]= 0
while $i < $rows ::{
    $j [int]= 0
    while $j < $cols ::{
        f.util.print("Cell (" + f.str.str($i) + "," + f.str.str($j) + ")")
        $j = $j + 1
    }
    $i = $i + 1
}
```

10.5.6 Safety: Maximum Iteration Limit

To prevent infinite loops, the interpreter enforces a maximum iteration count per loop. The limit is controlled by `syst.flag.max_loop_iter` (default: 100000). If a loop exceeds this limit, a `Value Error` is raised:

Example:

```
syst.flag.max_loop_iter 500
$i [int]= 0
while $i < 1000 ::{
    $i = $i + 1
}
f.util.print($i) # never reached | Value Error at 501 iterations
```

The limit applies independently to each loop. Nested loops each have their own counter.

10.5.7 Infinite Loops

An infinite loop can be written with `True` as the condition. Use `break` to exit:

Example:

```
$count [int]= 0
while True ::{
    $count = $count + 1
    if $count == 10 ::{
        break
    }
}
f.util.print($count) #c# Prints: 10
```


10.6 Goto Statements

Goto statements provide unconditional jumps to labeled positions:

EBNF:

```
gotomarker = "gotomarker", id
gotojump   = "gotojump", id
```

Example:

```
gotojump skip_section
f.util.print("This will not be printed")
gotomarker skip_section
f.util.print("Jumped here")
```

Warning – Deprecated Usage Goto statements are **strongly discouraged**. They make code difficult to read, debug, and maintain. Prefer structured control flow (**if**, **for**, **while**, **break**, **continue**) for all new code. Gotos are retained for backward compatibility only. Gotos may cross loop boundaries (e.g., jump from outside a loop into the loop body) but may not cross function boundaries.

10.7 End of Program Statements

EBNF:

```
halt = "halt"
noop = "skip"
```

- **halt** – Terminates program execution and exports all created files
- **skip** – No operation; program continues to next statement

10.8 Output and Logging

EBNF:

```
echo_statement = "echo", variable
writetolog      = "writetolog", string_expression
```

Example:

```
f.util.print("Hello, EDADSL!")
writetolog "Board design completed successfully"
```

- **echo** – Keyword that prints a variable reference to the console. Only accepts a variable, not an arbitrary expression. Use `f.util.print()` for expression output.
- **writetolog** – Keyword that appends a message to the log file

Chapter 11

Functions

11.1 Built-in Functions

Important: All built-in function calls use a two-level namespace: `f.namespace.function`. This is defined in the formal grammar (Chapter 3):

```
function_call_built_in = "f", ".", namespace, ".", id, "(",  
                        [{ data, ",", }, data], ")"  
namespace = "math" | "trig" | "hyp" | "cplx" | "log"  
            | "special" | "str" | "list" | "tnsr"  
            | "stat" | "linalg" | "ee" | "int" | "util"  
            | "seq"
```

Correct usage: `f.math.abs(-3.2)`, `f.trig.sin($x)`, `f.tnsr.zeros([3,3])`

Incorrect: `f.math.abs(-3.2)`, `f.trig.sin($x)`, `abs(-3.2)`

`f.util.print()` takes a **single** argument. Use `f.str.str()` for string concatenation:

```
f.util.print("value = " + $x)           # OK  
f.util.print(f.str.str("value = ", $x)) # OK  
f.util.print("value = ", $x)           # WRONG - two arguments
```

11.1.1 Mathematical Functions

Absolute Value Function

The Absolute Value Function (denoted as `abs`) returns the Absolute Value of the Argument or the Magnitude of a Vector.

Morphology of the Absolute Value Function:

EBNF:

```
"f.math.abs", "(", argument, [ ",", "vec", "=",  
                             boolean_data_type ], ")"
```

If the Type of the Argument does not match the expected Type (a Numeric Value or a List of Numeric Values) a Value Error is raised.

Example:

```
f.math.abs(-3.2)
```

This Example would return a value of 3.2.

The Argument may also be a List of Numeric Values. If This is the case the Function is iterated over each Element.

Example:

```
f.math.abs([-3, 0.2, 0, -7])
```

This Example would return [3, 0.2, 0, 7].

If `vec = True` is passed a List shall be passed as the Argument and the Function will return Magnitude of the List as an n-dimensional Vector, where n is the Length of the List.

Example:

```
f.math.abs( [ 3, -4 ], vec = True )
```

This Example would return 5.0.

There are no Domain-Restrictions for the function inside $\mathbb{R}^n$.

Edge Cases – Absolute Value

- A **Type Error** is raised if the argument is not `int`, `real`, `complex`, or a list of numeric values.
- A **Type Error** is raised if `vec = True` and the argument is not a list.
- A **Type Error** is raised if `vec = True` and the list contains complex numbers (vector magnitude is only defined for real vectors).

Complex Overload When the Argument is a Complex Number $z = a + bi$, `abs` returns the Modulus (Magnitude) of the Complex Number, defined as:

$$|a + bi| = \sqrt{a^2 + b^2}$$

The Result is always a Real Number, even if the Argument is Complex.

Domain: All Complex Numbers ($\mathbb{C}$).

Example:

```
f.math.abs(3 + 4i)    #c# Returns 5.0
f.math.abs(1 + 1i)    #c# Returns approximately 1.41421
f.math.abs(0 - 5i)    #c# Returns 5.0
```

Parallel Equivalent Function

The Parallel Equivalent Function (denoted as `parallel`) computes the parallel combination of a list of values. This is the standard formula for parallel resistances, conductances, or any set of admittances.

Morphology of the Parallel Equivalent Function:

EBNF:

```
"f.math.parallel", "(", argument, ")"
```

If the Type of the Argument does not match the expected Type (a List of Numeric Values) a Value Error is raised.

The Argument must be a List of Numeric Values (`int`, `real`, or `complex`). The function computes:

$$\text{parallel}([v_1, v_2, \dots, v_n]) = \frac{1}{\frac{1}{v_1} + \frac{1}{v_2} + \dots + \frac{1}{v_n}}$$

If any Element in the List is Zero, the Function immediately Returns Zero.

If any Element is Infinite, it is Ignored (does not contribute to the Result).

The Return Type is determined by the Numerical Value of the Result:

- If the Result is Numerically Equal to an Integer, Return `int`
- Otherwise, if the Result is Numerically Equal to a Real, Return `real`
- Otherwise, Return `complex`

Examples:

```
f.math.parallel([100, 100])      ## Returns 50 (int)
f.math.parallel([100, 200, 300]) ## Returns 54 (int, softint)
f.math.parallel([1e3, 2.2e3])    ## Returns 687.5 (real)
f.math.parallel([100, 0])        ## Returns 0 (any zero)
f.math.parallel([100, 200, 0, 300]) ## Returns 0 (any zero)
f.math.parallel([100, inf])      ## Returns 100 (inf ignored)
f.math.parallel([100, 200, inf]) ## Returns 66.666... (inf ignored)
```

Complex Overload When the List contains Complex Numbers, the function computes the parallel combination using Complex Arithmetic. If the Result has a Zero Imaginary Part, it is Returned as `real`.

Example:

```
f.math.parallel([100 + 0i, 200 + 0i])
    ## Returns 66.666... (real, imag=0)
f.math.parallel([3 + 4i, 1 - 2i])      ## Returns complex result
```

Domain: All Lists of Numeric Values. Empty Lists raise a Value Error. Lists with a single Element return that Element unchanged (after `softint(softreal())`).

Signum Function

The Signum Function (denoted as `sign`) returns the Sign of the Argument.

Morphology of the Signum Function:

EBNF:

```
"f.math.sign", "(", argument, ")"
```

If the Type of the Argument does not match the expected Type (a Numeric Value or a List of Numeric Values) a Value Error is raised.

Example:

```
f.math.sign(-3.2)
```

This Example would return a value of -1 .

The Argument may also be a List of Numeric Values. If This is the case the Function is iterated over each Element.

Example:

```
f.math.sign([-3, 0.2, 0, -7])
```

This Example would return $[-1, 1, 0, -1]$.

There are no Domain-Restrictions for the function inside $\mathbb{R}^n$.

Complex Overload When the Argument is a Complex Number $z = a + bi$, **sign** returns the unit complex $z/|z|$. For $z = 0$, **sign** returns 0 (int).

$$\text{sign}(z) = \begin{cases} \frac{z}{|z|} & z \neq 0 \\ 0 & z = 0 \end{cases}$$

Domain: All Complex Numbers ($\mathbb{C}$). Examples:

```
f.math.sign(3 + 4i)      #c# Returns approximately 0.6 + 0.8i
f.math.sign(0 + 0i)      #c# Returns 0 (int)
f.math.sign(1 + 0i)      #c# Returns 1 + 0i
```

Sine Function

Computes the sine of an angle. Angles are interpreted as radians by default.

Morphology:

EBNF:

```
"f.trig.sin", "(", ( int | real | complex ), [ ",", "unit", "=", string_type ], ")"
```

- Operands: int, real, complex, or list of numeric values.
- Optional unit: "rad" (default) or "deg".
- Result: real for int/real input, complex for complex input. For list input, the function is applied element-wise.
- Domain: All $\mathbb{R}$ and all $\mathbb{C}$.
- A Type Error is raised if the argument is not a numeric value or list of numeric values.

Examples:

```
f.trig.sin(1.570796327)      #c# Returns approximately 1.0
f.trig.sin([0, 30, 90], unit="deg") #c# Returns [0, 0.5, 1]
f.trig.sin(1 + 2i)          #c# Returns approximately 3.1658 - 1.9596i
```

Cosine Function

Computes the cosine of an angle. Angles are interpreted as radians by default.

Morphology:

EBNF:

```
"f.trig.cos", "(", ( int | real | complex ), [ ",", "unit", "=", string_type ], ")"
```

- Operands: `int`, `real`, `complex`, or list of numeric values.
- Optional unit: `"rad"` (default) or `"deg"`.
- Result: `real` for `int/real` input, `complex` for `complex` input. For list input, the function is applied element-wise.
- Domain: All $\mathbb{R}$ and all $\mathbb{C}$.
- A `Type Error` is raised if the argument is not a numeric value or list of numeric values.

Examples:

```
f.trig.cos(0)                ## Returns 1.0
f.trig.cos([0, 60, 90], unit="deg")  ## Returns [1, 0.5, 0]
f.trig.cos(1 + 2i)            ## Returns approximately 2.0327 + 3.0519i
```

Tangent Function

Computes the tangent of an angle. Angles are interpreted as radians by default.

Morphology:

EBNF:

```
"f.trig.tan", "(", ( int | real | complex ), [ ",", "unit", "=", string_type ], ")"
```

- Operands: `int`, `real`, `complex`, or list of numeric values.
- Optional unit: `"rad"` (default) or `"deg"`.
- Result: `real` for `int/real` input, `complex` for `complex` input. For list input, the function is applied element-wise.
- Domain for real: $\{x \in \mathbb{R} : \cos x \neq 0\}$. A `Division by Zero Error` is raised when $\cos x = 0$.
- Domain for complex: all $\mathbb{C}$ where $\cos z \neq 0$.
- A `Type Error` is raised if the argument is not a numeric value or list of numeric values.

Examples:

```
f.trig.tan(0)                ## Returns 0.0
f.trig.tan([0, 45, 90], unit="deg")  ## Returns [0, 1, inf]
f.trig.tan(0 + 1i)            ## Returns approximately 0.0 + 0.7616i
```

Cotangent Function

Computes the cotangent $\cot(x) = \frac{1}{\tan(x)} = \frac{\cos(x)}{\sin(x)}$.

Morphology:

EBNF:

```
"f.trig.cot", "(", ( int | real | complex ), [ ",", "unit", "=", string_type ], ")"
```

- Operands: `int`, `real`, `complex`, or list of numeric values.
- Optional unit: `"rad"` (default) or `"deg"`.
- Result: `real` for `int/real` input, `complex` for `complex` input.
- A Division by Zero Error is raised when $\sin x = 0$.

Examples:

```
f.trig.cot(0.7854)          ## Returns approximately 1.0  (pi/4)
f.trig.cot([45, 90], unit="deg")  ## Returns [1.0, 0.0]
```

Secant Function

Computes the secant $\sec(x) = \frac{1}{\cos(x)}$.

Morphology:

EBNF:

```
"f.trig.sec", "(", ( int | real | complex ), [ ",", "unit", "=", string_type ], ")"
```

- Operands: `int`, `real`, `complex`, or list of numeric values.
- Optional unit: `"rad"` (default) or `"deg"`.
- Result: `real` for `int/real` input, `complex` for `complex` input.
- A Division by Zero Error is raised when $\cos x = 0$.

Examples:

```
f.trig.sec(0)              ## Returns 1.0
f.trig.sec([0, 60], unit="deg")  ## Returns [1.0, 2.0]
```

Cosecant Function

Computes the cosecant $\csc(x) = \frac{1}{\sin(x)}$.

Morphology:

EBNF:

```
"f.trig.csc", "(", ( int | real | complex ), [ ",", "unit", "=", string_type ], ")"
```

- Operands: `int`, `real`, `complex`, or list of numeric values.
- Optional unit: `"rad"` (default) or `"deg"`.
- Result: `real` for `int/real` input, `complex` for `complex` input.

- A `Division by Zero Error` is raised when $\sin x = 0$.

Examples:

```
f.trig.csc(1.5708)           #c# Returns approximately 1.0 (pi/2)
f.trig.csc([30, 90], unit="deg")  #c# Returns [2.0, 1.0]
```

Complex Trigonometric Formulas

For complex $z = a + bi$, the trigonometric functions are computed via analytic continuation:

$$\sin(a + bi) = \sin(a) \cosh(b) + i \cos(a) \sinh(b)$$

$$\cos(a + bi) = \cos(a) \cosh(b) - i \sin(a) \sinh(b)$$

$$\tan(a + bi) = \frac{\sin(a + bi)}{\cos(a + bi)}$$

Arcsine Function

Computes the inverse sine. Angles are interpreted as radians by default.

Morphology:

EBNF:

```
("f.trig.asin"), "(", ( int | real | complex ),
  [ ",", "unit", "=", string_type ], ")"
```

- Operands: `int`, `real`, or list of numeric values.
- Optional unit: `"rad"` (default) or `"deg"`.
- Result: `real`.
- Domain: $[-1, 1]$. A `Value Error` is raised for $|x| > 1$.
- A `Type Error` is raised if the argument is not a numeric value or list of numeric values.

Examples:

```
f.trig.asin(0)           #c# Returns 0.0
f.trig.asin(1)           #c# Returns approximately 1.5708 (pi/2)
f.trig.asin([0, 0.5, 1], unit="deg")  #c# Returns [0, 30, 90]
```

Arccosine Function

Computes the inverse cosine. Angles are interpreted as radians by default.

Morphology:

EBNF:

```
("f.trig.acos"), "(", ( int | real | complex ),
  [ ",", "unit", "=", string_type ], ")"
```

- Operands: `int`, `real`, or list of numeric values.

- Optional unit: "rad" (default) or "deg".
- Result: `real`.
- Domain: $[-1, 1]$. A `Value Error` is raised for $|x| > 1$.
- A `Type Error` is raised if the argument is not a numeric value or list of numeric values.

Examples:

```
f.trig.acos(1)           ## Returns 0.0
f.trig.acos(0)           ## Returns approximately 1.5708 (pi/2)
f.trig.acos([0, 0.5, 1], unit="deg")  ## Returns [90, 60, 0]
```

Arctangent Function

Computes the inverse tangent. Angles are interpreted as radians by default.

Morphology:

EBNF:

```
("f.trig.atan"), "(", ( int | real | complex ),
  [ ",", "unit", "=", string_type ], ")"
```

- Operands: `int`, `real`, or list of numeric values.
- Optional unit: "rad" (default) or "deg".
- Result: `real`.
- Domain: all $\mathbb{R}$. No domain restrictions.
- A `Type Error` is raised if the argument is not a numeric value or list of numeric values.

Examples:

```
f.trig.atan(0)           ## Returns 0.0
f.trig.atan(1)           ## Returns approximately 0.7854 (pi/4)
f.trig.atan([0, 1, inf], unit="deg")  ## Returns [0, 45, 90]
```

Arccotangent Function

Computes the inverse cotangent $\operatorname{acot}(x) = \frac{\pi}{2} - \operatorname{atan}(x)$.

Morphology:

EBNF:

```
"f.trig.acot", "(", ( int | real ), [ ",", "unit", "=", string_type ], ")"
```

- Operands: `int` or `real`.
- Optional unit: "rad" (default) or "deg".
- Result: `real`. Domain: all $\mathbb{R}$.

Examples:

```
f.trig.acot(1)           ## Returns approximately 0.7854 (pi/4)
f.trig.acot(inf)         ## Returns 0.0
```

Arcsecant Function

Computes the inverse secant $\text{asec}(x) = \text{acos}(1/x)$.

Morphology:

EBNF:

```
"f.trig.asec", "(", ( int | real ), [ ",", "unit", "=", string_type ], ")"
```

- Operands: `int` or `real`.
- Optional unit: `"rad"` (default) or `"deg"`.
- Result: `real`.
- A Value Error is raised for $|x| < 1$ (out of domain).

Examples:

```
f.trig.asec(1)           ## Returns 0.0
f.trig.asec(2)           ## Returns approximately 1.0472 (pi/3)
```

Arccosecant Function

Computes the inverse cosecant $\text{acsc}(x) = \text{asin}(1/x)$.

Morphology:

EBNF:

```
"f.trig.acsc", "(", ( int | real ), [ ",", "unit", "=", string_type ], ")"
```

- Operands: `int` or `real`.
- Optional unit: `"rad"` (default) or `"deg"`.
- Result: `real`.
- A Value Error is raised for $|x| < 1$ (out of domain).

Examples:

```
f.trig.acsc(1)           ## Returns approximately 1.5708 (pi/2)
f.trig.acsc(2)           ## Returns approximately 0.5236 (pi/6)
```

Hyperbolic Sine Function

Computes the hyperbolic sine of the argument.

Morphology:

EBNF:

```
"f.hyp.sinh", "(", ( int | real | complex ), ")"
```

- Operands: `int`, `real`, `complex`, or list of numeric values.
- Result: `real` for `int/real` input, `complex` for `complex` input.
- Domain: all $\mathbb{R}$ and all $\mathbb{C}$. No restrictions.

- A **Type Error** is raised if the argument is not a numeric value or list of numeric values.

Examples:

```
f.hyp.sinh(0)           ## Returns 0.0
f.hyp.sinh(1.443635475) ## Returns approximately 2.0
f.hyp.sinh(1 + 2i)      ## Returns approximately -0.4891 + 0.9889i
```

Hyperbolic Cosine Function

Computes the hyperbolic cosine of the argument.

Morphology:

EBNF:

```
"f.hyp.cosh", "(", ( int | real | complex ), ")"
```

- Operands: `int`, `real`, `complex`, or list of numeric values.
- Result: `real` for `int/real` input, `complex` for `complex` input.
- Domain: all $\mathbb{R}$ and all $\mathbb{C}$. No restrictions.
- A **Type Error** is raised if the argument is not a numeric value or list of numeric values.

Examples:

```
f.hyp.cosh(0)           ## Returns 1.0
f.hyp.cosh(1)           ## Returns approximately 1.5431
f.hyp.cosh(1 + 2i)      ## Returns approximately -0.4891 + 0.9889i
```

Hyperbolic Tangent Function

Computes the hyperbolic tangent of the argument.

Morphology:

EBNF:

```
"f.hyp.tanh", "(", ( int | real | complex ), ")"
```

- Operands: `int`, `real`, `complex`, or list of numeric values.
- Result: `real` for `int/real` input, `complex` for `complex` input.
- Domain for `real`: all $\mathbb{R}$. For `complex`: all $\mathbb{C}$ where $\cosh z \neq 0$.
- A **Type Error** is raised if the argument is not a numeric value or list of numeric values.

Examples:

```
f.hyp.tanh(0)           ## Returns 0.0
f.hyp.tanh(1)           ## Returns approximately 0.7616
f.hyp.tanh(0 + 1i)      ## Returns approximately 0.0 + 0.5524i
```

Hyperbolic Cotangent Function

Computes the hyperbolic cotangent $\coth(x) = \frac{\cosh(x)}{\sinh(x)}$.

Morphology:

EBNF:

```
"f.hyp.coth", "(", ( int | real | complex ), ")"
```

- Operands: int, real, or complex.
- Result: same type as input.
- A Division by Zero Error is raised at $x = 0$.

Examples:

```
f.hyp.coth(1)          ## Returns approximately 1.3130
```

Hyperbolic Secant Function

Computes the hyperbolic secant $\operatorname{sech}(x) = \frac{1}{\cosh(x)}$.

Morphology:

EBNF:

```
"f.hyp.sech", "(", ( int | real | complex ), ")"
```

- Operands: int, real, or complex.
- Result: same type as input.
- No domain restrictions.

Examples:

```
f.hyp.sech(0)          ## Returns 1.0
f.hyp.sech(1)          ## Returns approximately 0.6481
```

Hyperbolic Cosecant Function

Computes the hyperbolic cosecant $\operatorname{csch}(x) = \frac{1}{\sinh(x)}$.

Morphology:

EBNF:

```
"f.hyp.csch", "(", ( int | real | complex ), ")"
```

- Operands: int, real, or complex.
- Result: same type as input.
- A Division by Zero Error is raised at $x = 0$.

Examples:

```
f.hyp.csch(1)          ## Returns approximately 0.8509
```

Complex Hyperbolic Formulas

For complex $z = a + bi$, the hyperbolic functions are computed as:

$$\sinh(a + bi) = \sinh(a) \cos(b) + i \cosh(a) \sin(b)$$

$$\cosh(a + bi) = \cosh(a) \cos(b) + i \sinh(a) \sin(b)$$

$$\tanh(a + bi) = \frac{\sinh(a + bi)}{\cosh(a + bi)}$$

Equivalently: $\sinh(z) = -i \sin(iz)$, $\cosh(z) = \cos(iz)$, $\tanh(z) = \sinh(z) / \cosh(z)$.

Hyperbolic Arcsine Function

Computes the inverse hyperbolic sine.

Morphology:

EBNF:

```
("f.hyp.asinh"), "(", ( int | real | complex ), ")"
```

- Operands: `int`, `real`, `complex`, or list of numeric values.
- Result: `real` for `int/real` input, `complex` for `complex` input.
- Domain: all $\mathbb{R}$ and all $\mathbb{C}$. No restrictions.
- A `Type Error` is raised if the argument is not a numeric value or list of numeric values.

Examples:

```
f.hyp.asinh(0)          #c# Returns 0.0
f.hyp.asinh(1)          #c# Returns approximately 0.8814
f.hyp.asinh(1 + 2i)     #c# Returns approximately 1.4694 + 1.0987i
```

Hyperbolic Arccosine Function

Computes the inverse hyperbolic cosine.

Morphology:

EBNF:

```
("f.hyp.acosh"), "(", ( int | real | complex ), ")"
```

- Operands: `int`, `real`, `complex`, or list of numeric values.
- Result: `real` for `int/real` input, `complex` for `complex` input.
- Domain for real: $[1, \infty)$. A `Value Error` is raised for $x < 1$.
- Domain for complex: all $\mathbb{C}$ (using principal branch).
- A `Type Error` is raised if the argument is not a numeric value or list of numeric values.

Examples:

```
f.hyp.acosh(1)          #c# Returns 0.0
f.hyp.acosh(2)          #c# Returns approximately 1.3170
f.hyp.acosh(1 + 2i)     #c# Returns approximately 1.5286 + 1.1437i
```

Hyperbolic Arctangent Function

Computes the inverse hyperbolic tangent.

Morphology:

EBNF:

```
("f.hyp.atanh"), "(", ( int | real | complex ), ")"
```

- Operands: `int`, `real`, `complex`, or list of numeric values.
- Result: `real` for `int`/`real` input, `complex` for `complex` input.
- Domain for `real`: $(-1, 1)$. A `Value Error` is raised for $|x| \geq 1$.
- Domain for `complex`: all $\mathbb{C}$ where $z \neq \pm 1$.
- A `Type Error` is raised if the argument is not a numeric value or list of numeric values.

Examples:

```
f.hyp.atanh(0)           ## Returns 0.0
f.hyp.atanh(0.5)         ## Returns approximately 0.5493
f.hyp.atanh(0.5 + 0i)    ## Returns approximately 0.5493 + 0.0i
```

Inverse Hyperbolic Cotangent Function

Computes $\operatorname{acoth}(x) = \frac{1}{2} \ln\left(\frac{x+1}{x-1}\right)$.

Morphology:

EBNF:

```
"f.hyp.acoth", "(", ( int | real | complex ), ")"
```

- Operands: `int`, `real`, or `complex`.
- Result: same type as input.
- A `Value Error` is raised for `real` $|x| \leq 1$ (out of domain).

Examples:

```
f.hyp.acoth(2)           ## Returns approximately 0.5493
```

Inverse Hyperbolic Secant Function

Computes $\operatorname{asech}(x) = \operatorname{acosh}(1/x)$.

Morphology:

EBNF:

```
"f.hyp.asech", "(", ( int | real | complex ), ")"
```

- Operands: `int`, `real`, or `complex`.
- Result: same type as input.
- A `Value Error` is raised for `real` $x \leq 0$ or $x > 1$ (out of domain).

Examples:

```
f.hyp.asech(1)           ## Returns 0.0
```

Inverse Hyperbolic Cosecant Function

Computes $\operatorname{acsch}(x) = \operatorname{asch}(1/x)$.

Morphology:

EBNF:

```
"f.hyp.acsch", "(", ( int | real | complex ), ")"
```

- Operands: `int`, `real`, or `complex`.
- Result: same type as input.
- A `Value Error` is raised at $x = 0$ (singularity).

Examples:

```
f.hyp.acsch(1)          #c# Returns approximately 0.8814
```

Complex Hyperbolic Inverse Formulas

For complex arguments, the inverse hyperbolic functions are computed via the complex logarithm:

$$\operatorname{asinh}(z) = \ln\left(z + \sqrt{z^2 + 1}\right)$$

$$\operatorname{acosh}(z) = \ln\left(z + \sqrt{z^2 - 1}\right)$$

$$\operatorname{atanh}(z) = \frac{1}{2} \ln\left(\frac{1+z}{1-z}\right)$$

Exponential Function

Returns e raised to the power of the argument.

Morphology:

EBNF:

```
"f.log.exp", "(", ( int | real | complex ), ")"
```

- Operands: `int`, `real`, `complex`, or list of numeric values.
- Result: `real` for `int`/`real` input, `complex` for `complex` input.
- Domain: all $\mathbb{R}$ and all $\mathbb{C}$.
- A `Type Error` is raised if the argument is not a numeric value or list of numeric values.

For complex $z = a + bi$, computed via Euler's Formula: $e^{a+bi} = e^a(\cos b + i \sin b)$. Examples:

```
f.log.exp(0)          #c# Returns 1.0
f.log.exp(1)          #c# Returns approximately 2.71828
f.log.exp(0 + 3.14159i) #c# Returns approximately -1 + 0.0i (Euler's Identity)
f.log.exp(1 + 1i)      #c# Returns approximately 1.4687 + 2.2874i
```

Square Root Function

Returns the principal square root of the argument.

Morphology:

EBNF:

```
"f.log.sqrt", "(", ( int | real | complex ), ")"
```

- Operands: **int**, **real**, **complex**, or list of numeric values.
- Result: **real** for non-negative **int**/**real** input, **complex** for negative **real** or **complex** input.
- Domain for **real**: $[0, \infty)$ returns **real**. Negative **real** input returns **complex** (principal branch).
- Domain for **complex**: all $\mathbb{C}$, principal branch $-\pi < \arg(\sqrt{z}) \leq \pi$.
- A **Type Error** is raised if the argument is not a numeric value or list of numeric values.

Examples:

```
f.log.sqrt(9)           ## Returns 3.0
f.log.sqrt(2)           ## Returns approximately 1.41421
f.log.sqrt(-1)          ## Returns 0 + 1i
f.log.sqrt(-4)          ## Returns 0 + 2i
f.log.sqrt(3 + 4i)       ## Returns approximately 2 + 1i
f.log.sqrt(0)           ## Returns 0.0
```

Power Function

Computes exponentiation and optional modular exponentiation.

Morphology:

EBNF:

```
"f.log.pow", "(", ( int | real | complex ), ",", ( int | real | complex ), [ ",", int ],
```

- `f.log.pow(a, b)`: returns a^b . Both arguments **int**, **real**, or **complex**.
- `f.log.pow(a, b, c)`: returns $a^b \bmod c$ (modular exponentiation). **a**, **c** must be **int**, **c** must be > 0 , **b** can be negative (computes modular inverse).
- Return type: **int** when both arguments are **int** and the result is exact; **real** otherwise.
- Without **c**: negative exponents on **int** produce **real**.
- With **c**: negative **b** computes $a^b \bmod c$ via modular multiplicative inverse (exists when $\gcd(a, c) = 1$).
- For complex z_1, z_2 : $z_1^{z_2} = e^{z_2 \ln z_1}$ (principal branch). 0^z only defined for $\operatorname{Re}(z) > 0$.
- A **Type Error** is raised if arguments are not numeric.

Examples:


```

f.log.pow(2, 3)          #c# Returns 8 (int)
f.log.pow(9, 0.5)        #c# Returns 3.0 (real)
f.log.pow(3, 4, 7)       #c# Returns 4  (81 mod 7)
f.log.pow(2, 32, 100)    #c# Returns 96  (4294967296 mod 100)
f.log.pow(3, -1, 7)      #c# Returns 5  (modular inverse: 3*5=15==1 mod 7)
f.log.pow(2, 1 + 1i)     #c# Returns approximately 1.538 + 0.999i
f.log.pow(1 + 1i, 2)     #c# Returns 0 + 2i

```

Hyperoperation

Computes the n th hyperoperation: addition, multiplication, exponentiation, tetration, pentation, or hexation. Optionally applies modular reduction.

Morphology:

EBNF:

```

"f.log.hyperop", "(", int, ",", ( int | real | complex ), ",", ( int | real | complex ),
[ ",", int ], ")"

```

- **n** (int): hyperoperation level (0–6).
- **a**: left operand. Any numeric type.
- **b**: right operand. Any numeric type for $n \leq 3$; must be `int` for $n \geq 4$.
- **m** (int, optional): modulus. Must be > 0 . When present, each intermediate step is reduced mod **m**.

Levels:

n	Name	Definition
0	Successor	$a + 1$ (b ignored)
1	Addition	$a + b$
2	Multiplication	$a \times b$
3	Exponentiation	a^b
4	Tetration	${}^b a$ (iterated exponentiation)
5	Pentation	(iterated tetration)
6	Hexation	(iterated pentation)

- `int` $\times$ `int` returns `int` (arbitrary precision). With modulus, result is in $[0, m)$.
- Any `real` operand returns `real`.
- Any `complex` operand returns `complex`. Modulus not supported for complex operands.
- A `Type Error` is raised if **n** is not `int`, or if $n \geq 4$ and **b** is not `int`.
- A `Type Error` is raised if $n < 0$ or $n > 6$.
- A `Type Error` is raised if **m** is provided and is not `int`, or if **a** is `complex`.
- A `Value Error` is raised if $n \geq 3$ and $a = 0$ and $b = 0$.
- A `Value Error` is raised if $n \geq 4$ and $a = 0$ and $b \leq 0$.

- A Value Error is raised if $n \geq 4$ and $b < 0$.
- A Value Error is raised if $m \leq 0$.
- A Value Error is raised if the result exceeds 2^{20} bits (only when no modulus is specified).

Examples:

```
f.log.hyperop(0, 5, 99)      #c# Returns 6          (successor, b ignored)
f.log.hyperop(1, 3, 4)      #c# Returns 7          (addition)
f.log.hyperop(2, 3, 4)      #c# Returns 12         (multiplication)
f.log.hyperop(3, 2, 10)     #c# Returns 1024        (exponentiation)
f.log.hyperop(4, 2, 4)      #c# Returns 65536       (tetration: 2^2^2^2)
f.log.hyperop(4, 3, 3)      #c# Returns 7625597484987 (3^3^3)
f.log.hyperop(5, 2, 3)      #c# Returns 65536       (pentation)
f.log.hyperop(6, 2, 3)      #c# Returns 65536       (hexation)
f.log.hyperop(4, 1+1i, 2)   #c# Returns approximately 0 + 2i
f.log.hyperop(3, 2.5, 3)    #c# Returns 15.625      (real)

# With modulus
f.log.hyperop(3, 2, 10, 1000) #c# Returns 24        (2^10 mod 1000 = 1024 mod 1000)
f.log.hyperop(4, 2, 8, 10000) #c# Returns 5536      (2 ^^ 8 mod 10000)
f.log.hyperop(4, 3, 3, 100)   #c# Returns 87        (3^3^3 mod 100 = 7625597484987 mod 100)
f.log.hyperop(3, 7, 256, 256) #c# Returns 0         (7^256 mod 256 = 0)
f.log.hyperop(2, 13, 17, 10)  #c# Returns 1         (13*17 mod 10 = 221 mod 10)
f.log.hyperop(4, 2, 4, 1)     #c# Returns 0         (anything mod 1 = 0)
```

Hyperroot

Computes the hyperroot: the inverse of a hyperoperation. Finds x such that $\text{hyperop}(n, x, b) = y$.

Morphology:

EBNF:

```
"f.log.nthhyprt", "(", ( int | real | complex ), ",", int, ",", int, ")"
```

- y : the value to find the hyperroot of. Any numeric type.
- n (int): hyperoperation level (3–6).
- b (int): iteration count. Must be ≥ 1 .

Relation to `f.log.hyperop`:

- `f.log.nthhyprt(y, 3, b)` = b th root of $y = y^{1/b}$ (inverse of exponentiation).
- `f.log.nthhyprt(y, 4, b)` = b th hyperroot of y (inverse of tetration).
- `f.log.nthhyprt(y, 5, b)` = inverse of pentation.
- `f.log.nthhyprt(y, 6, b)` = inverse of hexation.

- Returns real for int/real y ; complex for complex y .
- A Type Error is raised if n or b is not int, or if $n < 3$ or $n > 6$.
- A Value Error is raised if $b < 1$.
- A Value Error is raised if $y \leq 0$ and $n \geq 4$ (tetration base must be positive for real result).

Examples:

```
f.log.nthhypert(27, 3, 3)      #c# Returns 3.0      (cube root of 27)
f.log.nthhypert(65536, 4, 2)   #c# Returns 4        (2nd hyperroot:  $x^x = 65536 \rightarrow x = 4$ )
f.log.nthhypert(7625597484987, 4, 3) #c# Returns 3 (3rd hyperroot:  $x^x^x = \dots$ )
f.log.nthhypert(16, 3, 4)      #c# Returns 2.0      (4th root of 16)
f.log.nthhypert(0 + 64i, 3, 3) #c# Returns approximately 2 + 2i (complex root)
```

Super-Logarithm

Computes the super-logarithm: the inverse of tetration. Finds x such that $b \uparrow\uparrow x = y$. “How many times must I tetrate b to reach y ?”

Morphology:

EBNF:

```
"f.log.slog", "(", ( int | real ), ",", ( int | real ), ")"
```

- b (int — real): the base of the tetration. Must be > 0 .
- y (int — real): the target value. Must be ≥ 1 for real result.

Relation to `f.log.hyperop`:

- $\text{slog}(b, y) = x$ means $b \uparrow\uparrow x = y$.
- Inverse of `f.log.hyperop(4, b, x)`.
- Returns real. Integer results are exact (e.g., $\text{slog}(2, 65536) = 4.0$ exactly).
- A Type Error is raised if arguments are not numeric.
- A Value Error is raised if $b \leq 0$ or $y < 1$.

Examples:

```
f.log.slog(2, 1)      #c# Returns 0.0      (2  $\uparrow\uparrow$  0 = 1)
f.log.slog(2, 2)      #c# Returns 1.0      (2  $\uparrow\uparrow$  1 = 2)
f.log.slog(2, 4)      #c# Returns 2.0      (2  $\uparrow\uparrow$  2 = 22 = 4)
f.log.slog(2, 16)     #c# Returns 3.0      (2  $\uparrow\uparrow$  3 = 2222 = 16)
f.log.slog(2, 65536)  #c# Returns 4.0      (2  $\uparrow\uparrow$  4 = 65536)
f.log.slog(3, 7625597484987) #c# Returns 3.0 (3  $\uparrow\uparrow$  3 = 3333)
f.log.slog(2, 3)      #c# Returns approximately 1.409 (between 1 and 2)
f.log.slog(10, 1000)  #c# Returns approximately 2.388 (between 2 and 3)
```

Natural Logarithm

Returns the natural logarithm (base e) of the argument.

Morphology:

EBNF:

```
"f.log.ln", "(", ( int | real | complex ), ")"
```

- Operands: `int` (must be > 0), `real` (must be > 0), `complex` ($\neq 0$).
- Result: `real` for positive `int`/`real` input, `complex` for `complex` input.
- A `Value Error` is raised for non-positive real arguments.
- A `Type Error` is raised if the argument is not a numeric value or list of numeric values.

Examples:

```
f.log.ln(1)           #c# Returns 0.0
f.log.ln(c.math.e)    #c# Returns approximately 1.0
f.log.ln(100)         #c# Returns approximately 4.6052
f.log.ln(1 + 1i)      #c# Returns approximately 0.3466 + 0.7854i
```

Base-10 Logarithm

Returns the base-10 logarithm of the argument.

Morphology:

EBNF:

```
"f.log.log10", "(", ( int | real | complex ), ")"
```

- Operands: `int` (must be > 0), `real` (must be > 0), `complex` ($\neq 0$).
- Result: `real` for positive `int`/`real` input, `complex` for `complex` input.
- A `Value Error` is raised for non-positive real arguments.
- A `Type Error` is raised if the argument is not a numeric value or list of numeric values.

Examples:

```
f.log10(1)           #c# Returns 0.0
f.log10(10)          #c# Returns 1.0
f.log10(1000)        #c# Returns 3.0
f.log10(1 + 1i)      #c# Returns approximately 0.1505 + 0.3411i
```

Expm1 Function

Computes $e^x - 1$ with high precision for small x , avoiding catastrophic cancellation.

Morphology:

EBNF:

```
"f.math.expm1", "(", ( int | real ), ")"
```

- Operand: `int` or `real`.
- Result: `real`.

Examples:

```
f.math.expm1(0)      #c# Returns 0.0
f.math.expm1(1)      #c# Returns approximately 1.7183 (e - 1)
f.math.expm1(0.001)  #c# Returns approximately 0.0010005 (more precise than exp(0.001))
```

Log1p Function

Computes $\ln(1 + x)$ with high precision for small x , avoiding catastrophic cancellation.

Morphology:

EBNF:

```
"f.math.log1p", "(", ( int | real ), ")"
```

- Operand: `int` or `real`.
- Result: `real`.
- A Value Error is raised for $x \leq -1$.

Examples:

```
f.math.log1p(0)      #c# Returns 0.0
f.math.log1p(1)      #c# Returns approximately 0.6931 (ln 2)
f.math.log1p(0.001)  #c# Returns approximately 0.0009995 (more precise than ln(1.001))
```

Exp2 Function

Returns 2^x . More precise and faster than `f.math.pow(2, x)`.

Morphology:

EBNF:

```
"f.math.exp2", "(", ( int | real ), ")"
```

- Operand: `int` or `real`.
- Result: `real`.

Examples:

```
f.math.exp2(0)      #c# Returns 1.0
f.math.exp2(1)      #c# Returns 2.0
f.math.exp2(10)     #c# Returns 1024.0
f.math.exp2(-1)     #c# Returns 0.5
```

Logaddexp Function

Returns $\ln(e^x + e^y)$, numerically stable. Prevents overflow when computing log of sum of exponentials.

Morphology:

EBNF:

```
"f.math.logaddexp", "(", real, ",", real, ")"
```

- Operands: `real`, `real`.
- Result: `real`.

Examples:

```
f.math.logaddexp(0, 0)      #c# Returns approximately 0.6931 (ln 2)
f.math.logaddexp(0, 1000)   #c# Returns approximately 1000.0 (no overflow)
```

Logaddexp2 Function

Returns $\log_2(2^x + 2^y)$, numerically stable.

Morphology:

EBNF:

```
"f.math.logaddexp2", "(", real, ",", real, ")"
```

- Operands: `real`, `real`.
- Result: `real`.

Examples:

```
f.math.logaddexp2(0, 0)     #c# Returns 1.0 (log2(2))
f.math.logaddexp2(0, 1000)  #c# Returns approximately 1000.0 (no overflow)
```

IsClose Function

Compares two scalar values for approximate equality using combined absolute and relative tolerance. This is the function form of the approximate equality operators (`~`, `~*`, `~+()`, `~*()`); unlike the operators, `f.math.isclose` can be passed as a function argument, used in lambdas, and has explicit tolerance parameters.

The comparison formula is:

$$|a - b| \leq \max(\text{atol}, \text{rtol} * \max(|a|, |b|))$$

The absolute tolerance `atol` dominates near zero (where relative tolerance would be meaningless). The relative tolerance `rtol` dominates for large values.

Morphology:

EBNF:

```
"f.math.isclose", "(", (int | real), ",", (int | real), [",", "rtol", "=", real], ["
```

- Operands: two values to compare (`int` or `real`), optional `rtol` (relative tolerance, default 1e-9), optional `atol` (absolute tolerance, default 0.0).

- Result: `boolean` (True if within tolerance, False otherwise).
- The tolerance values must be non-negative. A `Value Error` is raised if `rtol < 0` or `atol < 0`.

Relationship to Operators

- `f.math.isclose(a, b, atol=c)` is equivalent to $a \sim_{+(c)} b$
- `f.math.isclose(a, b, rtol=c)` is equivalent to $a \sim_{*(c)} b$
- `f.math.isclose(a, b)` uses `rtol=1e-9`, which is much tighter than the default `syst.flag.approx.epsilon = 1`

Examples

```
#c# Basic floating-point rounding
f.math.isclose(0.1 + 0.2, 0.3)           #c# Returns True   (0.1+0.2 = 0.30000000000000004)
f.math.isclose(1.0, 1.0)                 #c# Returns True
f.math.isclose(1.0, 2.0)                 #c# Returns False

#c# Tolerance tuning
f.math.isclose(0.1 + 0.2, 0.3, rtol=0)   #c# Returns False (strict: no relative tolerance)
f.math.isclose(1000.0, 1000.001)         #c# Returns False (rtol=1e-9 too small)
f.math.isclose(1000.0, 1000.001, rtol=1e-4) #c# Returns True  (1e-4 covers 0.001/1000)

#c# Absolute tolerance near zero
f.math.isclose(0.0, 1e-10)               #c# Returns False (atol=0, no absolute tolerance)
f.math.isclose(0.0, 1e-10, atol=1e-9)   #c# Returns True  (atol dominates near zero)

#c# Composability
$f [func]= L::($x [real], $y [real]) -> { f.math.isclose($x, $y, rtol=1e-6) }
$f(0.1 + 0.2, 0.3)                      #c# Returns True  (usable in lambdas)
```

AllClose Function

Compares two collections element-wise for approximate equality. Returns `True` if every corresponding pair of elements satisfies the tolerance check (same formula as `f.math.isclose`). Nested lists must have matching structure.

There is no operator equivalent for element-wise collection comparison.

Morphology:

EBNF:

```
"f.math.allclose", "(", list, ",", list, "[", ",", "rtol", "=", real, "[", ",", "atol", "=", real,
```

- Operands: two `lists` of the same length (nested lists must have matching structure), optional `rtol` (relative tolerance, default 1e-9), optional `atol` (absolute tolerance, default 0.0).
- Result: `boolean` (True if all element pairs are within tolerance).

- A **Type Error** is raised if the operands are not both lists.
- A **Value Error** is raised if the lists have different lengths.

Examples

```
#c# Basic usage
f.math.allclose([0.1, 0.2, 0.3], [0.1+0.2, 0.2+0.1, 0.3+0.0]) #c# Returns True
f.math.allclose([1.0, 2.0], [1.0, 2.0]) #c# Returns True
f.math.allclose([1.0, 2.0], [1.0, 2.1]) #c# Returns False

#c# Edge cases
f.math.allclose([], []) #c# Returns True (empty lists)
f.math.allclose([0.0, 1e-10], [0.0, 0.0], atol=1e-9) #c# Returns True

#c# Nested lists
f.math.allclose([ [1.0, 2.0], [3.0, 4.0] ], [ [1.0, 2.0], [3.0, 4.0] ]) #c# Returns True

#c# Typical workflow: compare two filter responses
$response_a [tensor]= compute_response($coeffs_a, $coeffs_b)
$response_b [tensor]= compute_response($coeffs_c, $coeffs_d)
f.math.allclose($response_a, $response_b, rtol=1e-6) #c# Returns True if responses are close
```

Real Part Function

Returns the real part of a complex number.

Morphology:

EBNF:

```
"f.cplx.real", "(", ( int | real | complex ), ")"
```

- Operands: **int** (promoted to **real**), **real**, **complex**.
- Result: **real**.
- For **int/real** input, the value is returned as **real**.
- A **Type Error** is raised for non-numeric arguments.

Examples:

```
f.cplx.real(3 + 4i) #c# Returns 3.0
f.cplx.real(-2i) #c# Returns 0.0
f.cplx.real(5) #c# Returns 5.0 (int promoted)
```

Imaginary Part Function

Returns the imaginary part of a complex number.

Morphology:

EBNF:

```
"f.cplx.imag", "(", ( int | real | complex ), ")"
```

- Operands: `int`, `real` (imaginary part is 0.0), `complex`.
- Result: `real`.
- A `Type Error` is raised for non-numeric arguments.

Examples:

```
f.cplx.imag(3 + 4i)    #c# Returns 4.0
f.cplx.imag(-2i)       #c# Returns -2.0
f.cplx.imag(5)         #c# Returns 0.0 (real has no imaginary part)
```

Angle Function

Returns the angle (argument) of a complex number in radians, in the range $(-\pi, \pi]$.

Morphology:

EBNF:

```
"f.cplx.angle", "(", ( int | real | complex ), ")"
```

- Operands: `int`, `real`, `complex`.
- Result: `real`.
- `angle(0)` returns 0.0.
- A `Type Error` is raised for non-numeric arguments.

Examples:

```
f.cplx.angle(1 + 0i)    #c# Returns 0.0
f.cplx.angle(0 + 1i)    #c# Returns 1.5708 (pi/2)
f.cplx.angle(-1 + 0i)   #c# Returns 3.14159 (pi)
f.cplx.angle(1 + 1i)    #c# Returns 0.7854 (pi/4)
```

Complex Conjugate Function

Returns the complex conjugate: $\overline{a + bi} = a - bi$.

Morphology:

EBNF:

```
"f.cplx.conj", "(", ( int | real | complex ), ")"
```

- Operands: `int`, `real` (self-conjugate), `complex`.
- Result: same type as input. Real numbers are self-conjugate.
- A `Type Error` is raised for non-numeric arguments.

Examples:

```
f.cplx.conj(3 + 4i)    #c# Returns 3 - 4i
f.cplx.conj(-2i)       #c# Returns 2i
f.cplx.conj(5)         #c# Returns 5 (real numbers are self-conjugate)
```

Polar Conversion Function

Converts a complex number to polar form, returning [magnitude, angle].

Morphology:

EBNF:

```
"f.cplx.polar", "(", ( int | real | complex ), ")"
```

- Operands: int, real, complex.
- Result: list of two real values: [|z|, arg(z)].
- The angle is in radians, in the range $(-\pi, \pi]$.
- A `Type Error` is raised for non-numeric arguments.

Examples:

```
f.cplx.polar(3 + 4i)      ## Returns [5.0, 0.9273]
f.cplx.polar(1 + 0i)      ## Returns [1.0, 0.0]
f.cplx.polar(0 + 1i)      ## Returns [1.0, 1.5708]
```

Rectangular Conversion Function

Creates a complex number from polar form (magnitude and angle in radians).

Morphology:

EBNF:

```
"f.cplx.rect", "(", real, ",", real, ")"
```

- Operands: magnitude (int or real), angle in radians (int or real).
- Result: complex.
- Computes $z = r(\cos \theta + i \sin \theta)$.
- A `Type Error` is raised if arguments are not numeric.

Examples:

```
f.cplx.rect(5, 0.9273)    ## Returns approximately 3 + 4i
f.cplx.rect(1, 0)         ## Returns 1 + 0i
f.cplx.rect(1, 1.5708)    ## Returns approximately 0 + 1i
```

Cis Function

Computes the cis function: $\text{cis}(\theta) = \cos(\theta) + i \cdot \sin(\theta) = e^{i\theta}$. Returns a complex number on the unit circle at angle θ (radians).

Morphology:

EBNF:

```
"f.cplx.cis", "(", ( int | real ), ")"
```

- Operands: angle in radians (int or real).

- Result: `complex`.
- Computes $\text{cis}(\theta) = \cos(\theta) + i \cdot \sin(\theta)$.
- A `Type Error` is raised if the argument is not `int` or `real`.

Examples:

```
f.cplx.cis(0)           #c# Returns 1 + 0i
f.cplx.cis(1.5708)      #c# Returns approximately 0 + 1i (pi/2)
f.cplx.cis(3.14159)     #c# Returns approximately -1 + 0i (pi)
f.cplx.cis(-1.5708)     #c# Returns approximately 0 - 1i (-pi/2)
```

Radians to Degrees Function

Converts an angle from radians to degrees.

Morphology:

EBNF:

```
"f.trig.rad2deg", "(", ( int | real ), ")"
```

- Operands: `int` or `real`.
- Result: `real`.
- Computes $\text{degrees} = \text{radians} \times \frac{180}{\pi}$.
- A `Type Error` is raised if the argument is not `int` or `real`.

Examples:

```
f.trig.rad2deg(3.14159) #c# Returns 180.0
f.trig.rad2deg(1.5708)  #c# Returns 90.0
f.trig.rad2deg(0)       #c# Returns 0.0
```

Degrees to Radians Function

Converts an angle from degrees to radians.

Morphology:

EBNF:

```
"f.trig.deg2rad", "(", ( int | real ), ")"
```

- Operands: `int` or `real`.
- Result: `real`.
- Computes $\text{radians} = \text{degrees} \times \frac{\pi}{180}$.
- A `Type Error` is raised if the argument is not `int` or `real`.

Examples:

```
f.trig.deg2rad(180)    #c# Returns 3.14159
f.trig.deg2rad(90)     #c# Returns 1.5708
f.trig.deg2rad(0)      #c# Returns 0.0
```

Floor Function

Rounds the argument down to the nearest integer.

Morphology:

EBNF:

```
"f.math.floor", "(", ( int | real ), ")"
```

- Operands: `int` (returned as-is), `real`.
- Result: `int`.
- Returns the largest integer $\leq$ the argument.
- A `Type Error` is raised if the argument is not `int` or `real`.

Examples:

```
f.math.floor(2.7)    #c# Returns 2
f.math.floor(-1.3)   #c# Returns -2
f.math.floor(5)      #c# Returns 5
```

Ceiling Function

Rounds the argument up to the nearest integer.

Morphology:

EBNF:

```
"f.math.ceil", "(", ( int | real ), ")"
```

- Operands: `int` (returned as-is), `real`.
- Result: `int`.
- Returns the smallest integer $\geq$ the argument.
- A `Type Error` is raised if the argument is not `int` or `real`.

Examples:

```
f.math.ceil(2.3)     #c# Returns 3
f.math.ceil(-2.7)    #c# Returns -2
f.math.ceil(5)       #c# Returns 5
```

Round Function

Rounds the argument to the nearest value. Without `digits`, rounds to the nearest integer. With `digits`, rounds to the specified number of decimal places. Negative `digits` rounds to the nearest 10^n : `digits=-1` rounds to nearest 10, `digits=-2` to nearest 100, etc.

Morphology:

EBNF:

```
"f.math.round", "(", ( int | real ), [",", "digits", "=", int], ")"
```

- Operands: value (`int` or `real`), optional `digits` (`int`, default 0).
- Result: `int` when no `digits` specified and input is `int`; `int` when `digits` ≥ 0 ; `real` when `digits` < 0 .
- Uses banker's rounding (round half to even).
- A `Type Error` is raised if the first argument is not `int` or `real`.
- A `Type Error` is raised if `digits` is not `int`.

Examples:

```
f.math.round(2.5)           ## Returns 2 (banker's rounding)
f.math.round(2.4)           ## Returns 2
f.math.round(3.14159, digits=2) ## Returns 3.14
f.math.round(3.14159, digits=0) ## Returns 3
f.math.round(3.14159, digits=-1) ## Returns 0
f.math.round(155, digits=-2)   ## Returns 200
```

E-Series Rounding Function

Rounds a value to the nearest standard E-series value. The E-series are logarithmically spaced value series used for resistors, capacitors, and inductors.

Morphology:

EBNF:

```
"f.math.eseries", "(", ( int | real ), ",", string, ["", "rm", "=", string], ")"
```

- Operands: value (`int` or `real`), series name (`string`: "E3", "E6", "E12", "E24", "E48", "E96", or "E192", case-insensitive), optional `rm` (`string`: "nearest" (default), "floor", or "ceil").
- Result: `real` in the selected series, scaled to the correct decade.
- Zero returns 0 directly (0 Ω resistors are valid components).
- A `Type Error` is raised if the first argument is not `int` or `real`.
- A `Type Error` is raised if the second argument is not a `string`.
- A `Value Error` is raised if the value is negative.
- A `Value Error` is raised if the series name is not one of the valid E-series.
- A `Value Error` is raised if the rounding mode is not one of "nearest", "floor", or "ceil".

Examples:

```
f.math.eseries(330, "E24")      ## Returns 330
f.math.eseries(350, "E24")      ## Returns 330
f.math.eseries(350, "E12")      ## Returns 330
f.math.eseries(350, "E6")       ## Returns 330
```

```

f.math.eseries(390, "E24")           #c# Returns 390
f.math.eseries(4.7e3, "E12")         #c# Returns 4700
f.math.eseries(1.5e3, "E3")          #c# Returns 2200
f.math.eseries(330, "e24")           #c# Returns 330 (case-insensitive)
f.math.eseries(0, "E24")             #c# Returns 0
f.math.eseries(330, "E5")            #c# Value Error
f.math.eseries(-100, "E24")          #c# Value Error
f.math.eseries("abc", "E24")         #c# Type Error
f.math.eseries(330, "E24", rm="floor") #c# Returns 330
f.math.eseries(350, "E24", rm="ceil") #c# Returns 390

```

E-Series Reference All values are base values within one decade (1.0 to 10). Multiply by 10^n for other decades. Every value in $E(n)$ is also a member of $E(2n)$.

E3 (20%): 1.0, 2.2, 4.7

E6 (20%): 1.0, 1.5, 2.2, 3.3, 4.7, 6.8

E12 (10%): 1.0, 1.2, 1.5, 1.8, 2.2, 2.7, 3.3, 3.9, 4.7, 5.6, 6.8, 8.2

E24 (5%): 1.0, 1.1, 1.2, 1.3, 1.5, 1.6, 1.8, 2.0, 2.2, 2.4, 2.7, 3.0, 3.3, 3.6, 3.9, 4.3, 4.7, 5.1, 5.6, 6.2, 6.8, 7.5, 8.2, 9.1

E48 (2%): 1.00, 1.05, 1.10, 1.15, 1.21, 1.27, 1.33, 1.40, 1.47, 1.54, 1.62, 1.69, 1.78, 1.87, 1.96, 2.05, 2.15, 2.26, 2.37, 2.49, 2.61, 2.74, 2.87, 3.01, 3.16, 3.32, 3.48, 3.65, 3.83, 4.02, 4.22, 4.42, 4.64, 4.87, 5.11, 5.36, 5.62, 5.90, 6.19, 6.49, 6.81, 7.15, 7.50, 7.87, 8.25, 8.66, 9.09, 9.53

E96 (1%): 1.00, 1.02, 1.05, 1.07, 1.10, 1.13, 1.15, 1.18, 1.21, 1.24, 1.27, 1.30, 1.33, 1.37, 1.40, 1.43, 1.47, 1.50, 1.54, 1.58, 1.62, 1.65, 1.69, 1.74, 1.78, 1.82, 1.87, 1.91, 1.96, 2.00, 2.05, 2.10, 2.15, 2.21, 2.26, 2.32, 2.37, 2.43, 2.49, 2.55, 2.61, 2.67, 2.74, 2.80, 2.87, 2.94, 3.01, 3.09, 3.16, 3.24, 3.32, 3.40, 3.48, 3.57, 3.65, 3.74, 3.83, 3.92, 4.02, 4.12, 4.22, 4.32, 4.42, 4.53, 4.64, 4.75, 4.87, 4.99, 5.11, 5.23, 5.36, 5.49, 5.62, 5.76, 5.90, 6.04, 6.19, 6.34, 6.49, 6.65, 6.81, 6.98, 7.15, 7.32, 7.50, 7.68, 7.87, 8.06, 8.25, 8.45, 8.66, 8.87, 9.09, 9.31, 9.53, 9.76

E192 (0.1%): 1.00, 1.01, 1.02, 1.04, 1.05, 1.06, 1.07, 1.09, 1.10, 1.11, 1.13, 1.14, 1.15, 1.17, 1.18, 1.20, 1.21, 1.23, 1.24, 1.26, 1.27, 1.29, 1.30, 1.32, 1.33, 1.35, 1.37, 1.38, 1.40, 1.42, 1.43, 1.45, 1.47, 1.49, 1.50, 1.52, 1.54, 1.56, 1.58, 1.60, 1.62, 1.64, 1.65, 1.67, 1.69, 1.72, 1.74, 1.76, 1.78, 1.80, 1.82, 1.84, 1.87, 1.89, 1.91, 1.93, 1.96, 1.98, 2.00, 2.03, 2.05, 2.08, 2.10, 2.13, 2.15, 2.18, 2.21, 2.23, 2.26, 2.29, 2.32, 2.34, 2.37, 2.40, 2.43, 2.46, 2.49, 2.52, 2.55, 2.58, 2.61, 2.64, 2.67, 2.71, 2.74, 2.77, 2.80, 2.84, 2.87, 2.91, 2.94, 2.98, 3.01, 3.05, 3.09, 3.12, 3.16, 3.20, 3.24, 3.28, 3.32, 3.36, 3.40, 3.44, 3.48, 3.52, 3.57, 3.61, 3.65, 3.70, 3.74, 3.79, 3.83, 3.88, 3.92, 3.97, 4.02, 4.07, 4.12, 4.17, 4.22, 4.27, 4.32, 4.37, 4.42, 4.48, 4.53, 4.59, 4.64, 4.70, 4.75, 4.81, 4.87, 4.93, 4.99, 5.05, 5.11, 5.17, 5.23, 5.30, 5.36, 5.42, 5.49, 5.56, 5.62, 5.69, 5.76, 5.83, 5.90, 5.97, 6.04, 6.12, 6.19, 6.26, 6.34, 6.42, 6.49, 6.57, 6.65, 6.73, 6.81, 6.90, 6.98, 7.06, 7.15, 7.23, 7.32, 7.41, 7.50, 7.59, 7.68, 7.77, 7.87, 7.96, 8.06, 8.16, 8.25, 8.35, 8.45, 8.56, 8.66, 8.76, 8.87, 8.98, 9.09, 9.20, 9.31, 9.42, 9.53, 9.65, 9.76, 9.88

Minimum Function

Returns the minimum value in a list. For tensors, an optional **axis** parameter reduces along that axis.

Morphology:

EBNF:

"f.math.min", "(", list, [",", "axis", "=", int], ")"

- Operands: **list** of comparable elements, or **tensor**. Optional **axis**: **int** (tensor only).
- Result: the smallest element in the list/array.
- For tensors with **axis**, reduces along the specified axis, returning an tensor.
- A **Value Error** is raised if the list is empty.
- A **Type Error** is raised if the argument is not a list or tensor.
- A **Type Error** is raised if elements are not comparable (e.g., mixing **int** and **string**).

Examples:

```
f.math.min([3, 1, 4, 1, 5])           ## Returns 1
f.math.min([[3,1],[4,2]], axis=0)      ## Returns [3, 1]
f.math.min([[3,1],[4,2]], axis=1)      ## Returns [1, 2]
```

Maximum Function

Returns the maximum value in a list. For tensors, an optional **axis** parameter reduces along that axis.

Morphology:

EBNF:

"f.math.max", "(", list, [",", "axis", "=", int], ")"

- Operands: **list** of comparable elements, or **tensor**. Optional **axis**: **int** (tensor only).
- Result: the largest element in the list/array.
- For tensors with **axis**, reduces along the specified axis, returning an tensor.
- A **Value Error** is raised if the list is empty.
- A **Type Error** is raised if the argument is not a list or tensor.
- A **Type Error** is raised if elements are not comparable.

Examples:

```
f.math.max([3, 1, 4, 1, 5])           ## Returns 5
f.math.max([[3,1],[4,2]], axis=0)      ## Returns [4, 2]
f.math.max([[3,1],[4,2]], axis=1)      ## Returns [3, 4]
```

Inverse Tangent Function based on a Vector Argument

The **atan2** function computes the arctangent of y/x using the signs of both arguments to determine the correct quadrant.

Morphology:

EBNF:

"f.trig.atan2", "(", argument, ",", argument, ")"

Edge Cases

- Operands: both arguments must be `int` or `real`.
- `atan2(0, 0)` returns `0.0`.
- A `Type Error` is raised if either argument is not `int` or `real`.

Example:

```
f.atan2(1, 0)    ## Returns approximately 1.5708 (pi/2)
f.atan2(0, 1)    ## Returns 0.0
```

Norm Function

Computes the p-norm of a vector or matrix.

Morphology:

EBNF:

```
"f.tnsr.norm", "(", ( list | list[list] ), [",", "p", "=", ( real | "fro" ) ], ")"
```

- Operands: `list` of `int/real` (vector), `list[list]` of `int/real` (matrix), optional `p` (`int`, `real`, or `"fro"`, default `2`).
- Result: `real`.

Vector norms:

- `p = 0` – count of non-zero elements.
- `p = 1` – L1/Manhattan norm.
- `p = 2` – L2/Euclidean norm (default).
- Any real $p \geq 2$ – Lp norm.
- `p = -1` – L-infinity/Chebyshev norm (max absolute value).
- A `Value Error` is raised for $p < -1$ or $-1 < p < 0$.

Matrix norms:

- `p = 1` – maximum absolute column sum.
- `p = -1` – minimum absolute column sum.
- `p = 2` (default) – spectral norm (largest singular value).
- `p = -2` – smallest singular value.
- `p = "fro"` – Frobenius norm ($\sqrt{\sum_{i,j} a_{ij}^2}$).
- A `Value Error` is raised for unsupported matrix `p` values.
- A `Type Error` is raised if the argument is not a list of numeric values or a list of lists of numeric values.

Examples:

```
# Vector norms
f.tnsr.norm([3, 4])           #c# Returns 5.0 (L2 norm)
f.tnsr.norm([3, 4], p=1)      #c# Returns 7 (L1 norm)
f.tnsr.norm([3, 4], p=0)      #c# Returns 2 (L0 norm)
f.tnsr.norm([-3, 4, -2], p=-1) #c# Returns 4 (L-inf norm)

# Matrix norms
f.tnsr.norm([[1,2],[3,4]])     #c# Returns approximately 5.4649 (spectral norm)
f.tnsr.norm([[1,2],[3,4]], p="fro") #c# Returns approximately 5.4772 (Frobenius)
f.tnsr.norm([[1,2],[3,4]], p=1) #c# Returns 7 (max column sum: |3|+|1|=4, |4|+|2|=6)
f.tnsr.norm([[1,2],[3,4]], p=-1) #c# Returns 3 (min column sum)
f.tnsr.norm([[1,2],[3,4]], p=-2) #c# Returns approximately 0.1010 (smallest singular value)
```

Normalize Function

Returns a unit vector in the direction of the input vector: $\hat{\mathbf{x}} = \mathbf{x} / \|\mathbf{x}\|_p$.

Morphology:

EBNF:

```
"f.tnsr.normalize", "(", list, ["", "p", "=", real], ")"
```

- Operands: list of int or real values, optional p (int or real, default 2).
- Result: list of real values.
- Uses the same norm as `f.tnsr.norm` for the denominator.
- A Division by Zero Error is raised if the norm of the vector is zero.
- A Type Error is raised if the argument is not a list of numeric values.

Examples:

```
f.tnsr.normalize([3, 4])       #c# Returns [0.6, 0.8] (L2)
f.tnsr.normalize([3, 4], p=1)  #c# Returns [0.43, 0.57] (L1)
f.tnsr.normalize([6, 8], p=2)  #c# Returns [0.6, 0.8] (L2)
```

Relationship between `abs(vec=True)` and `norm` The function `abs(v, vec=True)` is equivalent to `norm(v)` (L2 norm by default).

Clamp

The `clamp` function limits a value to a specified range.

Morphology:

EBNF:

```
"f.math.clamp", "(", ( int | real | list | tensor ), ",", ( int | real ), ",", ( int | r
```

`f.math.clamp(x, min, max)` returns min if $x < \text{min}$, max if $x > \text{max}$, otherwise x . Operates element-wise on lists and tensors.

Edge Cases

- A **Value Error** is raised if `min > max`.
- A **Type Error** is raised if all three arguments are not the same numeric type (`int`, `real`).

Examples:

```
f.math.clamp(5, 0, 10)           ## Returns 5
f.math.clamp(-3, 0, 10)          ## Returns 0
f.math.clamp(15, 0, 10)          ## Returns 10
f.math.clamp([-2, 5, 12], 0, 10) ## Returns [0, 5, 10]
f.math.clamp(|[-2, 5, 12], 0, 10) ## Returns |[0, 5, 10]
f.math.clamp(|[[-3, 7],[15, 2]], 0, 10) ## Returns |[ [0, 7],[10, 2]]
```

Linear Interpolation

The `lerp` function performs linear interpolation between two values.

Morphology:

EBNF:

```
"f.math.lerp", "(", ( int | real ), ",", ( int | real ), ",", real, [",", "clamp", "=",
```

- Operands: `a` (start value), `b` (end value), `t` (interpolation factor), optional `clamp` (`bool`, default `True`).
- Result: `real` (always `real`, even when `a` and `b` are `int`).
- Computes $a + (b - a) \cdot t$.
- When `clamp = True` (default), `t` is clamped to $[0, 1]$ and the result is always between `a` and `b`.
- When `clamp = False`, extrapolation is allowed for any `t`.
- A **Type Error** is raised if arguments are not numeric or `clamp` is not `bool`.

Examples:

```
f.math.lerp(0, 10, 0.5)           ## Returns 5.0
f.math.lerp(0, 10, 0.0)           ## Returns 0.0
f.math.lerp(0, 10, 1.0)           ## Returns 10.0
f.math.lerp(0, 10, 1.5)           ## Returns 10.0 (clamped by default)
f.math.lerp(0, 10, 1.5, clamp=False) ## Returns 15.0 (extrapolated)
f.math.lerp(0, 10, -0.5, clamp=False) ## Returns -5.0 (extrapolated)
```

Is Finite Function

The `f.math.isFinite` function checks whether a value is finite (not NaN and not $\pm\infty$).

Morphology:

EBNF:

```
"f.math.isFinite", "(", ( int | real | complex ), ")"
```

- Result: `bool` — `True` if the value is finite, `False` otherwise.
- `int` values are always finite.

Examples:

```
f.math.isFinite(42)      #c# True
f.math.isFinite(3.14)    #c# True
f.math.isFinite(0.0 / 0.0) #c# False (NaN)
f.math.isFinite(1.0 / 0.0) #c# False (inf)
```

Is NaN Function

The `f.math.isNaN` function checks whether a value is NaN (Not a Number).
Morphology:

EBNF:

```
"f.math.isNaN", "(", ( int | real | complex ), ")"
```

- Result: `bool` — `True` if the value is NaN, `False` otherwise.
- `int` values are never NaN.

Examples:

```
f.math.isNaN(42)      #c# False
f.math.isNaN(0.0 / 0.0) #c# True
f.math.isNaN(3.14)    #c# False
```

Unit in the Last Place Function

The `f.math.ulp` function returns the unit in the last place of a floating-point number — the gap between `x` and its next representable neighbor. Useful for numerical precision analysis.
Morphology:

EBNF:

```
"f.math.ulp", "(", real, ")"
```

- Operands: `real`.
- Result: `real`.
- A `Value Error` is raised if the argument is not finite.

Examples:

```
f.math.ulp(1.0)      #c# 2.220446049250313e-16 (for f64)
f.math.ulp(1000.0)   #c# 1.1368683772161603e-13
f.math.ulp(0.0)      #c# 5e-324 (smallest subnormal)
```

Machine Epsilon Function

The `f.math.epsilon` function returns the machine epsilon — the smallest positive value ϵ such that $1.0 + \epsilon \neq 1.0$.

Morphology:

EBNF:

```
"f.math.epsilon", "(", ")"
```

- Result: `real` — approximately 2.22×10^{-16} for IEEE 754 f64.

Examples:

```
$f eps = f.math.epsilon()
#c# 2.220446049250313e-16
1.0 + $eps / 2 == 1.0    #c# True (below precision threshold)
1.0 + $eps == 1.0        #c# False (above precision threshold)
```

Truncate

The `trunc` function truncates the decimal part of a number, returning the integer part toward zero.

Morphology:

EBNF:

```
"f.math.trunc", "(", ( int | real ), ")"
```

- Operands: `int` (returned unchanged), `real`, `complex`
- Result: `int`
- For complex numbers, truncates real and imaginary parts separately and returns complex.

Examples:

```
f.math.trunc(3.7)          #c# Returns 3
f.math.trunc(-3.7)         #c# Returns -3
f.math.trunc(3.2 + 4.8i)   #c# Returns 3 + 4i
```

Cube Root

The `cbrrt` function returns the real cube root of a number.

Morphology:

EBNF:

```
"f.math.cbrrt", "(", ( int | real ), ")"
```

- Operands: `int`, `real`
- Result: `real`
- Returns the real cube root (negative inputs yield negative results, unlike $\wedge$).

Examples:

```
f.math.cbrt(27)          #c# Returns 3.0
f.math.cbrt(-8)          #c# Returns -2.0
f.math.cbrt(0)           #c# Returns 0.0
```

Nth Root

The `nthrt` function returns the n -th root of a value, computed as $x^{1/n}$. Supports negative and complex n (reciprocal and complex roots), though **integer** $n \geq 1$ **is encouraged**.

Morphology:

EBNF:

```
"f.math.nthrt", "(", ( int | real | complex ), ",", ( int | real | complex ), ")"
```

- Operands: `int`, `real`, or `complex` for both n and x .
- Result: `complex` if either operand is `complex`, otherwise `real`.
- Positive n : returns $x^{1/n}$. For odd integer n with negative real x , returns the negative real root (like `cbrt`). For even n with negative x , returns `complex`.
- Negative n : returns $x^{1/n} = 1/x^{1/|n|}$ (reciprocal root).
- $n = 0$: raises a `Division by Zero Error`.
- $x = 0$, $n > 0$: returns `0.0`.
- $x = 0$, $n \leq 0$: raises a `Division by Zero Error`.
- A `Type Error` is raised if either argument is not numeric.

Examples:

```
f.math.nthrt(2, 9)       #c# Returns 3.0 (square root)
f.math.nthrt(3, 27)      #c# Returns 3.0 (cube root)
f.math.nthrt(4, 16)      #c# Returns 2.0 (fourth root)
f.math.nthrt(-2, 9)      #c# Returns 0.333... (reciprocal: 1/sqrt(9))
f.math.nthrt(3, -8)      #c# Returns -2.0 (real odd root)
f.math.nthrt(2, -4)      #c# Returns 0 + 2i (complex even root)
f.math.nthrt(1, 42)      #c# Returns 42.0 (first root)
f.math.nthrt(0.5, 4)     #c# Returns 16.0 (real n)
f.math.nthrt(2+0i, 9)    #c# Returns 3 + 0i (complex n)
```

Base-2 Logarithm

The `log2` function returns the base-2 logarithm.

Morphology:

EBNF:

```
"f.log.log2", "(", ( int | real ), ")"
```

- Operands: `int` (must be > 0), `real` (must be > 0)

- Result: **real**
- A Value Error is raised for arguments ≤ 0 .

Examples:

```
f.log.log2(8)           #c# Returns 3.0
f.log.log2(1)           #c# Returns 0.0
f.log.log2(1024)        #c# Returns 10.0
```

General Logarithm

Returns the logarithm of the argument to an arbitrary base, computed as $\log_b(x) = \frac{\ln(x)}{\ln(b)}$.
Morphology:

EBNF:

```
"f.log.log", "(", ( int | real ), ",", ( int | real | complex ), ")"
```

- **base**: **int** or **real** (must be > 0 and $\neq 1$).
- **x**: **int** (must be > 0), **real** (must be > 0), **complex** ($\neq 0$).
- Result: **real** for positive **int**/**real** input, **complex** for **complex** input.
- A Value Error is raised if **base** ≤ 0 or **base** = 1.
- A Value Error is raised for non-positive real arguments or zero complex argument.

Examples:

```
f.log.log(10, 100)      #c# Returns 2.0
f.log.log(2, 8)         #c# Returns 3.0
f.log.log(c.math.e, 1)   #c# Returns 0.0
f.log.log(10, 1 + 1i)    #c# Returns approximately 0.6505 + 0.3411i
```

Hypotenuse

The **hypot** function computes $\sqrt{a^2 + b^2}$, the Euclidean distance / hypotenuse.
Morphology:

EBNF:

```
"f.math.hypot", "(", ( int | real ), ",", ( int | real ), ")"
```

- Operands: two numeric values (**int** or **real**)
- Result: **real**
- Avoids intermediate overflow/underflow for large/small values.

Examples:

```
f.math.hypot(3, 4)      #c# Returns 5.0
f.math.hypot(5, 12)     #c# Returns 13.0
f.math.hypot(0, 1)      #c# Returns 1.0
```

Sinc Function

The `sinc` function computes the normalized sinc function $\text{sinc}(x) = \frac{\sin(\pi x)}{\pi x}$ for $x \neq 0$, and 1 for $x = 0$.

Morphology:

EBNF:

```
"f.math.sinc", "(", ( int | real ), ")"
```

- Operands: `int` or `real`
- Result: `real`
- $\text{sinc}(0) = 1.0$. Zeros at all non-zero integers.

Examples:

```
f.math.sinc(0)           #c# Returns 1.0
f.math.sinc(0.5)         #c# Returns approximately 0.6366
f.math.sinc(1)           #c# Returns 0.0
f.math.sinc(2)           #c# Returns 0.0
```

Tanc Function

The `tanc` function computes $\text{tanc}(x) = \frac{\tan(x)}{x}$ for $x \neq 0$, and 1 for $x = 0$.

Morphology:

EBNF:

```
"f.math.tanc", "(", ( int | real ), ")"
```

- Operands: `int` or `real`
- Result: `real`
- $\text{tanc}(0) = 1.0$. Raises a Division by Zero Error at $x = \pm\pi/2$ (where $\tan$ is undefined).

Examples:

```
f.math.tanc(0)           #c# Returns 1.0
f.math.tanc(0.5)         #c# Returns approximately 1.0450
f.math.tanc(1)           #c# Returns approximately 1.5574
```

Factorial

The `factorial` function returns $n! = n \times (n - 1) \times \cdots \times 1$.

Morphology:

EBNF:

```
"f.math.factorial", "(", int, ")"
```

- Operands: `int` (must be ≥ 0)
- Result: `int`

- $0! = 1$. A Value Error is raised for negative arguments.

Examples:

```
f.math.factorial(0)      #c# Returns 1
f.math.factorial(5)      #c# Returns 120
f.math.factorial(10)     #c# Returns 3628800
```

Greatest Common Divisor

Computes the greatest common divisor of two integers.

Morphology:

EBNF:

```
"f.math.gcd", "(", int, ",", int, ")"
```

- Operands: two `int` values.
- Result: `int` (≥ 0).
- $\gcd(0, 0) = 0$. $\gcd(a, 0) = |a|$. $\gcd(a, b) = \gcd(b, a \bmod b)$.

Examples:

```
f.math.gcd(12, 8)       #c# Returns 4
f.math.gcd(0, 5)        #c# Returns 5
f.math.gcd(7, 13)       #c# Returns 1
```

Least Common Multiple

Computes the least common multiple of two integers.

Morphology:

EBNF:

```
"f.math.lcm", "(", int, ",", int, ")"
```

- Operands: two `int` values.
- Result: `int` (≥ 0).
- $\text{lcm}(0, 0) = 0$. $\text{lcm}(a, 0) = 0$. $\text{lcm}(a, b) = |a \cdot b| / \gcd(a, b)$.

Examples:

```
f.math.lcm(4, 6)        #c# Returns 12
f.math.lcm(0, 5)        #c# Returns 0
f.math.lcm(7, 13)       #c# Returns 91
```


Double Factorial

Computes the double factorial $n!! = n \times (n - 2) \times (n - 4) \times \dots$.

Morphology:

EBNF:

```
"f.math.doubleFactorial", "(", int, ")"
```

- Operand: `int` (≥ 0).
- Result: `int`.
- $0!! = 1$, $1!! = 1$, $5!! = 15$, $6!! = 48$.
- A `Value Error` is raised for negative arguments.

Examples:

```
f.math.doubleFactorial(0)    ## Returns 1
f.math.doubleFactorial(5)    ## Returns 15  (5 * 3 * 1)
f.math.doubleFactorial(6)    ## Returns 48  (6 * 4 * 2)
```

Falling Factorial

Computes the falling factorial $n^{\underline{k}} = n \times (n - 1) \times \dots \times (n - k + 1)$.

Morphology:

EBNF:

```
"f.math.fallingFactorial", "(", int, ",", int, ")"
```

- Operands: n (`int`), k (`int` ≥ 0).
- Result: `int`.
- $n^0 = 1$. $n^1 = n$. $n^n = n!$.
- A `Value Error` is raised if $k < 0$.

Examples:

```
f.math.fallingFactorial(5, 3)  ## Returns 60  (5 * 4 * 3)
f.math.fallingFactorial(5, 0)  ## Returns 1
```

Rising Factorial

Computes the rising factorial (Pochhammer symbol) $n^{\overline{k}} = n \times (n + 1) \times \dots \times (n + k - 1)$.

Morphology:

EBNF:

```
"f.math.risingFactorial", "(", int, ",", int, ")"
```

- Operands: n (`int`), k (`int` ≥ 0).
- Result: `int`.

- $n^{\bar{0}} = 1$. $n^{\bar{1}} = n$. $n^{\bar{k}} = (n + k - 1)! / (n - 1)!$.
- A Value Error is raised if $k < 0$.

Examples:

```
f.math.risingFactorial(3, 4)    #c# Returns 360  (3 * 4 * 5 * 6)
f.math.risingFactorial(1, 5)    #c# Returns 120  (1 * 2 * 3 * 4 * 5)
```

Binomial Coefficient

Computes $\binom{n}{k} = \frac{n!}{k!(n-k)!}$. Same as `f.math.comb`.

Morphology:

EBNF:

```
"f.math.binomial", "(", int, ",", int, ")"
```

- Operands: n (`int` ≥ 0), k (`int` ≥ 0).
- Result: `int`.
- $\binom{n}{0} = 1$, $\binom{n}{1} = n$, $\binom{n}{n} = 1$.
- A Value Error is raised if $k > n$ or $n < 0$ or $k < 0$.

Examples:

```
f.math.binomial(5, 3)    #c# Returns 10
f.math.binomial(10, 2)   #c# Returns 45
```

Multinomial Coefficient

Computes $\binom{n}{k_1, k_2, \dots, k_m} = \frac{n!}{k_1! k_2! \dots k_m!}$ where $n = \sum k_i$.

Morphology:

EBNF:

```
"f.math.multinomial", "(", list, ")"
```

- Operand: `list[int]` (≥ 0).
- Result: `int`.
- The sum of all elements is n .
- A Value Error is raised if any element is negative.

Examples:

```
f.math.multinomial([1, 2, 3])    #c# Returns 60  (6! / (1!*2!*3!))
f.math.multinomial([2, 2])       #c# Returns 6   (4! / (2!*2!))
f.math.multinomial([5])          #c# Returns 1
```

Modular Inverse

Computes the modular multiplicative inverse x such that $a \cdot x \equiv 1 \pmod{m}$.

Morphology:

EBNF:

```
"f.math.modInverse", "(", int, ",", int, ")"
```

- Operands: a (int), m (int, > 0).
- Result: int in range $[0, m)$.
- A Value Error is raised if $\gcd(a, m) \neq 1$ (no inverse exists) or $m \leq 0$.

Examples:

```
f.math.modInverse(3, 7)      ## Returns 5   (3*5 = 15 = 1 mod 7)
f.math.modInverse(10, 17)   ## Returns 12  (10*12 = 120 = 1 mod 17)
```

Divides

Tests whether a divides b evenly ($a \mid b$).

Morphology:

EBNF:

```
"f.math.divides", "(", int, ",", int, ")"
```

- Operands: a (int), b (int).
- Result: bool. Returns True if $b \bmod a = 0$.
- A Value Error is raised if $a = 0$.

Examples:

```
f.math.divides(3, 9)        ## Returns True
f.math.divides(3, 10)       ## Returns False
f.math.divides(1, 7)        ## Returns True (1 divides everything)
```

Permutations

Computes the number of ordered selections: $P(n, k) = \frac{n!}{(n-k)!}$.

Morphology:

EBNF:

```
"f.math.perm", "(", int, ",", int, ")"
```

- Operands: n (int ≥ 0), k (int ≥ 0).
- Result: int.
- $P(n, 0) = 1$, $P(n, 1) = n$.
- A Value Error is raised if $k > n$ or $n < 0$ or $k < 0$.

Examples:

```
f.math.perm(5, 3)          ## Returns 60
f.math.perm(10, 2)         ## Returns 90
f.math.perm(5, 0)          ## Returns 1
```

Combinations

Computes the number of unordered selections: $C(n, k) = \frac{n!}{k!(n-k)!}$.

Morphology:

EBNF:

```
"f.math.comb", "(", int, ",", int, ")"
```

- Operands: n (`int` ≥ 0), k (`int` ≥ 0).
- Result: `int`.
- $C(n, 0) = 1$, $C(n, 1) = n$, $C(n, n) = 1$.
- A **Value Error** is raised if $k > n$ or $n < 0$ or $k < 0$.

Examples:

```
f.math.comb(5, 3)    #c# Returns 10
f.math.comb(10, 2)   #c# Returns 45
f.math.comb(5, 5)    #c# Returns 1
```

Partitions

Computes the number of integer partitions of n , i.e. the number of ways to write n as a sum of positive integers (order does not matter).

Morphology:

EBNF:

```
"f.math.partitions", "(", int, ")"
```

- Operand: n (`int` ≥ 0).
- Result: `int`.
- $p(0) = 1$, $p(1) = 1$, $p(5) = 7$.
- A **Value Error** is raised if $n < 0$.

Examples:

```
f.math.partitions(0)  #c# Returns 1
f.math.partitions(5)  #c# Returns 7  (5, 4+1, 3+2, 3+1+1, 2+2+1, 2+1+1+1, 1+1+1+1+1)
f.math.partitions(10) #c# Returns 42
```

Compositions

Computes the number of compositions of n , i.e. the number of ways to write n as an ordered sum of positive integers. Equal to 2^{n-1} for $n \geq 1$.

Morphology:

EBNF:

```
"f.math.compositions", "(", int, ")"
```

- Operand: n (`int` ≥ 0).
- Result: `int`.
- $c(0) = 1$, $c(1) = 1$, $c(n) = 2^{n-1}$ for $n \geq 1$.
- A Value Error is raised if $n < 0$.

Examples:

```
f.math.compositions(0)    #c# Returns 1
f.math.compositions(1)    #c# Returns 1
f.math.compositions(4)    #c# Returns 8  (2^3)
```

Gray Code

Computes the n -th value in the binary reflected Gray code sequence.
Morphology:

EBNF:

```
"f.math.grayCode", "(", int, ")"
```

- Operand: n (`int` ≥ 0).
- Result: `int`. Formula: $G(n) = n \oplus \lfloor n/2 \rfloor$.
- Adjacent Gray code values differ by exactly one bit.
- A Value Error is raised if $n < 0$.

Examples:

```
f.math.grayCode(0)       #c# Returns 0
f.math.grayCode(1)       #c# Returns 1
f.math.grayCode(2)       #c# Returns 3
f.math.grayCode(3)       #c# Returns 2
f.math.grayCode(4)       #c# Returns 6
```

Derangements

Computes the number of derangements $!n$ (permutations with no fixed points).
Morphology:

EBNF:

```
"f.math.derangements", "(", int, ")"
```

- Operand: n (`int` ≥ 0).
- Result: `int`.
- $!0 = 1$, $!1 = 0$, $!2 = 1$, $!3 = 2$, $!4 = 9$.
- A Value Error is raised if $n < 0$.

Examples:

```
f.math.derangements(0)    #c# Returns 1
f.math.derangements(1)    #c# Returns 0
f.math.derangements(4)    #c# Returns 9
f.math.derangements(10)   #c# Returns 1334961
```

Gamma Function

The `gamma` function returns $\Gamma(x) = (x - 1)!$ for positive integers, extended to real and complex numbers.

Morphology:

EBNF:

```
"f.special.gamma", "(", ( int | real ), ")"
```

- Operands: int, real, complex
- Result: real or complex
- $\Gamma(1) = 1$, $\Gamma(0.5) = \sqrt{\pi}$. Poles at non-positive integers raise a Value Error.

Examples:

```
f.special.gamma(1)          #c# Returns 1.0
f.special.gamma(0.5)        #c# Returns approximately 1.77245
f.special.gamma(5)          #c# Returns 24.0 (= 4!)
```

Log-Gamma Function

The `lgamma` function returns $\ln|\Gamma(x)|$, the natural logarithm of the absolute value of the gamma function. Useful for large factorials without overflow.

Morphology:

EBNF:

```
"f.special.lgamma", "(", ( int | real ), ")"
```

- Operands: int, real
- Result: real
- Always returns a non-negative value.

Examples:

```
f.special.lgamma(1)          #c# Returns 0.0
f.special.lgamma(5)          #c# Returns approximately 3.17805 (= ln(24))
f.special.lgamma(10)         #c# Returns approximately 12.8018 (= ln(362880))
```

11.1.2 String Functions

String Conversion

The `str` function converts its arguments to a string.
Morphology:

EBNF:

```
"f.str.str", "(", { argument }, ")"
```

Example:

```
f.str.str("Hello")          ## Returns "Hello"
f.str.str(42)                ## Returns "42"
f.str.str("Value: ", 3.14)   ## Returns "Value: 3.14"
f.str.str()                  ## Returns "" (empty string)
```

With zero arguments, `f.str.str()` returns an empty string.

String Formatting

The `format` function formats a value using printf-style format specifiers.
Morphology:

EBNF:

```
"f.str.format", "(", argument, ",", string_type, ")"
```

Format specifiers:

- `%d` – Integer decimal
- `%f` – Fixed-point real (default 6 decimal places)
- `%e` – Scientific notation
- `%g` – Shortest of `%f` or `%e`
- `%s` – String
- `%x` – Hexadecimal (integers)
- `%o` – Octal (integers)
- `%b` – Binary (integers)

Width and precision: `%<width>f, %<width>.<precision>f`

Edge Cases

- A **Type Error** is raised if the value type does not match the format specifier (e.g. `f.str.format("abc", "%d")`).

Examples:

<code>f.str.format(3.14159, "%.2f")</code>	<code>#c# Returns "3.14"</code>
<code>f.str.format(42, "%d")</code>	<code>#c# Returns "42"</code>
<code>f.str.format(42, "%5d")</code>	<code>#c# Returns " 42"</code>
<code>f.str.format(0xff, "%x")</code>	<code>#c# Returns "ff"</code>
<code>f.str.format(3.14159, "%4.1f")</code>	<code>#c# Returns " 3.1"</code>
<code>f.str.format(1234567, "%e")</code>	<code>#c# Returns "1.234567e+06"</code>

Uppercase Function

Converts a string to uppercase.

Morphology:

EBNF:

`"f.str.upperc", "(", string, ")"`

- Operands: `string`.
- Result: `string`.
- Empty string returns empty string.
- A `Type Error` is raised if the argument is not a `string`.

Examples:

<code>f.str.upperc("hello")</code>	<code>#c# Returns "HELLO"</code>
<code>f.str.upperc("")</code>	<code>#c# Returns ""</code>
<code>f.str.upperc("abc123")</code>	<code>#c# Returns "ABC123"</code>

Lowercase Function

Converts a string to lowercase.

Morphology:

EBNF:

`"f.str.lowerc", "(", string, ")"`

- Operands: `string`.
- Result: `string`.
- Empty string returns empty string.
- A `Type Error` is raised if the argument is not a `string`.

Examples:

<code>f.str.lowerc("HELLO")</code>	<code>#c# Returns "hello"</code>
<code>f.str.lowerc("")</code>	<code>#c# Returns ""</code>
<code>f.str.lowerc("ABC123")</code>	<code>#c# Returns "abc123"</code>

String Character Access

Returns a single character at the given index. Strings support index and slice operations, similar to lists.

Syntax:

`str[i]` Returns the character at index `i`

- Index is zero-based.
- Negative indices count from the end: `str[-1]` is the last character.
- `str[-0]` equals `str[0]` (negative zero equals zero).
- A `Value Error` is raised if the index is out of bounds.

Examples:

```
"hello"[0]      #c# Returns "h"
"hello"[-1]     #c# Returns "o"
```

String Slice

Returns a substring using start:end:step slicing.

Syntax:

```
str[start]      Single character access
str[start:end]  Returns str[start] through str[end-1]
str[start:end:step] Returns every step-th character from start to end
```

- `start` defaults to 0, `end` defaults to string length, `step` defaults to 1.
- Negative `step` reverses the direction.
- Out-of-range indices are clamped (not errors).

Examples:

```
"hello"[1:4]      #c# Returns "ell"
"hello"[0:5:2]    #c# Returns "hlo"
"hello"[::-1]     #c# Returns "olleh"
```

String Length Function

Returns the number of characters in a string.

Morphology:

EBNF:

```
"f.str.length", "(", string, ")"
```

- Operands: `string`.
- Result: `int`.
- `f.str.length("")` returns 0.

- A **Type Error** is raised if the argument is not a **string**.

Examples:

```
f.str.length("")      #c# Returns 0
f.str.length("hello") #c# Returns 5
```

Unicode Code Point Function

Returns the Unicode code point of the character at the given index.

Morphology:

EBNF:

```
"f.str.charCodeAt", "(", string, ",", int, ")"
```

- Operands: **string**, **int** (index).
- Result: **int** (Unicode code point).
- A **Value Error** is raised if the index is out of bounds.
- A **Type Error** is raised if the first argument is not a **string** or the second is not an **int**.

Examples:

```
f.str.charCodeAt("A", 0)    #c# Returns 65
f.str.charCodeAt("hello", 4) #c# Returns 111
```

Contains Function

Returns **True** if the string contains the substring.

Morphology:

EBNF:

```
"f.str.contains", "(", string, ",", string, ")"
```

- Operands: **string**, **string** (substring).
- Result: **bool**.
- `f.str.contains("hello", "")` returns **True** (empty string is contained in any string).
- A **Type Error** is raised if either argument is not a **string**.

Examples:

```
f.str.contains("hello world", "world") #c# Returns True
f.str.contains("hello", "xyz")         #c# Returns False
f.str.contains("hello", "")            #c# Returns True
```

IndexOf Function

Returns the index of the first occurrence of the substring, or -1 if not found.
Morphology:

EBNF:

```
"f.str.indexOf", "(", string, ",", string, ")"
```

- Operands: `string`, `string` (substring).
- Result: `int`.
- Returns -1 if the substring is not found.
- A `Type Error` is raised if either argument is not a `string`.

Examples:

```
f.str.indexOf("hello", "l")      ## Returns 2
f.str.indexOf("hello", "xyz")    ## Returns -1
```

LastIndexOf Function

Returns the index of the last occurrence of the substring, or -1 if not found.
Morphology:

EBNF:

```
"f.str.lastIndexOf", "(", string, ",", string, ")"
```

- Operands: `string`, `string` (substring).
- Result: `int`.
- Returns -1 if the substring is not found.
- A `Type Error` is raised if either argument is not a `string`.

Examples:

```
f.str.lastIndexOf("hello", "l")  ## Returns 3
f.str.lastIndexOf("hello", "xyz") ## Returns -1
```

Count Function

Returns the number of non-overlapping occurrences of the substring.
Morphology:

EBNF:

```
"f.str.count", "(", string, ",", string, ")"
```

- Operands: `string`, `string` (substring).
- Result: `int`.
- Occurrences do not overlap: `f.util.count("aaa", "aa")` returns 1.

- A **Type Error** is raised if either argument is not a **string**.

Examples:

```
f.util.count("hello", "l")      ## Returns 2
f.util.count("aaa", "aa")      ## Returns 1 (non-overlapping)
```

StartsWith Function

Returns **True** if the string starts with the given prefix.

Morphology:

EBNF:

```
"f.str.startsWith", "(", string, ",", string, ")"
```

- Operands: **string**, **string** (prefix).
- Result: **bool**.
- A **Type Error** is raised if either argument is not a **string**.

Examples:

```
f.str.startsWith("hello", "hel")  ## Returns True
f.str.startsWith("hello", "xyz")  ## Returns False
```

EndsWith Function

Returns **True** if the string ends with the given suffix.

Morphology:

EBNF:

```
"f.str.endsWith", "(", string, ",", string, ")"
```

- Operands: **string**, **string** (suffix).
- Result: **bool**.
- A **Type Error** is raised if either argument is not a **string**.

Examples:

```
f.str.endsWith("hello", "llo")    ## Returns True
f.str.endsWith("hello", "xyz")    ## Returns False
```

Replace Function

Replaces the first occurrence of the search string with the replacement string.

Morphology:

EBNF:

```
"f.str.replace", "(", string, ",", string, ",", string, ")"
```

- Operands: **string** (haystack), **string** (search), **string** (replacement).

- Result: **string**.
- Only the first occurrence is replaced. Use **f.replaceAll** for all occurrences.
- A **Type Error** is raised if any argument is not a **string**.

Examples:

```
f.str.replace("hello world", "world", "there")  #c# Returns "hello there"
f.str.replace("aabbcc", "a", "x")              #c# Returns "xabbcc"
```

ReplaceAll Function

Replaces all occurrences of the search string with the replacement string.

Morphology:

EBNF:

```
"f.str.replaceAll", "(", string, ",", string, ",", string, ")"
```

- Operands: **string** (haystack), **string** (search), **string** (replacement).
- Result: **string**.
- A **Type Error** is raised if any argument is not a **string**.

Examples:

```
f.str.replaceAll("aabbcc", "a", "x")  #c# Returns "xxbbcc"
```

Insert Function

Inserts a string before the given index.

Morphology:

EBNF:

```
"f.str.insert", "(", string, ",", int, ",", string, ")"
```

- Operands: **string**, **int** (index), **string** (insertion).
- Result: **string**.
- If the index is less than 0, it is clamped to 0 (insert at beginning).
- If the index exceeds the string length, it is clamped to the string length (insert at end).
- A **Type Error** is raised if the first or third argument is not a **string** or the second is not an **int**.

Examples:

```
f.str.insert("ace", 1, "b")  #c# Returns "abce"
f.str.insert("abc", 0, "x")  #c# Returns "xabc"
f.str.insert("abc", 3, "x")  #c# Returns "abcx"
f.str.insert("abc", 99, "x") #c# Returns "abcx" (clamped to end)
```

Remove Function

Removes characters from a string by index range or by value.

Morphology:

EBNF:

```
"f.str.remove", "(", string, ",", data, ",", data, ["", "mode", "=", string], ")"
```

- Operands: `string`, second/third operands depend on mode, optional mode (`string`: `"index"` (default) or `"value"`).
- mode=`"index"`: `f.str.remove(str, start, end)` removes characters from index `start` up to (not including) `end`.
- mode=`"value"`: `f.str.remove(str, search, pos, mode="value")` removes the first occurrence of `search` starting at position `pos`.
- Out-of-bounds indices are clamped to the string length.
- A `Type Error` is raised if the first argument is not a `string`.

Examples:

```
f.str.remove("abcdef", 2, 4)           #c# Returns "abef"
f.str.remove("hello world", 5, 6)      #c# Returns "helloworld"
f.str.remove("aabbcc", "bb", 0, mode="value") #c# Returns "aacc"
```

ToUpperCase Function

Converts a string to uppercase. Identical in behavior to `f.str.upperc`.

Morphology:

EBNF:

```
"f.str.toUpperCase", "(", string, ")"
```

- Operands: `string`.
- Result: `string`.
- A `Type Error` is raised if the argument is not a `string`.

Examples:

```
f.str.toUpperCase("hello") #c# Returns "HELLO"
```

ToLowerCase Function

Converts a string to lowercase. Identical in behavior to `f.str.lowerc`.

Morphology:

EBNF:

```
"f.str.toLowerCase", "(", string, ")"
```

- Operands: `string`.

- Result: `string`.
- A `Type Error` is raised if the argument is not a `string`.

Examples:

```
f.str.toLowerCase("HELLO")  #c# Returns "hello"
```

Capitalize Function

Returns the string with the first character uppercased and the rest lowercased.

Morphology:

EBNF:

```
"f.str.capitalize", "(", string, ")"
```

- Operands: `string`.
- Result: `string`.
- Empty string returns empty string.
- A `Type Error` is raised if the argument is not a `string`.

Examples:

```
f.str.capitalize("hello")      #c# Returns "Hello"
f.str.capitalize("hELLO")      #c# Returns "Hello"
f.str.capitalize("")           #c# Returns ""
```

Title Case Function

Returns the string with the first character of each word uppercased and the rest lowercased.

Words are delimited by whitespace.

Morphology:

EBNF:

```
"f.str.titleCase", "(", string, ")"
```

- Operands: `string`.
- Result: `string`.
- A `Type Error` is raised if the argument is not a `string`.

Returns the string with the first character of each word uppercased.

```
f.str.titleCase("hello world")  #c# Returns "Hello World"
f.str.titleCase("foo bar baz")  #c# Returns "Foo Bar Baz"
```

Swap Case Function

Swaps the case of each character in the string (uppercase $\leftrightarrow$ lowercase).

Morphology:

EBNF:

```
"f.str.swapCase", "(", string, ")"
```

- Operands: `string`.
- Result: `string`.
- Non-alphabetic characters are unchanged.
- A `Type Error` is raised if the argument is not a `string`.

Examples:

```
f.str.swapCase("Hello World")    #c# Returns "hELLO wORLD"  
f.str.swapCase("ABC123")         #c# Returns "abc123"
```

Trim Function

Removes leading and/or trailing whitespace from a string.

Morphology:

EBNF:

```
"f.str.trim", "(", string, ")"
```

- Operands: `string`.
- Result: `string`.
- Removes spaces, tabs, and newlines from both ends.
- A `Type Error` is raised if the argument is not a `string`.

Examples:

```
f.str.trim("  hello  ")          #c# Returns "hello"
```

TrimStart Function

Removes leading whitespace from a string.

Morphology:

EBNF:

```
"f.str.trimStart", "(", string, ")"
```

- Operands: `string`.
- Result: `string`.
- Removes spaces, tabs, and newlines from the beginning.
- A `Type Error` is raised if the argument is not a `string`.

Examples:

```
f.trimStart("  hello  ") #c# Returns "hello  "
```


TrimEnd Function

Removes trailing whitespace from a string.

Morphology:

EBNF:

```
"f.str.trimEnd", "(", string, ")"
```

- Operands: `string`.
- Result: `string`.
- Removes spaces, tabs, and newlines from the end.
- A `Type Error` is raised if the argument is not a `string`.

Examples:

```
f.trimEnd(" hello ")  ## Returns " hello"
```

Split Function

Splits a string into a list of substrings. If no delimiter is given, splits on whitespace (one or more whitespace characters are treated as a single delimiter).

Morphology:

EBNF:

```
"f.str.split", "(", string, [",", string], ")"
```

- Operands: `string`, optional delimiter (`string`).
- Result: list of `string`.
- `f.str.split("")` returns `[""]` (a list containing one empty string).
- If the delimiter is not found, returns a list containing the entire string.
- A `Type Error` is raised if the argument is not a `string`.

Examples:

```
f.str.split("hello world")      ## Returns ["hello", "world"]
f.str.split("a,b,c", ",")       ## Returns ["a", "b", "c"]
f.str.split("one two three")    ## Returns ["one", "two", "three"]
```

SplitLines Function

Splits a string on newline boundaries.

Morphology:

EBNF:

```
"f.str.splitLines", "(", string, ")"
```

- Operands: `string`.

- Result: list of **string**.
- Handles both
n and
r
n line endings.
- A **Type Error** is raised if the argument is not a **string**.

Examples:

```
f.str.splitLines("line1\nline2\nline3")  ## Returns ["line1", "line2", "line3"]
f.str.splitLines("single")                ## Returns ["single"]
```

Join Function

Joins a list of strings with the given separator.

Morphology:

EBNF:

```
"f.str.join", "(", list, ",", string, ")"
```

- Operands: list of **string**, separator (**string**).
- Result: **string**.
- A **Type Error** is raised if the list contains non-string elements.
- A **Type Error** is raised if the separator is not a **string**.

Examples:

```
f.str.join(["a", "b", "c"], ",")  ## Returns "a,b,c"
f.str.join(["hello", "world"], " ") ## Returns "hello world"
f.str.join(["a", "b"], "")        ## Returns "ab"
```

Partition Function

Splits at the first occurrence of the separator, returning a 3-element list: [before, separator, after].

Morphology:

EBNF:

```
"f.str.partition", "(", string, ",", string, ")"
```

- Operands: **string**, **string** (separator).
- Result: list of 3 **string** elements.
- If the separator is not found, returns [original, "", ""].
- A **Type Error** is raised if either argument is not a **string**.

Examples:

```
f.str.partition("hello world", " ")  ## Returns ["hello", " ", "world"]
f.str.partition("abc", "x")          ## Returns ["abc", "", ""]
```

Format Hex Function

Formats an integer as a zero-padded hexadecimal string of the given width.
Morphology:

EBNF:

```
"f.str.formatHex", "(", int, ",", int, ")"
```

- Operands: value (`int`), width (`int`).
- Result: `string`.
- Output is lowercase hex, zero-padded to the specified width.
- A `Value Error` is raised for negative values.
- A `Type Error` is raised if arguments are not `int`.

Examples:

```
f.str.formatHex(255, 4)    #c# Returns "00ff"  
f.str.formatHex(10, 2)    #c# Returns "0a"
```

Format Binary Function

Formats an integer as a zero-padded binary string of the given width.
Morphology:

EBNF:

```
"f.str.formatBinary", "(", int, ",", int, ")"
```

- Operands: value (`int`), width (`int`).
- Result: `string`.
- Output is binary digits, zero-padded to the specified width.
- A `Value Error` is raised for negative values.
- A `Type Error` is raised if arguments are not `int`.

Examples:

```
f.str.formatBinary(10, 8)    #c# Returns "00001010"  
f.str.formatBinary(255, 8)  #c# Returns "11111111"
```

Is Empty Function

Returns `True` if the string has length 0.
Morphology:

EBNF:

```
"f.str.isEmpty", "(", string, ")"
```

- Operands: `string`.

- Result: bool.
- A Type Error is raised if the argument is not a **string**.

Examples:

```
f.str.isEmpty("")      #c# Returns True
f.str.isEmpty("hello")  #c# Returns False
```

Is Blank Function

Returns **True** if the string is empty or contains only whitespace.

Morphology:

EBNF:

```
"f.str.isBlank", "(", string, ")"
```

- Operands: **string**.
- Result: bool.
- A Type Error is raised if the argument is not a **string**.

Examples:

```
f.str.isBlank("")      #c# Returns True
f.str.isBlank(" ")     #c# Returns True
f.str.isBlank("hello")  #c# Returns False
```

Is Numeric Function

Returns **True** if the string represents any number (int, real, or complex).

Morphology:

EBNF:

```
"f.str.isNumeric", "(", string, ")"
```

- Operands: **string**.
- Result: bool.
- Returns **False** for empty strings and whitespace-only strings.
- "1+2i" returns **True** (complex), "3.14" returns **True** (real), "42" returns **True** (int).
- A Type Error is raised if the argument is not a **string**.

Examples:

```
f.str.isNumeric("42")   #c# Returns True
f.str.isNumeric("3.14") #c# Returns True
f.str.isNumeric("1+2i") #c# Returns True
f.str.isNumeric("abc")  #c# Returns False
f.str.isNumeric("")     #c# Returns False
```

Is Integer Function

Returns **True** if the string represents an integer.

Morphology:

EBNF:

```
"f.str.isInteger", "(", string, ")"
```

- Operands: **string**.
- Result: **bool**.
- Returns **False** for empty strings, reals, and complex numbers.
- A **Type Error** is raised if the argument is not a **string**.

Examples:

```
f.str.isInteger("42")      #c# Returns True
f.str.isInteger("3.14")    #c# Returns False
f.str.isInteger("1+2i")    #c# Returns False
```

Is Real Function

Returns **True** if the string represents a real number (not complex).

Morphology:

EBNF:

```
"f.str.isReal", "(", string, ")"
```

- Operands: **string**.
- Result: **bool**.
- "3.0" returns **True**, "1+2i" returns **False**.
- Returns **False** for empty strings.
- A **Type Error** is raised if the argument is not a **string**.

Examples:

```
f.str.isReal("3.14")      #c# Returns True
f.str.isReal("1+2i")      #c# Returns False
```

Is Identifier Function

Returns **True** if the string is a valid EDADSL identifier (starts with letter or underscore, followed by letters, digits, or underscores). Keywords (e.g., **"if"**, **"return"**, **"function"**) are syntactically valid identifiers and return **True**.

Morphology:

EBNF:

```
"f.str.isIdentifier", "(", string, ")"
```

- Operands: `string`.
- Result: `bool`.
- A `Type Error` is raised if the argument is not a `string`.

Examples:

```
f.str.isIdentifier("hello")    #c# Returns True
f.str.isIdentifier("_foo")     #c# Returns True
f.str.isIdentifier("if")      #c# Returns True  (keyword is syntactically valid)
f.str.isIdentifier("123abc")   #c# Returns False
f.str.isIdentifier("my-var")   #c# Returns False
```

Is Valid UTF-8 Function

Returns `True` if the string is valid UTF-8.

Morphology:

EBNF:

```
"f.str.isValidUTF8", "(", string, ")"
```

- Operands: `string`.
- Result: `bool`.
- All string literals in EDADSL are UTF-8; this function validates whether a byte sequence (if accessible) forms valid UTF-8.
- A `Type Error` is raised if the argument is not a `string`.

Examples:

```
f.str.isValidUTF8("hello")    #c# Returns True
```

Regex Functions

- A `Value Error` is raised by all four regex functions if the pattern is not a valid regular expression.

Regex Match

EBNF:

```
"f.str.regexMatch", "(", string, ",", string, ")"
```

Returns `True` if the string matches the regular expression pattern (full match).

```
f.str.regexMatch("hello123", "[a-z]+[0-9]+") #c# Returns True
f.str.regexMatch("hello", "[0-9]+")          #c# Returns False
```

Regex Find

EBNF:

```
"f.str.regexFind", "(", string, ",", string, ")"
```

Returns the first match as a string, or null if no match.

```
f.str.regexFind("abc123def", "[0-9]+")    ## Returns "123"
f.str.regexFind("hello", "[0-9]+")        ## Returns null
```

Regex FindAll

EBNF:

```
"f.str.regexFindAll", "(", string, ",", string, ")"
```

Returns a list of all non-overlapping matches.

```
f.str.regexFindAll("a1b2c3", "[0-9]+")    ## Returns ["1", "2", "3"]
f.str.regexFindAll("hello", "[0-9]+")      ## Returns []
```

Regex Replace

EBNF:

```
"f.str.regexReplace", "(", string, ",", string, ",", string, ")"
```

Replaces all matches of the pattern with the replacement string.

```
f.str.regexReplace("abc123def456", "[0-9]+", "X")  ## Returns "abcXdefX"
f.str.regexReplace("hello", "[0-9]+", "X")          ## Returns "hello"
```

Encoding Functions

Bytes Conversion

EBNF:

```
"f.str.toBytes", "(", string, ")"
```

- Converts a string to a list of integer byte values using UTF-8 encoding. Multi-byte characters produce multiple byte values (e.g. `f.str.toBytes("€")` returns `[0xE2, 0x82, 0xAC]`).

Examples:

```
f.str.toBytes("AB")          ## Returns [65, 66]
f.str.toBytes("€")           ## Returns [0xE2, 0x82, 0xAC]
```

FromBytes

EBNF:

```
"f.str.fromBytes", "(", list, ")"
```

- Converts a list of integer byte values (0–255) back to a string via UTF-8 decoding. A `Value Error` is raised if any byte value is outside 0–255 or if the resulting bytes do not form valid UTF-8.

```
f.str.fromBytes([65, 66])    ## Returns "AB"
f.str.fromBytes([0xE2, 0x82, 0xAC])  ## Returns "€"
```

Base64

EBNF:

```
("f.str.base64Encode" | "f.str.base64Decode"), "(", string, ")"
```

- A Value Error is raised by `f.base64Decode` if the input is not valid base64.

```
f.str.base64Encode("hello")      #c# Returns "aGVsbG8="
f.str.base64Decode("aGVsbG8=")   #c# Returns "hello"
```

Hex Encode/Decode

EBNF:

```
("f.str.hexEncode" | "f.str.hexDecode"), "(", string, ")"
```

- `f.str.hexEncode(string)` – converts each byte to its two-digit hex representation
- `f.str.hexDecode(string)` – converts hex string back to bytes/string. A Value Error is raised for odd-length strings or invalid hex characters.

```
f.str.hexEncode("AB")           #c# Returns "4142"
f.str.hexDecode("4142")         #c# Returns "AB"
```

Identifier Functions

Case Checks

EBNF:

```
("f.str.isUpper" | "f.str.isLower" | "f.str.isAlpha" | "f.str.isDigit" | "f.str.isAlphanu", string, ")"
```

- `f.isUpper` – True if all cased characters are uppercase
- `f.isLower` – True if all cased characters are lowercase
- `f.isAlpha` – True if all characters are letters
- `f.isDigit` – True if all characters are digits
- `f.isAlphanumeric` – True if all characters are letters or digits
- All five functions return True for empty strings (vacuously true).

```
f.str.isUpper("HELLO")          #c# Returns True
f.str.isUpper("Hello")          #c# Returns False
f.str.isUpper("")               #c# Returns True  (empty string)
f.str.isLower("hello")          #c# Returns True
f.str.isAlpha("abc")            #c# Returns True
f.str.isAlpha("abc123")         #c# Returns False
f.str.isDigit("123")            #c# Returns True
f.str.isDigit("12.3")           #c# Returns False
f.str.isAlphanumeric("abc123")  #c# Returns True
```


Sanitize Identifier

EBNF:

```
"f.str.sanitizeIdentifier", "(", string, ")"
```

Converts a string to a valid identifier by replacing invalid characters with underscores and collapsing consecutive underscores.

- If the result starts with a digit, an underscore is prepended.

```
f.str.sanitizeIdentifier("my-var")      #c# Returns "my_var"
f.str.sanitizeIdentifier("  spaces  ")  #c# Returns "spaces"
f.str.sanitizeIdentifier("a!!b")        #c# Returns "a_b"
f.str.sanitizeIdentifier("123abc")      #c# Returns "_123abc"
```

11.1.3 Vectorization

All scalar math, trigonometric, and complex functions automatically operate on tensors element-wise. When a tensor is passed as an argument, the function returns a tensor of the same shape with the operation applied to each element.

Supported Functions

- **f.math:** abs, sqrt, exp, log, log10, log2, exp2, log1p, expm1, floor, ceil, round, sign, cbrt, square, reciprocal, isnan, isinf, angle, real, imag, conj, fmod
- **f.trig:** sin, cos, tan, cot, sec, csc, asin, acos, atan, atan2, sinh, cosh, tanh, asinh, acosh, atanh
- **f.cplx:** conj, real, imag

Edge Cases

- Complex results: `f.math.sqrt(|[-1, 4, 9])` returns `|[0+1i, 2.0, 3.0]`
- Undefined results: `f.math.log(|[0, 1, 2])` returns `|[null, 0.0, 0.693]`
- Empty tensor: `f.math.abs(|[])` returns `|[]`

Examples

```
f.math.abs(|[-3, 4, -5])      #c# Returns |[3, 4, 5]
f.math.sqrt(|[1, 4, 9, 16])   #c# Returns |[1.0, 2.0, 3.0, 4.0]
f.math.sqrt(|[-1, 0, 4])      #c# Returns |[0+1i, 0.0, 2.0]
f.math.exp(|[0, 1, 2])        #c# Returns |[1.0, 2.718, 7.389]
f.math.log(|[1, 2.718])       #c# Returns |[0.0, 1.0]
f.math.sin(|[0, 3.14159/2])    #c# Returns |[0.0, 1.0]
f.math.floor(|[1.2, 2.7, 3.0]) #c# Returns |[1, 2, 3]
f.math.isnan(|[1.0, #nan, 3.0]) #c# Returns |[false, true, false]
```

11.1.4 Tensor Indexing

Tensor indexing provides flexible access to elements, sub-tensors, and slices of multi-dimensional tensors. Tensors support arbitrary dimensionality (1D, 2D, 3D, ..., nD).

Element Access

Returns a scalar at the specified position.

EBNF:

expression, "[" , indexing_expr , "]"

- $\$a[i]$ — element at index i (1D $\rightarrow$ scalar)
- $\$a[i, j]$ — element at $[i, j]$ (2D $\rightarrow$ scalar)
- $\$a[i, j, k]$ — element at $[i, j, k]$ (3D $\rightarrow$ scalar)
- $\$a[i, j, k, \dots]$ — element at position (nD $\rightarrow$ scalar)
- $\$a[-i]$ — i -th element from end

Examples:

```
 $\$a$  [tensor]= |[10, 20, 30, 40, 50]
 $\$a[0]$                                 ## Returns 10
 $\$a[-1]$                              ## Returns 50

 $\$b$  [tensor]= |[[1,2,3],[4,5,6],[7,8,9]]
 $\$b[0, 1]$                             ## Returns 2
 $\$b[2, 2]$                             ## Returns 9
```

Sub-tensor Access

Returns a sub-tensor along the first axis.

- $\$a[i]$ — sub-tensor at position i (2D $\rightarrow$ row, 3D $\rightarrow$ 2D slice)
- $\$a[-i]$ — i -th sub-tensor from end

Examples:

```
 $\$b$  [tensor]= |[[1,2,3],[4,5,6],[7,8,9]]
 $\$b[0]$                                 ## Returns |[1, 2, 3] (row 0)
 $\$b[1]$                                 ## Returns |[4, 5, 6] (row 1)
 $\$b[-1]$                              ## Returns |[7, 8, 9] (last row)

 $\$c$  [tensor]= |[[[1,2],[3,4]],[[5,6],[7,8]]]
 $\$c[0]$                                 ## Returns |[1,2],[3,4]] (2D slice)
 $\$c[1]$                                 ## Returns |[5,6],[7,8]] (2D slice)
```

Slice Access

Returns a sub-tensor (copy, not view).

- `$a[start:end]` — elements from start to end (exclusive)
- `$a[start:end:step]` — every step-th element
- `$a[:]` — copy of entire tensor
- `$a[:end]` — from beginning to end
- `$a[start:]` — from start to end
- `$a[::2]` — every 2nd element
- `$a[::-1]` — reversed tensor

Examples:

```
$a [tensor]= |[10, 20, 30, 40, 50]
$a[1:4]                ## Returns |[20, 30, 40]
$a[:3]                 ## Returns |[10, 20, 30]
$a[2:]                 ## Returns |[30, 40, 50]
$a[::2]                ## Returns |[10, 30, 50]
$a[::-1]               ## Returns |[50, 40, 30, 20, 10]
$a[:]                  ## Returns |[10, 20, 30, 40, 50] (copy)
```

```
$b [tensor]= |[[1,2,3],[4,5,6],[7,8,9]]
$b[0:2]            ## Returns |[[1,2,3],[4,5,6]] (rows 0-1)
$b[0:2, 1:3]       ## Returns |[[2,3],[5,6]] (sub-matrix)
$b[:, 0]           ## Returns |[1, 4, 7] (all rows, col 0)
```

Index List Access

Returns sub-tensor at specified indices.

- `$a[[0, 2, 4]]` — elements/rows at indices 0, 2, 4
- `$a[[-1, -2]]` — last two elements/rows

Examples:

```
$a [tensor]= |[10, 20, 30, 40, 50]
$a[[0, 2, 4]]        ## Returns |[10, 30, 50]

$b [tensor]= |[[1,2,3],[4,5,6],[7,8,9]]
$b[[0, 2]]           ## Returns |[[1,2,3],[7,8,9]] (rows 0 and 2)
$b[[-1]]             ## Returns |[7, 8, 9] (last row)
```

Boolean Indexing

Returns 1D tensor of elements where condition is True.

- `$a[condition]` — elements where condition is True
- `$a[condition] = value` — set matching elements
- `$a[condition] = other[condition]` — set from another tensor

Conditions: any expression returning a boolean tensor:

- Comparisons: `$a > 3`, `$a == 0`, `$a != 5`
- Approximate equality: `$a $~ 3`
- Compound: `$a > 0 && $a < 10`
- Vectorized functions: `f.math.isinf($a)`, `f.math.isnan($a)`

Examples:

```
$a [tensor]= |[1, 5, 3, 8, 2]
$a[$a > 3]                                     ## Returns |[5, 8] (1D tensor)
$a[$a > 3] = 0                                 ## $a is now |[1, 0, 3, 0, 2]
$a[$a == 0] = |[10, 20]                       ## $a is now |[1, 10, 3, 20, 2]
```

```
$f [tensor]= |[1.0, #inf, 3.0, #nan, 5.0]
$f[f.math.isinf($f)]                         ## Returns |[inf]
$f[f.math.isnan($f)]                         ## Returns |[nan]
```

Ellipsis Indexing

Shorthand for remaining axes.

- `$a[..., 0]` — first element along last axis for all preceding axes
- `$a[0, ...]` — first element along first axis for all remaining axes
- `$a[..., i, j]` — indices i, j along last two axes

Examples:

```
$c [tensor]= |[[[[1,2],[3,4]],[[5,6],[7,8]]]
$c[..., 0]                                     ## Returns |[1,3],[5,7]] (first along last axis)
$c[..., 0, 1]                                 ## Returns |[3, 7]
$c[0, ...]                                    ## Returns |[1,2],[3,4]] (same as $c[0])
```

Safe Indexing (?[])

Returns null if the left operand is null, otherwise performs normal indexing.

- `$a?[i]` — safe element access — null if `$a` is null
- `$a?[i, j]` — safe multi-dim element — null if `$a` is null

- `$a?[start:end]` — safe slice — null if `$a` is null
- `$a?[start:end:step]` — safe slice with step — null if `$a` is null
- `$a?[[0, 2, 4]]` — safe index list — null if `$a` is null
- `$a?[condition]` — safe boolean indexing — null if `$a` is null
- `$a?[..., 0]` — safe ellipsis — null if `$a` is null

Behavior:

- If left operand is null → returns null (no Runtime Error)
- If left operand is not null → performs normal indexing
- Type Error / Value Error still raised if indexing is invalid (wrong type, out of bounds)

Examples:

```
$mat = null
$mat?[0, 1]           #c# Returns null (no error)
$mat?[0:2]            #c# Returns null (no error)
$mat?[0:2, 0]         #c# Returns null (no error)
$mat?[$mat > 0]       #c# Returns null (no error)

$mat [tensor]= |[[1,2],[3,4]]
$mat?[0, 1]           #c# Returns 2 (normal indexing)
$mat?[0:2, 0]         #c# Returns |[1, 3]
```

Assignment

All indexing operations support assignment (in-place modification).

- `$a[i] = val` — set single element
- `$a[i, j] = val` — set element at `[i, j]`
- `$a[start:end] = vals` — set range
- `$a[mask] = vals` — set matching elements

Safe assignment:

- `$a?[i] = val` — safe assignment — no-op if `$a` is null

Examples:

```
$a [tensor]= |[10, 20, 30, 40, 50]
$a[0] = 99           #c# $a is now |[99, 20, 30, 40, 50]
$a[1:3] = |[99, 99]  #c# $a is now |[99, 99, 99, 40, 50]
$a[$a > 40] = 0      #c# $a is now |[99, 99, 99, 40, 0]

$b [tensor]= |[[1,2,3],[4,5,6],[7,8,9]]
$b[0] = |[10, 20, 30] #c# Replace row 0
```

```
$b[1, 2] = 99                                ## Set element at [1,2]
```

```
# Safe assignment
```

```
$c = null
```

```
$c?[0] = 99                                ## No-op ($c is null)
```

Edge Cases

- Empty tensor: `$a[0] → Value Error`
- Out of bounds: `$a[100] → Value Error`
- Negative out of bounds: `$a[-100] → Value Error`
- Empty result: `$a[0:0] → |[] (empty tensor)`
- Empty mask: `$a[$a > 100] → |[] (empty tensor)`
- Wrong mask length: `$a[| [true, false]]` on 5-element tensor `→ Value Error`
- Invalid index type: `$a["hello"] → Type Error`
- Safe indexing only suppresses null left operand errors; type/value errors still raised

Semantics

- Slicing returns copies, not views (consistent with EDADSL copy-on-assignment semantics)
- Boolean indexing returns 1D tensor (not flattened)
- Index list returns sub-tensor (preserves dimensionality)
- Ellipsis is syntactic sugar for remaining axes

11.1.5 List and Set Functions

Concatenation

The `concat` function combines all arguments into a single list.

Morphology:

EBNF:

```
"f.list.concat", "(", { argument }, ")"
```

Example:

```
f.list.concat([1, 2], [3, 4])
```

```
## Returns [1, 2, 3, 4]
```

```
f.list.concat("a", "b", "c")
```

```
## Returns ["a", "b", "c"]
```

Push

The `push` function appends an element to the end of a list.

Morphology:

EBNF:

```
"f.list.push", "(", list, ",", data, ")"
```

- Operands: list, any data type matching the list element type
- Returns the modified list (the original list is also mutated)

Examples:

```
f.list.push([1, 2, 3], 4)      ## Returns [1, 2, 3, 4]
$f = [10, 20]
f.list.push($f, 30)           ## $f is now [10, 20, 30]
```

Pop

The `pop` function removes and returns the last element of a list.

Morphology:

EBNF:

```
"f.list.pop", "(", list, ")"
```

- Operands: list (must not be empty)
- Returns the removed element
- A Value Error is raised if the list is empty

Examples:

```
f.list.pop([1, 2, 3])          ## Returns 3, list is now [1, 2]
$f = [10, 20, 30]
f.list.pop($f)                 ## Returns 30, $f is now [10, 20]
```

Head

The `head` function returns the first `n` elements of a list.

Morphology:

EBNF:

```
"f.list.head", "(", list, ",", int, ")"
```

- Operands: list, int ($n \geq 0$)
- Returns a new list containing the first `n` elements
- If `n` exceeds the list length, returns the entire list (no error)
- A Value Error is raised if `n < 0`.

Examples:

```
f.list.head([1, 2, 3, 4, 5], 3) ## Returns [1, 2, 3]
f.list.head([1, 2, 3], 10)      ## Returns [1, 2, 3]
f.list.head([1, 2, 3], 0)       ## Returns []
```

Tail

The **tail** function returns the last **n** elements of a list.

Morphology:

EBNF:

```
"f.list.tail", "(", list, ",", int, ")"
```

- Operands: list, int ($n \geq 0$)
- Returns a new list containing the last **n** elements
- If **n** exceeds the list length, returns the entire list (no error)
- A Value Error is raised if $n < 0$.

Examples:

```
f.list.tail([1, 2, 3, 4, 5], 3)  ## Returns [3, 4, 5]
f.list.tail([1, 2, 3], 10)      ## Returns [1, 2, 3]
f.list.tail([1, 2, 3], 0)       ## Returns []
```

Type Conversion

List to Set The **set** function converts a list to a set. All unique elements are preserved; duplicate elements are removed and order is lost.

Morphism:

EBNF:

```
"f.list.set", "(", list, ")"
```

Example:

```
f.list.set([Q1, Q2, R5])  ## Returns {Q1, Q2, R5}
```

Set to List The **list** function converts a set to a list. The order is determined by the lexicographic order of the canonical string representations of the elements.

Morphology:

EBNF:

```
"f.list.fromSet", "(", set, ")"
```

Example:

```
f.list.fromSet({Q1, Q2})  ## Returns [Q1, Q2] (order from *e)
```

Copy and Clone

Shallow Copy The **copy** function creates a shallow copy of a value. For lists, a new list is created containing the same elements (but nested structures are shared by reference). For scalars (int, real, complex, bool, string), the value is returned unchanged (scalars are immutable).

Morphology:

EBNF:

```
"f.list.copy", "(", data, ")"
```

- Operands: any
- Result: shallow copy of the input

Examples:

```
$a = [1, 2, 3]
$b = f.list.copy($a)      #c# $b is a new list [1, 2, 3]
$b[0] = 99                #c# $a is still [1, 2, 3]
```

Deep Clone The `clone` function creates a deep copy of a value. All nested structures are recursively copied, so modifications to the clone never affect the original. For scalars (int, real, complex, bool, string), the value is returned unchanged (scalars are immutable).

Morphology:

EBNF:

```
"f.list.clone", "(", data, ")"
```

- Operands: any
- Result: deep copy of the input

Examples:

```
$a = [[1, 2], [3, 4]]
$b = f.list.clone($a)      #c# $b is a fully independent copy
$b[0][0] = 99              #c# $a is still [[1, 2], [3, 4]]
```

11.1.6 Nddarray Functions

Functions for operating on tensors (and lists). These functions extend the built-in math and list functions with array-aware behavior.

Array Creation

Zeros The `zeros` function creates an array filled with zeros.

Morphology:

EBNF:

```
"f.tnsr.zeros", "(", shape, ")"
```

- Operands: int (length for 1D), or list of ints (dimensions for nD)
- Result: tensor filled with 0
- A `Value Error` is raised for negative dimensions.

Examples:

```
f.tnsr.zeros(5)           #c# Returns [0, 0, 0, 0, 0]
f.tnsr.zeros([2, 3])      #c# Returns [[0,0,0],[0,0,0]]
```

Ones The `ones` function creates an array filled with ones.

Morphology:

EBNF:

```
"f.tnsr.ones", "(", shape, ")"
```

- Operands: int (length for 1D), or list of ints (dimensions for nD)
- Result: tensor filled with 1
- A `Value Error` is raised for negative dimensions.

Examples:

```
f.tnsr.ones(4)           ## Returns [1, 1, 1, 1]
f.tnsr.ones([2, 2])      ## Returns [[1,1],[1,1]]
```

Identity Matrix The `identity` function creates an $n \times n$ identity matrix.

Morphology:

EBNF:

```
"f.tnsr.identity", "(", int, ")"
```

- Operands: int ($n \geq 0$)
- Result: tensor ($n \times n$) with 1s on the diagonal and 0s elsewhere
- `f.tnsr.identity(0)` returns a 0×0 matrix `[[[]]]`.

Examples:

```
f.tnsr.identity(1)       ## Returns [[1]]
f.tnsr.identity(3)       ## Returns [[1,0,0],[0,1,0],[0,0,1]]
```

Full The `full` function creates a tensor filled with a constant value.

Morphology:

EBNF:

```
"f.tnsr.full", "(", list[int], ",", ( int | real | complex ), ")"
```

- First argument: shape (list of ints, e.g., `[3, 4]`)
- Second argument: fill value
- Result: tensor of specified shape filled with value
- A `Value Error` is raised for negative dimensions.

Examples:

```
f.tnsr.full([3, 4], 0)   ## Returns [[0,0,0,0],[0,0,0,0],[0,0,0,0]]
f.tnsr.full([2, 3], 5.5) ## Returns [[5.5,5.5,5.5],[5.5,5.5,5.5]]
f.tnsr.full([5], #inf)   ## Returns [inf, inf, inf, inf, inf]
f.tnsr.full([2, 2], 1+2i) ## Returns [[1+2i,1+2i],[1+2i,1+2i]]
```

Diagonal The `diag` function creates a diagonal matrix from a list, or extracts the diagonal from a 2D tensor.

Morphology:

EBNF:

```
"f.tnsr.diag", "(", argument, ")"
```

- 1D input: creates square matrix with input on diagonal, zeros elsewhere
- 2D input: extracts diagonal as 1D list
- Result: tensor (creation) or list (extraction)

Examples:

```
f.tnsr.diag([1, 2, 3])           #c# Returns [[1,0,0],[0,2,0],[0,0,3]]
f.tnsr.diag([5])                 #c# Returns [[5]]
f.tnsr.diag(|[[1,2,3],[4,5,6],[7,8,9]]) #c# Returns [1, 5, 9]
f.tnsr.diag(|[[1,2,3,4],[5,6,7,8]]) #c# Returns [1, 6]
```

Diagonal Flat The `diagflat` function creates a 2D tensor with the input flattened on the diagonal.

Morphology:

EBNF:

```
"f.tnsr.diagflat", "(", argument, ")"
```

- Flattens input and creates square matrix with values on diagonal
- Result: 2D tensor

Examples:

```
f.tnsr.diagflat([1, 2, 3])       #c# Returns [[1,0,0],[0,2,0],[0,0,3]]
f.tnsr.diagflat(|[1, 2], [3])    #c# Returns [[1,0,0,0],[0,2,0,0],[0,0,3,0],[0,0,
```

Lower Triangle The `tril` function returns the lower triangle of a tensor.

Morphology:

EBNF:

```
"f.tnsr.tril", "(", tensor, ")"
```

- Returns elements on and below the main diagonal
- Elements above diagonal become 0

Examples:

```
f.tnsr.tril(|[[1,2,3],[4,5,6],[7,8,9]]) #c# Returns [[1,0,0],[4,5,0],[7,8,9]]
f.tnsr.tril(|[[1,2],[3,4]])             #c# Returns [[1,0],[3,4]]
```

Upper Triangle The `triu` function returns the upper triangle of a tensor.

Morphology:

EBNF:

```
"f.tnsr.triu", "(" , tensor, ")"
```

- Returns elements on and above the main diagonal
- Elements below diagonal become 0

Examples:

```
f.tnsr.triu(|[[1,2,3],[4,5,6],[7,8,9]]) #c# Returns [[1,2,3],[0,5,6],[0,0,9]]
f.tnsr.triu(|[[1,2],[3,4]])             #c# Returns [[1,2],[0,4]]
```

Eye The `eye` function creates an identity matrix with optional diagonal offset.

Morphology:

EBNF:

```
"f.tnsr.eye", "(" , int, [ ",", "k", "=", int ], ")"
```

- First argument: size ($n \times n$ matrix)
- Optional k: diagonal offset (0=main, positive=above, negative=below)
- Result: tensor with 1s on specified diagonal, 0s elsewhere
- A `Value Error` is raised if $n \leq 0$.

Examples:

```
f.tnsr.eye(3)           #c# Returns [[1,0,0],[0,1,0],[0,0,1]]
f.tnsr.eye(2)           #c# Returns [[1,0],[0,1]]
f.tnsr.eye(3, k=1)      #c# Returns [[0,1,0],[0,0,1],[0,0,0]]
f.tnsr.eye(3, k=-1)     #c# Returns [[0,0,0],[1,0,0],[0,1,0]]
f.tnsr.eye(4, k=2)      #c# Returns [[0,0,1,0],[0,0,0,1],[0,0,0,0],[0,0,0,0]]
```

Dimension Manipulation

Stack The `stack` function stacks tensors along a new axis.

Morphology:

EBNF:

```
"f.tnsr.stack", "(" , list[tensor], [ ",", "axis", "=", int ], ")"
```

- First argument: list of tensors (same shape)
- Optional axis: dimension along which to stack (default 0)
- Result: tensor with one additional dimension
- A `Value Error` is raised if list is empty, shapes differ, or axis is out of range.

Examples:

```
f.tnsr.stack(|[[1,2], |[3,4], |[5,6]]) #c# Returns [[1,2],[3,4],[5,6]]
f.tnsr.stack(|[[1,2], |[3,4]], axis=1)  #c# Returns [[1,3],[2,4]]
```

Vertical Stack The `vstack` function stacks tensors vertically (row-wise).

Morphology:

EBNF:

```
"f.tnsr.vstack", "(", list[tensor], ")"
```

- All tensors must have same number of columns
- Result: concatenated tensor along first axis
- A `Value Error` is raised if list is empty or columns differ.

Examples:

```
f.tnsr.vstack([| [1,2], | [3,4], | [5,6]])    #c# Returns [[1,2],[3,4],[5,6]]
f.tnsr.vstack([| [1,2,3], | [4,5,6]])          #c# Returns [[1,2,3],[4,5,6]]
```

Horizontal Stack The `hstack` function stacks tensors horizontally (column-wise).

Morphology:

EBNF:

```
"f.tnsr.hstack", "(", list[tensor], ")"
```

- All tensors must have same number of rows
- For 1D: concatenates along first axis
- A `Value Error` is raised if list is empty or rows differ.

Examples:

```
f.tnsr.hstack([| [1,2], | [3,4]])              #c# Returns [1,2,3,4]
f.tnsr.hstack([| [1,2], | [3,4]], | [5,6], | [7,8]]) #c# Returns [[1,2,5,7],[3,4,6,8]]
```

Swap Axes The `swapaxes` function swaps two axes of a tensor.

Morphology:

EBNF:

```
"f.tnsr.swapaxes", "(", tensor, ",", int, ",", int, ")"
```

- Swaps the two specified axes
- A `Value Error` is raised if axes are out of range.

Examples:

```
f.tnsr.swapaxes(| [[1,2,3],[4,5,6]], 0, 1)    #c# Returns [[1,4],[2,5],[3,6]]
f.tnsr.swapaxes(| [[1,2],[3,4]], 0, 1)         #c# Returns [[1,3],[2,4]]
```

Expand Dimensions The `expandDims` function inserts a new dimension of size 1.
Morphology:

EBNF:

```
"f.tnsr.expandDims", "(", tensor, ",", int, ")"
```

- Second argument: axis where new dimension is inserted
- Negative axis counts from end
- A `Value Error` is raised if axis is out of range.

Examples:

```
f.tnsr.expandDims([1,2,3], 0)      ## Returns [[1,2,3]]
f.tnsr.expandDims([1,2,3], 1)      ## Returns [[1],[2],[3]]
f.tnsr.expandDims([1,2],[3,4], 2)  ## Returns [[[1,2]],[[3,4]]]
```

Squeeze The `squeeze` function removes dimensions of size 1.
Morphology:

EBNF:

```
"f.tnsr.squeeze", "(", tensor, ")"
```

- Removes all dimensions with size 1
- Returns copy if no dimensions to squeeze

Examples:

```
f.tnsr.squeeze([1,2,3])           ## Returns [1,2,3]
f.tnsr.squeeze([1],[2],[3])       ## Returns [1,2,3]
f.tnsr.squeeze([1,2])             ## Returns [1,2]
f.tnsr.squeeze([1,2],[3,4])       ## Returns [[1,2],[3,4]]
```

Flip The `flip` function flips tensor along specified axis.
Morphology:

EBNF:

```
"f.tnsr.flip", "(", tensor, [ ",", "axis", "=", int ], ")"
```

- Optional axis: dimension to flip (default 0)
- A `Value Error` is raised if axis is out of range.

Examples:

```
f.tnsr.flip([1,2,3,4,5])          ## Returns [5,4,3,2,1]
f.tnsr.flip([1,2,3],[4,5,6], axis=0) ## Returns [[4,5,6],[1,2,3]]
f.tnsr.flip([1,2,3],[4,5,6], axis=1) ## Returns [[3,2,1],[6,5,4]]
```

Flip Vertical The `flipud` function flips vertically (up-down). Equivalent to `flip(tensor, axis=0)`.

Morphology:

EBNF:

```
"f.tnsr.flipud", "(", tensor, ")"
```

Examples:

```
f.tnsr.flipud([1,2,3,4,5])          ## Returns [5,4,3,2,1]
f.tnsr.flipud([[1,2],[3,4]])        ## Returns [[3,4],[1,2]]
```

Flip Horizontal The `fliplr` function flips horizontally (left-right). Equivalent to `flip(tensor, axis=1)`.

Morphology:

EBNF:

```
"f.tnsr.fliplr", "(", tensor, ")"
```

Examples:

```
f.tnsr.fliplr([[1,2,3],[4,5,6]])     ## Returns [[3,2,1],[6,5,4]]
f.tnsr.fliplr([1,2,3,4,5])          ## Returns [5,4,3,2,1]
```

Rotate 90 The `rot90` function rotates tensor 90° counterclockwise k times.

Morphology:

EBNF:

```
"f.tnsr.rot90", "(", tensor, [ ",", "k", "=", int ], ")"
```

- Optional k: number of 90° rotations (default 1)
- Negative k: clockwise rotation
- Must be 2D tensor
- A `Type Error` is raised for non-2D tensors.

Examples:

```
f.tnsr.rot90([[1,2,3],[4,5,6]])      ## Returns [[3,6],[2,5],[1,4]]
f.tnsr.rot90([[1,2,3],[4,5,6]], k=2) ## Returns [[6,5,4],[3,2,1]]
f.tnsr.rot90([[1,2],[3,4]], k=-1)    ## Returns [[2,4],[1,3]]
```

Repeat The `repeat` function repeats each element specified number of times.

Morphology:

EBNF:

```
"f.tnsr.repeat", "(", tensor, ",", int, ")"
```

- Second argument: repeat count per element
- Returns flattened result

- A Value Error is raised if $\text{count} \leq 0$.

Examples:

```
f.tnsr.repeat(|[1,2,3], 3)    ## Returns [1,1,1,2,2,2,3,3,3]
f.tnsr.repeat(|[1,2], 2)     ## Returns [1,1,2,2]
```

Tile The tile function tiles tensor by repeating it along each axis.
Morphology:

EBNF:

```
"f.tnsr.tile", "(", tensor, ",", ( int | list[int] ), ")"
```

- Single int: repeat that many times along all axes
- List: repeat along each axis respectively
- A Value Error is raised for negative reps.

Examples:

```
f.tnsr.tile(|[1,2,3], 2)          ## Returns [1,2,3,1,2,3]
f.tnsr.tile(|[[1,2],[3,4]], [2, 3]) ## Returns [[1,2,1,2,1,2],[3,4,3,4,3,4],[1,2,1,2,1,2,1,2]]
```

Pad The pad function pads tensor with values.
Morphology:

EBNF:

```
"f.tnsr.pad", "(", tensor, ",", ( int | list[int] ), [ ",", "constant", "=", real ], ")"
```

- Second argument: padding amount ([before, after] for 1D, [[top,bottom],[left,right]] for 2D)
- Optional constant: fill value (default 0)
- A Value Error is raised for negative padding.

Examples:

```
f.tnsr.pad(|[1,2,3], [2, 1])          ## Returns [0,0,1,2,3,0]
f.tnsr.pad(|[1,2,3], [1, 2], constant=9) ## Returns [9,1,2,3,9,9]
f.tnsr.pad(|[[1,2],[3,4]], [[1,1],[0,0]]) ## Returns [[0,0],[1,2],[3,4],[0,0]]
```

Append The append function appends second tensor to first (flattened).
Morphology:

EBNF:

```
"f.tnsr.append", "(", tensor, ",", tensor, ")"
```

- Returns 1D tensor

Examples:

```
f.tnsr.append(|[1,2], |[3,4,5])      ## Returns [1,2,3,4,5]
f.tnsr.append(|[[1,2]], |[[3,4]])    ## Returns [1,2,3,4]
```


Insert The `insert` function inserts values at specified position.
Morphology:

EBNF:

```
"f.tnsr.insert", "(" , tensor, ",", int, ",", tensor, ")"
```

- Negative index counts from end
- Returns new tensor (original unchanged)
- A `Value Error` is raised if index is out of range.

Examples:

```
f.tnsr.insert(|[1,2,5], 2, |[3,4])    ## Returns [1,2,3,4,5]
f.tnsr.insert(|[1,2,3], -1, |[10])    ## Returns [1,2,10,3]
```

Delete The `delete` function removes element at specified position.
Morphology:

EBNF:

```
"f.tnsr.delete", "(" , tensor, ",", int, ")"
```

- Negative index counts from end
- Returns new tensor (original unchanged)
- A `Value Error` is raised if index is out of range.

Examples:

```
f.tnsr.delete(|[1,2,3,4,5], 2)        ## Returns [1,2,4,5]
f.tnsr.delete(|[1,2,3,4,5], -1)       ## Returns [1,2,3,4]
```

Trim Zeros The `trimZeros` function removes leading and trailing zeros.
Morphology:

EBNF:

```
"f.tnsr.trimZeros", "(" , tensor, ")"
```

- Keeps internal zeros
- Returns empty tensor if all zeros

Examples:

```
f.tnsr.trimZeros(|[0,0,1,2,0,3,0,0])  ## Returns [1,2,0,3]
f.tnsr.trimZeros(|[0,0,0])            ## Returns []
f.tnsr.trimZeros(|[1,2,3])            ## Returns [1,2,3]
```

Edge Cases – Reduction Functions

- `f.stat.sum([])` returns 0 (additive identity).
- `f.stat.prod([])` returns 1 (multiplicative identity).
- `f.stat.mean([])`, `f.stat.std([])`, `f.stat.var([])`, `f.stat.median([])` raise a `Value Error` on empty lists.
- `f.util.sort([])`, `f.util.reverse([])`, `f.util.unique([])` return `[]` on empty lists.
- A `Type Error` is raised if the argument is not a list.

Reduction Functions

Reduction functions collapse an tensor along an axis. When `axis` is omitted, the operation is applied to the flattened array (all elements). When `axis = 0`, the operation is applied row-wise (collapses rows). When `axis = 1`, the operation is applied column-wise (collapses columns).

Sum

EBNF:

```
"f.stat.sum", "(", argument, [ ",", "axis", "=", int ], ")"
```

```
f.stat.sum([1, 2, 3])           #c# Returns 6
f.stat.sum([[1,2],[3,4]])       #c# Returns 10 (flattened)
f.stat.sum([[1,2],[3,4]], axis=0) #c# Returns [4, 6] (row-wise)
f.stat.sum([[1,2],[3,4]], axis=1) #c# Returns [3, 7] (column-wise)
```

Product

EBNF:

```
"f.stat.prod", "(", argument, [ ",", "axis", "=", int ], ")"
```

```
f.stat.prod([2, 3, 4])          #c# Returns 24
f.stat.prod([[2,3],[4,5]], axis=0) #c# Returns [8, 15]
f.stat.prod([[2,3],[4,5]], axis=1) #c# Returns [6, 20]
```

Mean

EBNF:

```
"f.stat.mean", "(", argument, [ ",", "axis", "=", int ], ")"
```

```
f.stat.mean([1, 2, 3, 4])       #c# Returns 2.5
f.stat.mean([[1,2],[3,4]], axis=0) #c# Returns [2.0, 3.0]
f.stat.mean([[1,2],[3,4]], axis=1) #c# Returns [1.5, 3.5]
```

Standard Deviation

EBNF:

```
"f.stat.std", "(", argument, [ ",", "axis", "=", int ], ")"
```

```
f.stat.std([1, 2, 3, 4, 5])      ## Returns ~1.4142
```

```
f.stat.std([1, 1, 1, 1])        ## Returns 0.0
```

Variance Computes the population variance (divides by N , not $N - 1$).

EBNF:

```
"f.stat.var", "(", argument, [ ",", "axis", "=", int ], ")"
```

```
f.stat.var([1, 2, 3, 4, 5])     ## Returns 2.0
```

```
f.stat.var([1, 1, 1, 1])       ## Returns 0.0
```

Median

EBNF:

```
"f.stat.median", "(", argument, ")"
```

```
f.stat.median([3, 1, 4, 1, 5])  ## Returns 3
```

```
f.stat.median([1, 2, 3, 4])     ## Returns 2.5 (average of middle two)
```

Percentile

EBNF:

```
"f.stat.percentile", "(", argument, ",", real, ")"
```

The second argument is a percentage (0–100).

Edge Cases – Percentile

- A **Value Error** is raised if the list is empty.
- A **Value Error** is raised if the percentage is outside 0–100.

```
f.stat.percentile([10, 20, 30, 40, 50], 50)  ## Returns 30
```

```
f.stat.percentile([10, 20, 30, 40, 50], 0)   ## Returns 10
```

```
f.stat.percentile([10, 20, 30, 40, 50], 100) ## Returns 50
```

Mode Returns the most frequently occurring value(s) in a list. If multiple values tie for the highest frequency, returns a sorted list of all modes.

EBNF:

```
"f.stat.mode", "(", list, ")"
```

Edge Cases – Mode

- A Value Error is raised if the list is empty.
- A Type Error is raised if the argument is not a list.
- If all values occur equally often, returns a sorted list of all unique values.

```
f.stat.mode([1, 2, 2, 3, 3, 3])    #c# Returns 3 (single mode)
f.stat.mode([1, 1, 2, 2, 3])      #c# Returns [1, 2] (bimodal)
f.stat.mode([1, 2, 3])           #c# Returns [1, 2, 3] (all equally frequent)
```

Quantile Returns the value at quantile q (in $[0, 1]$). Equivalent to `f.stat.percentile(x, q * 100)`.

EBNF:

```
"f.stat.quantile", "(", list, ",", real, ")"
```

Edge Cases – Quantile

- A Value Error is raised if the list is empty.
- A Value Error is raised if the quantile is outside 0–1.
- A Type Error is raised if the first argument is not a list.

```
f.stat.quantile([10, 20, 30, 40, 50], 0.5) #c# Returns 30
f.stat.quantile([10, 20, 30, 40, 50], 0.0) #c# Returns 10
f.stat.quantile([10, 20, 30, 40, 50], 1.0) #c# Returns 50
f.stat.quantile([10, 20, 30, 40, 50], 0.25) #c# Returns 20
```

Root Mean Square

EBNF:

```
"f.stat.rms", "(", argument, ")"
```

Computes $\sqrt{\frac{1}{n} \sum_{i=1}^n x_i^2}$. Useful for AC signal analysis and power calculations.

- A Value Error is raised if the list is empty.

```
f.stat.rms([1, -1, 1, -1])    #c# Returns 1.0
f.stat.rms([3, 4])           #c# Returns 3.5355...
f.stat.rms([0, 0, 0])        #c# Returns 0.0
```

Peak

EBNF:

```
"f.stat.peak", "(", argument, [ ",", "mode", "=", string ], ")"
```

Modes:

- "p2p" (default) – Peak-to-peak: $\max(x) - \min(x)$

- "abs" – Maximum absolute value: $\max(|x|)$
- "pos" – Maximum positive value: $\max(x)$
- "neg" – Minimum negative value: $\min(x)$

```
f.stat.peak([1, -3, 2, -5, 4])           ## Returns 9 (p2p: 4 - (-5))
f.stat.peak([1, -3, 2, -5, 4], mode="abs") ## Returns 5
f.stat.peak([1, -3, 2, -5, 4], mode="pos") ## Returns 4
f.stat.peak([1, -3, 2, -5, 4], mode="neg") ## Returns -5
```

Edge Cases – Peak

- A Value Error is raised if the list is empty.
- A Value Error is raised if an invalid mode is specified.

Search Functions

Edge Cases – Search Functions

- A Value Error is raised if the list is empty.

Index of Minimum

EBNF:

```
"f.stat.argmin", "(", argument, ")"
```

```
f.stat.argmin([5, 3, 1, 4])  ## Returns 2 (index of 1)
```

Index of Maximum

EBNF:

```
"f.stat.argmax", "(", argument, ")"
```

```
f.stat.argmax([5, 3, 1, 4])  ## Returns 0 (index of 5)
```

Binary Search The `searchsorted` function finds the index where a value should be inserted to maintain sort order.

Morphology:

EBNF:

```
"f.util.searchsorted", "(", list, ",", (int | real), [ ",", string ], ")"
```

- First argument: sorted list
- Second argument: value to insert
- Optional side: "left" (default) or "right"
- Returns index where value should be inserted
- List must be sorted; behavior is undefined for unsorted lists.

Examples:

```
f.util.searchsorted([1, 2, 3, 4, 5], 3)           #c# Returns 2
f.util.searchsorted([1, 2, 3, 4, 5], 3, "left")    #c# Returns 2
f.util.searchsorted([1, 2, 3, 4, 5], 3, "right")   #c# Returns 3
f.util.searchsorted([1, 2, 4, 5], 3)               #c# Returns 2
f.util.searchsorted([], 5)                         #c# Returns 0
```

Partial Sort The `partition` function rearranges a list so the `k`th element is in its sorted position.

Morphology:

EBNF:

```
"f.util.partition", "(", list, ",", int, ")"
```

- Returns modified list (original unchanged)
- Elements before `k`th are `it`; elements after are `it`
- A `Value Error` is raised if `k` is out of range or list is empty.

Examples:

```
f.util.partition([3, 1, 4, 1, 5, 9, 2, 6], 3)      #c# Returns [1,1,2,3,4,5,6,9]
f.util.partition([5, 3, 1, 4, 2], 0)               #c# Returns [1,2,3,4,5]
f.util.partition([5, 3, 1, 4, 2], 4)               #c# Returns [1,2,3,4,5]
```

Set Intersection The `intersect` function returns sorted list of common elements (with duplicates removed).

Morphology:

EBNF:

```
"f.util.intersect", "(", list, ",", list, ")"
```

Examples:

```
f.util.intersect([1, 2, 3, 4], [3, 4, 5, 6])      #c# Returns [3, 4]
f.util.intersect([1, 1, 2], [1, 2, 2])            #c# Returns [1, 2]
f.util.intersect([1, 2], [3, 4])                  #c# Returns []
```

Set Union The `union` function returns sorted list of unique elements from both lists.

Morphology:

EBNF:

```
"f.util.union", "(", list, ",", list, ")"
```

Examples:

```
f.util.union([1, 2, 3], [3, 4, 5])                #c# Returns [1, 2, 3, 4, 5]
f.util.union([1, 1, 2], [2, 3, 3])                #c# Returns [1, 2, 3]
f.util.union([], [1, 2])                          #c# Returns [1, 2]
```

Set Difference The `setdiff` function returns elements in first list but not in second.
Morphology:

EBNF:

```
"f.util.setdiff", "(", list, ",", list, ")"
```

Examples:

```
f.util.setdiff([1, 2, 3, 4], [3, 4, 5, 6])    ## Returns [1, 2]
f.util.setdiff([1, 2, 3], [1, 2, 3])         ## Returns []
f.util.setdiff([1, 2, 3], [])                ## Returns [1, 2, 3]
```

Is In The `isin` function returns boolean list indicating membership in second list.
Morphology:

EBNF:

```
"f.util.isin", "(", list, ",", list, ")"
```

Examples:

```
f.util.isin([1, 2, 3, 4], [2, 4, 6])         ## Returns [false, true, false, true]
f.util.isin([1, 2, 3], [1, 2, 3])            ## Returns [true, true, true]
f.util.isin([1, 2], [])                      ## Returns [false, false]
```

Is In (Flattened) The `in1d` function tests membership after flattening the first list.
Morphology:

EBNF:

```
"f.util.in1d", "(", list, ",", list, ")"
```

Examples:

```
f.util.in1d([1, 2, 3], [2, 3, 4])            ## Returns [false, true, true]
f.util.in1d([1,2],[3,4], [2,4])              ## Returns [false, true, false, true]
```

Select The `select` function selects values from choices based on conditions.
Morphology:

EBNF:

```
"f.util.select", "(", list[lamba], ",", list, [ ",", "default", "=", data ], ")"
```

- First argument: list of conditions (lambdas returning bool)
- Second argument: list of choices (same length as conditions)
- Optional default: value for elements matching no condition
- A `Value Error` is raised if condition and choice lists differ in length.

Examples:

```
f.util.select(
  [L::($x [int])->{ $x > 0 }, L::($x [int])->{ $x < 0 }],
  [1, -1],
  default=0
) on [1, -2, 3]
## Returns [1, -1, 1]

f.util.select([L::($x [int])->{ $x > 5 }], [100]) on [1, 2, 3]
## Returns [100, 100, 100]

f.util.select([L::($x [int])->{ $x > 10 }], [999], default=0) on [1, 2, 3]
## Returns [0, 0, 0]
```

Compress The `compress` function selects elements where boolean mask is True.
Morphology:

EBNF:

```
"f.util.compress", "(", list[bool], ",", list, ")"
```

- A Value Error is raised if lists differ in length.

Examples:

```
f.util.compress([true, false, true, false], [10, 20, 30, 40]) ## Returns [10, 30]
f.util.compress([true, true, true], [1, 2, 3]) ## Returns [1, 2, 3]
f.util.compress([false, false, false], [1, 2, 3]) ## Returns []
```

Advanced Indexing

Place The `place` function modifies a tensor in-place, placing values where mask is True.
Morphology:

EBNF:

```
"f.util.place", "(", tensor, ",", tensor[bool], ",", tensor, ")"
```

- Modifies first tensor in-place, returns null
- A Value Error is raised if mask and data lengths differ.

Examples:

```
$a [tensor]= |[0, 0, 0, 0, 0]
f.util.place($a, |[true, false, true, false, true], |[10, 20, 30])
## $a is now |[10, 0, 20, 0, 30]

$b [tensor]= |[1, 2, 3, 4, 5]
f.util.place($b, |[false, false, false, false, false], |[99, 99, 99, 99, 99])
## $b is unchanged |[1, 2, 3, 4, 5]
```


Put The `put` function modifies a tensor in-place, placing values at specified indices.
Morphology:

EBNF:

```
"f.util.put", "(", tensor, ",", list[int], ",", tensor, ")"
```

- Modifies first tensor in-place, returns null
- Indices can repeat; last write wins
- A `Value Error` is raised if any index is out of range.

Examples:

```
$a [tensor]= |[0, 0, 0, 0, 0]
f.util.put($a, [0, 2, 4], |[10, 20, 30])
## $a is now |[10, 0, 20, 0, 30]

$b [tensor]= |[1, 2, 3, 4, 5]
f.util.put($b, [1, 1, 3], |[100, 200, 300])
## $b is now |[1, 200, 3, 300, 5]
```

Take The `take` function returns a new tensor with elements at specified indices.
Morphology:

EBNF:

```
"f.util.take", "(", tensor, ",", list[int], ")"
```

- Indices can repeat
- A `Value Error` is raised if any index is out of range.

Examples:

```
f.util.take(|[10, 20, 30, 40, 50], [0, 2, 4])    ## Returns |[10, 30, 50]
f.util.take(|[10, 20, 30, 40, 50], [4, 3, 2, 1]) ## Returns |[50, 40, 30, 20]
f.util.take(|[[1,2],[3,4],[5,6]], [0, 2])      ## Returns |[[1,2],[5,6]]
```

Choose The `choose` function selects elements from choice lists based on indices.
Morphology:

EBNF:

```
"f.util.choose", "(", list[int], ",", list[list], ")"
```

- Element `i` is from choice list at index given by first argument
- A `Value Error` is raised if index is out of range or choice lists differ in length.

Examples:

```
f.util.choose([0, 1, 2, 1], [[10, 11], [20, 21], [30, 31]])
## Returns |[10, 21, 30, 21]

f.util.choose([0, 0, 0], [[1, 2, 3]])
## Returns |[1, 1, 1]
```

Index Mesh The `ix_` function creates open mesh for cross-indexing.

Morphology:

EBNF:

```
"f.util.ix_", "(", list[int], ",", list[int], ")"
```

- Returns [rowIndices, colIndices] as 1D tensors
- Used for selecting sub-matrices with tensor indexing

Examples:

```
$f [tensor]= |[[1, 2, 3], [4, 5, 6], [7, 8, 9]]
$rowIndices, $colIndices = f.util.ix_([0, 2], [1, 2])
$f[$rowIndices, $colIndices]          #c# Returns |[[2, 3], [8, 9]]

$rowIndices, $colIndices = f.util.ix_([0, 1], [0])
$f[$rowIndices, $colIndices]          #c# Returns |[[1], [4]]
```

Diagonal Indices The `diagIndices` function returns indices of main diagonal.

Morphology:

EBNF:

```
"f.util.diagIndices", "(", int, ")"
```

- Returns [rowIndices, colIndices]
- A Value Error is raised if $n \leq 0$.

Examples:

```
f.util.diagIndices(3)          #c# Returns [[0, 1, 2], [0, 1, 2]]

$a [tensor]= |[[1,2,3],[4,5,6],[7,8,9]]
$rows, $cols = f.util.diagIndices(3)
$a[$rows, $cols]               #c# Returns |[1, 5, 9]
```

Lower Triangle Indices The `trilIndices` function returns indices of lower triangle (including diagonal).

Morphology:

EBNF:

```
"f.util.trilIndices", "(", int, ")"
```

- Returns [rowIndices, colIndices]
- A Value Error is raised if $n \leq 0$.

Examples:

```
f.util.trilIndices(3)          #c# Returns [[0,1,1,2,2,2], [0,0,1,0,1,2]]

$a [tensor]= |[[1,2,3],[4,5,6],[7,8,9]]
$rows, $cols = f.util.trilIndices(3)
$a[$rows, $cols]               #c# Returns |[1, 4, 5, 7, 8, 9]
```

Upper Triangle Indices The `triuIndices` function returns indices of upper triangle (including diagonal).

Morphology:

EBNF:

```
"f.util.triuIndices", "(", int, ")"
```

- Returns [rowIndices, colIndices]
- A Value Error is raised if $n \leq 0$.

Examples:

```
f.util.triuIndices(3)      ## Returns [[0,0,0,1,1,2], [0,1,2,1,2,2]]
```

```
$a [tensor]= |[[1,2,3],[4,5,6],[7,8,9]]
```

```
$rows, $cols = f.util.triuIndices(3)
```

```
$a[$rows, $cols]          ## Returns |[1, 2, 3, 5, 6, 9]
```

Shape Functions

Flatten

EBNF:

```
"f.tnsr.flatten", "(", argument, ")"
```

Flattens one level of nesting. Already-flat lists are returned unchanged. Empty lists return [].

```
f.tnsr.flatten([[1,2],[3,4],[5,6]])  ## Returns [1, 2, 3, 4, 5, 6]
f.tnsr.flatten([[1,2,3]])             ## Returns [1, 2, 3]
f.tnsr.flatten([1, 2, 3])             ## Returns [1, 2, 3] (already flat)
f.tnsr.flatten([])                   ## Returns []
```

Shape Returns the shape of a tensor as a list of ints. Empty tensors | [] return [] (0D).

EBNF:

```
"f.tnsr.shape", "(", argument, ")"
```

```
f.tnsr.shape(|[1, 2, 3])              ## Returns [3]
f.tnsr.shape(|[[1,2],[3,4]])          ## Returns [2, 2]
f.tnsr.shape(|[[[1]]])                ## Returns [1, 1, 1]
f.tnsr.shape(|[[[1,2],[3,4]],[[5,6],[7,8]]]]) ## Returns [2, 2, 2, 2]
f.tnsr.shape(|[])                     ## Returns [] (0D)
```

Dimensions Returns the number of dimensions (axes) of a tensor. Empty tensors | [] return 0 (0D).

EBNF:

```
"f.tnsr.dims", "(", argument, ")"
```

```
f.tnsr.dims(|[1, 2, 3])               ## Returns 1
f.tnsr.dims(|[[1,2],[3,4]])           ## Returns 2
f.tnsr.dims(|[[[1]]])                 ## Returns 3
f.tnsr.dims(|[])                      ## Returns 0 (0D)
```

Data Type Returns the element type as a string, or null for empty tensors.

EBNF:

```
"f.tnsr.dtype", "(", argument, ")"
```

```
f.tnsr.dtype([1, 2, 3])          #c# Returns "int"
f.tnsr.dtype([1.0, 2.5])        #c# Returns "real"
f.tnsr.dtype([true, false])     #c# Returns "bool"
f.tnsr.dtype([1, 2.0])          #c# Returns "mixed"
f.tnsr.dtype([])                #c# Returns null
```

Size Returns the total number of elements in a tensor. Empty tensors `[]` return 0.

EBNF:

```
"f.tnsr.size", "(", argument, ")"
```

```
f.tnsr.size([1, 2, 3])          #c# Returns 3
f.tnsr.size([[1,2],[3,4]])      #c# Returns 4
f.tnsr.size([[[1,2],[3,4]],[[5,6],[7,8]]]) #c# Returns 8
f.tnsr.size([])                #c# Returns 0
```

Memory Returns the runtime memory usage of a tensor in bytes. Measured via runtime introspection — no compile-time limit.

EBNF:

```
"f.tnsr.tmem", "(", argument, ")"
```

```
f.tnsr.tmem([1, 2, 3])          #c# Returns 12 (3 int × 4 bytes)
f.tnsr.tmem([1.0, 2.0])        #c# Returns 16 (2 real × 8 bytes)
f.tnsr.tmem([[1,2],[3,4]])      #c# Returns 16 (4 int × 4 bytes)
f.tnsr.tmem([])                #c# Returns 0 (0D)
```

Cumulative Operations

Cumulative Sum Returns the cumulative sum of elements along the given axis.

Morphology:

EBNF:

```
"f.tnsr.csum", "(", argument, [ ",", "axis", "=", int ], ")"
```

- Operands: list or tensor, optional axis (default: last axis)
- Result: list or tensor of cumulative sums
- A **Value Error** is raised if axis is out of bounds.

Examples:

```
f.tnsr.csum([1, 2, 3])          #c# Returns [1, 3, 6]
f.tnsr.csum([1.0, 2.0, 3.0])    #c# Returns [1.0, 3.0, 6.0]
f.tnsr.csum([true, true, false]) #c# Returns [1, 2, 2]
f.tnsr.csum([])                 #c# Returns []
f.tnsr.csum([[1,2],[3,4]], axis=0) #c# Returns [[1,2],[4,6]]
f.tnsr.csum([[1,2],[3,4]], axis=1) #c# Returns [[1,3],[3,7]]
```

Cumulative Product Returns the cumulative product of elements along the given axis.
Morphology:

EBNF:

```
"f.tnsr.cprod", "(", argument, [ ",", "axis", "=", int ], ")"
```

- Operands: list or tensor, optional axis (default: last axis)
- Result: list or tensor of cumulative products
- A Value Error is raised if axis is out of bounds.

Examples:

```
f.tnsr.cprod([1, 2, 3])          ## Returns [1, 2, 6]
f.tnsr.cprod([0, 1, 2])          ## Returns [0, 0, 0]
f.tnsr.cprod([])                 ## Returns []
```

Difference Returns the nth discrete difference along the given axis.
Morphology:

EBNF:

```
"f.tnsr.diff", "(", argument, [ ",", "n", "=", int ], [ ",", "axis", "=", int ], ")"
```

- Operands: list or tensor, optional n (default: 1), optional axis (default: last axis)
- Result: list or tensor with difference applied
- A Value Error is raised if $n < 0$ or axis is out of bounds.

Examples:

```
f.tnsr.diff([1, 2, 4, 7])          ## Returns [1, 2, 3]
f.tnsr.diff([1, 2, 4, 7], n=2)      ## Returns [1, 1]
f.tnsr.diff([1])                   ## Returns [] (single element)
f.tnsr.diff([])                     ## Returns []
f.tnsr.diff([[1,2],[4,7]], axis=1)  ## Returns [[1],[3]]
```

Sort Returns a new sorted list without modifying the original. The sort is stable: equal elements preserve their original relative order. An optional lambda comparator can be provided for custom ordering (see Higher-Order Functions section).

Morphology:

EBNF:

```
"f.util.sort", "(", argument, [ ",", lambda ], ")"
```

- Operands: list of comparable elements (all same type), optional comparator lambda
- Result: new list with elements in sorted order
- A Type Error is raised if elements are of mixed incomparable types.

Examples:

```
f.util.sort([5, 3, 1, 4])          ## Returns [1, 3, 4, 5]
f.util.sort([3.1, 2.7, 1.4])        ## Returns [1.4, 2.7, 3.1]
f.util.sort([5, 1, 3], L::($a [int], $b [int])->{ $b - $a })
## Returns [5, 3, 1] (descending)
```

Argsort Returns a list of indices that would sort the list. The returned indices can be used to reorder another list in the same order without sorting it directly, which is useful when multiple lists share the same sort key (e.g., sorting coordinates by x-value while keeping y-values aligned). The sort is stable: equal elements preserve their original relative order. An optional lambda comparator can be provided for custom ordering (see Higher-Order Functions section).

Morphology:

EBNF:

```
"f.util.argsort", "(", argument, [ ",", lambda ], ")"
```

- Operands: list of comparable elements (all same type), optional comparator lambda
- Result: list of ints (indices into the original list)
- A `Type Error` is raised if elements are of mixed incomparable types.

Examples:

```
f.util.argsort([5, 3, 1, 4])      #c# Returns [2, 1, 3, 0]
f.util.argsort([1, 2, 3])         #c# Returns [0, 1, 2]  (already sorted)
f.util.argsort([3, 2, 1])         #c# Returns [2, 1, 0]  (reverse sorted)
f.util.argsort([])                #c# Returns []
f.util.argsort([5, 3, 1], L::($a [int], $b [int])->{ $b - $a })
#c# Returns [0, 2, 1]  (descending)
f.util.argsort([3, 1, 3])         #c# Returns [1, 0, 2]  (stable)
```

Reverse Returns a new reversed list without modifying the original.

Morphology:

EBNF:

```
"f.util.reverse", "(", argument, ")"
```

- Operands: list or tensor
- Result: new list/tensor with elements in reverse order

Examples:

```
f.util.reverse([1, 2, 3])         #c# Returns [3, 2, 1]
f.util.reverse([[1,2],[3,4]])     #c# Returns [[3,4],[1,2]]
```

Unique Returns a new list with duplicate elements removed. The order of first occurrence is preserved.

Morphology:

EBNF:

```
"f.util.unique", "(", argument, ")"
```

- Operands: list
- Result: new list with unique elements in order of first appearance

Examples:

```
f.util.unique([1, 2, 2, 3, 3, 3])    ## Returns [1, 2, 3]
f.util.unique([3, 1, 2, 1])          ## Returns [3, 1, 2]
```

Linear Algebra Functions

Edge Cases – Linear Algebra Functions

- A `Type Error` is raised if the argument is not an `tensor` (matrix).
- A `Value Error` is raised for `f.linalg.det`, `f.linalg.inv`, `f.linalg.eig`, `f.linalg.eigvec`, `f.linalg.lu`, `f.linalg.qr`, `f.linalg.trace`, `f.linalg.cond` if the matrix is not square.
- A `Value Error` is raised for `f.linalg.cond` if the matrix is singular.
- A `Value Error` is raised for `f.tnsr.normalize` if the vector has zero magnitude.

Determinant

EBNF:

```
"f.linalg.det", "(", argument, ")"
```

```
f.linalg.det([[1,2],[3,4]])    ## Returns -2.0
f.linalg.det([[2,0],[0,3]])    ## Returns 6.0
```

Matrix Inverse

EBNF:

```
"f.linalg.inv", "(", argument, ")"
```

Returns the inverse matrix as a list of lists. A `Division by Zero Error` is raised if the matrix is singular.

```
f.linalg.inv([[1,2],[3,4]])    ## Returns [[-2.0, 1.0],[1.5, -0.5]]
```

Eigenvalues

EBNF:

```
"f.linalg.eig", "(", argument, ")"
```

Returns a list of eigenvalues.

```
f.linalg.eig([[2,0],[0,3]])    ## Returns [2, 3]
f.linalg.eig([[1,2],[2,1]])    ## Returns [3, -1]
```

Eigenvectors

EBNF:

```
"f.linalg.eigvec", "(", argument, ")"
```

Returns a list of eigenvectors (one per eigenvalue, as rows).

```
f.linalg.eigvec([[2,0],[0,3]])    ## Returns [[1,0],[0,1]]
```

Singular Value Decomposition

EBNF:

```
"f.linalg.svd", "(", argument, ")"
```

Returns a list of 3 lists: [U, S, V] where U and V are orthogonal matrices and S is the list of singular values. Supports non-square matrices: for an $m \times n$ input, U is $m \times m$, S has $\min(m, n)$ values, and V is $n \times n$.

```
f.linalg.svd([[1,0],[0,2]])  
    ## Returns [[[1,0],[0,1]], [1, 2], [[1,0],[0,1]]]
```

LU Decomposition

EBNF:

```
"f.linalg.lu", "(", argument, ")"
```

Returns a list of 2 lists: [L, U] where L is lower triangular and U is upper triangular. Supports non-square matrices.

```
f.linalg.lu([[2,1],[4,3]])    ## Returns L and U such that L .* U = original
```

QR Decomposition

EBNF:

```
"f.linalg.qr", "(", argument, ")"
```

Returns a list of 2 lists: [Q, R] where Q is orthogonal and R is upper triangular. Supports non-square matrices.

```
f.linalg.qr([[1,1],[1,-1]])    ## Returns Q and R such that Q .* R = original
```

Cholesky Decomposition

EBNF:

```
"f.linalg.chol", "(", argument, ")"
```

Returns the lower triangular matrix L such that $L \cdot L.T = \text{original}$.

Edge Cases

- A `Type Error` is raised if the argument is not a list of lists (tensor).
- A `Value Error` is raised if the matrix is not square.
- A `Value Error` is raised if the matrix is not symmetric.
- A `Value Error` is raised if the matrix is not positive definite.

```
f.linalg.chol([[4,2],[2,3]])    ## Returns [[2,0],[1,1.7321]]  
f.linalg.chol([[1,2],[3,4]])    ## Value Error (not symmetric)  
f.linalg.chol([[1,2,3]])        ## Value Error (not square)
```


Trace

EBNF:

```
"f.linalg.trace", "(", argument, ")"
```

```
f.linalg.trace([[1,2],[3,4]])    ## Returns 5
```

```
f.linalg.trace([[2,0],[0,3]])    ## Returns 5
```

Matrix Rank

EBNF:

```
"f.linalg.rank", "(", argument, ")"
```

```
f.linalg.rank([[1,2],[3,4]])    ## Returns 2
```

```
f.linalg.rank([[1,2],[2,4]])    ## Returns 1 (rows are linearly dependent)
```

Condition Number

EBNF:

```
"f.linalg.cond", "(", argument, ")"
```

Returns the ratio of the largest to smallest singular value. A high condition number indicates numerical instability.

```
f.linalg.cond([[1,0],[0,1]])    ## Returns 1.0 (well-conditioned)
```

```
f.linalg.cond([[1,0],[0,1e-10]]) ## Returns 1e10 (ill-conditioned)
```

Pseudo-Inverse

EBNF:

```
"f.linalg.pinv", "(", argument, ")"
```

Returns the Moore–Penrose pseudo-inverse. Works for non-square or singular matrices.

```
f.linalg.pinv([[1,2],[3,4]])    ## Returns [[-2, 1],[1.5, -0.5]]
```

Least Squares The `lstsq` function computes the least-squares solution to $A\mathbf{x} = \mathbf{b}$.
Morphology:

EBNF:

```
"f.linalg.lstsq", "(", tensor, ",", tensor, ")"
```

- Returns $[x, \text{residuals}, \text{rank}, \text{sv}]$ where:
 - x : solution vector
 - residuals: sum of squared residuals (empty if rank $\geq n$ or overdetermined)
 - rank: rank of A
 - sv: singular values of A
- Works for overdetermined (more equations than unknowns) and underdetermined systems.

Examples:

```
f.linalg.lstsq([[1,1],[1,2],[1,3]], [1,2,2])  
## Returns [0.667, 0.5, [], 2, [3.172, 0.265]]
```

```
f.linalg.lstsq([[1,0],[0,1]], [3,4])  
## Returns [3, 4, [], 2, [1.0, 1.0]]
```

Cross Product The `cross` function computes the vector cross product.

Morphology:

EBNF:

```
"f.linalg.cross", "(", tensor, ",", tensor, ")"
```

- Both arguments must be 3-element vectors
- A `Value Error` is raised if vectors don't have exactly 3 elements.

Examples:

```
f.linalg.cross([1, 0, 0], [0, 1, 0]) ## Returns [0, 0, 1]  
f.linalg.cross([1, 2, 3], [4, 5, 6]) ## Returns [-3, 6, -3]  
f.linalg.cross([1, 0, 0], [1, 0, 0]) ## Returns [0, 0, 0]
```

Outer Product The `outer` function computes the outer product of two vectors.

Morphology:

EBNF:

```
"f.linalg.outer", "(", tensor, ",", tensor, ")"
```

- Flattens inputs to 1D vectors
- Returns 2D tensor of shape (m, n) where $m = \text{len}(a)$, $n = \text{len}(b)$

Examples:

```
f.linalg.outer([1, 2, 3], [4, 5]) ## Returns [[4,5],[8,10],[12,15]]  
f.linalg.outer([1, 0], [0, 1]) ## Returns [[0,1],[0,0]]
```

Kronecker Product The `kron` function computes the Kronecker product.

Morphology:

EBNF:

```
"f.linalg.kron", "(", tensor, ",", tensor, ")"
```

- Returns 2D tensor

Examples:

```
f.linalg.kron([[1,2],[3,4]], [[0,1],[1,0]]) ## Returns [[0,1,0,2],[1,0,2,0],[0,3,0,4],[3,0,4,0]]  
f.linalg.kron([[1,0],[0,1]], [[1,2],[3,4]]) ## Returns [[1,2,0,0],[3,4,0,0],[0,0,1,2],[0,0,2,1]]
```

Tensor Dot The `tensor_dot` function computes tensor contraction over specified axes.
Morphology:

EBNF:

```
"f.linalg.tensor_dot", "(", tensor, ",", tensor, ",", list[int], ")"
```

- Third argument: `[axesA, axesB]` — axes to contract over
- Result: tensor with contracted axes removed

Examples:

```
f.linalg.tensor_dot([[1,2],[3,4]], [[5,6],[7,8]], [[0],[0]])  
## Returns [[19, 22], [43, 50]]
```

```
f.linalg.tensor_dot([[1,2],[3,4]], [[5,6],[7,8]], [[1],[0]])  
## Returns [[17, 23], [39, 53]]
```

Hermitian Eigendecomposition The `eigh` function computes eigendecomposition optimized for Hermitian (symmetric) matrices.

Morphology:

EBNF:

```
"f.linalg.eigh", "(", argument, ")"
```

- Returns `[eigenvalues, eigenvectors]`
- Eigenvalues are returned in ascending order
- More efficient and numerically stable than `eig` for symmetric matrices
- A `Value Error` is raised if the matrix is not symmetric.

Examples:

```
f.linalg.eigh([[2,1],[1,2]])    ## Returns [1, 3], [[-0.707, 0.707], [0.707, 0.707]]  
f.linalg.eigh([[4,0],[0,9]])    ## Returns [4, 9], [[1,0],[0,1]]
```

Sign-Log Determinant The `slogdet` function returns the sign and natural log of the determinant.

Morphology:

EBNF:

```
"f.linalg.slogdet", "(", argument, ")"
```

- Returns `[sign, logabsdet]` where:
 - sign: -1, 0, or 1
 - logabsdet: natural log of absolute value of determinant
- More numerically stable than computing `det()` directly for large matrices

Examples:

```
f.linalg.slogdet([[1,2],[3,4]])    ## Returns [-1, 0.693]  
f.linalg.slogdet([[2,0],[0,3]])    ## Returns [1, 1.792]  
f.linalg.slogdet([[1,0],[0,0]])    ## Returns [0, -inf]
```

Multi-Matrix Multiply The `multiDot` function performs chained matrix multiplication.
Morphology:

EBNF:

```
"f.linalg.multiDot", "(", list[tensor], ")"
```

- Optimizes multiplication order for efficiency
- Requires at least 2 matrices
- A `Value Error` is raised if fewer than 2 matrices provided or dimensions incompatible.

Examples:

```
f.linalg.multiDot([[[1,2],[3,4]], [[5,6],[7,8]]])  
## Returns [[19, 22], [43, 50]]
```

```
f.linalg.multiDot([[[1,2],[3,4]], [[5,6],[7,8]], [[1,0],[0,1]]])  
## Returns [[19, 22], [43, 50]]
```

Statistical Functions

Histogram

EBNF:

```
"f.stat.histogram", "(", argument, ",", int, ")"
```

Returns a list of 2 lists: `[edges, counts]` where `edges` has `n + 1` values and `counts` has `n` values.

Edge Cases – Histogram

- A `Type Error` is raised if the first argument is not a list.
- A `Value Error` is raised if `n <= 0`.
- A `Value Error` is raised if the list is empty.

```
f.stat.histogram([1, 2, 3, 4, 5], 3)  
## Returns [[1.0, 2.333, 3.667, 5.0], [2, 1, 2]]
```

Weighted Average The `weightedMean` function computes the weighted average: $\frac{\sum(v_i \cdot w_i)}{\sum w_i}$.
Morphology:

EBNF:

```
"f.stat.weightedMean", "(", list, ",", list, ")"
```

- First argument: values (list of int, real, or complex).
- Second argument: weights (list of int or real, same length as values).
- Result: `real` or `complex`.
- Negative weights are allowed.

Examples:

```
f.stat.weightedMean([10, 20, 30], [1, 2, 3])      ## Returns 23.333...
f.stat.weightedMean([1.0, 2.0, 3.0], [0.5, 0.5, 1.0]) ## Returns 2.25
f.stat.weightedMean([100], [1])                  ## Returns 100
f.stat.weightedMean([10, 20], [1, -1])            ## Returns -10
```

Edge Cases:

- A **Type Error** is raised if either argument is not a list.
- A **Type Error** is raised if values contain non-numeric types.
- A **Value Error** is raised if lists are empty.
- A **Value Error** is raised if lists have different lengths.
- A **Value Error** is raised if sum of weights is zero.

Linear System Solver

The `solve` function solves the linear system $A\mathbf{x} = \mathbf{b}$ and returns $\mathbf{x}$.
Morphology:

EBNF:

```
"f.linalg.solve", "(", tensor, ",", tensor, ")"
```

- Operands: A (tensor, square coefficient matrix), b (tensor, right-hand side vector or matrix)
- Result: tensor $\mathbf{x}$ such that $A\mathbf{x} = \mathbf{b}$
- A must be square and non-singular (determinant $\neq 0$)
- A **Division by Zero Error** is raised if A is singular
- A **Value Error** is raised if A and b have incompatible dimensions

Examples:

```
f.linalg.solve([[2,1],[5,7]], [11,13])      ## Returns [7.333, -3.667]
f.linalg.solve([[1,0],[0,1]], [3,4])        ## Returns [3, 4]
```

11.1.7 Integer Sequence Functions

Functions in the `f.seq` namespace compute mathematical integer sequences. All functions use 0-based indexing and return a single value at index n . Negative indices raise a **Value Error**.

Fibonacci

Computes the n -th Fibonacci number: $F(0) = 0$, $F(1) = 1$, $F(n) = F(n - 1) + F(n - 2)$.
Morphology:

EBNF:

```
"f.seq.fibonacci", "(", int, ")"
```

- Operands: non-negative `int` ($n \geq 0$).
- Result: `int`.
- A `Value Error` is raised if $n < 0$.

Examples:

```
f.seq.fibonacci(0)    #c# Returns 0
f.seq.fibonacci(1)    #c# Returns 1
f.seq.fibonacci(10)   #c# Returns 55
f.seq.fibonacci(20)   #c# Returns 6765
```

Lucas

Computes the n -th Lucas number: $L(0) = 2$, $L(1) = 1$, $L(n) = L(n - 1) + L(n - 2)$.
Morphology:

EBNF:

```
"f.seq.lucas", "(", int, ")"
```

- Operands: non-negative `int` ($n \geq 0$).
- Result: `int`.
- A `Value Error` is raised if $n < 0$.

Examples:

```
f.seq.lucas(0)        #c# Returns 2
f.seq.lucas(1)        #c# Returns 1
f.seq.lucas(10)       #c# Returns 123
f.seq.lucas(20)       #c# Returns 15127
```

Pell

Computes the n -th Pell number: $P(0) = 0$, $P(1) = 1$, $P(n) = 2 \cdot P(n - 1) + P(n - 2)$.
Morphology:

EBNF:

```
"f.seq.pell", "(", int, ")"
```

- Operands: non-negative `int` ($n \geq 0$).
- Result: `int`.

- A Value Error is raised if $n < 0$.

Examples:

```
f.seq.pell(0)      #c# Returns 0
f.seq.pell(1)      #c# Returns 1
f.seq.pell(10)     #c# Returns 2378
f.seq.pell(20)     #c# Returns 195025
```

Tribonacci

Computes the n -th Tribonacci number: $T(0) = 0$, $T(1) = 0$, $T(2) = 1$, $T(n) = T(n-1) + T(n-2) + T(n-3)$.

Morphology:

EBNF:

```
"f.seq.tribonacci", "(", int, ")"
```

- Operands: non-negative int ($n \geq 0$).
- Result: int.
- A Value Error is raised if $n < 0$.

Examples:

```
f.seq.tribonacci(0)  #c# Returns 0
f.seq.tribonacci(1)  #c# Returns 0
f.seq.tribonacci(2)  #c# Returns 1
f.seq.tribonacci(10) #c# Returns 44
f.seq.tribonacci(20) #c# Returns 35890
```

Catalan

Computes the n -th Catalan number: $C(0) = 1$, $C(n) = \sum_{i=0}^{n-1} C(i) \cdot C(n-1-i)$.

Morphology:

EBNF:

```
"f.seq.catalan", "(", int, ")"
```

- Operands: non-negative int ($n \geq 0$).
- Result: int.
- A Value Error is raised if $n < 0$.

Examples:

```
f.seq.catalan(0)     #c# Returns 1
f.seq.catalan(1)     #c# Returns 1
f.seq.catalan(10)    #c# Returns 16796
f.seq.catalan(20)    #c# Returns 6564120420
```

Bell

Computes the n -th Bell number (total number of partitions of a set of n elements): $B(0) = 1$.
Morphology:

EBNF:

```
"f.seq.bell", "(", int, ")"
```

- Operands: non-negative `int` ($n \geq 0$).
- Result: `int`.
- A `Value Error` is raised if $n < 0$.

Examples:

```
f.seq.bell(0)    #c# Returns 1
f.seq.bell(1)    #c# Returns 1
f.seq.bell(10)   #c# Returns 115975
f.seq.bell(20)   #c# Returns 51724158235372
```

Bernoulli

Computes the n -th Bernoulli number: $B(0) = 1$, $B(1) = -1/2$, $B(n) = 0$ for odd $n > 1$.
Morphology:

EBNF:

```
"f.seq.bernoulli", "(", int, ")"
```

- Operands: non-negative `int` ($n \geq 0$).
- Result: `real`.
- A `Value Error` is raised if $n < 0$.
- A warning is issued for odd $n > 1$ (function returns 0).

Examples:

```
f.seq.bernoulli(0)    #c# Returns 1.0
f.seq.bernoulli(1)    #c# Returns -0.5
f.seq.bernoulli(2)    #c# Returns 0.16667
f.seq.bernoulli(10)   #c# Returns 0.0757576
```

Euler Number

Computes the n -th Euler number (alternating permutations): $E(0) = 1$.
Morphology:

EBNF:

```
"f.seq.eulerNumber", "(", int, ")"
```

- Operands: non-negative `int` ($n \geq 0$).

- Result: `int`.
- A `Value Error` is raised if $n < 0$.

Examples:

```
f.seq.eulerNumber(0)    #c# Returns 1
f.seq.eulerNumber(1)    #c# Returns 1
f.seq.eulerNumber(2)    #c# Returns 0
f.seq.eulerNumber(10)   #c# Returns -50521
```

Stirling Numbers of the First Kind

Computes the unsigned Stirling number of the first kind $|s(n, k)|$ (number of permutations of n elements with exactly k cycles).

Morphology:

EBNF:

```
"f.seq.stirling1", "(", int, ",", int, ")"
```

- Operands: non-negative `int` ($n \geq 0, k \geq 0$).
- Result: `int`.
- A `Value Error` is raised if $n < 0$ or $k < 0$.
- Returns 0 if $k > n$ (with a warning).
- $|s(0, 0)| = 1, |s(n, 0)| = 0$ for $n > 0$.

Examples:

```
f.seq.stirling1(0, 0)    #c# Returns 1
f.seq.stirling1(5, 0)    #c# Returns 0
f.seq.stirling1(5, 1)    #c# Returns 24
f.seq.stirling1(5, 5)    #c# Returns 1
f.seq.stirling1(5, 3)    #c# Returns 35
```

Stirling Numbers of the Second Kind

Computes the Stirling number of the second kind $S(n, k)$ (number of ways to partition a set of n elements into k non-empty subsets).

Morphology:

EBNF:

```
"f.seq.stirling2", "(", int, ",", int, ")"
```

- Operands: non-negative `int` ($n \geq 0, k \geq 0$).
- Result: `int`.
- A `Value Error` is raised if $n < 0$ or $k < 0$.
- Returns 0 if $k > n$ (with a warning).

- $S(0, 0) = 1$, $S(n, 0) = 0$ for $n > 0$.

Examples:

```
f.seq.stirling2(0, 0)    #c# Returns 1
f.seq.stirling2(5, 0)    #c# Returns 0
f.seq.stirling2(5, 1)    #c# Returns 1
f.seq.stirling2(5, 5)    #c# Returns 1
f.seq.stirling2(5, 3)    #c# Returns 25
```

Partition Number

Computes the number of integer partitions of n : $P(0) = 1$.
Morphology:

EBNF:

```
"f.seq.partitionNumber", "(", int, ")"
```

- Operands: non-negative int ($n \geq 0$).
- Result: int.
- A Value Error is raised if $n < 0$.

Examples:

```
f.seq.partitionNumber(0)    #c# Returns 1
f.seq.partitionNumber(1)    #c# Returns 1
f.seq.partitionNumber(10)   #c# Returns 42
f.seq.partitionNumber(20)   #c# Returns 627
```

Harmonic Number

Computes the n -th harmonic number: $H(0) = 0$, $H(n) = 1 + 1/2 + \dots + 1/n$.
Morphology:

EBNF:

```
"f.seq.harmonicNumber", "(", int, ")"
```

- Operands: non-negative int ($n \geq 0$).
- Result: real.
- A Value Error is raised if $n < 0$.

Examples:

```
f.seq.harmonicNumber(0)    #c# Returns 0.0
f.seq.harmonicNumber(1)    #c# Returns 1.0
f.seq.harmonicNumber(10)   #c# Returns 2.92897
f.seq.harmonicNumber(100)  #c# Returns 5.18738
```

11.1.8 Higher-Order Functions

Higher-order functions take function values (lambdas or function references) as arguments. They operate on lists and tensors. See Section 8.1.13 for lambda syntax and Section 5 for the function data type.

Edge Cases – Higher-Order Functions

- A **Type Error** is raised if the second argument is not a function value (lambda or function reference).
- A **Type Error** is raised if the first argument is not a list.
- A **Runtime Error** is raised if the function value is `null`.
- `f.util.reduce` raises a **Value Error** if the list is empty and no initial value is provided (third argument).

Map

The `map` function applies a lambda to each element of a list and returns a new list with the results. The input list is not modified.

Morphology

EBNF:

```
"f.util.map", "(", list, ",", lambda, ")"
```

Type Overloading

- `map(list[T], L::($x [T])->{...}) → list[U]` (T and U can be different types)
- Works on any list: flat, nested, mixed types
- The lambda parameter type must match the element type

Examples

```
f.util.map([1, 2, 3], L::($x [int])->{ $x * 2 })  
## Returns [2, 4, 6]
```

```
f.util.map([1, 2, 3], L::($x [int])->{ $x > 2 })  
## Returns [False, False, True] (type change: int -> bool)
```

```
f.util.map(["a", "bb", "ccc"], L::($x [string])->{ f.util.len($x) })  
## Returns [1, 2, 3] (type change: string -> int)
```

```
f.util.map([1, 2, 3], L::($x [int])->{  
    $y [int] = $x * 10  
    $y + 1  
})
```

#c# Returns [11, 21, 31] (multi-line lambda)

```
f.util.map([], L::($x [int])->{ $x * 2 })
#c# Returns [] (empty list)
```

Error Conditions

- Type mismatch: lambda parameter type does not match element type raises a Runtime Error

Filter

The `filter` function returns a new list containing only elements for which the lambda returns True. The input list is not modified.

Morphology

EBNF:

```
"f.util.filter", "(", list, ",", lambda, ")"
```

Type Overloading

- `filter(list[T], L::($x [T])->{bool}) → list[T]` (return type is the same as input)
- The lambda must return bool

Examples

```
f.util.filter([1, 2, 3, 4, 5], L::($x [int])->{ $x % 2 == 0 })
#c# Returns [2, 4]
```

```
f.util.filter([1, 2, 3, 4, 5], L::($x [int])->{ $x > 10 })
#c# Returns [] (no matches)
```

```
f.util.filter(["a", "bb", "ccc", "dd"], L::($x [string])->{ f.util.len($x) > 2 })
#c# Returns ["ccc"]
```

```
f.util.filter([-1, -2, 3, -4, 5], L::($x [int])->{ $x > 0 })
#c# Returns [3, 5]
```

```
f.util.filter([1, 2, 3], L::($x [int])->{ $x > 5 })
#c# Returns [] (empty result)
```

```
f.util.filter([], L::($x [int])->{ $x > 0 })
#c# Returns [] (empty input)
```

Error Conditions

- Type mismatch: lambda parameter type does not match element type raises a Runtime Error
- Lambda does not return `bool` raises a Type Error

Reduce

The `reduce` function folds a list to a single value by applying a binary lambda cumulatively. The lambda receives an accumulator and the current element. The `init` value is always required.

Morphology

EBNF:

```
"f.util.reduce", "(", list, ",", lambda, ",", expression, ")"
```

Type Overloading

- `reduce(list[T], L::($acc [U], $x [T])->{U}, init[U]) → U` (accumulator type U can differ from element type T)
- The `init` value type must match the accumulator type

Examples

```
#c# Sum integers
```

```
f.util.reduce([1, 2, 3, 4], L::($acc [int], $x [int])->{ $acc + $x }, 0)
```

```
#c# Returns 10
```

```
#c# Product integers
```

```
f.util.reduce([2, 3, 4], L::($acc [int], $x [int])->{ $acc * $x }, 1)
```

```
#c# Returns 24
```

```
#c# Find maximum
```

```
f.util.reduce([3, 1, 4, 1, 5], L::($acc [int], $x [int])->{  
    ($acc > $x)?->($acc)!($x)  
}, 0)
```

```
#c# Returns 5
```

```
#c# Reduce with string concatenation
```

```
f.util.reduce(["a", "b", "c"],  
    L::($acc [string], $x [string])->{ $acc + $x }, "")
```

```
#c# Returns "abc"
```

```
#c# Reduce on empty list returns init
```

```
f.util.reduce([], L::($acc [int], $x [int])->{ $acc + $x }, 42)
```

```
#c# Returns 42
```

```
#c# Flatten with reduce (using concat)
f.util.reduce([[1,2],[3,4]],
  L::($acc [list], $x [list])->{ f.list.concat($acc, $x) }, [])
#c# Returns [1, 2, 3, 4]
```

Error Conditions

- Type mismatch: lambda parameter types do not match accumulator/element types raises a Runtime Error
- Lambda does not return a type coercible to the accumulator type raises a Type Error

Where

The **where** function returns a list of **indices** (ints) for which the lambda returns **True**. Indices are 0-based. When two additional lists are provided, it returns **values** from the first list where the condition is **True**, and from the second list where **False**.

Morphology

EBNF:

```
"f.util.where", "(", list, ",", lambda, [ ",", list, ",", list ], ")"
```

Type Overloading

- `where(list[T], L::($x [T])->{bool}) → list[int]`
- `where(list[T], L::($x [T])->{bool}, list[A], list[B]) → list[A|B]`
- The lambda must return `bool`
- In value-selection mode, both value lists must have the same length as the input list.

Examples — Index Mode

```
f.util.where([-1, 3, -2, 5], L::($x [int])->{ $x > 0 })
#c# Returns [1, 3]
```

```
f.util.where([10, 20, 30], L::($x [int])->{ $x >= 20 })
#c# Returns [1, 2]
```

```
f.util.where([1, 2, 3], L::($x [int])->{ $x > 10 })
#c# Returns [] (no matches)
```

```
f.util.where(["a", "bb", "ccc"], L::($x [string])->{ f.util.len($x) >= 2 })
#c# Returns [1, 2]
```

```
f.util.where([], L::($x [int])->{ $x > 0 })
#c# Returns [] (empty input)
```

Examples — Value Selection Mode

```
f.util.where([1, 2, 3], L::($x [int])->{ $x > 1 },
  |[10, 20, 30], |[100, 200, 300])
#c# Returns |[100, 20, 30]
```

```
f.util.where([1, 2, 3], L::($x [int])->{ $x > 1 },
  |["a", "b", "c"], |["x", "y", "z"])
#c# Returns |["x", "b", "c"]
```

```
f.util.where([1, 2, 3], L::($x [int])->{ $x > 5 },
  |[10, 20, 30], |[100, 200, 300])
#c# Returns |[100, 200, 300] (all False)
```

```
f.util.where([], L::($x [int])->{ $x > 0 }, [], [])
#c# Returns []
```

Error Conditions

- Type mismatch: lambda parameter type does not match element type raises a Runtime Error
- Lambda does not return bool raises a Type Error
- In value-selection mode, lists of different lengths raises a Value Error

Nonzero

The `nonzero` function returns a list of indices where elements are non-zero. For boolean lists, returns indices where `True`. For complex lists, nonzero if `real` $\neq 0$ OR `imag` $\neq 0$.

Morphology:

EBNF:

```
"f.util.nonzero", "(" , list , ")"
```

- Operands: list of bool, int, real, or complex.
- Result: `list[int]` of 0-based indices.

Examples:

```
f.util.nonzero([0, 5, 0, 3, 0])      #c# Returns [1, 3]
f.util.nonzero([1, 2, 3, 4])         #c# Returns [0, 1, 2, 3]
f.util.nonzero([0, 0, 0])            #c# Returns []
f.util.nonzero([])                   #c# Returns []
f.util.nonzero([0, 1+0i, 0, 2+3i])   #c# Returns [1, 3]
f.util.nonzero([true, false, true])  #c# Returns [0, 2]
```

Edge Cases:

- A `Type Error` is raised if the argument is not a list.

Count

The `count` function returns the number of elements for which the lambda returns `True`.

Morphology

EBNF:

```
"f.util.count", "(", list, ",", lambda, ")"
```

Type Overloading

- `count(list[T], L::($x [T])->{bool}) → int`
- The lambda must return `bool`

Examples

```
f.util.count([1, 2, 3, 4, 5], L::($x [int])->{ $x > 3 })  
## Returns 2
```

```
f.util.count([True, False, True, True], L::($x [bool])->{ $x })  
## Returns 3
```

```
f.util.count([1, 2, 3], L::($x [int])->{ $x > 10 })  
## Returns 0
```

```
f.util.count([], L::($x [int])->{ $x > 0 })  
## Returns 0
```

Error Conditions

- Type mismatch: lambda parameter type does not match element type raises a Runtime Error
- Lambda does not return `bool` raises a Type Error

Find

The `find` function returns the first element for which the lambda returns `True`. If no element matches, a Value Error is raised.

Morphology

EBNF:

```
"f.util.find", "(", list, ",", lambda, ")"
```

Type Overloading

- `find(list[T], L::($x [T])->{bool}) → T` (return type is the same as element type)
- The lambda must return `bool`

Examples

```
f.util.find([1, 2, 3, 4, 5], L::($x [int])->{ $x > 2 })  
## Returns 3
```

```
f.util.find([1, 2, 3, 4, 5], L::($x [int])->{ $x == 4 })  
## Returns 4
```

```
f.util.find(["a", "bb", "ccc"], L::($x [string])->{ f.util.len($x) > 1 })  
## Returns "bb"
```

```
f.util.find([1, 2, 3, 4, 5], L::($x [int])->{ $x > 10 })  
## Value Error: no matching element
```

```
f.util.find([], L::($x [int])->{ $x > 0 })  
## Value Error: no matching element
```

Error Conditions

- Value Error: no element matches the lambda condition
- Type mismatch: lambda parameter type does not match element type raises a Runtime Error
- Lambda does not return bool raises a Type Error

Sort with Comparator

The `sort` function returns a new sorted list. Without a comparator, default sort order is ascending. An optional lambda comparator can be provided for custom ordering (see Sort paragraph above for EBNF).

Comparator Lambda The comparator lambda receives two elements (`$a` and `$b`) and must return a numeric value:

- **Negative** value (e.g., `$a - $b`): `$a` comes before `$b`
- **Zero**: order unchanged (stable sort)
- **Positive** value (e.g., `$b - $a`): `$b` comes before `$a`

This is equivalent to the `cmp` convention used in many languages. The simplest ascending comparator is `$a - $b`, and descending is `$b - $a`.

Type Overloading

- `sort(list[T]) → list[T]` (default ascending)
- `sort(list[T], L::($a [T], $b [T])->{int}) → list[T]` (custom comparator, must return int or real)

Examples

```
#c# Default sort (ascending)
f.util.sort([5, 1, 3])
#c# Returns [1, 3, 5]

#c# Ascending with comparator
f.util.sort([5, 1, 3], L::($a [int], $b [int])->{ $a - $b })
#c# Returns [1, 3, 5]

#c# Descending with comparator
f.util.sort([5, 1, 3], L::($a [int], $b [int])->{ $b - $a })
#c# Returns [5, 3, 1]

#c# Sort strings by length
f.util.sort(["bb", "a", "ccc"],
  L::($a [string], $b [string])->{ f.util.len($a) - f.util.len($b) })
#c# Returns ["a", "bb", "ccc"]

#c# Sort reals ascending
f.util.sort([3.14, 1.41, 2.72], L::($a [real], $b [real])->{ $a - $b })
#c# Returns [1.41, 2.72, 3.14]

#c# Sort with nested sort key (sort by absolute value)
f.util.sort([-5, 3, -1, 4], L::($a [int], $b [int])->{ f.math.abs($a) - f.math.abs($b) })
#c# Returns [-1, 3, 4, -5]

#c# Sort preserves original (returns new list)
$list [list]= [5, 3, 1]
$sorted [list]= f.util.sort($list)
#c# $list is still [5, 3, 1], $sorted is [1, 3, 5]
```

Error Conditions

- Type mismatch: comparator parameter types do not match element types raises a Runtime Error
- Comparator does not return a numeric type raises a Type Error

Arity

The `arity` function returns the number of parameters a function value accepts. This is useful for introspection and validation before calling a function value stored in a variable.

EBNF:

```
"f.util.arity", "(", func_value, ")"
```

Type Overloading

- `arity(func) → int`
- Works on any function value: built-in references, user-defined references, and lambdas

Examples

```
f.util.arity(f.math.sin)           #c# Returns 1
f.util.arity(f.math.pow)           #c# Returns 2
f.util.arity(f.math.convert)       #c# Returns 4
f.util.arity(L::($x [int])->{ $x * 2 }) #c# Returns 1
f.util.arity(L::($a [int], $b [int])->{ $a + $b }) #c# Returns 2
```

Error Conditions

- Type Error: argument is not a function value
- Runtime Error: argument is null

Assert

The `assert` statement checks a boolean condition and raises an error if the condition is `False`. It is available both as a keyword statement and as a built-in function `f.util.assert`.

Keyword Form

EBNF:

```
assertion = "assert", boolean_expression, ":", string_type
           , [";", error_type]
```

```
error_type = "Assertion Error" | "aerr"
            | "Type Error"      | "terr"
            | "Value Error"     | "verr"
            | "Division by Zero Error" | "0div"
            | "Runtime Error"   | "rerr"
            | "Permission Error" | "perm"
```

Function Form

EBNF:

```
"f.util.assert", "(", boolean_expression, ","
                  , string_type, [";", error_type], ")"
```

Semantics

- The boolean expression is evaluated
- If `True`: execution continues (no-op)
- If `False`: raises the specified error type with the message string

- Default error type (when omitted): **Assertion Error**
- The `error_type` accepts both full names and short aliases

Error Type Aliases

Full Name	Alias
"Assertion Error"	"aerr"
"Type Error"	"terr"
"Value Error"	"verr"
"Division by Zero Error"	"0div"
"Runtime Error"	"rerr"

Examples

```
# Keyword form
assert $x > 0 :: "x must be positive"
assert $x > 0 :: "x must be positive" ; "terr"
assert $freq > 0 :: "freq must be positive" ; "0div"

# Function form
f.util.assert($x > 0, "x must be positive")
f.util.assert($x > 0, "x must be positive", "terr")
f.util.assert($freq > 0, "freq must be positive", "0div")

# Short aliases
assert $val != 0 :: "val must be nonzero" ; "aerr"
assert $len > 0 :: "list must not be empty" ; "rerr"
```

Error Conditions

- **Assertion Error** (default): condition is **False**
- The error message is the string provided as the second argument
- The error type string must be one of the 5 valid types or their aliases; an invalid error type string raises a **Value Error**

Try/Except

The `try/except` construct allows structured error handling. Code in the `try` block is executed; if an error occurs, matching `otherwise` blocks catch it. An optional `except` block runs always, regardless of success or failure.

Syntax

EBNF:

```
try_except = "try", ":", code_block
            , { "otherwise", [error_type], ":", code_block }
```

```

        , ["except", "::", code_block]

error_type = "Assertion Error" | "aerr"
            | "Type Error"       | "terr"
            | "Value Error"      | "verr"
            | "Division by Zero Error" | "Odiv"
            | "Runtime Error"    | "rerr"
            | "Permission Error" | "perm"

```

Reuses the `error_type` rule defined for `assert`.

Semantics

- Execute the `try` block sequentially
- If no error: skip all `otherwise` blocks, execute `except` if present, continue
- If error: find the first matching `otherwise` (by error type or catch-all), execute its block, then execute `except` if present, continue
- If error occurs and no matching `otherwise`: the error propagates (halts as normal), but `except` still runs first
- Bare `otherwise` (no error type) catches all errors
- Multiple `otherwise` blocks are checked in order; first match wins

Variable Scope Variables declared in the `try` block are visible in all `otherwise` and `except` blocks. Variables declared in an `otherwise` block are visible in the `except` block.

Examples

```

# Basic try/except
try ::{
    $result [real]= f.math.sqrt($x)
} otherwise "Value Error" ::{
    $result [real]= 0.0
}

# Multiple error types
try ::{
    $inv [real]= 1.0 / $x
} otherwise "Division by Zero Error" ::{
    f.util.print("division by zero")
    $inv [real]= 0.0
} otherwise "Value Error" ::{
    f.util.print("invalid value")
    $inv [real]= 0.0
}

```

```

# Catch-all otherwise
try ::{
    $data [list]= f.list.fromFile("input.csv")
} otherwise ::{
    f.util.print("failed to load, using defaults")
    $data [list]= []
}

# With except (always runs)
try ::{
    $result [real]= f.math.sqrt($x)
} otherwise "Value Error" ::{
    $result [real]= 0.0
} except ::{
    f.util.print("attempted sqrt calculation")
}

# Nested try
try ::{
    try ::{
        $val [real]= 1.0 / $x
    } otherwise "Division by Zero Error" ::{
        $val [real]= 0.0
    }
    $result [real]= f.math.sqrt($val)
} otherwise "Value Error" ::{
    $result [real]= 0.0
}

```

Error Conditions

- The `error_type` string must be one of the 5 valid types or their aliases; an invalid error type string raises a `Value Error`
- If an error occurs and no matching `otherwise` exists, the error propagates after the `except` block (if present)

11.1.9 Signal Processing Functions

Functions for frequency-domain analysis of signals.

Fast Fourier Transform

The `fft` function computes the Discrete Fourier Transform of a real-valued signal.

EBNF:

```
"f.ee.fft", "(", list, ")"
```

Type Overloading

- `fft(list[int]) → tensor[complex]`
- `fft(list[real]) → tensor[complex]`
- Output length equals input length
- Works on any signal length (not restricted to powers of 2)

Examples

```
#c# FFT of a simple signal
f.ee.fft([1, 0, 1, 0])    #c# Returns |[2+0i, 0+0i, 2+0i, 0+0i]

#c# FFT of a single impulse
f.ee.fft([1, 0, 0, 0])    #c# Returns |[1+0i, 1+0i, 1+0i, 1+0i]

#c# FFT of a DC signal (constant value)
f.ee.fft([5, 5, 5, 5])    #c# Returns |[20+0i, 0+0i, 0+0i, 0+0i]

#c# Extract magnitude spectrum
$freq [tensor]= f.ee.fft([1, 0, 1, 0])
$mags [tensor]= f.math.abs($freq)
#c# $mags = |[2.0, 0.0, 2.0, 0.0]
```

Error Conditions

- Empty list raises a Value Error

Inverse Fast Fourier Transform

The `ifft` function computes the Inverse Discrete Fourier Transform, reconstructing a real signal from complex frequency components.

EBNF:

```
"f.ee.ifft", "(", list, ")"
```

Type Overloading

- `ifft(tensor[complex]) → tensor[real]`
- Output length equals input length
- Imaginary parts are discarded (reconstructed signal is real)

Examples

```
#c# Round-trip: fft then ifft
$signal [tensor]= |[1, 0, 1, 0]
$freq [tensor]= f.ee.fft($signal)
$reconstructed [tensor]= f.ee.ifft($freq)
#c# $reconstructed is approximately |[1, 0, 1, 0]

#c# ifft of DC component
f.ee.ifft(|[4+0i, 0+0i, 0+0i, 0+0i])    #c# Returns |[1, 1, 1, 1]

#c# ifft of impulse in frequency domain
f.ee.ifft(|[1+0i, 1+0i, 1+0i, 1+0i])    #c# Returns |[1, 0, 0, 0]
```

Error Conditions

- Empty list raises a Value Error

FFT Utilities

FFT Shift Shifts the zero-frequency component to the center of the spectrum. Useful for visualizing FFT output where DC component should be in the middle.

EBNF:

```
"f.ee.fftshift", "(", list, ")"
```

- Operands: list or tensor
- Result: shifted list or tensor of same length

```
f.ee.fftshift(|[0, 1, 2, 3])          #c# Returns |[2, 3, 0, 1]
f.ee.fftshift(|[0, 1, 2, 3, 4])       #c# Returns |[3, 4, 0, 1, 2]
f.ee.fftshift(|[5])                   #c# Returns |[5]
f.ee.fftshift(|[])                    #c# Returns |[]
```

Inverse FFT Shift Undoes fftshift, moving zero-frequency component back to beginning.

EBNF:

```
"f.ee.ifftshift", "(", list, ")"
```

- Operands: list or tensor
- Result: unshifted list or tensor of same length

```
f.ee.ifftshift(|[2, 3, 0, 1])         #c# Returns |[0, 1, 2, 3]
f.ee.ifftshift(|[3, 4, 0, 1, 2])      #c# Returns |[0, 1, 2, 3, 4]
f.ee.ifftshift(|[5])                   #c# Returns |[5]
f.ee.ifftshift(|[])                    #c# Returns |[]
```


FFT Frequencies Returns the frequency bins for an FFT output. Pair with `fftshift` to get frequency axis for centered spectrum.

EBNF:

```
"f.ee.fftfreq", "(", int, [",", "d", "=", real], ")"
```

- Operands: `n` (number of samples), optional `d` (sample spacing, default 1.0)
- Result: tensor of `n` frequency bins
- A `Value Error` is raised if `n < 0`.

```
f.ee.fftfreq(4)           ## Returns | [0.0, 0.25, -0.5, -0.25]
f.ee.fftfreq(4, d=0.001) ## Returns | [0.0, 250.0, -500.0, -250.0]
f.ee.fftfreq(5)           ## Returns | [0.0, 0.2, 0.4, -0.4, -0.2]
f.ee.fftfreq(1)           ## Returns | [0.0]
f.ee.fftfreq(0)           ## Returns | []
```

Real FFT

Computes the one-dimensional Discrete Fourier Transform for *real-valued* input, returning only the non-negative frequency terms. For real input, the DFT exhibits Hermitian symmetry: $X[k] = X[n - k]^*$. The `rfft` function exploits this by computing only the first $(n/2) + 1$ points (for even n), discarding the redundant negative-frequency mirror. This yields approximately 33% speedup over `f.ee.fft` for real signals.

Output Length For input of length n :

- If n is even: output length is $(n/2) + 1$ (includes DC and Nyquist)
- If n is odd: output length is $(n + 1)/2$ (includes DC, no Nyquist)

Comparison to `fft`

For real input `[0, 1, 0, 0]`:

```
f.ee.fft(|[0, 1, 0, 0])      ## Returns | [1+0j, 0-1j, -1+0j, 0+1j]   (4 points)
f.ee.rfft(|[0, 1, 0, 0])    ## Returns | [1+0j, 0-1j, -1+0j]       (3 points)
```

`Output[0]` (DC) and `Output[n/2]` (Nyquist) are real-valued

`Output[k]` for $0 < k < n/2$ are complex

Morphology:

EBNF:

```
"f.ee.rfft", "(", list, [",", int], ")"
```

- Operands: input signal (list of `int` or `real`), optional output length n (`int`).
- Result: `list[complex]` of length $(n/2) + 1$ (even n) or $(n + 1)/2$ (odd n).
- If n is given, the input is zero-padded (with zeros) or truncated to length n before transforming.

- If $n < \text{input length}$, input is truncated. If $n > \text{input length}$, input is zero-padded.
- A `Type Error` is raised if any input element is `complex`.

Examples

```
f.ee.rfft(|[0, 1, 0, 0])      ## Returns |[1+0j, 0-1j, -1+0j]
f.ee.rfft(|[1, 2, 3])        ## Returns |[6+0j, -1.5+0.866j]
f.ee.rfft(|[1])              ## Returns |[1+0j]
f.ee.rfft(|[])               ## Returns |[]
```

Zero-Padding and Truncation

```
f.ee.rfft(|[0, 1, 0, 0], 8)    ## Zero-pads to 8, returns 5 complex points
f.ee.rfft(|[0, 1, 0, 0, 0, 0], 4) ## Truncates to 4, returns 3 complex points
```

Typical Workflow

```
# Analyze frequency content of a real signal
$signal [tensor]= |[0.0, 1.0, 0.0, -1.0, 0.0, 1.0, 0.0, -1.0]
$$ [tensor]= f.ee.rfft($signal)
$freqs [tensor]= f.ee.rfftfreq(8, d=0.001)
$power [tensor]= f.math.abs($$) ^ 2
```

Inverse Real FFT

Computes the inverse of `f.ee.rfft`. Recovers a real-valued signal from its non-negative frequency representation. The input must be in the form returned by `rfft`: real-valued DC component followed by complex positive-frequency terms in order of increasing frequency.

Output Length For input of length m :

- Default output length: $2 \times (m - 1)$
- If n is specified, output has exactly n points
- For n output points, $(n/2) + 1$ input points are needed (even n)

Morphology:

EBNF:

```
"f.ee.irfft", "(", list, [",", int], ")"
```

- Operands: `list[complex]` (Hermitian-symmetric form from `rfft`), optional output length n (`int`).
- Result: `list[real]` of length n .
- Roundtrip identity: `f.ee.irfft(f.ee.rfft(a), f.list.len(a)) == a` (within numerical precision).

Examples

```
f.ee.irfft(|[1+0j, 0-1j, -1+0j])    #c# Returns |[0.0, 1.0, 0.0, 0.0]
f.ee.irfft(|[6+0j, -1.5+0.866j])    #c# Returns |[1.0, 2.0, 3.0]
f.ee.irfft(|[4+0j])                 #c# Returns |[4.0]
```

Roundtrip Verification

```
$orig [tensor]= |[1.0, 2.0, 3.0, 4.0]
$fwd [tensor]= f.ee.rfft($orig)
$inv [tensor]= f.ee.irfft($fwd, 4)
# $inv == |[1.0, 2.0, 3.0, 4.0] (within numerical precision)
```

Resampling via Fourier Interpolation

```
# Resample from 4 points to 8 points (zero-padding in frequency domain)
$$ [tensor]= f.ee.rfft(|[1.0, 2.0, 3.0, 4.0])
$resampled [tensor]= f.ee.irfft($$, 8)
# $resampled has 8 points (interpolated from original 4)
```

Real FFT Frequencies

Returns the frequency bins for use with `f.ee.rfft`. Unlike `f.ee.fftfreq` which returns both positive and negative frequencies, `rfftfreq` returns only the non-negative frequencies (DC through Nyquist). Pair with `f.ee.fftshift` to center the spectrum.

Relationship to `fftfreq`

```
# fftfreq returns both positive and negative frequencies (n points)
f.ee.fftfreq(4)                #c# Returns |[0.0, 0.25, -0.5, -0.25]

# rfftfreq returns only non-negative frequencies (n/2+1 points)
f.ee.rfftfreq(4)                #c# Returns |[0.0, 0.25, 0.5]
```

Morphology:

EBNF:

```
"f.ee.rfftfreq", "(", int, [",", "d", "=", real], ")"
```

- Operands: `n` (original signal length), optional `d` (sample spacing, default 1.0).
- Result: `list[real]` of $(n/2) + 1$ frequency bins (only non-negative frequencies).
- Units: frequencies are in cycles per unit of `d`. For `d` in seconds, result is in Hz.

Examples

```
f.ee.rfftfreq(4)                #c# Returns |[0.0, 0.25, 0.5]
f.ee.rfftfreq(4, d=0.001)       #c# Returns |[0.0, 250.0, 500.0]
f.ee.rfftfreq(5)                #c# Returns |[0.0, 0.2, 0.4]
f.ee.rfftfreq(1)                #c# Returns |[0.0]
f.ee.rfftfreq(0)                #c# Returns |[]
```

Complete Example

```
# Analyze a 1kHz sine wave sampled at 8kHz
$fs [int]= 8000
$N [int]= 64
$t [tensor]= |[f.math.range(0, $N)] / $fs
$signal [tensor]= f.trig.sin(2 * f.math.pi * 1000 * $t)
$$ [tensor]= f.ee.rfft($signal)
$freqs [tensor]= f.ee.rfftfreq($N, d=1.0/$fs)
$mag [tensor]= f.math.abs($$)
# $mag peaks at index where $freqs == 1000.0
```

11.1.10 Debug Functions

Debug functions (`dbg.*`) provide runtime inspection, tracing, and profiling. All debug functions are no-ops when `syst.flag.debug_mode` is `False` (default). Enable with:

```
syst.flag.debug_mode True
```

Debug Print Function

The `dbg.print` function outputs a variable's value and type. The destination is controlled by the `mode` parameter or the global `debug_output` flag.

EBNF:

```
"dbg.print", "(", variable, [",", string_type], ")"
```

Modes

- "console" (default) – Output to console
- "log" – Output to debug log file
- "both" – Output to console and debug log file

Examples

```
dbg.print($voltage)
dbg.print($voltage, "log")
dbg.print($voltage, "both")
```

Breakpoint Function

The `dbg.breakpoint` function pauses execution and prints the current state (variables, stack). An optional condition can be provided.

EBNF:

```
"dbg.breakpoint", "(", [boolean_expression], ")"
```

Examples

```
debug.breakpoint()  
debug.breakpoint($voltage > 5.0)  
debug.breakpoint($count == 10)
```

If a condition is provided, execution only pauses when the condition is **True**. When `debug_mode` is **False**, breakpoints are no-ops.

Graph Dump Function

The `debug.graph` function dumps the full electrical graph state (parts, nets, connections, pins).

EBNF:

```
"debug.graph", "(", ")"
```

Examples

```
debug.graph()
```

Output includes all parts with models and parameters, all nets with tags, all connections, and all pin assignments.

Resource Monitor Function

The `debug.resources` function prints current resource usage: memory, CPU time elapsed, and object counts (parts, nets, connections, pins).

EBNF:

```
"debug.resources", "(", ")"
```

Examples

```
debug.resources()
```

Execution Trace Function

The `debug.trace` function logs an execution trace entry with a timestamp and optional message. Trace entries are written to the debug log file.

EBNF:

```
"debug.trace", "(", [string_type], ")"
```

Examples

```
debug.trace("Entering main loop")  
debug.trace("Before constraint solve")  
debug.trace()
```

Profiling Function

The `dbg.profile` function measures the execution time of a function call. It returns the elapsed time in seconds and optionally prints the result.

EBNF:

```
"dbg.profile", "(", function_call, [",", string_type], ")"
```

Examples

```
dbg.profile(f.util.sort($data))
dbg.profile(f.linalg.inv($matrix), "log")
dbg.profile(f.ee.fft($signal), "both")
```

Output includes the function name, execution time in seconds, and the return value.

Memory Inspection Function

The `dbg.memory` function reports detailed memory usage: total heap, live objects, and per-type breakdown.

EBNF:

```
"dbg.memory", "(", ")"
```

Examples

```
dbg.memory()
```

Output includes total allocated bytes, live object count by type, and peak memory usage.

CPU Usage Function

The `dbg.cpu` function returns the current CPU usage as a percentage. Unlike `dbg.resources` which reports cumulative CPU time, this reports instantaneous usage.

EBNF:

```
"dbg.cpu", "(", ")"
```

Examples

```
dbg.cpu()           #c# Returns 42.5 (percent)
```

Variable Watch Function

The `dbg.watch` function registers a variable for watching. Changes to the watched variable are logged to the debug log file at each breakpoint.

EBNF:

```
"dbg.watch", "(", variable, ")"
```

Examples

```
dbg.watch($voltage)
dbg.watch($current)
dbg.breakpoint($count == 10)    #c# Logs all watched variable changes
```

Multiple variables can be watched. Watch entries persist until the program terminates.

Call Stack Function

The `dbg.stack` function prints the current call stack, showing function names and line numbers.

EBNF:

```
"dbg.stack", "(", ")"
```

Examples

```
dbg.stack()
```

Output shows the chain of nested function calls from the current execution point back to the entry point.

Variable Inspector Function

The `dbg.inspect` function outputs detailed information about a variable: its type, value, memory footprint, and metadata. Richer than `dbg.print`.

EBNF:

```
"dbg.inspect", "(", variable, ")"
```

Examples

```
dbg.inspect($matrix)
dbg.inspect($signal)
```

Output includes:

- Type and dimensions (for lists/tensors)
- Element types and count
- Memory footprint in bytes
- Value summary (truncated for large structures)

11.1.11 System Functions

System functions (`f.sys.*`) provide runtime introspection and configuration. Unlike `syst.flag` assignments (which are declarative and appear before the program body), system functions are callable at runtime.

Get Configuration Function

The `f.sys.getConfig` function retrieves a named configuration value. Configuration keys are strings referencing runtime settings.

Morphology:

```
"f.sys.getConfig", "(", string, ")"
```

Returns: The value associated with the given key (type depends on the key).

Raises: `Runtime Error` if the key is not recognized.

```
$f max_iter = f.sys.getConfig("max_loop_iter")
$f dbg = f.sys.getConfig("debug_mode")
```

Set Configuration Function

The `f.sys.setConfig` function assigns a value to a named configuration key at runtime.

Morphology:

```
"f.sys.setConfig", "(", string, ",", data, ")"
```

Returns: `None`.

Raises: `Runtime Error` if the key is read-only or not recognized.

```
f.sys.setConfig("max_loop_iter", 5000)
f.sys.setConfig("debug_mode", True)
```

Environment Variable Function

The `f.sys.environment` function retrieves the value of an environment variable by name.

Morphology:

```
"f.sys.environment", "(", string, ")"
```

Returns: `string` — the value of the environment variable.

Raises: `Runtime Error` if the variable is not set.

```
$f home = f.sys.environment("HOME")
$f path = f.sys.environment("PATH")
```

Set Environment Function

The `f.sys.setEnvironment` function sets an environment variable for the current runtime session.

Morphology:

```
"f.sys.setEnvironment", "(", string, ",", string, ")"
```

Returns: `None`.

```
f.sys.setEnvironment("EDA_LIB", "/opt/eda/lib")
```


Version Function

The `f.sys.version` function returns the language or runtime version string.
Morphology:

```
"f.sys.version", "(", ")"
```

Returns: `string` — version identifier (e.g. "0.1.0").

```
$f ver = f.sys.version()
```

Features Function

The `f.sys.features` function returns a list of supported runtime feature identifiers.
Morphology:

```
"f.sys.features", "(", ")"
```

Returns: `list[string]` — feature identifiers enabled in the current runtime.

```
$f feats = f.sys.features()
```

11.1.12 System Variables

System variables (`$$sys_*`) provide read-only access to hardware state and configuration. Unlike `syst.flag` assignments (declarative) or `f.sys.*` functions (callable), system variables are accessed like regular variables but cannot be assigned.

EBNF:

```
sys_var = "$$", sys_var_name
```

```
sys_var_name = "sys_mem_SHA512"  
              | "sys_temp_", id  
              | "sys_temp_available_sensors"
```

- All `$$sys_*` variables are **read-only**
- Assignment (`$$sys_* = val`) raises a `Type Error`
- Compound assignment (`$$sys_* += val`) raises a `Type Error`
- Lazy binding (`$x :Lt= $$sys_*`) *is allowed*
- Available in all scopes (main body, functions, lambdas)
- Returns null if hardware doesn't support the feature
- Unknown sensor names return null (no error)

Memory Hash Variable

The `$$sys_mem_SHA512` variable returns a SHA-512 hash of the current system memory state. This is a **live value** that may change between reads as memory state changes.

- Type: `string`
- Returns: SHA-512 hash hex string
- Returns `null` if memory introspection is unavailable

Examples:

```
$hash [string]= $$sys_mem_SHA512
f.util.print(f.str.str("Memory hash: ", $hash))
```

```
# Compare memory state between two points
$hash1 [string]= $$sys_mem_SHA512
# ... some code ...
$hash2 [string]= $$sys_mem_SHA512
if $hash1 != $hash2 ::{
    f.util.print("Memory state changed")
}
```

Temperature Sensor Variables

Temperature sensor variables provide read-only access to hardware temperature readings. Sensor names are validated at runtime; unknown names return `null`.

- Type: `real` (temperature in °C)
- Returns `null` if sensor is unavailable
- Common sensor names: `cpu`, `gpu`, `ambient`
- Any identifier is valid (unknown names return `null`)

Examples:

```
$cpu_temp [real]= $$sys_temp_cpu
if $cpu_temp != null ::{
    f.util.print(f.str.str("CPU temp: ", $cpu_temp))
}
```

```
$gpu_temp [real]= $$sys_temp_gpu
if $gpu_temp != null ::{
    f.util.print(f.str.str("GPU temp: ", $gpu_temp))
}
```

```
# Unknown sensor returns null (no error)
$unknown [real]= $$sys_temp_foo
f.util.print($unknown)    #c# prints "null"
```

Available Sensors Variable

The `$$sys_temp_available_sensors` variable returns a list of available temperature sensor names.

- Type: `list[string]`
- Returns `[]` (empty list) if no sensors are available

Examples:

```
$sensors [list]= $$sys_temp_available_sensors
for $s E $sensors ::{
    f.util.print(f.str.str("Sensor: ", $s))
}
```

```
# Check if a specific sensor is available
$sensors [list]= $$sys_temp_available_sensors
if "cpu" E $sensors ::{
    $temp [real]= $$sys_temp_cpu
    f.util.print(f.str.str("CPU: ", $temp))
}
```

Comparison to Other System Access

- `syst.flag.*` — Declarative flags, set before program body, configure runtime behavior
- `f.sys.*` — Callable functions, return runtime information, can be used in expressions
- `$$sys_*` — Read-only variables, access hardware state, live values

11.1.13 Time Functions

Time functions (`f.time.*`) provide timestamps, sleep, and named timers. Timers are useful for profiling solver performance or measuring execution time of code sections.

Epoch Nanoseconds Function

The `f.time.now` function returns the current time as nanoseconds since the Unix epoch (1970-01-01T00:00:00Z).

Morphology:

EBNF:

```
"f.time.now", "(", ")"
```

- Result: `int` — nanoseconds since epoch.

Examples:

```
$f t = f.time.now()
#c# 1753036200000000000 (varies)
```

ISO 8601 Timestamp Function

The `f.time.nowISO` function returns the current time as an ISO 8601 string. Convenience wrapper for `f.time.nsToISO8601(f.time.now())`.

Morphology:

EBNF:

```
"f.time.nowISO", "(", ")"
```

- Result: `string` — ISO 8601 formatted timestamp (e.g. "2025-07-20T14:30:00Z").

Examples:

```
$f ts = f.time.nowISO()
#c# "2025-07-20T14:30:00Z" (varies)
```

Nanoseconds to ISO 8601 Function

The `f.time.nsToISO8601` function converts a nanosecond timestamp to an ISO 8601 string. Morphology:

EBNF:

```
"f.time.nsToISO8601", "(", int, ")"
```

- Operands: `int` — nanoseconds since epoch.
- Result: `string` — ISO 8601 formatted timestamp.
- A `Value Error` is raised if the argument is negative.

Examples:

```
f.time.nsToISO8601(1753036200000000000) #c# "2025-07-20T14:30:00Z"
f.time.nsToISO8601(f.time.now())         #c# "2025-07-20T14:30:00Z" (current time)
```

Sleep Function

The `f.time.sleep` function pauses execution for the given number of milliseconds. Morphology:

EBNF:

```
"f.time.sleep", "(", ( int | real ), ")"
```

- Operands: `int` or `real` — milliseconds to sleep.
- Result: `None`.
- A `Value Error` is raised if the argument is negative.

Examples:

```
f.time.sleep(100)    #c# Pauses for 100 ms
f.time.sleep(0.5)    #c# Pauses for 0.5 ms
```

Timer Start Function

The `f.time.timerStart` function starts or resets a named timer. If no name is given, the default timer is used.

Morphology:

EBNF:

```
"f.time.timerStart", "(", [string], ")"
```

- Operands: optional `string` — timer name (default: `"default"`).
- Result: `None`.

Examples:

```
f.time.timerStart()           ## Starts default timer
f.time.timerStart("placement") ## Starts timer named "placement"
f.time.timerStart("routing")   ## Starts timer named "routing"
```

Timer Stop Function

The `f.time.timerStop` function stops a named timer.

Morphology:

EBNF:

```
"f.time.timerStop", "(", [string], ")"
```

- Operands: optional `string` — timer name (default: `"default"`).
- Result: `None`.
- A `Runtime Error` is raised if the timer has not been started.

Examples:

```
f.time.timerStop()           ## Stops default timer
f.time.timerStop("placement") ## Stops timer named "placement"
```

Timer Read Function

The `f.time.timerRead` function returns the elapsed time in milliseconds since the named timer was started.

Morphology:

EBNF:

```
"f.time.timerRead", "(", [string], ")"
```

- Operands: optional `string` — timer name (default: `"default"`).
- Result: `real` — elapsed milliseconds.
- A `Runtime Error` is raised if the timer has not been started.

Examples:

```

f.time.timerStart("placement")
# ... work ...
f.time.timerStop("placement")
$f elapsed = f.time.timerRead("placement")
#c# 1234.56 (varies)

# Multiple timers are independent
f.time.timerStart("a")
f.time.timerStart("b")
# ... work ...
f.time.timerStop("a")
f.time.timerRead("a")    #c# 500.0 (varies)
f.time.timerRead("b")    #c# Runtime Error (not stopped yet)

```

11.1.14 Random Number Functions

Random number functions (`f.rng.*`) provide pseudo-random number generation for Monte Carlo analysis, randomized test vectors, and solver seeding. Use `f.rng.seed` for reproducible results.

Seed Function

The `f.rng.seed` function sets the random number generator seed. Calling with the same seed produces identical sequences.

Morphology:

EBNF:

```
"f.rng.seed", "(", int, ")"
```

- Operands: `int` — seed value.
- Result: `None`.

Examples:

```
f.rng.seed(42)
```

Random Function

The `f.rng.random` function returns a random real in $[0, 1)$.

Morphology:

EBNF:

```
"f.rng.random", "(", ")"
```

- Result: `real` — value in $[0, 1)$.

Examples:

```

f.rng.seed(42)
f.rng.random()          #c# 0.37454 (reproducible with seed 42)

```

Random Integer Function

The `f.rng.randInt` function returns a random integer in `[min, max]` (inclusive both ends).
Morphology:

EBNF:

```
"f.rng.randInt", "(", int, ",", int, ")"
```

- Operands: `min (int)`, `max (int)`.
- Result: `int` — value in `[min, max]`.
- A Value Error is raised if `min > max`.

Examples:

```
f.rng.seed(42)
$f v = f.rng.randInt(1, 100)    #c# 73 (reproducible with seed 42)
```

Random Real Function

The `f.rng.randReal` function returns a random real in `[min, max)` (half-open interval).
Morphology:

EBNF:

```
"f.rng.randReal", "(", real, ",", real, ")"
```

- Operands: `min (real)`, `max (real)`.
- Result: `real` — value in `[min, max)`.
- A Value Error is raised if `min > max`.

Examples:

```
f.rng.seed(42)
$f r = f.rng.randReal(0.0, 10.0)  #c# 3.7454 (reproducible with seed 42)
```

Uniform Function

The `f.rng.uniform` function returns a random real in `[min, max]` (closed interval, inclusive both ends).
Morphology:

EBNF:

```
"f.rng.uniform", "(", real, ",", real, ")"
```

- Operands: `min (real)`, `max (real)`.
- Result: `real` — value in `[min, max]`.
- A Value Error is raised if `min > max`.

Examples:

```
f.rng.seed(42)
$f u = f.rng.uniform(0.0, 1.0)    #c# 0.37454 (reproducible with seed 42)
```

Normal Distribution Function

The `f.rng.normal` function returns a Gaussian-distributed random sample.
Morphology:

EBNF:

```
"f.rng.normal", "(", [real], [real], ")"
```

- Operands: optional `mean` (`real`, default 0.0), optional `stddev` (`real`, default 1.0).
- Result: `real`.
- A Value Error is raised if `stddev < 0`.

Examples:

```
f.rng.seed(42)
$f n = f.rng.normal()           #c# -0.5823 (mean=0, stddev=1)
$f n = f.rng.normal(100.0, 15.0) #c# 91.234 (mean=100, stddev=15)
```

Choice Function

The `f.rng.choice` function returns a random element from a list.
Morphology:

EBNF:

```
"f.rng.choice", "(", list, ")"
```

- Operands: `list` (non-empty).
- Result: element from the list.
- A Value Error is raised if the list is empty.

Examples:

```
f.rng.seed(42)
$f c = f.rng.choice(["R", "C", "L"]) #c# "C" (reproducible with seed 42)
```

Shuffle Function

The `f.rng.shuffle` function returns a shuffled copy of a list. The original list is unchanged.
Morphology:

EBNF:

```
"f.rng.shuffle", "(", list, ")"
```

- Operands: `list`.
- Result: `list` — shuffled copy.

Examples:

```
f.rng.seed(42)
$f s = f.rng.shuffle([1, 2, 3, 4, 5]) #c# [3, 1, 5, 2, 4]
```


Random Boolean Function

The `f.rng.bool` function returns a random boolean with 50/50 probability.

Morphology:

EBNF:

```
"f.rng.bool", "(", ")"
```

- Result: `bool` — True or False.

Examples:

```
f.rng.seed(42)
```

```
$f b = f.rng.bool()    #c# True (reproducible with seed 42)
```

Cryptographically Secure Random Function

The `f.rng.csprng` function returns a cryptographically secure pseudo-random integer of the specified bit width. Unlike other `f.rng.*` functions, this is not affected by `f.rng.seed` and produces non-deterministic output.

Morphology:

EBNF:

```
"f.rng.csprng", "(", int, ")"
```

- Operands: `int` — bit width (must be > 0).
- Result: `int` — random integer with the specified number of bits.
- A Value Error is raised if the argument is ≤ 0 .

Examples:

```
$f uid = f.rng.csprng(128)    #c# 2894839274893728472983748923748923 (varies, 128-bit)
```

```
$f key = f.rng.csprng(256)    #c# (varies, 256-bit)
```

Random Prime Function

The `f.rng.randomPrime` function returns a random prime number in `[min, max]`. Useful for cryptographic key generation.

Morphology:

EBNF:

```
"f.rng.randomPrime", "(", int, ",", int, ")"
```

- Operands: `min` (`int`, must be ≥ 2), `max` (`int`).
- Result: `int` — random prime in `[min, max]`.
- A Value Error is raised if `min < 2` or if no prime exists in the range.

Examples:

```
f.rng.seed(42)
```

```
$f p = f.rng.randomPrime(100, 200)    #c# 173 (reproducible with seed 42)
```

```
$f p = f.rng.randomPrime(2, 10)       #c# 5 (one of: 2, 3, 5, 7)
```

11.1.15 Hash Functions

Hash functions (`f.hash.*`) provide integrity checking, tamper detection, deduplication, and data fingerprinting. Useful for PCB design verification, supply chain integrity, and secure hardware validation.

General Hash Function

The `f.hash.hash` function computes a hash value using the algorithm specified by `syst.flag.default_hash` (default: "sha512").

Morphology:

EBNF:

```
"f.hash.hash", "(", ( string | list[int] ), [",", ( "hex" | "bytes" | "int" ) ], ")"
```

- Operands: `string` or `list[int]` (raw bytes), optional output format ("hex" (default), "bytes", "int").
- Result: `string` (hex), `list[int]` (bytes), or `int` depending on format.

Examples:

```
f.hash.hash("Hello World")           ## "a591a6d40bf420404a011733cfb7b190d62c65bf0b"
f.hash.hash("Hello World", "hex")      ## same as above
f.hash.hash("Hello World", "int")      ## 10102030405060708091011... (integer)
f.hash.hash("Hello World", "bytes")    ## [165, 145, 166, 212, ...]
```

Hash Bytes Function

The `f.hash.hashBytes` function computes a hash of raw byte data.

Morphology:

EBNF:

```
"f.hash.hashBytes", "(", list[int], ")"
```

- Operands: `list[int]` — byte values (0–255).
- Result: `string` — hex-encoded hash.

Examples:

```
f.hash.hashBytes([0x48, 0x65, 0x6c, 0x6c, 0x6f]) ## same hash as "Hello"
```

FNV-1a Hash Function

The `f.hash.fnv1a` function computes a non-cryptographic FNV-1a hash. Fast, good distribution.

Morphology:

EBNF:

```
"f.hash.fnv1a", "(", ( string | list[int] ), [",", ( "int" | "bytes" | "int" ) ], ")"
```

- Operands: `string` or `list[int]`, optional output format ("int" (default), "hex", "bytes").

- Result: `int` (default), `string` (hex), or `list[int]` (bytes).

Examples:

```
f.hash.fnv1a("Hello World")           ## 2550413245 (int, default)
f.hash.fnv1a("Hello World", "hex")     ## "982414e5" (hex string)
f.hash.fnv1a("Hello World", "bytes")   ## [152, 36, 20, 229]
```

MurmurHash Function

The `f.hash.murmurHash` function computes a non-cryptographic MurmurHash. Fast, good distribution for hash tables.

Morphology:

EBNF:

```
"f.hash.murmurHash", "(", ( string | list[int] ), [",", ( "hex" | "bytes" | "int" ) ], "
```

- Operands: `string` or `list[int]`, optional output format ("`int`" (default), "`hex`", "`bytes`").
- Result: `int` (default), `string` (hex), or `list[int]` (bytes).

Examples:

```
f.hash.murmurHash("Hello World")       ## (int, varies)
f.hash.murmurHash("Hello World", "hex") ## (hex string, varies)
f.hash.murmurHash("Hello World", "bytes") ## (byte array, varies)
```

xxHash Function

The `f.hash.xxHash` function computes a non-cryptographic xxHash. Extremely fast.

Morphology:

EBNF:

```
"f.hash.xxHash", "(", ( string | list[int] ), [",", ( "hex" | "bytes" | "int" ) ], ")"
```

- Operands: `string` or `list[int]`, optional output format ("`int`" (default), "`hex`", "`bytes`").
- Result: `int` (default), `string` (hex), or `list[int]` (bytes).

Examples:

```
f.hash.xxHash("Hello World")           ## (int, varies)
f.hash.xxHash("Hello World", "hex")     ## (hex string, varies)
f.hash.xxHash("Hello World", "bytes")   ## (byte array, varies)
```

CRC-32 Checksum Function

The `f.hash.crc32` function computes a CRC-32 checksum. Common in hardware data integrity verification.

Morphology:

EBNF:

```
"f.hash.crc32", "(", ( string | list[int] ), [",", ( "hex" | "bytes" | "int" ) ], ")"
```

- Operands: `string` or `list[int]`, optional output format ("`int`" (default), "`hex`", "`bytes`").
- Result: `int` (32-bit unsigned, default), `string` (hex), or `list[int]` (bytes).

Examples:

<code>f.hash.crc32("Hello World")</code>	<code>#c# 3603590121</code> (int, default)
<code>f.hash.crc32("Hello World", "hex")</code>	<code>#c# "d5663949"</code> (hex string)
<code>f.hash.crc32("Hello World", "bytes")</code>	<code>#c# [213, 102, 57, 73]</code>

Adler-32 Checksum Function

The `f.hash.adler32` function computes an Adler-32 checksum. Faster than CRC-32, weaker error detection.

Morphology:

EBNF:

```
"f.hash.adler32", "(", ( string | list[int] ), [",", ( "hex" | "bytes" | "int" ) ], ")"
```

- Operands: `string` or `list[int]`, optional output format ("`int`" (default), "`hex`", "`bytes`").
- Result: `int` (32-bit unsigned, default), `string` (hex), or `list[int]` (bytes).

Examples:

<code>f.hash.adler32("Hello World")</code>	<code>#c# 466698496</code> (int, default)
<code>f.hash.adler32("Hello World", "hex")</code>	<code>#c# "1bd42d50"</code> (hex string)
<code>f.hash.adler32("Hello World", "bytes")</code>	<code>#c# [27, 212, 45, 80]</code>

MD5 Hash Function

The `f.hash.md5` function computes an MD5 digest. Legacy — use for compatibility only, not for security.

Morphology:

EBNF:

```
"f.hash.md5", "(", ( string | list[int] ), [",", ( "hex" | "bytes" | "int" ) ], ")"
```

- Operands: `string` or `list[int]`, optional output format ("`hex`" (default), "`bytes`", "`int`").
- Result: `string` (hex, default), `list[int]` (bytes), or `int`.

Examples:

<code>f.hash.md5("Hello World")</code>	<code>#c# "b10a8db164e0754105b7a99be72e3fe5"</code> (hex,
<code>f.hash.md5("Hello World", "int")</code>	<code>#c#</code> (integer value)
<code>f.hash.md5("Hello World", "bytes")</code>	<code>#c# [177, 10, 141, 177, ...]</code>

SHA-1 Hash Function

The `f.hash.sha1` function computes a SHA-1 digest. Legacy — use for compatibility only, not for security.

Morphology:

EBNF:

```
"f.hash.sha1", "(", ( string | list[int] ), [",", ( "hex" | "bytes" | "int" ) ], ")"
```

- Operands: `string` or `list[int]`, optional output format (`"hex"` (default), `"bytes"`, `"int"`).
- Result: `string` (hex, default), `list[int]` (bytes), or `int`.

Examples:

<code>f.hash.sha1("Hello World")</code>	<code>#c# "0a4d55a8d778e5022fab701977c5d840bbc486d"</code>
<code>f.hash.sha1("Hello World", "int")</code>	<code>#c#</code> (integer value)
<code>f.hash.sha1("Hello World", "bytes")</code>	<code>#c# [10, 77, 85, 168, ...]</code>

SHA-224 Hash Function

The `f.hash.sha224` function computes a SHA-224 digest.

Morphology:

EBNF:

```
"f.hash.sha224", "(", ( string | list[int] ), [",", ( "hex" | "bytes" | "int" ) ], ")"
```

- Operands: `string` or `list[int]`, optional output format (`"hex"` (default), `"bytes"`, `"int"`).
- Result: `string` (hex, default), `list[int]` (bytes), or `int`.

Examples:

<code>f.hash.sha224("Hello World")</code>	<code>#c# "c8b073b548cd38f5e0eea5727bea4657"</code> (hex)
<code>f.hash.sha224("Hello World", "int")</code>	<code>#c#</code> (integer value)
<code>f.hash.sha224("Hello World", "bytes")</code>	<code>#c# [200, 176, 115, 181, ...]</code>

SHA-256 Hash Function

The `f.hash.sha256` function computes a SHA-256 digest.

Morphology:

EBNF:

```
"f.hash.sha256", "(", ( string | list[int] ), [",", ( "hex" | "bytes" | "int" ) ], ")"
```

- Operands: `string` or `list[int]`, optional output format ("`hex`" (default), "`bytes`", "`int`").
- Result: `string` (hex, default), `list[int]` (bytes), or `int`.

Examples:

```
f.hash.sha256("Hello World")           #c# "a591a6d40bf420404a011733cfb7b190d62c6"
f.hash.sha256("Hello World", "int")     #c# (integer value)
f.hash.sha256("Hello World", "bytes")   #c# [165, 145, 166, 212, ...]
```

SHA-384 Hash Function

The `f.hash.sha384` function computes a SHA-384 digest.

Morphology:

EBNF:

```
"f.hash.sha384", "(", ( string | list[int] ), [",", ( "hex" | "bytes" | "int" ) ], ")"
```

- Operands: `string` or `list[int]`, optional output format ("`hex`" (default), "`bytes`", "`int`").
- Result: `string` (hex, default), `list[int]` (bytes), or `int`.

Examples:

```
f.hash.sha384("Hello World")           #c# "14fdb4453b92a0c18562b5a9705826b45a3ef"
f.hash.sha384("Hello World", "int")     #c# (integer value)
f.hash.sha384("Hello World", "bytes")   #c# [20, 253, 180, 69, ...]
```

SHA-512 Hash Function

The `f.hash.sha512` function computes a SHA-512 digest.

Morphology:

EBNF:

```
"f.hash.sha512", "(", ( string | list[int] ), [",", ( "hex" | "bytes" | "int" ) ], ")"
```

- Operands: `string` or `list[int]`, optional output format ("`hex`" (default), "`bytes`", "`int`").
- Result: `string` (hex, default), `list[int]` (bytes), or `int`.

Examples:

```
f.hash.sha512("Hello World")           #c# "861844d6704e8573fec34d967e20bcfef3d42"
f.hash.sha512("Hello World", "int")     #c# (integer value)
f.hash.sha512("Hello World", "bytes")   #c# [134, 24, 68, 214, ...]
```

SHA-3 256 Hash Function

The `f.hash.sha3_256` function computes a SHA-3 256-bit digest.

Morphology:

EBNF:

```
"f.hash.sha3_256", "(", ( string | list[int] ), [",", ( "hex" | "bytes" | "int" ) ], ")"
```

- Operands: `string` or `list[int]`, optional output format (`"hex"` (default), `"bytes"`, `"int"`).
- Result: `string` (hex, default), `list[int]` (bytes), or `int`.

Examples:

```
f.hash.sha3_256("Hello World")          #c# "644bcc7e56197e90906997b5657bd..." (
f.hash.sha3_256("Hello World", "int")    #c# (integer value)
f.hash.sha3_256("Hello World", "bytes")  #c# [100, 75, 204, 126, ...]
```

SHA-3 512 Hash Function

The `f.hash.sha3_512` function computes a SHA-3 512-bit digest.

Morphology:

EBNF:

```
"f.hash.sha3_512", "(", ( string | list[int] ), [",", ( "hex" | "bytes" | "int" ) ], ")"
```

- Operands: `string` or `list[int]`, optional output format (`"hex"` (default), `"bytes"`, `"int"`).
- Result: `string` (hex, default), `list[int]` (bytes), or `int`.

Examples:

```
f.hash.sha3_512("Hello World")          #c# (128-char hex, default)
f.hash.sha3_512("Hello World", "int")    #c# (integer value)
f.hash.sha3_512("Hello World", "bytes")  #c# (byte array)
```

BLAKE2b Hash Function

The `f.hash.blake2b` function computes a BLAKE2b digest. High performance, cryptographic.

Morphology:

EBNF:

```
"f.hash.blake2b", "(", ( string | list[int] ), [",", ( "hex" | "bytes" | "int" ) ], ")"
```

- Operands: `string` or `list[int]`, optional output format (`"hex"` (default), `"bytes"`, `"int"`).
- Result: `string` (hex, default), `list[int]` (bytes), or `int`.

Examples:

```
f.hash.blake2b("Hello World")          #c# (128-char hex, default)
f.hash.blake2b("Hello World", "int")    #c# (integer value)
f.hash.blake2b("Hello World", "bytes")  #c# (byte array)
```

BLAKE2s Hash Function

The `f.hash.blake2s` function computes a BLAKE2s digest. High performance, cryptographic.

Morphology:

EBNF:

```
"f.hash.blake2s", "(", ( string | list[int] ), [",", ( "hex" | "bytes" | "int" ) ], ")"
```

- Operands: `string` or `list[int]`, optional output format (`"hex"` (default), `"bytes"`, `"int"`).
- Result: `string` (hex, default), `list[int]` (bytes), or `int`.

Examples:

```
f.hash.blake2s("Hello World")           #c# (64-char hex, default)
f.hash.blake2s("Hello World", "int")      #c# (integer value)
f.hash.blake2s("Hello World", "bytes")    #c# (byte array)
```

BLAKE3 Hash Function

The `f.hash.blake3` function computes a BLAKE3 digest. Extremely fast, cryptographic.

Morphology:

EBNF:

```
"f.hash.blake3", "(", ( string | list[int] ), [",", ( "hex" | "bytes" | "int" ) ], ")"
```

- Operands: `string` or `list[int]`, optional output format (`"hex"` (default), `"bytes"`, `"int"`).
- Result: `string` (hex, default), `list[int]` (bytes), or `int`.

Examples:

```
f.hash.blake3("Hello World")           #c# (64-char hex, default)
f.hash.blake3("Hello World", "int")      #c# (integer value)
f.hash.blake3("Hello World", "bytes")    #c# (byte array)
```

11.1.16 Probability Distributions

Probability distribution functions (`f.stat.*`) provide PDF/PMF, CDF, inverse CDF, and random sampling for common statistical distributions. Useful for Monte Carlo tolerance analysis, reliability modeling, and statistical design validation.

Normal PDF

The `f.stat.normalPDF` function computes the probability density of the normal (Gaussian) distribution at a given point.

EBNF:

```
"f.stat.normalPDF", "(", real, ",", real, ",", real, ")"
```


- Operands: `x` (evaluation point), `mean`, `stddev` (must be > 0).
- Result: `real`.
- A Value Error is raised if `stddev` ≤ 0 .

```
f.stat.normalPDF(0, 0, 1)    #c# 0.39894
f.stat.normalPDF(1, 0, 1)    #c# 0.24197
```

Normal CDF

The `f.stat.normalCDF` function computes the cumulative probability $P(X \leq x)$ for the normal distribution.

EBNF:

```
"f.stat.normalCDF", "(", real, ",", real, ",", real, ")"
```

- Operands: `x`, `mean`, `stddev` (must be > 0).
- Result: `real` (in $[0, 1]$).
- A Value Error is raised if `stddev` ≤ 0 .

```
f.stat.normalCDF(0, 0, 1)    #c# 0.5
f.stat.normalCDF(1.96, 0, 1) #c# 0.975
```

Normal Inverse CDF

The `f.stat.normalInvCDF` function computes the quantile (inverse CDF) for the normal distribution.

EBNF:

```
"f.stat.normalInvCDF", "(", real, ",", real, ",", real, ")"
```

- Operands: `p` (probability, in $(0, 1)$), `mean`, `stddev` (must be > 0).
- Result: `real`.
- A Value Error is raised if `p` ≤ 0 , `p` ≥ 1 , or `stddev` ≤ 0 .

```
f.stat.normalInvCDF(0.5, 0, 1)    #c# 0.0
f.stat.normalInvCDF(0.975, 0, 1)  #c# 1.96
```

Normal Random

The `f.stat.normalRandom` function returns a random sample from the normal distribution.

EBNF:

```
"f.stat.normalRandom", "(", real, ",", real, ")"
```

- Operands: `mean`, `stddev` (must be > 0).
- Result: `real`.
- A Value Error is raised if `stddev` ≤ 0 .

```
f.rng.seed(42)
$f v = f.stat.normalRandom(0, 1)    #c# -0.5823 (varies)
```

Exponential PDF

The `f.stat.exponentialPDF` function computes the probability density of the exponential distribution.

EBNF:

```
"f.stat.exponentialPDF", "(", real, ",", real, ")"
```

- Operands: `x` (must be ≥ 0), `lambda` (rate, must be > 0).
- Result: `real`.
- Returns 0 for $x < 0$.
- A Value Error is raised if $\text{lambda} \leq 0$.

```
f.stat.exponentialPDF(0, 1)    #c# 1.0  
f.stat.exponentialPDF(1, 1)    #c# 0.36788
```

Exponential CDF

The `f.stat.exponentialCDF` function computes the cumulative probability for the exponential distribution.

EBNF:

```
"f.stat.exponentialCDF", "(", real, ",", real, ")"
```

- Operands: `x`, `lambda` (must be > 0).
- Result: `real` (in $[0, 1]$).
- A Value Error is raised if $\text{lambda} \leq 0$.

```
f.stat.exponentialCDF(0, 1)    #c# 0.0  
f.stat.exponentialCDF(1, 1)    #c# 0.63212
```

Exponential Inverse CDF

The `f.stat.exponentialInvCDF` function computes the quantile for the exponential distribution.

EBNF:

```
"f.stat.exponentialInvCDF", "(", real, ",", real, ")"
```

- Operands: `p` (probability, in $(0, 1)$), `lambda` (must be > 0).
- Result: `real`.
- A Value Error is raised if $p \leq 0$, $p \geq 1$, or $\text{lambda} \leq 0$.

```
f.stat.exponentialInvCDF(0.5, 1)  #c# 0.69315  
f.stat.exponentialInvCDF(0.9, 1)  #c# 2.30259
```

Exponential Random

The `f.stat.exponentialRandom` function returns a random sample from the exponential distribution.

EBNF:

```
"f.stat.exponentialRandom", "(", real, ")"
```

- Operands: `lambda` (must be > 0).
- Result: `real`.
- A Value Error is raised if `lambda` ≤ 0 .

```
f.rng.seed(42)
```

```
$f v = f.stat.exponentialRandom(1)    ## 0.4869 (varies)
```

Gamma PDF

The `f.stat.gammaPDF` function computes the probability density of the gamma distribution.

EBNF:

```
"f.stat.gammaPDF", "(", real, ",", real, ",", real, ")"
```

- Operands: `x` (must be > 0), `shape` (must be > 0), `rate` (must be > 0).
- Result: `real`.
- Returns 0 for $x \leq 0$.
- A Value Error is raised if `shape` ≤ 0 or `rate` ≤ 0 .

```
f.stat.gammaPDF(1, 1, 1)    ## 1.0
```

```
f.stat.gammaPDF(2, 2, 1)    ## 0.27067
```

Gamma CDF

The `f.stat.gammaCDF` function computes the cumulative probability for the gamma distribution.

EBNF:

```
"f.stat.gammaCDF", "(", real, ",", real, ",", real, ")"
```

- Operands: `x`, `shape` (must be > 0), `rate` (must be > 0).
- Result: `real` (in $[0, 1]$).
- A Value Error is raised if `shape` ≤ 0 or `rate` ≤ 0 .

```
f.stat.gammaCDF(1, 1, 1)    ## 0.63212
```

```
f.stat.gammaCDF(2, 2, 1)    ## 0.59399
```

Gamma Inverse CDF

The `f.stat.gammaInvCDF` function computes the quantile for the gamma distribution.

EBNF:

```
"f.stat.gammaInvCDF", "(", real, ",", real, ",", real, ")"
```

- Operands: `p` (probability, in $(0, 1)$), `shape` (must be > 0), `rate` (must be > 0).
- Result: `real`.
- A Value Error is raised if $p \leq 0$, $p \geq 1$, $shape \leq 0$, or $rate \leq 0$.

```
f.stat.gammaInvCDF(0.5, 1, 1)    #c# 0.69315
```

Gamma Random

The `f.stat.gammaRandom` function returns a random sample from the gamma distribution.

EBNF:

```
"f.stat.gammaRandom", "(", real, ",", real, ")"
```

- Operands: `shape` (must be > 0), `rate` (must be > 0).
- Result: `real`.
- A Value Error is raised if $shape \leq 0$ or $rate \leq 0$.

```
f.rng.seed(42)
```

```
$f v = f.stat.gammaRandom(2, 1)    #c# 1.2345 (varies)
```

Beta PDF

The `f.stat.betaPDF` function computes the probability density of the beta distribution.

EBNF:

```
"f.stat.betaPDF", "(", real, ",", real, ",", real, ")"
```

- Operands: `x` (in $[0, 1]$), `alpha` (must be > 0), `beta` (must be > 0).
- Result: `real`.
- Returns 0 for $x < 0$ or $x > 1$.
- A Value Error is raised if $alpha \leq 0$ or $beta \leq 0$.

```
f.stat.betaPDF(0.5, 1, 1)    #c# 1.0
```

```
f.stat.betaPDF(0.5, 2, 2)    #c# 1.5
```

Beta CDF

The `f.stat.betaCDF` function computes the cumulative probability for the beta distribution.

EBNF:

```
"f.stat.betaCDF", "(", real, ",", real, ",", real, ")"
```

- Operands: `x`, `alpha` (must be > 0), `beta` (must be > 0).
- Result: `real` (in $[0, 1]$).
- A Value Error is raised if `alpha` ≤ 0 or `beta` ≤ 0 .

```
f.stat.betaCDF(0.5, 1, 1)    #c# 0.5  
f.stat.betaCDF(0.5, 2, 2)    #c# 0.5
```

Beta Inverse CDF

The `f.stat.betaInvCDF` function computes the quantile for the beta distribution.

EBNF:

```
"f.stat.betaInvCDF", "(", real, ",", real, ",", real, ")"
```

- Operands: `p` (probability, in $(0, 1)$), `alpha` (must be > 0), `beta` (must be > 0).
- Result: `real`.
- A Value Error is raised if `p` ≤ 0 , `p` ≥ 1 , `alpha` ≤ 0 , or `beta` ≤ 0 .

```
f.stat.betaInvCDF(0.5, 1, 1)    #c# 0.5
```

Beta Random

The `f.stat.betaRandom` function returns a random sample from the beta distribution.

EBNF:

```
"f.stat.betaRandom", "(", real, ",", real, ")"
```

- Operands: `alpha` (must be > 0), `beta` (must be > 0).
- Result: `real` (in $[0, 1]$).
- A Value Error is raised if `alpha` ≤ 0 or `beta` ≤ 0 .

```
f.rng.seed(42)  
$f v = f.stat.betaRandom(2, 5)    #c# 0.3456 (varies)
```

Weibull PDF

The `f.stat.weibullPDF` function computes the probability density of the Weibull distribution.

EBNF:

```
"f.stat.weibullPDF", "(", real, ",", real, ",", real, ")"
```

- Operands: `x` (must be ≥ 0), `shape` (must be > 0), `scale` (must be > 0).
- Result: `real`.
- Returns 0 for $x < 0$.
- A Value Error is raised if `shape` ≤ 0 or `scale` ≤ 0 .

```
f.stat.weibullPDF(1, 1, 1)    ## 0.36788
f.stat.weibullPDF(1, 2, 1)    ## 0.73576
```

Weibull CDF

The `f.stat.weibullCDF` function computes the cumulative probability for the Weibull distribution.

EBNF:

```
"f.stat.weibullCDF", "(", real, ",", real, ",", real, ")"
```

- Operands: `x`, `shape` (must be > 0), `scale` (must be > 0).
- Result: `real` (in $[0, 1]$).
- A Value Error is raised if `shape` ≤ 0 or `scale` ≤ 0 .

```
f.stat.weibullCDF(1, 1, 1)    ## 0.63212
f.stat.weibullCDF(1, 2, 1)    ## 0.39347
```

Weibull Inverse CDF

The `f.stat.weibullInvCDF` function computes the quantile for the Weibull distribution.

EBNF:

```
"f.stat.weibullInvCDF", "(", real, ",", real, ",", real, ")"
```

- Operands: `p` (probability, in $(0, 1)$), `shape` (must be > 0), `scale` (must be > 0).
- Result: `real`.
- A Value Error is raised if `p` ≤ 0 , `p` ≥ 1 , `shape` ≤ 0 , or `scale` ≤ 0 .

```
f.stat.weibullInvCDF(0.5, 1, 1)  ## 0.69315
```

Weibull Random

The `f.stat.weibullRandom` function returns a random sample from the Weibull distribution.

EBNF:

```
"f.stat.weibullRandom", "(", real, ",", real, ")"
```

- Operands: `shape` (must be > 0), `scale` (must be > 0).
- Result: `real`.
- A Value Error is raised if `shape` ≤ 0 or `scale` ≤ 0 .

```
f.rng.seed(42)
```

```
$f v = f.stat.weibullRandom(2, 1)    #c# 0.8765 (varies)
```

Logistic PDF

The `f.stat.logisticPDF` function computes the probability density of the logistic distribution.

EBNF:

```
"f.stat.logisticPDF", "(", real, ",", real, ",", real, ")"
```

- Operands: `x`, `mu` (location), `s` (scale, must be > 0).
- Result: `real`.
- A Value Error is raised if `s` ≤ 0 .

```
f.stat.logisticPDF(0, 0, 1)    #c# 0.25
```

```
f.stat.logisticPDF(1, 0, 1)    #c# 0.19661
```

Logistic CDF

The `f.stat.logisticCDF` function computes the cumulative probability for the logistic distribution.

EBNF:

```
"f.stat.logisticCDF", "(", real, ",", real, ",", real, ")"
```

- Operands: `x`, `mu`, `s` (must be > 0).
- Result: `real` (in $(0, 1)$).
- A Value Error is raised if `s` ≤ 0 .

```
f.stat.logisticCDF(0, 0, 1)    #c# 0.5
```

```
f.stat.logisticCDF(2, 0, 1)    #c# 0.88080
```

Logistic Inverse CDF

The `f.stat.logisticInvCDF` function computes the quantile for the logistic distribution.

EBNF:

```
"f.stat.logisticInvCDF", "(", real, ",", real, ",", real, ")"
```

- Operands: `p` (probability, in $(0, 1)$), `mu`, `s` (must be > 0).
- Result: `real`.
- A Value Error is raised if $p \leq 0$, $p \geq 1$, or $s \leq 0$.

```
f.stat.logisticInvCDF(0.5, 0, 1)    #c# 0.0  
f.stat.logisticInvCDF(0.9, 0, 1)    #c# 2.19722
```

Logistic Random

The `f.stat.logisticRandom` function returns a random sample from the logistic distribution.

EBNF:

```
"f.stat.logisticRandom", "(", real, ",", real, ")"
```

- Operands: `mu`, `s` (must be > 0).
- Result: `real`.
- A Value Error is raised if $s \leq 0$.

```
f.rng.seed(42)  
$f v = f.stat.logisticRandom(0, 1)  #c# -0.3456 (varies)
```

Cauchy PDF

The `f.stat.cauchyPDF` function computes the probability density of the Cauchy distribution.

EBNF:

```
"f.stat.cauchyPDF", "(", real, ",", real, ",", real, ")"
```

- Operands: `x`, `x0` (location), `gamma` (scale, must be > 0).
- Result: `real`.
- A Value Error is raised if $gamma \leq 0$.

```
f.stat.cauchyPDF(0, 0, 1)    #c# 0.31831  
f.stat.cauchyPDF(1, 0, 1)    #c# 0.15915
```


Cauchy CDF

The `f.stat.cauchyCDF` function computes the cumulative probability for the Cauchy distribution.

EBNF:

```
"f.stat.cauchyCDF", "(", real, ",", real, ",", real, ")"
```

- Operands: `x`, `x0`, `gamma` (must be > 0).
- Result: `real` (in $(0, 1)$).
- A Value Error is raised if $\text{gamma} \leq 0$.

```
f.stat.cauchyCDF(0, 0, 1)    #c# 0.5  
f.stat.cauchyCDF(1, 0, 1)    #c# 0.75
```

Cauchy Inverse CDF

The `f.stat.cauchyInvCDF` function computes the quantile for the Cauchy distribution.

EBNF:

```
"f.stat.cauchyInvCDF", "(", real, ",", real, ",", real, ")"
```

- Operands: `p` (probability, in $(0, 1)$), `x0`, `gamma` (must be > 0).
- Result: `real`.
- A Value Error is raised if $p \leq 0$, $p \geq 1$, or $\text{gamma} \leq 0$.

```
f.stat.cauchyInvCDF(0.5, 0, 1)  #c# 0.0  
f.stat.cauchyInvCDF(0.75, 0, 1) #c# 1.0
```

Cauchy Random

The `f.stat.cauchyRandom` function returns a random sample from the Cauchy distribution.

EBNF:

```
"f.stat.cauchyRandom", "(", real, ",", real, ")"
```

- Operands: `x0`, `gamma` (must be > 0).
- Result: `real`.
- A Value Error is raised if $\text{gamma} \leq 0$.

```
f.rng.seed(42)  
$f v = f.stat.cauchyRandom(0, 1)    #c# -1.2345 (varies)
```

Student's t PDF

The `f.stat.studentTPDF` function computes the probability density of the Student's t distribution.

EBNF:

```
"f.stat.studentTPDF", "(", real, ",", real, ")"
```

- Operands: `x`, `df` (degrees of freedom, must be > 0).
- Result: `real`.
- A Value Error is raised if `df` ≤ 0 .

```
f.stat.studentTPDF(0, 1)    #c# 0.31831 (Cauchy)
```

```
f.stat.studentTPDF(0, 10)   #c# 0.37589
```

Student's t CDF

The `f.stat.studentTCDF` function computes the cumulative probability for the Student's t distribution.

EBNF:

```
"f.stat.studentTCDF", "(", real, ",", real, ")"
```

- Operands: `x`, `df` (must be > 0).
- Result: `real` (in $(0, 1)$).
- A Value Error is raised if `df` ≤ 0 .

```
f.stat.studentTCDF(0, 10)    #c# 0.5
```

```
f.stat.studentTCDF(1.812, 10) #c# 0.95
```

Student's t Inverse CDF

The `f.stat.studentTInvCDF` function computes the quantile for the Student's t distribution.

EBNF:

```
"f.stat.studentTInvCDF", "(", real, ",", real, ")"
```

- Operands: `p` (probability, in $(0, 1)$), `df` (must be > 0).
- Result: `real`.
- A Value Error is raised if `p` ≤ 0 , `p` ≥ 1 , or `df` ≤ 0 .

```
f.stat.studentTInvCDF(0.5, 10)    #c# 0.0
```

```
f.stat.studentTInvCDF(0.95, 10)   #c# 1.812
```

Student's t Random

The `f.stat.studentTRandom` function returns a random sample from the Student's t distribution.

EBNF:

```
"f.stat.studentTRandom", "(", real, ")"
```

- Operands: `df` (must be > 0).
- Result: `real`.
- A Value Error is raised if `df` ≤ 0 .

```
f.rng.seed(42)
```

```
$f v = f.stat.studentTRandom(5)    ## -0.7890 (varies)
```

Chi-Square PDF

The `f.stat.chiSquarePDF` function computes the probability density of the chi-square distribution.

EBNF:

```
"f.stat.chiSquarePDF", "(", real, ",", real, ")"
```

- Operands: `x` (must be ≥ 0), `df` (degrees of freedom, must be > 0).
- Result: `real`.
- Returns 0 for $x < 0$.
- A Value Error is raised if `df` ≤ 0 .

```
f.stat.chiSquarePDF(1, 1)    ## 0.24197
```

```
f.stat.chiSquarePDF(2, 2)    ## 0.18394
```

Chi-Square CDF

The `f.stat.chiSquareCDF` function computes the cumulative probability for the chi-square distribution.

EBNF:

```
"f.stat.chiSquareCDF", "(", real, ",", real, ")"
```

- Operands: `x`, `df` (must be > 0).
- Result: `real` (in $[0, 1]$).
- A Value Error is raised if `df` ≤ 0 .

```
f.stat.chiSquareCDF(1, 1)    ## 0.68269
```

```
f.stat.chiSquareCDF(2, 2)    ## 0.63212
```

Chi-Square Inverse CDF

The `f.stat.chiSquareInvCDF` function computes the quantile for the chi-square distribution.

EBNF:

```
"f.stat.chiSquareInvCDF", "(", real, ",", real, ")"
```

- Operands: `p` (probability, in $(0, 1)$), `df` (must be > 0).
- Result: `real`.
- A Value Error is raised if $p \leq 0$, $p \geq 1$, or $df \leq 0$.

```
f.stat.chiSquareInvCDF(0.5, 1)    #c# 0.45494
```

Chi-Square Random

The `f.stat.chiSquareRandom` function returns a random sample from the chi-square distribution.

EBNF:

```
"f.stat.chiSquareRandom", "(", real, ")"
```

- Operands: `df` (must be > 0).
- Result: `real`.
- A Value Error is raised if $df \leq 0$.

```
f.rng.seed(42)
```

```
$f v = f.stat.chiSquareRandom(3)    #c# 2.3456 (varies)
```

F-Distribution PDF

The `f.stat.fDistributionPDF` function computes the probability density of the F-distribution.

EBNF:

```
"f.stat.fDistributionPDF", "(", real, ",", real, ",", real, ")"
```

- Operands: `x` (must be ≥ 0), `df1` (numerator df, must be > 0), `df2` (denominator df, must be > 0).
- Result: `real`.
- Returns 0 for $x < 0$.
- A Value Error is raised if $df1 \leq 0$ or $df2 \leq 0$.

```
f.stat.fDistributionPDF(1, 5, 10)    #c# 0.31831 (approx)
```

F-Distribution CDF

The `f.stat.fDistributionCDF` function computes the cumulative probability for the F-distribution.

EBNF:

```
"f.stat.fDistributionCDF", "(", real, ",", real, ",", real, ")"
```

- Operands: `x`, `df1`, `df2` (both must be > 0).
- Result: `real` (in $[0, 1]$).
- A Value Error is raised if $df1 \leq 0$ or $df2 \leq 0$.

```
f.stat.fDistributionCDF(1, 5, 10)    #c# 0.5 (approx)
```

F-Distribution Inverse CDF

The `f.stat.fDistributionInvCDF` function computes the quantile for the F-distribution.

EBNF:

```
"f.stat.fDistributionInvCDF", "(", real, ",", real, ",", real, ")"
```

- Operands: `p` (probability, in $(0, 1)$), `df1`, `df2` (both must be > 0).
- Result: `real`.
- A Value Error is raised if $p \leq 0$, $p \geq 1$, $df1 \leq 0$, or $df2 \leq 0$.

```
f.stat.fDistributionInvCDF(0.5, 5, 10)    #c# 0.976 (approx)
```

F-Distribution Random

The `f.stat.fDistributionRandom` function returns a random sample from the F-distribution.

EBNF:

```
"f.stat.fDistributionRandom", "(", real, ",", real, ")"
```

- Operands: `df1` (must be > 0), `df2` (must be > 0).
- Result: `real`.
- A Value Error is raised if $df1 \leq 0$ or $df2 \leq 0$.

```
f.rng.seed(42)
```

```
$f v = f.stat.fDistributionRandom(5, 10)    #c# 1.2345 (varies)
```

Uniform PDF

The `f.stat.uniformPDF` function computes the probability density of the continuous uniform distribution.

EBNF:

```
"f.stat.uniformPDF", "(", real, ",", real, ",", real, ")"
```

- Operands: `x`, `a` (lower bound), `b` (upper bound, must be $> a$).
- Result: `real` — $1/(b - a)$ if $a \leq x \leq b$, else 0.
- A Value Error is raised if $a \geq b$.

```
f.stat.uniformPDF(0.5, 0, 1)    #c# 1.0  
f.stat.uniformPDF(2, 0, 1)      #c# 0.0
```

Uniform CDF

The `f.stat.uniformCDF` function computes the cumulative probability for the continuous uniform distribution.

EBNF:

```
"f.stat.uniformCDF", "(", real, ",", real, ",", real, ")"
```

- Operands: `x`, `a`, `b` (must be $> a$).
- Result: `real` (in $[0, 1]$).
- A Value Error is raised if $a \geq b$.

```
f.stat.uniformCDF(0.5, 0, 1)    #c# 0.5  
f.stat.uniformCDF(1, 0, 1)      #c# 1.0
```

Uniform Inverse CDF

The `f.stat.uniformInvCDF` function computes the quantile for the continuous uniform distribution.

EBNF:

```
"f.stat.uniformInvCDF", "(", real, ",", real, ",", real, ")"
```

- Operands: `p` (probability, in $(0, 1)$), `a`, `b` (must be $> a$).
- Result: `real`.
- A Value Error is raised if $p \leq 0$, $p \geq 1$, or $a \geq b$.

```
f.stat.uniformInvCDF(0.5, 0, 1)  #c# 0.5  
f.stat.uniformInvCDF(0.9, 0, 10) #c# 9.0
```

Uniform Random

The `f.stat.uniformRandom` function returns a random sample from the continuous uniform distribution.

EBNF:

```
"f.stat.uniformRandom", "(", real, ",", real, ")"
```

- Operands: `a` (lower bound), `b` (upper bound, must be $> a$).
- Result: `real`.
- A Value Error is raised if $a \geq b$.

```
f.rng.seed(42)
```

```
$f v = f.stat.uniformRandom(0, 1)    #c# 0.3745 (varies)
```

Binomial PMF

The `f.stat.binomialPMF` function computes the probability mass of the binomial distribution.

EBNF:

```
"f.stat.binomialPMF", "(", int, ",", int, ",", real, ")"
```

- Operands: `k` (number of successes, $0 \leq k \leq n$), `n` (trials, must be ≥ 0), `p` (probability, in $[0, 1]$).
- Result: `real`.
- Returns 0 if $k < 0$ or $k > n$.
- A Value Error is raised if $n < 0$ or $p < 0$ or $p > 1$.

```
f.stat.binomialPMF(3, 10, 0.5)    #c# 0.11719
```

```
f.stat.binomialPMF(0, 10, 0.5)    #c# 0.00098
```

Binomial CDF

The `f.stat.binomialCDF` function computes the cumulative probability for the binomial distribution.

EBNF:

```
"f.stat.binomialCDF", "(", int, ",", int, ",", real, ")"
```

- Operands: `k`, `n` (must be ≥ 0), `p` (in $[0, 1]$).
- Result: `real` (in $[0, 1]$).
- A Value Error is raised if $n < 0$ or $p < 0$ or $p > 1$.

```
f.stat.binomialCDF(3, 10, 0.5)    #c# 0.17188
```

```
f.stat.binomialCDF(5, 10, 0.5)    #c# 0.62305
```

Binomial Inverse CDF

The `f.stat.binomialInvCDF` function computes the quantile for the binomial distribution.

EBNF:

```
"f.stat.binomialInvCDF", "(", real, ",", int, ",", real, ")"
```

- Operands: `p` (probability, in $(0, 1)$), `n` (must be ≥ 0), `p_succ` (in $[0, 1]$).
- Result: `int`.
- A Value Error is raised if $p \leq 0$, $p \geq 1$, $n < 0$, or $p_succ < 0$ or $p_succ > 1$.

```
f.stat.binomialInvCDF(0.5, 10, 0.5)    #c# 5
```

Binomial Random

The `f.stat.binomialRandom` function returns a random sample from the binomial distribution.

EBNF:

```
"f.stat.binomialRandom", "(", int, ",", real, ")"
```

- Operands: `n` (trials, must be ≥ 0), `p` (probability, in $[0, 1]$).
- Result: `int`.
- A Value Error is raised if $n < 0$ or $p < 0$ or $p > 1$.

```
f.rng.seed(42)
```

```
$f v = f.stat.binomialRandom(10, 0.5)    #c# 6 (varies)
```

Poisson PMF

The `f.stat.poissonPMF` function computes the probability mass of the Poisson distribution.

EBNF:

```
"f.stat.poissonPMF", "(", int, ",", real, ")"
```

- Operands: `k` (events, must be ≥ 0), `lambda` (rate, must be > 0).
- Result: `real`.
- Returns 0 for $k < 0$.
- A Value Error is raised if $k < 0$ or $lambda \leq 0$.

```
f.stat.poissonPMF(3, 5)    #c# 0.14037
```

```
f.stat.poissonPMF(0, 5)    #c# 0.00674
```


Poisson CDF

The `f.stat.poissonCDF` function computes the cumulative probability for the Poisson distribution.

EBNF:

```
"f.stat.poissonCDF", "(", int, ",", real, ")"
```

- Operands: `k`, `lambda` (must be > 0).
- Result: `real` (in $[0, 1]$).
- A Value Error is raised if $k < 0$ or $lambda \leq 0$.

```
f.stat.poissonCDF(3, 5)    #c# 0.26503  
f.stat.poissonCDF(5, 5)    #c# 0.61596
```

Poisson Inverse CDF

The `f.stat.poissonInvCDF` function computes the quantile for the Poisson distribution.

EBNF:

```
"f.stat.poissonInvCDF", "(", real, ",", real, ")"
```

- Operands: `p` (probability, in $(0, 1)$), `lambda` (must be > 0).
- Result: `int`.
- A Value Error is raised if $p \leq 0$, $p \geq 1$, or $lambda \leq 0$.

```
f.stat.poissonInvCDF(0.5, 5)    #c# 5
```

Poisson Random

The `f.stat.poissonRandom` function returns a random sample from the Poisson distribution.

EBNF:

```
"f.stat.poissonRandom", "(", real, ")"
```

- Operands: `lambda` (must be > 0).
- Result: `int`.
- A Value Error is raised if $lambda \leq 0$.

```
f.rng.seed(42)  
$f v = f.stat.poissonRandom(5)    #c# 4 (varies)
```

Geometric PMF

The `f.stat.geometricPMF` function computes the probability mass of the geometric distribution (number of failures before first success).

EBNF:

```
"f.stat.geometricPMF", "(", int, ",", real, ")"
```

- Operands: `k` (failures, must be ≥ 0), `p` (success probability, in $(0, 1]$).
- Result: `real`.
- A `Value Error` is raised if $k < 0$, $p \leq 0$, or $p > 1$.

```
f.stat.geometricPMF(0, 0.5)    #c# 0.5  
f.stat.geometricPMF(2, 0.5)    #c# 0.125
```

Geometric CDF

The `f.stat.geometricCDF` function computes the cumulative probability for the geometric distribution.

EBNF:

```
"f.stat.geometricCDF", "(", int, ",", real, ")"
```

- Operands: `k`, `p` (in $(0, 1]$).
- Result: `real` (in $[0, 1]$).
- A `Value Error` is raised if $k < 0$, $p \leq 0$, or $p > 1$.

```
f.stat.geometricCDF(0, 0.5)    #c# 0.5  
f.stat.geometricCDF(2, 0.5)    #c# 0.875
```

Geometric Inverse CDF

The `f.stat.geometricInvCDF` function computes the quantile for the geometric distribution.

EBNF:

```
"f.stat.geometricInvCDF", "(", real, ",", real, ")"
```

- Operands: `p_prob` (probability, in $(0, 1)$), `p_succ` (success probability, in $(0, 1]$).
- Result: `int`.
- A `Value Error` is raised if $p_prob \leq 0$, $p_prob \geq 1$, $p_succ \leq 0$, or $p_succ > 1$.

```
f.stat.geometricInvCDF(0.5, 0.5)    #c# 1
```

Geometric Random

The `f.stat.geometricRandom` function returns a random sample from the geometric distribution.

EBNF:

```
"f.stat.geometricRandom", "(", real, ")"
```

- Operands: `p` (success probability, in $(0, 1]$).
- Result: `int`.
- A Value Error is raised if $p \leq 0$ or $p > 1$.

```
f.rng.seed(42)
```

```
$f v = f.stat.geometricRandom(0.5)    ## 1 (varies)
```

Negative Binomial PMF

The `f.stat.negativeBinomialPMF` function computes the probability mass of the negative binomial distribution (number of failures before r successes).

EBNF:

```
"f.stat.negativeBinomialPMF", "(", int, ",", int, ",", real, ")"
```

- Operands: `k` (failures, must be ≥ 0), `r` (target successes, must be > 0), `p` (success probability, in $(0, 1]$).
- Result: `real`.
- A Value Error is raised if $k < 0$, $r \leq 0$, $p \leq 0$, or $p > 1$.

```
f.stat.negativeBinomialPMF(3, 5, 0.5)    ## 0.15625
```

Negative Binomial CDF

The `f.stat.negativeBinomialCDF` function computes the cumulative probability for the negative binomial distribution.

EBNF:

```
"f.stat.negativeBinomialCDF", "(", int, ",", int, ",", real, ")"
```

- Operands: `k`, `r` (must be > 0), `p` (in $(0, 1]$).
- Result: `real` (in $[0, 1]$).
- A Value Error is raised if $k < 0$, $r \leq 0$, $p \leq 0$, or $p > 1$.

```
f.stat.negativeBinomialCDF(3, 5, 0.5)    ## 0.5
```

Negative Binomial Inverse CDF

The `f.stat.negativeBinomialInvCDF` function computes the quantile for the negative binomial distribution.

EBNF:

```
"f.stat.negativeBinomialInvCDF", "(", real, ",", int, ",", real, ")"
```

- Operands: `p_prob` (probability, in $(0, 1)$), `r` (must be > 0), `p_succ` (in $(0, 1]$).
- Result: `int`.
- A Value Error is raised if `p_prob` ≤ 0 , `p_prob` ≥ 1 , `r` ≤ 0 , `p_succ` ≤ 0 , or `p_succ` > 1 .

```
f.stat.negativeBinomialInvCDF(0.5, 5, 0.5)    #c# 4
```

Negative Binomial Random

The `f.stat.negativeBinomialRandom` function returns a random sample from the negative binomial distribution.

EBNF:

```
"f.stat.negativeBinomialRandom", "(", int, ",", real, ")"
```

- Operands: `r` (target successes, must be > 0), `p` (success probability, in $(0, 1]$).
- Result: `int`.
- A Value Error is raised if `r` ≤ 0 , `p` ≤ 0 , or `p` > 1 .

```
f.rng.seed(42)
```

```
$f v = f.stat.negativeBinomialRandom(5, 0.5)    #c# 3 (varies)
```

11.1.17 Numerical Analysis

Numerical analysis functions (`f.math.*`) provide root finding, interpolation, calculus, and ODE solving for engineering computations. All functions in this section accept function references (lambdas or named functions) as arguments.

When to Use

- **Root finding** — solving $f(x) = 0$ for circuit operating points, bias calculations, threshold detection.
- **Interpolation** — lookup tables, component model approximation, sensor calibration curves.
- **Calculus** — filter response integration, sensitivity analysis, power/energy calculations.
- **ODE solving** — transient circuit analysis, SPICE-like simulation, thermal modeling, PLL loop dynamics.

Newton's Method

The `f.math.newton` function finds a root of $f(x) = 0$ using Newton's method (also known as the Newton-Raphson method). Requires the derivative $f'(x)$ to be computed automatically via automatic differentiation.

Algorithm: Starting from initial guess x_0 , iterate $x_{n+1} = x_n - f(x_n)/f'(x_n)$ until $|f(x_n)| < \text{tol}$ or `maxIter` is reached.

Convergence: Quadratic convergence when near a root and $f'(x) \neq 0$. May diverge if the initial guess is far from a root, or if $f'(x)$ is near zero.

Behavior: Finds ONE root. The root found depends on the initial guess x_0 . Different initial guesses may converge to different roots. If f has multiple roots, the user must try different starting points.

EBNF:

```
"f.math.newton", "(", func, ",", real, ["", real], ["", int], ")"
```

- Operands: `$f` (function $f(x)$), `x0` (initial guess), optional `tol` (real, default 10^{-10}), optional `maxIter` (int, default 100).
- Result: `real` — root approximation where $|f(x)| < \text{tol}$.
- A `Runtime Error` is raised if $f'(x) = 0$ during any iteration (division by zero).
- A `Value Error` is raised if no convergence within `maxIter` iterations, if `tol` ≤ 0 , or if `maxIter` ≤ 0 .
- A `Type Error` is raised if `$f` is not a function reference.

When to Prefer Newton's Method Fast convergence when you have a good initial guess and the function is smooth. Not suitable for functions with discontinuities or zero derivatives near the root.

```
$f sq = func($x) $x * $x - 2
f.math.newton($sq, 1.0)      #c# 1.41421 (sqrt(2), found from guess 1.0)
f.math.newton($sq, -1.0)     #c# -1.41421 (sqrt(2), found from guess -1.0)
f.math.newton($sq, 1.0, 1e-8) #c# 1.41421 (custom tolerance)
```

Bisection Method

The `f.math.bisection` function finds a root of $f(x) = 0$ using the bisection method. Requires $f(a)$ and $f(b)$ to have opposite signs (Intermediate Value Theorem guarantees a root exists in $[a, b]$).

Algorithm: Repeatedly halve the interval $[a, b]$ by checking the sign of f at the midpoint. Converges to a root within `tol`.

Convergence: Linear convergence. Always converges if $f(a)$ and $f(b)$ have opposite signs and f is continuous. Maximum $\lceil \log_2((b-a)/\text{tol}) \rceil$ iterations.

Behavior: Finds ONE root in the interval $[a, b]$. If multiple roots exist in the interval, returns one of them (not specified which). The result is always within `tol` of the true root.

EBNF:

```
"f.math.bisection", "(", func, ",", real, ",", real, ["", real], ")"
```

- Operands: `$f` (function $f(x)$), `a` (lower bound), `b` (upper bound), optional `tol` (real, default 10^{-10}).
- Result: `real` — root approximation within `tol` of the true root.
- A `Value Error` is raised if $f(a)$ and $f(b)$ have the same sign (no guaranteed root), if $a = b$, if no convergence within internal limit (~ 1000 iterations), or if $\text{tol} \leq 0$.
- A `Type Error` is raised if `$f` is not a function reference.

When to Prefer Bisection Most robust method. Guaranteed to converge if a sign change exists. Use when you know the root is bracketed but the function is not smooth, or when Newton's method fails.

```
$f sq = func($x) $x * $x - 2
f.math.bisection($sq, 0, 2)          #c# 1.41421 (sqrt(2))
f.math.bisection($sq, 0, 2, 1e-8)    #c# 1.41421 (custom tolerance)
```

Secant Method

The `f.math.secant` function finds a root of $f(x) = 0$ using the secant method. A finite-difference approximation of Newton's method that does not require computing $f'(x)$.

Algorithm: Starting from two points x_0, x_1 , iterate $x_{n+1} = x_n - f(x_n) \cdot (x_n - x_{n-1}) / (f(x_n) - f(x_{n-1}))$.

Convergence: Superlinear convergence (order ≈ 1.618). Faster than bisection, slower than Newton. May fail if $f(x_n) = f(x_{n-1})$ (zero denominator).

Behavior: Finds ONE root. The root found depends on both initial points x_0 and x_1 . Not guaranteed to converge (unlike bisection).

EBNF:

```
"f.math.secant", "(", func, ",", real, ",", real, [",", real], [",", int], ")"
```

- Operands: `$f` (function $f(x)$), `x0`, `x1` (two initial points), optional `tol` (real, default 10^{-10}), optional `maxIter` (int, default 100).
- Result: `real` — root approximation.
- A `Runtime Error` is raised if $f(x_0) = f(x_1)$ (zero denominator during first step).
- A `Value Error` is raised if no convergence within `maxIter` iterations, if $\text{tol} \leq 0$, or if $\text{maxIter} \leq 0$.
- A `Type Error` is raised if `$f` is not a function reference.

When to Prefer Secant Method When you cannot compute $f'(x)$ analytically but need faster convergence than bisection. Good general-purpose method when Newton's automatic differentiation is not available or too expensive.

```
$f sq = func($x) $x * $x - 2
f.math.secant($sq, 0, 2)          #c# 1.41421 (sqrt(2))
f.math.secant($sq, 0, 2, 1e-8)    #c# 1.41421 (custom tolerance)
```

Brent's Method

The `f.math.brent` function finds a root of $f(x) = 0$ using Brent's method. Combines the robustness of bisection with the speed of secant and inverse quadratic interpolation.

Algorithm: Maintains a bracketing interval $[a, b]$. Attempts inverse quadratic interpolation when safe, falls back to secant, and uses bisection as a last resort. Guarantees progress at every step.

Convergence: Superlinear convergence in practice. Guaranteed to converge (like bisection) if the root is bracketed. Typically faster than bisection, sometimes faster than secant.

Behavior: Finds ONE root in the interval $[a, b]$. Combines the best properties of bisection (guaranteed convergence) and secant (fast convergence). Recommended as the default root-finding method when a bracket is known.

EBNF:

```
"f.math.brent", "(", func, ",", real, ",", real, ["", real], ")"
```

- Operands: `$f` (function $f(x)$), `a` (lower bound), `b` (upper bound), optional `tol` (real, default 10^{-10}).
- Result: `real` — root approximation.
- A `Value Error` is raised if $f(a)$ and $f(b)$ have the same sign (no guaranteed root), if $a = b$, if no convergence within internal limit (~ 1000 iterations), or if `tol` ≤ 0 .
- A `Type Error` is raised if `$f` is not a function reference.

When to Prefer Brent's Method Recommended default when you have a bracketed interval. More robust than secant, faster than bisection. Use this unless you have a specific reason to prefer another method.

```
$f sq = func($x) $x * $x - 2
f.math.brent($sq, 0, 2)          #c# 1.41421 (sqrt(2))
f.math.brent($sq, 0, 2, 1e-8) #c# 1.41421 (custom tolerance)
```

Root-Finding Method Comparison

Method	Convergence	Needs <code>\$f'</code>	Needs Bracket	Guaranteed
Newton	Quadratic	Yes	No	No
Bisection	Linear	No	Yes	Yes
Secant	Superlinear (ϕ)	No	No	No
Brent	Superlinear	No	Yes	Yes

Interpolation

The `f.math.interpolate` function estimates y values between (or beyond) known data points.

Linear interpolation: For each interval $[x_i, x_{i+1}]$, computes the straight-line segment between (x_i, y_i) and (x_{i+1}, y_{i+1}) . Continuous but not smooth (kinks at data points). Always uses the two nearest data points.

Polynomial interpolation: Fits a single polynomial of degree $n - 1$ through all n data points using Lagrange interpolation. Exact at all data points. Smooth everywhere. May exhibit Runge’s oscillation for equally-spaced points with large n (use spline instead for $n > 10$).

Extrapolation: Both methods extrapolate beyond the data range. Linear extrapolation extends the last segment. Polynomial extrapolation follows the fitted polynomial. No error is raised for out-of-range x , but extrapolation accuracy may be poor.

EBNF:

```
"f.math.interpolate", "(", list[real], ",", list[real], ",", real, [",", string], ")"
```

- Operands: **xs** (x-coordinates, must be sorted ascending, unique), **ys** (y-coordinates), **x** (evaluation point), optional **method** ("linear" (default) or "polynomial").
- Result: **real** — interpolated or extrapolated y-value.
- A **Value Error** is raised if `len(xs) != len(ys)`, if fewer than 2 data points, if fewer than 4 points with "polynomial" method, if x values are not unique, or if x values are not sorted.
- A **Type Error** is raised if data lists contain non-numeric values.

Runge’s Phenomenon For equally-spaced data points, high-degree polynomial interpolation can oscillate wildly near the edges. Use "linear" or `f.math.spline` instead for more than 10 data points.

Linear interpolation

```
f.math.interpolate([0, 1, 2], [0, 1, 4], 1.5)           #c# 2.25
f.math.interpolate([0, 1, 2], [0, 1, 4], 0.5)           #c# 0.5
f.math.interpolate([0, 1, 2], [0, 1, 4], 3.0)           #c# 7.0 (extrapolated)
```

Polynomial interpolation (exact for quadratic data)

```
f.math.interpolate([0, 1, 2], [0, 1, 4], 1.5, "polynomial") #c# 2.25
f.math.interpolate([0, 1, 2, 3], [0, 1, 4, 9], 2.5, "polynomial") #c# 6.25
```

Spline Interpolation

The `f.math.spline` function performs cubic spline interpolation. Fits piecewise cubic polynomials between data points such that the result is continuous and has continuous first and second derivatives at all interior data points.

Advantages over polynomial interpolation: No Runge’s oscillation. Smooth (continuous f , f' , f''). Stable for large numbers of data points.

Boundary conditions:

- "natural" (default) — Zero second derivative at the endpoints: $f''(x_0) = 0$, $f''(x_n) = 0$. Produces the “smoothest” curve.
- "clamp" — Zero first derivative at the endpoints: $f'(x_0) = 0$, $f'(x_n) = 0$. Curve is flat at the boundaries.

- **"periodic"** — The function wraps around: $f(x_0) = f(x_n)$, $f'(x_0) = f'(x_n)$, $f''(x_0) = f''(x_n)$. For periodic data.

Extrapolation: Beyond the data range, the spline extends using the boundary slope of the nearest segment. No error is raised, but accuracy may be poor.

EBNF:

```
"f.math.spline", "(", list[real], ",", list[real], ",", real, [",", string], ")"
```

- Operands: **xs** (x-coordinates, must be sorted ascending, unique), **ys** (y-coordinates), **x** (evaluation point), optional **boundary** ("natural" (default), "clamp", or "periodic").
- Result: **real** — interpolated or extrapolated y-value.
- A **Value Error** is raised if $\text{len}(\text{xs}) \neq \text{len}(\text{ys})$, if fewer than 3 data points, if x values are not unique, if x values are not sorted, or if **boundary** is unknown.
- A **Type Error** is raised if data lists contain non-numeric values.

Natural spline

```
f.math.spline([0, 1, 2], [0, 1, 4], 1.5) #c# 2.25
```

Clamped spline (flat at boundaries)

```
f.math.spline([0, 1, 2], [0, 1, 4], 1.5, "clamp") #c# 2.25
```

Periodic spline

```
f.math.spline([0, 1, 2, 3], [0, 1, 0, -1], 4.0, "periodic") #c# 0.0 (wraps to start)
```

Adaptive Integration

The `f.math.integrateAdaptive` function computes the definite integral $\int_a^b f(x) dx$ using adaptive Simpson quadrature. The algorithm subdivides intervals where the integrand changes rapidly, concentrating computation where it matters most.

Algorithm: Recursively apply Simpson's rule to each subinterval. If the estimated error exceeds `tol`, split the subinterval and recurse. Stops when the estimated total error is below `tol`.

Accuracy: Typically achieves the requested tolerance. The actual error is bounded by the estimated error. For smooth functions, the error is much smaller than the tolerance.

Behavior:

- If $a = b$, returns 0.0 (zero-width interval).
- If $a > b$, computes $-\int_b^a f(x) dx$ (reversed bounds).
- The function may be called many times (adaptive subdivision). Ensure f is efficient.

EBNF:

```
"f.math.integrateAdaptive", "(", func, ",", real, ",", real, [",", real], ")"
```

- Operands: **\$f** (function $f(x)$), **a** (lower bound), **b** (upper bound), optional **tol** (**real**, default 10^{-8}).

- Result: `real` — approximate integral $\int_a^b f(x) dx$.
- A `Runtime Error` is raised if the integral fails to converge (e.g., oscillatory or singular integrand).
- A `Value Error` is raised if `tol` ≤ 0 .
- A `Type Error` is raised if `$f` is not a function reference.

```
$f sq = func($x) $x * $x
f.math.integrateAdaptive($sq, 0, 1)          ## 0.33333 (1/3, exact)
f.math.integrateAdaptive($sq, 0, 1, 1e-6)    ## 0.33333 (custom tolerance)
f.math.integrateAdaptive($sq, 1, 0)          ## -0.33333 (reversed bounds)

# Integral of sin(x) from 0 to pi = 2.0
$f sinx = func($x) f.math.sin($x)
f.math.integrateAdaptive($sinx, 0, c.math.pi) ## 2.0
```

Numerical Differentiation

The `f.math.differentiateNumerically` function computes the first derivative $f'(x)$ using central differences: $f'(x) \approx (f(x+h) - f(x-h))/(2h)$.

Accuracy: Central differences have error $O(h^2)$, which is more accurate than forward differences ($O(h)$). The optimal step size balances truncation error ($O(h^2)$) against floating-point roundoff error ($O(\epsilon/h)$), typically $h \approx \sqrt{\epsilon} \cdot |x| \approx 10^{-5}$ for unit-magnitude x .

Behavior:

- If $h < 0$, uses $|h|$ (no error, documented).
- If $x = 0$, uses h as the absolute step (not relative).
- The function is called twice per evaluation ($f(x+h)$ and $f(x-h)$).

EBNF:

```
"f.math.differentiateNumerically", "(", func, ",", real, ["", real], ")"
```

- Operands: `$f` (function $f(x)$), `x` (evaluation point), optional `h` (step size, default 10^{-5}).
- Result: `real` — approximate first derivative $f'(x)$.
- A `Value Error` is raised if $h = 0$.
- A `Type Error` is raised if `$f` is not a function reference.

```
$f sq = func($x) $x * $x
f.math.differentiateNumerically($sq, 3)      ## 6.0 (f'(3) = 2*3 = 6)
f.math.differentiateNumerically($sq, 3, 1e-8) ## 6.0 (smaller step, same result)
f.math.differentiateNumerically($sq, 0)      ## 0.0 (f'(0) = 0)
```

ODE Solver

The `f.math.odeSolve` function solves a system of ordinary differential equations $\frac{dy}{dt} = f(t, y)$ over a time span $[t_0, t_{End}]$ with initial conditions $y(t_0) = y_0$.

Output: Returns a list of $[t, y]$ pairs sampled at regular intervals. For non-adaptive methods, the step count is 1000 (plus the start point), giving 1001 points. For the adaptive "rkf45" method, the step count and spacing vary based on the local error estimate.

Stiff systems: Stiff ODEs have widely separated time scales and require implicit methods for stability. Explicit methods (Euler, Heun, RK2/3/4) will either require extremely small step sizes or produce unstable (divergent) results when applied to stiff systems. Use implicit methods ("eulerBack", "midpointImp", "glrk*") for stiff problems.

\$dydt Signature

- For 1D (y_0 is real): `$dydt(t, y) → real`.
- For systems (y_0 is `list[real]`): `$dydt(t, y) → list[real]`.
- Return type must match y_0 type. A `Type Error` is raised on mismatch.

EBNF:

```
"f.math.odeSolve", "(", func, ",", real, ",", real, ",", ( real | list[real] ), [",", st
```

- Operands: `$dydt` (derivative function), `t0` (start time), `tEnd` (end time), `y0` (initial conditions: `real` for 1D, `list[real]` for systems), optional `method` (string, default "rk4").
- Result: `list[list[real]]` — trajectory as `[[t0, y0], [t1, y1], ...]`. Always includes the start point.
- If $t_0 = t_{End}$, returns `[[t0, y0]]` (single point).
- If $t_0 > t_{End}$, integrates backwards in time (no error).
- A `Runtime Error` is raised if the solver fails (e.g., stiff system with explicit method, or step failure).
- A `Value Error` is raised if `y0` is empty or if `method` is unknown.
- A `Type Error` is raised if `$dydt` is not a function reference.

Explicit Methods Explicit methods compute y_{n+1} directly from known values. Fast per step but may require very small step sizes for stiff systems.

- "euler" — Forward Euler, order 1. Simplest method. $y_{n+1} = y_n + h \cdot f(t_n, y_n)$. First-order accurate: error $O(h)$. Not suitable for stiff systems. Use only for quick estimates or non-stiff problems.
- "heun" — Heun's method (improved Euler), order 2. Predictor-corrector: predicts with Euler, corrects with average slope. Second-order accurate: error $O(h^2)$. Better than Euler for same step size.

- **"rk2"** — Midpoint method (RK2), order 2. Evaluates the slope at the midpoint of the interval. Second-order accurate: error $O(h^2)$. More stable than Heun for oscillatory problems.
- **"rk3"** — Kutta's third-order method (RK3), order 3. Three-stage method. Third-order accurate: error $O(h^3)$. Good balance of accuracy and cost.
- **"rk4"** — Classic fourth-order Runge-Kutta (RK4), order 4. (default) Four-stage method. Fourth-order accurate: error $O(h^4)$. The standard choice for non-stiff ODEs. Good accuracy for moderate computational cost.
- **"rkf45"** — Runge-Kutta-Fehlberg, order 4(5) with adaptive step size. Computes both 4th and 5th order solutions and uses the difference to estimate local error. Adjusts step size to keep the error below a target tolerance (default 10^{-6}). Step size bounds: minimum 10^{-12} , maximum $(t_{End} - t_0)/10$. May take fewer or more than 1000 steps depending on the problem.

Implicit Methods Implicit methods solve an equation at each step (typically using Newton iteration). More expensive per step but unconditionally stable for stiff systems.

- **"eulerBack"** — Backward Euler, order 1. $y_{n+1} = y_n + h \cdot f(t_{n+1}, y_{n+1})$. First-order accurate. A-stable (stable for all step sizes). The simplest stiff-safe method. Recommended for very stiff systems where accuracy is not critical.
- **"midpointImp"** — Implicit Midpoint, order 2. Symplectic method (preserves energy for Hamiltonian systems). Second-order accurate. A-stable. Good for long-time integration of conservative systems.
- **"glrk2"** — Gauss-Legendre RK of order 2 (implicit midpoint rule). Second-order accurate. A-stable. Symplectic. One-stage implicit method.
- **"glrk4"** — Gauss-Legendre RK of order 4. Fourth-order accurate. A-stable. Symplectic. Two-stage implicit method. Good balance of accuracy and stability for moderate-stiffness systems.
- **"glrk6"** — Gauss-Legendre RK of order 6. Sixth-order accurate. A-stable. Symplectic. Three-stage implicit method. High accuracy for smooth solutions.
- **"glrk8"** — Gauss-Legendre RK of order 8. Eighth-order accurate. A-stable. Symplectic. Four-stage implicit method. Highest available accuracy. Use when extreme precision is required.

Method Selection Guide

Method	Order	Adaptive	Stiff-safe
euler	1	No	No
heun	2	No	No
rk2	2	No	No
rk3	3	No	No
rk4	4	No	No
rkf45	4(5)	Yes	No
eulerBack	1	No	Yes
midpointImp	2	No	Yes
glrk2	2	No	Yes
glrk4	4	No	Yes
glrk6	6	No	Yes
glrk8	8	No	Yes

Guidelines:

- Start with "rk4" for non-stiff problems. It is reliable and well-understood.
- Use "rkf45" when you need automatic step size control or when the solution has varying time scales.
- Switch to "eulerBack" or "glrk4" if "rk4" produces unstable (divergent) results.
- Use "glrk8" only when high accuracy is required and the function evaluations are cheap.
- For Hamiltonian/conservative systems, prefer "midpointImp" or "glrk*" (symplectic methods preserve energy).

Examples

```
# 1D: Exponential decay dy/dt = -y, y(0) = 1
$f decay = func($t, $y) -$y
$f sol = f.math.odeSolve($decay, 0, 5, 1.0)
# sol = [[0, 1.0], [0.005, 0.995], ..., [5.0, 0.00674]]

# Same with backward Euler (stiff-safe)
$f sol = f.math.odeSolve($decay, 0, 5, 1.0, "eulerBack")

# Same with adaptive RK45
$f sol = f.math.odeSolve($decay, 0, 5, 1.0, "rkf45")
# sol may have more or fewer than 1001 points

# System: Lotka-Volterra (predator-prey)
$f lotka = func($t, $y) [
  1.5 * $y[0] - 1.0 * $y[0] * $y[1],
  -3.0 * $y[1] + 1.0 * $y[0] * $y[1]
]
$f sol = f.math.odeSolve($lotka, 0, 10, [1.0, 1.0])
```

```
# sol = [[0, [1.0, 1.0]], [0.01, [1.015, 0.970]], ..., [10, [...]]]

# Backwards in time
$f sol = f.math.odeSolve($decay, 5, 0, 0.01)
# Integrates from t=5 to t=0
```

11.1.18 Shared Functions

Assert Function

The `f.util.assert()` function checks a boolean condition and raises an error if `False`. See Section 11.1.8 for the full documentation (keyword and function forms, error type aliases, and examples).

Length/Size Function

The `len` function returns the length of a list or string, or the size of a set.
Morphology:

EBNF:

```
"f.util.len", "(", ( list | string | set ), ")"
```

Edge Cases – Collection Utilities

- A `Type Error` is raised if the argument to `f.util.len` is not a list, string, or set.
- A `Type Error` is raised if the arguments to `f.list.concat` are not all lists or all strings.
- A `Type Error` is raised if the argument to `f.list.fromSet` is not a set.
- A `Type Error` is raised if the argument to `f.list.set` is not a list.

Examples:

```
f.util.len([1, 2, 3])      #c# Returns 3
f.util.len("hello")        #c# Returns 5
f.util.len({Q1, Q2, R5})   #c# Returns 3
```

11.1.19 Special Functions

Print Function

The `f.util.print()` function outputs its argument to the console. Unlike `echo`, it can evaluate expressions and function calls directly.

Morphology:

EBNF:

```
"f.util.print", "(", argument, ")"
```

Example:

```
f.util.print("Hello, World!")
f.util.print(3 + 2 * 5)    #c# Outputs: 13
```

Type Function

The **type** function returns the type of an object as a string.

Morphology:

EBNF:

```
"f.util.type", "(", argument, ")"
```

Example:

```
f.util.type(42)          #c# Returns "int"
f.util.type(3.14)        #c# Returns "real"
f.util.type("hello")     #c# Returns "string"
f.util.type(Q1)          #c# Returns "part"
```

Reference Function

The **ref** function converts a string containing a part reference into the referenced part.

Morphology:

EBNF:

```
"f.util.ref", "(", string, ")"
```

Example:

```
f.util.ref("R1")        #c# Returns the part referenced as R1
```

No-Operation Function

The **noop** function is the identity function. It returns its argument unchanged.

Morphology:

EBNF:

```
"f.util.noop", "(", argument, ")"
```

Example:

```
f.util.noop(42)          #c# Returns 42
f.util.noop("hello")     #c# Returns "hello"
f.util.noop(Q1)          #c# Returns Q1
```

Use **noop** as a placeholder in queries where no transformation is needed.

Soft Integer Conversion

The **softint** function converts its argument to an integer if it is either already an integer or is a real number numerically equal to an integer. Otherwise it returns the argument unchanged.

Morphology:

EBNF:

```
"f.util.softint", "(", argument, ")"
```

Examples:

```
f.util.softint(42)      #c# Returns 42 (already int)
f.util.softint(5.0)     #c# Returns 5 (real equal to int)
f.util.softint(3.7)     #c# Returns 3.7 (not equal to int, unchanged)
f.util.softint("hello") #c# Returns "hello" (not numeric, unchanged)
```

Soft Real Conversion

The `softreal` function converts its argument to a real if it is either already a real, an integer, or a complex number with zero imaginary part. Otherwise it returns the argument unchanged.

Morphology:

EBNF:

```
"f.util.softreal", "(", argument, ")"
```

Examples:

```
f.util.softreal(42)      #c# Returns 42.0 (int promoted to real)
f.util.softreal(3.14)    #c# Returns 3.14 (already real)
f.util.softreal(5 + 0i)  #c# Returns 5.0 (complex with zero imag part)
f.util.softreal(3 + 4i)  #c# Returns 3 + 4i (complex with non-zero imag, unchanged)
f.util.softreal("hello") #c# Returns "hello" (not numeric, unchanged)
```

11.1.20 Engineering Math Functions

Magnitude to dB Function

Computes $10 \cdot \log_{10}(x)$, converting a power ratio to decibels.

Morphology:

EBNF:

```
"f.ee.db", "(", ( int | real ), ")"
```

- Operands: `int` or `real` (must be > 0).
- Result: `real`.
- A `Value Error` is raised for $x \leq 0$.
- A `Type Error` is raised if the argument is not numeric.

Examples:

```
f.ee.db(1)      #c# Returns 0.0
f.ee.db(10)     #c# Returns 10.0
f.ee.db(100)    #c# Returns 20.0
f.ee.db(0.5)    #c# Returns approximately -3.0103
```


dB10 Function

Alias for `f.ee.db`: computes $10 \cdot \log_{10}(x)$. Same semantics, provided for clarity in power contexts.

Morphology:

EBNF:

```
"f.ee.db10", "(", ( int | real ), ")"
```

- Operands: `int` or `real` (must be > 0).
- Result: `real`.
- A Value Error is raised for $x \leq 0$.

Examples:

```
f.db10(10)      #c# Returns 10.0
f.db10(100)     #c# Returns 20.0
```

dB20 Function

Computes $20 \cdot \log_{10}(x)$, converting a voltage/current ratio to decibels.

Morphology:

EBNF:

```
"f.ee.db20", "(", ( int | real ), ")"
```

- Operands: `int` or `real` (must be > 0).
- Result: `real`.
- A Value Error is raised for $x \leq 0$.

Examples:

```
f.db20(1)       #c# Returns 0.0
f.db20(10)      #c# Returns 20.0
f.db20(0.001)   #c# Returns -60.0
```

dB to Magnitude Function

Converts a decibel value back to a magnitude (inverse of `f.ee.db`). Computes $10^{x/10}$.

Morphology:

EBNF:

```
"f.ee.fromDb", "(", ( int | real ), ")"
```

- Operands: `int` or `real`.
- Result: `real`.
- A Type Error is raised if the argument is not numeric.

Examples:

```
f.fromDb(0)     #c# Returns 1.0
f.fromDb(20)    #c# Returns 100.0
f.fromDb(-3)    #c# Returns approximately 0.5012
```

Voltage Divider Function

Computes $V_{\text{out}} = V_{\text{in}} \cdot \frac{r_2}{r_1 + r_2}$.

Morphology:

EBNF:

```
"f.ee.voltageDivider", "(", ( int | real ), ",", ( int | real ), ",", ( int | real ), ")"
```

- Operands: r_1 (int or real), r_2 (int or real), V_{in} (int or real).
- Result: real.
- A Division by Zero Error is raised if $r_1 + r_2 = 0$.
- A Type Error is raised if arguments are not numeric.

Examples:

```
f.ee.voltageDivider(1000, 1000, 5)    ## Returns 2.5  
f.ee.voltageDivider(1000, 2000, 3.3)  ## Returns 2.2
```

Current Divider Function

Computes $I_1 = I \cdot \frac{r_2}{r_1 + r_2}$ (current through r_1 in a parallel circuit).

Morphology:

EBNF:

```
"f.ee.currentDivider", "(", ( int | real ), ",", ( int | real ), ",", ( int | real ), ")"
```

- Operands: r_1 (int or real), r_2 (int or real), I (int or real).
- Result: real.
- A Division by Zero Error is raised if $r_1 + r_2 = 0$.
- A Type Error is raised if arguments are not numeric.

Examples:

```
f.ee.currentDivider(100, 100, 1)      ## Returns 0.5  
f.ee.currentDivider(100, 300, 0.01)   ## Returns 0.0075
```

LC Resonance Frequency Function

Computes $f = \frac{1}{2\pi\sqrt{LC}}$.

Morphology:

EBNF:

```
"f.ee.lcFrequency", "(", ( int | real ), ",", ( int | real ), ")"
```

- Operands: L (inductance, int or real), C (capacitance, int or real).
- Result: real (frequency in Hz).
- A Value Error is raised if either argument is ≤ 0 .

- A **Type Error** is raised if arguments are not numeric.

Examples:

```
f.ee.lcFrequency(0.001, 0.000001)    ## Returns approximately 5032.9
f.ee.lcFrequency(10e-6, 100e-12)     ## Returns approximately 503292.1
```

Reactance Function

Computes the reactance $X = \omega L - \frac{1}{\omega C}$ for a series LC circuit.

Morphology:

EBNF:

```
"f.ee.reactance", "(", ( int | real ), ",", ( int | real ), ",", ( int | real ), ")"
```

- Operands: frequency (int or real), L (inductance, int or real), C (capacitance, int or real).
- Result: real.
- A **Division by Zero Error** is raised if frequency = 0 or $C = 0$.
- A **Type Error** is raised if arguments are not numeric.

Examples:

```
f.ee.reactance(1000, 0.001, 0.000001)  ## Returns approximately 5.36
```

Unit Conversion Function

Converts a numeric value from one unit to another within a specified physical dimension. The dimension parameter scopes the conversion, making cross-dimension conversions (e.g. Farad to °F) impossible by design. Temperature conversions are offset-based. Logarithmic power units (dBm, dBW) are supported within the **"power"** dimension.

Morphology:

EBNF:

```
"f.math.convert", "(", ( int | real ), ",", string, ",", string, ",", string, ")"
```

- Operands: value (int or real), dimension (string), from_unit (string), to_unit (string).
- Result: real.
- Supported dimensions and their unit strings are listed in Table 11.1.
- A **Value Error** is raised if the dimension string is not recognized, or if either unit string does not belong to the specified dimension.
- A **Type Error** is raised if the value is not numeric.

Supported dimensions and unit strings:

Examples:

Dimension	Unit Strings	Base
"length"	"pl" (Planck), "fm" (fermi), "pm", "A" (angstrom), "nm", "um" (micron), "mm", "cm", "m", "km", "mil", "pt" / "point", "in", "ft", "survey_foot", "yd", "mi", "survey_mile", "nmi", "au" / "astronomical_unit", "ly" / "light_year", "pc" / "parsec"	m
"mass"	"ug", "mg", "g" / "gram", "grain", "kg", "carat", "t" / "metric_ton", "oz" / "ounce", "troy_ounce", "lb" / "pound", "troy_pound", "stone", "blob" / "slinch", "slug", "short_ton", "long_ton", "u" / "m_u" / "amu" / "atomic_mass"	kg
"time"	"ps", "ns", "us", "ms", "s", "min", "h", "d", "wk", "yr", "Julian_year"	s
"frequency"	"mHz", "Hz", "kHz", "MHz", "GHz", "THz"	Hz
"angle"	"deg" / "degree", "rad", "grad", "arcmin" / "arcminute", "arcsec" / "arcsecond", "rev"	rad
"temperature"	"C", "F", "K", "R" (Rankine), "zero_Celsius", "degree_Fahrenheit"	K (offset)
"voltage"	"uV", "mV", "V", "kV", "MV", "dBV"	V
"current"	"pA", "nA", "uA", "mA", "A", "kA"	A
"resistance"	"uohm", "mohm", "ohm", "kohm", "Mohm", "Gohm"	Ω
"capacitance"	"pF", "nF", "uF", "mF", "F"	F
"inductance"	"nH", "uH", "mH", "H"	H
"charge"	"pC", "nC", "uC", "mC", "C", "Ah", "mAh"	C
"power"	"uW", "mW", "W", "kW", "MW", "hp" / "horsepower", "dBm", "dBW"	W
"energy"	"eV" / "electron_volt", "keV", "MeV", "erg", "J", "kJ", "cal" / "calorie_th", "calorie_IT", "kcal", "BTU" / "Btu_th", "Btu_IT", "ton_TNT"	J
"force"	"dyn" / "dyne", "N", "kN", "lbf" / "pound_force", "kgf" / "kilogram_force"	N
"pressure"	"Pa", "kPa", "MPa", "bar", "mbar", "atm" / "atmosphere", "psi", "torr" / "mmHg"	Pa
"area"	"mm2", "cm2", "m2", "km2", "in2", "ft2", "ac" / "acre", "ha" / "hectare", "mi2"	m ²
"volume"	"mm3", "cm3", "m3", "mL", "L" / "liter" / "litre", "in3", "ft3", "gal" / "gallon_US", "gallon_imp", "fl_oz" / "fluid_ounce_US", "fluid_ounce_imp", "bbl" / "barrel"	m ³
"velocity"	"m/s", "km/h" / "kmh", "mph", "kn" / "knot", "mach" / "speed_of_sound", "c"	m/s
"acceleration"	"m/s2", "g"	m/s ²
"voltage*time"	"Vs", "mVs", "uVs", "Wb"	V·s

Table 11.1: Supported dimensions and unit strings for `f.math.convert`.

Length: astronomical

```
f.math.convert(1, "length", "parsec", "light_year")    #c# Returns approximately 3.2616
f.math.convert(1, "length", "light_year", "km")        #c# Returns approximately 9.461e12
f.math.convert(1, "length", "au", "m")                 #c# Returns approximately 1.496e11
```

Length: imperial / PCB

```
f.math.convert(1, "length", "mile", "km")              #c# Returns approximately 1.609
f.math.convert(1, "length", "angstrom", "nm")          #c# Returns 0.1
f.math.convert(25.4, "length", "mil", "mm")            #c# Returns 25.4
f.math.convert(1, "length", "inch", "mm")              #c# Returns 25.4
f.math.convert(1, "length", "survey_foot", "ft")       #c# Returns approximately 1.000002
```

Mass

```
f.math.convert(1, "mass", "pound", "kg")               #c# Returns approximately 0.4536
f.math.convert(1, "mass", "stone", "lb")               #c# Returns 14.0
f.math.convert(1, "mass", "slug", "kg")                #c# Returns approximately 14.594
f.math.convert(1, "mass", "carat", "mg")               #c# Returns 200.0
f.math.convert(1, "mass", "troy_ounce", "g")           #c# Returns approximately 31.103
f.math.convert(1, "mass", "long_ton", "kg")            #c# Returns approximately 1016.0
```

```

f.math.convert(1, "mass", "short_ton", "kg")           #c# Returns approximately 907.2

# Time
f.math.convert(1, "time", "hour", "s")                #c# Returns 3600.0
f.math.convert(1, "time", "day", "hour")              #c# Returns 24.0
f.math.convert(1, "time", "Julian_year", "day")       #c# Returns 365.25

# Electrical
f.math.convert(100, "resistance", "ohm", "kohm")      #c# Returns 0.1
f.math.convert(1, "frequency", "MHz", "Hz")          #c# Returns 1000000.0
f.math.convert(30, "power", "dBm", "mW")             #c# Returns 1000.0
f.math.convert(1, "power", "horsepower", "W")         #c# Returns approximately 745.7

# Force
f.math.convert(1, "force", "lbf", "N")                #c# Returns approximately 4.448
f.math.convert(1, "force", "dyne", "N")              #c# Returns 1e-5

# Energy
f.math.convert(1, "energy", "electron_volt", "J")     #c# Returns approximately 1.602e-19
f.math.convert(1, "energy", "calorie_IT", "J")        #c# Returns approximately 4.1868
f.math.convert(1, "energy", "Btu_IT", "J")           #c# Returns approximately 1055.06
f.math.convert(1, "energy", "erg", "J")              #c# Returns 1e-7
f.math.convert(1, "energy", "ton_TNT", "kJ")         #c# Returns 4184.0

# Temperature
f.math.convert(100, "temperature", "C", "F")         #c# Returns 212.0
f.math.convert(0, "temperature", "C", "K")           #c# Returns 273.15

# Pressure
f.math.convert(1, "pressure", "atm", "psi")           #c# Returns approximately 14.696
f.math.convert(1, "pressure", "torr", "Pa")          #c# Returns approximately 133.322

# Velocity
f.math.convert(1, "velocity", "mach", "m/s")          #c# Returns approximately 340.3
f.math.convert(1, "velocity", "knot", "mph")          #c# Returns approximately 1.151

# Volume
f.math.convert(1, "volume", "gallon_imp", "L")        #c# Returns approximately 4.546
f.math.convert(1, "volume", "barrel", "L")           #c# Returns approximately 159.0

# Angle
f.math.convert(1, "angle", "rev", "deg")             #c# Returns 360.0
f.math.convert(90, "angle", "degree", "arcminute")   #c# Returns 5400.0

```

Wavelength to Frequency

Converts a wavelength (in meters) to optical frequency (in Hz) using $\nu = c/\lambda$, where $c = 299\,792\,458$ m/s.

Morphology:

EBNF:

```
"f.ee.lambda2nu", "(", ( int | real ), ")"
```

- Operands: wavelength in meters (**int** or **real**).
- Result: frequency in Hz (**real**).
- A **Division by Zero Error** is raised if the wavelength is 0.
- A **Type Error** is raised if the argument is not numeric.

Examples:

```
f.ee.lambda2nu(500e-9)      #c# Returns approximately 5.996e14 (green light)
f.ee.lambda2nu(1.55e-6)     #c# Returns approximately 1.934e14 (C-band telecom)
f.ee.lambda2nu(10.6e-6)     #c# Returns approximately 2.828e13 (CO2 laser)
```

Frequency to Wavelength

Converts an optical frequency (in Hz) to wavelength (in meters) using $\lambda = c/\nu$, where $c = 299\,792\,458$ m/s.

Morphology:

EBNF:

```
"f.ee.nu2lambda", "(", ( int | real ), ")"
```

- Operands: frequency in Hz (**int** or **real**).
- Result: wavelength in meters (**real**).
- A **Division by Zero Error** is raised if the frequency is 0.
- A **Type Error** is raised if the argument is not numeric.

Examples:

```
f.ee.nu2lambda(5.996e14)    #c# Returns approximately 5.000e-07 (green light)
f.ee.nu2lambda(1.934e14)    #c# Returns approximately 1.550e-06 (C-band telecom)
f.ee.nu2lambda(1e9)         #c# Returns approximately 0.2998 (1 GHz radio)
```

Error Function

Computes the Gauss error function $\text{erf}(x) = \frac{2}{\sqrt{\pi}} \int_0^x e^{-t^2} dt$.

Morphology:

EBNF:

```
"f.special.erf", "(", ( int | real ), ")"
```

- Operands: `int` or `real`.
- Result: `real`.
- Odd function: $\operatorname{erf}(-x) = -\operatorname{erf}(x)$.
- A `Type Error` is raised if the argument is not numeric.

Examples:

```
f.special.erf(0)           ## Returns 0.0
f.special.erf(1)           ## Returns approximately 0.8427
f.special.erf(-1)          ## Returns approximately -0.8427
```

Complementary Error Function

Computes $\operatorname{erfc}(x) = 1 - \operatorname{erf}(x)$.

Morphology:

EBNF:

```
"f.special.erfc", "(", ( int | real ), ")"
```

- Operands: `int` or `real`.
- Result: `real`.
- A `Type Error` is raised if the argument is not numeric.

Examples:

```
f.special.erfc(0)          ## Returns 1.0
f.special.erfc(1)          ## Returns approximately 0.1573
```

Imaginary Error Function

Computes $\operatorname{erfi}(x) = -i \cdot \operatorname{erf}(ix)$.

Morphology:

EBNF:

```
"f.special.erfi", "(", ( int | real ), ")"
```

- Operands: `int` or `real`.
- Result: `real`.
- Odd function: $\operatorname{erfi}(-x) = -\operatorname{erfi}(x)$.
- A `Type Error` is raised if the argument is not numeric.

Examples:

```
f.special.erfi(0)          ## Returns 0.0
f.special.erfi(1)          ## Returns approximately 1.6504
```

Bessel J Function

Bessel function of the first kind $J_n(x)$.

Morphology:

EBNF:

```
"f.special.besselJ", "(", int, ",", real, ")"
```

- Operands: order n (int), argument x (int or real).
- Result: real.
- A Type Error is raised if n is not int or x is not numeric.

Examples:

```
f.special.besselJ(0, 0)    #c# Returns 1.0
f.special.besselJ(0, 1)    #c# Returns approximately 0.7652
f.special.besselJ(1, 1)    #c# Returns approximately 0.4401
```

Bessel Y Function

Bessel function of the second kind $Y_n(x)$ (Weber/Neumann function).

Morphology:

EBNF:

```
"f.special.besselY", "(", int, ",", real, ")"
```

- Operands: order n (int), argument x (int or real).
- Result: real.
- A Value Error is raised for $x = 0$ (singularity).
- A Type Error is raised if n is not int or x is not numeric.

Examples:

```
f.special.besselY(0, 1)    #c# Returns approximately 0.0883
```

Bessel I Function

Modified Bessel function of the first kind $I_n(x)$.

Morphology:

EBNF:

```
"f.special.besselI", "(", int, ",", real, ")"
```

- Operands: order n (int), argument x (int or real).
- Result: real.
- A Type Error is raised if n is not int or x is not numeric.

Examples:

```
f.special.besselI(0, 1)    #c# Returns approximately 1.2661
```


Bessel K Function

Modified Bessel function of the second kind $K_n(x)$.

Morphology:

EBNF:

```
"f.special.besselK", "(", int, ",", real, ")"
```

- Operands: order n (`int`), argument x (`int` or `real`).
- Result: `real`.
- A `Value Error` is raised for $x = 0$ (singularity).
- A `Type Error` is raised if n is not `int` or x is not numeric.

Examples:

```
f.special.besselK(0, 1)    #c# Returns approximately 0.4210
```

Lambert W Function

Computes the Lambert W function, the inverse of xe^x .

Morphology:

EBNF:

```
"f.special.lambertW", "(", real, ["(", int, "], ")"
```

- Operands: x (`int` or `real`), optional branch k (`int`, default 0).
- Result: `real`.
- `f.special.lambertW(x)` returns the principal branch $W_0(x)$.
- `f.special.lambertW(x, k)` returns the branch $W_k(x)$.
- For the principal branch ($k = 0$), a `Value Error` is raised for $x < -1/e$ (domain restriction).
- A `Type Error` is raised if arguments are not numeric.

Examples:

```
f.special.lambertW(0)      #c# Returns 0.0
f.special.lambertW(1)      #c# Returns approximately 0.5671
f.special.lambertW(10, 0)  #c# Returns approximately 1.7455
```

Dilogarithm Function

Computes the dilogarithm $\text{Li}_2(x) = -\int_0^x \frac{\ln(1-t)}{t} dt$.

Morphology:

EBNF:

```
"f.special.Li2", "(", ( int | real | complex ), ")"
```

- Operands: `int`, `real`, or `complex`.
- Result: `real` for real input, `complex` for complex input.
- A `Type Error` is raised for non-numeric arguments.

Examples:

```
f.special.Li2(0)          #c# Returns 0.0
f.special.Li2(1)          #c# Returns approximately 1.6449 (= pi^2/6)
```

Trilogarithm Function

Computes the trilogarithm $\text{Li}_3(x) = \sum_{k=1}^{\infty} \frac{x^k}{k^3}$.

Morphology:

EBNF:

```
"f.special.Li3", "(", ( int | real | complex ), ")"
```

- Operands: `int`, `real`, or `complex`.
- Result: `real` for real input, `complex` for complex input.
- A `Type Error` is raised for non-numeric arguments.

Examples:

```
f.special.Li3(1)          #c# Returns approximately 1.2021 (= zeta(3))
```

General Polylogarithm Function

Computes the polylogarithm $\text{Li}_s(x) = \sum_{k=1}^{\infty} \frac{x^k}{k^s}$ for arbitrary order s .

Morphology:

EBNF:

```
"f.special.Li", "(", ( int | real | complex ), ",", ( int | real | complex ), ")"
```

- Operands: order s (`int` or `real`), argument x (`int`, `real`, or `complex`).
- Result: `real` for real input, `complex` for complex input.
- When $s = 2$, equivalent to `f.Li2`. When $s = 3$, equivalent to `f.Li3`.
- A `Type Error` is raised for non-numeric arguments.

Examples:

```
f.special.Li(2, 0.5)      #c# Returns approximately 0.5822
f.special.Li(2, 1)        #c# Returns approximately 1.6449 (= f.special.Li2(1))
f.special.Li(3, 1)        #c# Returns approximately 1.2021 (= f.special.Li3(1))
```

- `f.Li(s, x)` – polylogarithm $\text{Li}_s(x) = \sum_{k=1}^{\infty} \frac{x^k}{k^s}$
- s must be `int` (≥ 1), x can be `int`, `real`, or `complex`.
- `f.Li(2, x)` is equivalent to `f.Li2(x)`, `f.Li(3, x)` to `f.Li3(x)`.

```
f.special.Li(1, 0.5)      #c# Returns approximately 0.6931 (= -ln(1 - 0.5))
f.special.Li(2, 1)        #c# Returns approximately 1.6449 (= pi^2/6)
f.special.Li(4, 1)        #c# Returns approximately 1.0823 (= pi^4/90)
```

Digamma Function

Computes the digamma function $\psi(x) = \frac{d}{dx} \ln(\Gamma(x))$.

Morphology:

EBNF:

```
"f.special.digamma", "(", ( int | real ), ")"
```

- Operands: int or real.
- Result: real.
- A Value Error is raised for non-positive integers (poles of the gamma function).
- A Type Error is raised if the argument is not numeric.

Examples:

```
f.special.digamma(1)      ## Returns approximately -0.5772 (= -gamma)
f.special.digamma(2)      ## Returns approximately 0.4228
```

Trigamma Function

Computes the trigamma function $\psi'(x) = \frac{d^2}{dx^2} \ln(\Gamma(x))$.

Morphology:

EBNF:

```
"f.special.trigamma", "(", ( int | real ), ")"
```

- Operands: int or real.
- Result: real.
- A Value Error is raised for non-positive integers (poles).
- A Type Error is raised if the argument is not numeric.

Examples:

```
f.special.trigamma(1)      ## Returns approximately 1.6449 (= pi^2/6)
```

Beta Function

Computes the beta function $B(a, b) = \frac{\Gamma(a)\Gamma(b)}{\Gamma(a+b)}$.

Morphology:

EBNF:

```
"f.special.beta", "(", ( int | real ), ",", ( int | real ), ")"
```

- Operands: a (int or real), b (int or real).
- Result: real.
- Symmetric: $B(a, b) = B(b, a)$.
- A Value Error is raised if either argument is a non-positive integer (poles).

- A **Type Error** is raised if arguments are not numeric.

Examples:

```
f.special.beta(1, 1)      ## Returns 1.0
f.special.beta(2, 3)      ## Returns approximately 0.0833 (= 1/12)
```

Legendre P Function

Computes the Legendre polynomial $P_n(x)$.

Morphology:

EBNF:

```
"f.special.legendreP", "(", int, ",", real, ")"
```

- Operands: degree n (int ≥ 0), argument x (int or real).
- Result: real.
- A **Type Error** is raised if arguments are not the correct types.

Examples:

```
f.special.legendreP(0, 0.5)  ## Returns 1.0
f.special.legendreP(1, 0.5)  ## Returns 0.5
f.special.legendreP(2, 0.5)  ## Returns -0.125
```

Legendre Q Function

Computes the Legendre function of the second kind $Q_n(x)$.

Morphology:

EBNF:

```
"f.special.legendreQ", "(", int, ",", real, ")"
```

- Operands: degree n (int ≥ 0), argument x (int or real).
- Result: real.
- A **Value Error** is raised for $|x| \geq 1$ (singular at ± 1).
- A **Type Error** is raised if arguments are not the correct types.

Examples:

```
f.special.legendreQ(0, 2.0)  ## Returns approximately 0.5493
```

Associated Legendre Function

Computes the associated Legendre function $P_l^m(x)$.

Morphology:

EBNF:

```
"f.special.associatedLegendre", "(", int, ",", int, ",", real, ")"
```

- Operands: degree l (int ≥ 0), order m (int), argument x (int or real).
- Result: real.
- If $|m| > l$, the result is 0.0 (mathematical definition).
- A **Type Error** is raised if arguments are not the correct types.

Examples:

```
f.special.associatedLegendre(1, 0, 0.5)    #c# Returns 0.5
f.special.associatedLegendre(2, 1, 0.5)    #c# Returns approximately -0.9682
f.special.associatedLegendre(1, 2, 0.5)    #c# Returns 0.0 (|m| > l)
```

Hermite Polynomial Function

Computes the physicist's Hermite polynomial $H_n(x)$.

Morphology:

EBNF:

```
"f.special.hermite", "(", int, ",", real, ")"
```

- Operands: degree n (int ≥ 0), argument x (int or real).
- Result: real.
- A **Type Error** is raised if arguments are not the correct types.

Examples:

```
f.special.hermite(0, 1)    #c# Returns 1.0
f.special.hermite(1, 1)    #c# Returns 2.0
f.special.hermite(2, 1)    #c# Returns 2.0
```

Laguerre Polynomial Function

Computes the Laguerre polynomial $L_n(x)$.

Morphology:

EBNF:

```
"f.special.laguerre", "(", int, ",", real, ")"
```

- Operands: degree n (int ≥ 0), argument x (int or real).
- Result: real.
- A **Type Error** is raised if arguments are not the correct types.

Examples:

```
f.special.laguerre(0, 1)    #c# Returns 1.0
f.special.laguerre(1, 1)    #c# Returns 0.0
f.special.laguerre(2, 1)    #c# Returns 0.5
```

Chebyshev Polynomial Function

Computes the Chebyshev polynomial of the first kind $T_n(x)$.

Morphology:

EBNF:

```
"f.special.chebyshev", "(", int, ",", real, ")"
```

- Operands: degree n (`int` ≥ 0), argument x (`int` or `real`).
- Result: `real`.
- A `Type Error` is raised if arguments are not the correct types.

Examples:

```
f.special.chebyshev(0, 0.5) #c# Returns 1.0
f.special.chebyshev(1, 0.5) #c# Returns 0.5
f.special.chebyshev(2, 0.5) #c# Returns -0.5
```

Chebyshev Polynomial of the Second Kind

Computes the Chebyshev polynomial of the second kind $U_n(x)$.

Morphology:

EBNF:

```
"f.special.chebyshevU", "(", int, ",", real, ")"
```

- Operands: degree n (`int` ≥ 0), argument x (`int` or `real`).
- Result: `real`.
- $U_0(x) = 1$, $U_1(x) = 2x$.

Examples:

```
f.special.chebyshevU(0, 0.5) #c# Returns 1.0
f.special.chebyshevU(1, 0.5) #c# Returns 1.0
f.special.chebyshevU(2, 0.5) #c# Returns 3.0
```

Gegenbauer Polynomial

Computes the Gegenbauer (ultraspherical) polynomial $C_n^{(\alpha)}(x)$. For $\alpha = 0.5$ this is the Legendre polynomial; for $\alpha = 1$ this is the Chebyshev polynomial of the second kind.

Morphology:

EBNF:

```
"f.special.gegenbauer", "(", int, ",", ( int | real ), ",", ( int | real ), ")"
```

- Operands: degree n (`int` ≥ 0), parameter α (`int` or `real`), argument x (`int` or `real`).
- Result: `real`.

Examples:

```
f.special.gegenbauer(0, 1, 0.5)  ## Returns 1.0
f.special.gegenbauer(1, 1, 0.5)  ## Returns 1.0
```

Jacobi Polynomial

Computes the Jacobi polynomial $P_n^{(\alpha, \beta)}(x)$. Generalizes Legendre, Chebyshev, and Gegenbauer polynomials.

Morphology:

EBNF:

```
"f.special.jacobiP", "(", int, ",", ( int | real ), ",", ( int | real ), ",", ( int | re
```

- Operands: degree n (`int` ≥ 0), parameters α, β (`int` or `real`), argument x (`int` or `real`).
- Result: `real`.

Examples:

```
f.special.jacobiP(0, 0, 0, 0.5)  ## Returns 1.0
f.special.jacobiP(1, 1, 1, 0.5)  ## Returns 0.75
```

Zernike Polynomial

Computes the Zernike polynomial $R_n^m(r)$, used in optics for wavefront analysis over circular apertures.

Morphology:

EBNF:

```
"f.special.zernike", "(", int, ",", int, ",", ( int | real ), ")"
```

- Operands: radial order n (`int` ≥ 0), azimuthal frequency m (`int`, $|m| \leq n$), radius r (`int` or `real`).
- Result: `real`.
- A `Value Error` is raised if $|m| > n$ or $(n - m)$ is odd and negative.

Examples:

```
f.special.zernike(0, 0, 0.5)  ## Returns 1.0
f.special.zernike(1, 1, 0.5)  ## Returns 0.5
f.special.zernike(2, 0, 0.5)  ## Returns -0.125
```

Sine Integral Function

The `si` function computes the sine integral $\text{Si}(x) = \int_0^x \frac{\sin(t)}{t} dt$.

Morphology:

EBNF:

```
"f.special.si", "(", ( int | real ), ")"
```

- Operands: `int` or `real`
- Result: `real`
- $\text{Si}(0) = 0$. $\text{Si}(\infty) = \pi/2$. Odd function: $\text{Si}(-x) = -\text{Si}(x)$.

Examples:

```
f.special.si(0)           #c# Returns 0.0
f.special.si(1)           #c# Returns approximately 0.9461
f.special.si(3.14159)     #c# Returns approximately 1.8519 (near pi/2)
f.special.si(-1)          #c# Returns approximately -0.9461
```

Airy Function

Computes the Airy functions $\text{Ai}(x)$ and $\text{Bi}(x)$, solutions to the differential equation $y'' - xy = 0$. Airy functions arise in optics (Airy disk diffraction patterns), quantum mechanics (linear potential wells), and electromagnetic wave propagation near caustics.

Morphology:

EBNF:

```
"f.special.airy", "(", ( int | real ), ")"
```

- Operands: `int` or `real`.
- Result: list `[real, real, real, real]` — $[\text{Ai}(x), \text{Ai}'(x), \text{Bi}(x), \text{Bi}'(x)]$.
- For complex arguments, see `f.special.airyC`.
- A **Type Error** is raised if the argument is not numeric.

Examples:

```
f.special.airy(0)         #c# Returns [0.3551, -0.2588, 0.6149, 0.4483]
f.special.airy(1)         #c# Returns [0.1353, -0.1591, 1.2019, 0.9324]
f.special.airy(-1)        #c# Returns [0.5356, 0.1040, 0.1039, -0.4643]
```

- $\text{Ai}(0) \approx 0.3551$, $\text{Bi}(0) \approx 0.6149$.
- $\text{Ai}(x) \rightarrow 0$ and $\text{Bi}(x) \rightarrow \infty$ as $x \rightarrow +\infty$.
- For large negative x , both functions oscillate with decaying amplitude.

Airy Function (Complex)

Computes the Airy functions for complex arguments.

Morphology:

EBNF:

```
"f.special.airyC", "(", complex, ")"
```

- Operands: `int`, `real`, or `complex`.
- Result: list [`complex`, `complex`, `complex`, `complex`] — $[\text{Ai}(z), \text{Ai}'(z), \text{Bi}(z), \text{Bi}'(z)]$.
- For real arguments, prefer `f.special.airy` which returns `real` values.
- A `Type Error` is raised for non-numeric arguments.

Examples:

```
f.special.airyC(0)          #c# Returns [0.3551+0i, -0.2588+0i, 0.6149+0i, 0.4483+0i]
f.special.airyC(1+1i)       #c# Returns complex Airy values
```

Scaled Airy Function

Computes exponentially scaled Airy functions $e^{\frac{2}{3}x^{3/2}}\text{Ai}(x)$ and $e^{-\frac{2}{3}|x|^{3/2}}\text{Bi}(x)$ to avoid overflow for large arguments.

Morphology:

EBNF:

```
"f.special.airyScaled", "(", ( int | real ), ")"
```

- Operands: `int` or `real`.
- Result: list [`real`, `real`, `real`, `real`] — $[\text{AiS}(x), \text{AiS}'(x), \text{BiS}(x), \text{BiS}'(x)]$.
- For $x \geq 0$: $\text{AiS}(x) = e^{\frac{2}{3}x^{3/2}}\text{Ai}(x)$.
- For $x < 0$: $\text{AiS}(x) = \text{Ai}(x)$ (no scaling needed).
- $\text{BiS}(x) = e^{-\frac{2}{3}|x|^{3/2}}\text{Bi}(x)$ for all x .
- Useful when $|x|$ is large and unscaled `Ai` or `Bi` would overflow/underflow.
- A `Type Error` is raised if the argument is not numeric.

Examples:

```
f.special.airyScaled(0)      #c# Returns [0.3551, -0.2588, 0.6149, 0.4483]
f.special.airyScaled(5)      #c# Returns scaled values avoiding overflow
f.special.airyScaled(-5)     #c# Returns same as f.special.airy(-5) (no scaling for x < 0)
```

Airy Function Zeros

Computes zeros and values of the Airy functions Ai and Bi and their derivatives.

Morphology:

EBNF:

```
"f.special.airyZerosAi", "(", int, ")"
```

- Operands: count n (`int` ≥ 1).
- Result: list [`list`, `list`] — $[a_n, a'_n]$ where a_n are zeros of $\text{Ai}(x)$ and a'_n are zeros of $\text{Ai}'(x)$.
- A `Value Error` is raised if $n < 1$.
- A `Type Error` is raised if the argument is not `int`.

Examples:

```
f.special.airyZerosAi(1)    ## Returns [[-2.3381], [-1.0186]]
```

```
f.special.airyZerosAi(3)    ## Returns [[-2.3381, -4.0879, -5.5206], [-1.0186, -3.2482,
```

- Zeros of $\text{Ai}(x)$ are all negative (oscillatory regime).
- Zeros of $\text{Ai}'(x)$ interlace with zeros of $\text{Ai}(x)$.

Bi Airy Function Zeros

Computes zeros and values of the Airy functions Bi and Bi'.

Morphology:

EBNF:

```
"f.special.airyZerosBi", "(", int, ")"
```

- Operands: count n (`int` ≥ 1).
- Result: list [`list`, `list`] — $[b_n, b'_n]$ where b_n are zeros of $\text{Bi}(x)$ and b'_n are zeros of $\text{Bi}'(x)$.
- A `Value Error` is raised if $n < 1$.
- A `Type Error` is raised if the argument is not `int`.

Examples:

```
f.special.airyZerosBi(1)    ## Returns [[-1.1737], [-2.2945]]
```

```
f.special.airyZerosBi(3)    ## Returns [[-1.1737, -3.2711, -4.8307], [-2.2945, -4.1034,
```

- Zeros of $\text{Bi}(x)$ and $\text{Bi}'(x)$ are all negative.
- Used in quantum mechanical bound state problems with linear potentials.

Airy Integrals

Computes integrals of the Airy functions: $\int_0^x \text{Ai}(t) dt$ and $\int_0^x \text{Bi}(t) dt$.

Morphology:

EBNF:

```
"f.special.airyIntegral", "(", ( int | real ), ")"
```

- Operands: `int` or `real`.
- Result: list `[real, real]` — $[\int_0^x \text{Ai}(t) dt, \int_0^x \text{Bi}(t) dt]$.
- $\int_0^0 = [0.0, 0.0]$.
- A `Type Error` is raised if the argument is not numeric.

Examples:

```
f.special.airyIntegral(0)      ## Returns [0.0, 0.0]
f.special.airyIntegral(1)      ## Returns approximately [0.2018, 0.9640]
f.special.airyIntegral(-1)     ## Returns approximately [0.2310, -0.1199]
```

- Useful in diffraction theory and quantum scattering calculations.

Scorer's Function Hi

Computes the Scorer's function $\text{Hi}(z)$, a particular solution to the inhomogeneous Airy equation $y'' - xy = \frac{1}{\pi}$. Scorer's functions arise in diffraction theory, wave scattering, and electromagnetic propagation near caustics.

Morphology:

EBNF:

```
"f.special.scorerHi", "(", ( int | real ), ")"
```

- Operands: `x` (`int` or `real`).
- Result: `real` — value of $\text{Hi}(x)$.
- For complex arguments, see `f.special.scorerHiC`.

```
f.special.scorerHi(0)          ## Returns approximately 0.5
f.special.scorerHi(1)          ## Returns approximately 0.3016
f.special.scorerHi(-1)         ## Returns approximately 0.4643
```

Scorer's Function Gi

Computes the Scorer's function $\text{Gi}(z)$, the complementary particular solution to the inhomogeneous Airy equation. Related to Hi by $\text{Gi}(z) = \text{Bi}(z) \int_0^z \text{Ai}(t) dt - \text{Ai}(z) \int_0^z \text{Bi}(t) dt$.

Morphology:

EBNF:

```
"f.special.scorerGi", "(", ( int | real ), ")"
```

- Operands: `x` (`int` or `real`).

- Result: **real** — value of $\text{Gi}(x)$.
- For complex arguments, see `f.special.scorerGiC`.

```
f.special.scorerGi(0)      #c# Returns approximately 0.5
f.special.scorerGi(1)      #c# Returns approximately 0.1606
f.special.scorerGi(-1)     #c# Returns approximately 0.4643
```

Scorer's Function Hi (Complex)

Computes the Scorer's function $\text{Hi}(z)$ for complex arguments.

Morphology:

EBNF:

```
"f.special.scorerHiC", "(", complex, ")"
```

- Operands: z (**complex**).
- Result: **complex** — value of $\text{Hi}(z)$.
- For real arguments, prefer `f.special.scorerHi` which returns **real** values.

```
f.special.scorerHiC(0)      #c# Returns approximately 0.5+0i
f.special.scorerHiC(1+1i)   #c# Returns complex Scorer Hi value
```

Scorer's Function Gi (Complex)

Computes the Scorer's function $\text{Gi}(z)$ for complex arguments.

Morphology:

EBNF:

```
"f.special.scorerGiC", "(", complex, ")"
```

- Operands: z (**complex**).
- Result: **complex** — value of $\text{Gi}(z)$.
- For real arguments, prefer `f.special.scorerGi` which returns **real** values.

```
f.special.scorerGiC(0)      #c# Returns approximately 0.5+0i
f.special.scorerGiC(1+1i)   #c# Returns complex Scorer Gi value
```

Align Up Function

Rounds x up to the next multiple of **alignment**.

Morphology:

EBNF:

```
"f.int.alignUp", "(", int, ",", int, ")"
```

- Operands: x (**int**), **alignment** (**int**).
- Result: **int**.

- A Value Error is raised if alignment ≤ 0 .
- A Type Error is raised if arguments are not int.

Examples:

```
f.alignUp(7, 8)      #c# Returns 8
f.alignUp(9, 8)      #c# Returns 16
f.alignUp(8, 8)      #c# Returns 8 (already aligned)
```

Align Down Function

Rounds x down to the previous multiple of `alignment`.

Morphology:

EBNF:

```
"f.int.alignDown", "(", int, ",", int, ")"
```

- Operands: x (int), alignment (int).
- Result: int.
- A Value Error is raised if alignment ≤ 0 .
- A Type Error is raised if arguments are not int.

Examples:

```
f.alignDown(7, 8)    #c# Returns 0
f.alignDown(15, 8)   #c# Returns 8
```

Is Power of Two Function

Returns True if n is a power of two.

Morphology:

EBNF:

```
"f.int.isPowerOfTwo", "(", int, ")"
```

- Operands: int.
- Result: bool.
- A Type Error is raised if the argument is not int.

Examples:

```
f.isPowerOfTwo(8)    #c# Returns True
f.isPowerOfTwo(7)    #c# Returns False
```

Next Power of Two Function

Returns the smallest power of two $\geq n$.

Morphology:

EBNF:

```
"f.int.nextPowerOfTwo", "(", int, ")"
```

- Operands: `int` (must be > 0).
- Result: `int`.
- A `Value Error` is raised for $n \leq 0$.
- A `Type Error` is raised if the argument is not `int`.

Examples:

```
f.int.nextPowerOfTwo(7)    #c# Returns 8
f.int.nextPowerOfTwo(8)    #c# Returns 8 (already a power of two)
```

Popcount Function

Returns the number of set bits (1-bits) in the two's complement representation of n .

Morphology:

EBNF:

```
"f.int.popcount", "(", int, ")"
```

- Operands: `int`.
- Result: `int`.
- For negative numbers, operates on infinite-width two's complement (returns ∞).
- A `Type Error` is raised if the argument is not `int`.

Examples:

```
f.int.popcount(7)          #c# Returns 3  (binary 111)
f.int.popcount(8)          #c# Returns 1  (binary 1000)
```

Parity Function

Returns 0 if the number of 1-bits is even, 1 if odd.

Morphology:

EBNF:

```
"f.int.parity", "(", int, ")"
```

- Operands: `int`.
- Result: `int` (0 or 1).
- For negative numbers, operates on infinite-width two's complement.

- A **Type Error** is raised if the argument is not `int`.

Examples:

```
f.int.parity(7)          #c# Returns 1   (odd: three 1-bits)
f.int.parity(15)         #c# Returns 0   (even: four 1-bits)
```

Bit Length Function

Returns the number of bits required to represent n in binary (excluding the sign bit).

Morphology:

EBNF:

```
"f.int.bitLength", "(", int, ")"
```

- Operands: `int`.
- Result: `int`.
- `f.int.bitLength(0)` returns 0.
- For negative numbers, returns the bit length of the absolute value.
- A **Type Error** is raised if the argument is not `int`.

Examples:

```
f.int.bitLength(0)       #c# Returns 0
f.int.bitLength(8)       #c# Returns 4
f.int.bitLength(-8)      #c# Returns 4
```

Integer Log2 Function

Returns the integer floor of the base-2 logarithm. Equivalent to $\lfloor \log_2(n) \rfloor$.

Morphology:

EBNF:

```
"f.int.ilog2", "(", int, ")"
```

- Operands: `int` (must be > 0).
- Result: `int`.
- A **Value Error** is raised if the argument is ≤ 0 .
- A **Type Error** is raised if the argument is not `int`.

Examples:

```
f.int.ilog2(1)           #c# Returns 0   (2^0 = 1)
f.int.ilog2(8)           #c# Returns 3   (2^3 = 8)
f.int.ilog2(9)           #c# Returns 3   (floor of 3.17)
f.int.ilog2(1023)        #c# Returns 9   (2^9 = 512)
f.int.ilog2(1024)        #c# Returns 10  (2^10 = 1024)
```

Ceiling Log2 Function

Returns the smallest integer k such that $2^k \geq n$. Equivalent to $\lceil \log_2(n) \rceil$.
Morphology:

EBNF:

```
"f.int.ceilLog2", "(", int, ")"
```

- Operands: `int` (must be > 0).
- Result: `int`.
- A **Value Error** is raised if the argument is ≤ 0 .
- A **Type Error** is raised if the argument is not `int`.

Examples:

```
f.int.ceilLog2(1)      #c# Returns 0    (2^0 = 1)
f.int.ceilLog2(8)      #c# Returns 3    (2^3 = 8)
f.int.ceilLog2(9)      #c# Returns 4    (2^4 = 16 >= 9)
f.int.ceilLog2(1023)   #c# Returns 10   (2^10 = 1024)
f.int.ceilLog2(1024)   #c# Returns 10   (2^10 = 1024)
```

ToBase Function

Converts an integer to a string representation in the given base (2–36).
Morphology:

EBNF:

```
"f.int.toBase", "(", int, ",", int, ")"
```

- Operands: value (`int`), base (`int`, 2–36).
- Result: `string` (lowercase digits).
- Negative values produce a `"-"` prefix.
- A **Value Error** is raised for base outside 2–36.
- A **Type Error** is raised if arguments are not `int`.

Examples:

```
f.toBase(255, 16)      #c# Returns "ff"
f.toBase(255, 2)        #c# Returns "11111111"
f.toBase(-10, 16)       #c# Returns "-a"
```


FromBase Function

Parses a string in the given base to an integer.

Morphology:

EBNF:

```
"f.int.fromBase", "(", string, ",", int, ")"
```

- Operands: string (**string**), base (int, 2–36).
- Result: int.
- The "-" prefix is handled for negative values.
- A **Value Error** is raised for invalid characters or base outside 2–36.
- A **Type Error** is raised if the first argument is not a **string** or the second is not an **int**.

Examples:

```
f.fromBase("ff", 16)  #c# Returns 255
f.fromBase("1010", 2)  #c# Returns 10
f.fromBase("-a", 16)   #c# Returns -10
```

Bin Function

Converts an integer to a binary string (base 2).

Morphology:

EBNF:

```
"f.int.bin", "(", int, ")"
```

- Operands: int.
- Result: **string**.
- Equivalent to `f.toBase(x, 2)`.
- A **Type Error** is raised if the argument is not **int**.

Examples:

```
f.int.bin(10)          #c# Returns "1010"
f.int.bin(255)         #c# Returns "11111111"
```

Oct Function

Converts an integer to an octal string (base 8).

Morphology:

EBNF:

```
"f.int.oct", "(", int, ")"
```

- Operands: int.

- Result: `string`.
- Equivalent to `f.toBase(x, 8)`.
- A `Type Error` is raised if the argument is not `int`.

Examples:

```
f.int.oct(10)          #c# Returns "12"
```

Hex Function

Converts an integer to a hexadecimal string (base 16).

Morphology:

EBNF:

```
"f.int.hex", "(", int, ")"
```

- Operands: `int`.
- Result: `string` (lowercase).
- Equivalent to `f.toBase(x, 16)`.
- A `Type Error` is raised if the argument is not `int`.

Examples:

```
f.int.hex(255)         #c# Returns "ff"
```

FromBin Function

Parses a binary string to an integer.

Morphology:

EBNF:

```
"f.int.fromBin", "(", string, ")"
```

- Operands: `string`.
- Result: `int`.
- Equivalent to `f.fromBase(s, 2)`.
- A `Type Error` is raised if the argument is not a `string`.

Examples:

```
f.fromBin("1010")     #c# Returns 10
```

FromOct Function

Parses an octal string to an integer.

Morphology:

EBNF:

```
"f.int.fromOct", "(", string, ")"
```

- Operands: `string`.
- Result: `int`.
- Equivalent to `f.fromBase(s, 8)`.
- A `Type Error` is raised if the argument is not a `string`.

Examples:

```
f.fromOct("12")    #c# Returns 10
```

FromHex Function

Parses a hexadecimal string to an integer.

Morphology:

EBNF:

```
"f.int.fromHex", "(", string, ")"
```

- Operands: `string`.
- Result: `int`.
- Equivalent to `f.fromBase(s, 16)`.
- A `Type Error` is raised if the argument is not a `string`.

Examples:

```
f.fromHex("ff")    #c# Returns 255
```

Choice and Majority Functions

Choice (Ch)

EBNF:

```
"f.int.ch", "(", ( bool | int ), ",", ( bool | int ), ",", ( bool | int ), ")"
```

- Operands: three values of the same type
- For `bool`: $\text{Ch}(x, y, z) = (x \wedge y) \oplus (\neg x \wedge z)$ – if x then y else z
- For `int`: bitwise version – $\text{Ch}(x, y, z) = (x \& y) \wedge (b!&(x) \& z)$
- A `Type Error` is raised if the three arguments are not the same type, or if the type is not `bool` or `int`

Examples:

```
f.int.ch(True, True, False)    ## Returns True
f.int.ch(False, True, False)   ## Returns False
f.int.ch(True, False, True)    ## Returns False
f.int.ch(0xFF, 0x0F, 0x00)    ## Returns 0x0F (bitwise)
```

Majority (Maj)

EBNF:

```
"f.int.maj", "(", ( bool | int ), ",", ( bool | int ), ",", ( bool | int ), ")"
```

- Operands: three values of the same type
- For `bool`: $\text{Maj}(x, y, z) = (x \wedge y) \oplus (x \wedge z) \oplus (y \wedge z)$ – majority vote
- For `int`: bitwise version – $\text{Maj}(x, y, z) = (x \& y) \wedge (x \& z) \wedge (y \& z)$
- A `Type Error` is raised if the three arguments are not the same type, or if the type is not `bool` or `int`

Examples:

```
f.int.maj(True, True, False)    ## Returns True
f.int.maj(True, False, False)   ## Returns False
f.int.maj(False, True, True)    ## Returns True
f.int.maj(0xFF, 0x0F, 0x33)    ## Returns 0x33 (bitwise)
```

Is Prime

Tests whether an integer is prime.

Morphology:

EBNF:

```
"f.int.isPrime", "(", int, ")"
```

- Operand: `int`.
- Result: `bool`. Returns `False` for $n < 2$.

Examples:

```
f.int.isPrime(2)    ## Returns True
f.int.isPrime(17)   ## Returns True
f.int.isPrime(1)    ## Returns False
f.int.isPrime(-5)   ## Returns False
```

Next Prime

Returns the smallest prime strictly greater than n .

Morphology:

EBNF:

```
"f.int.nextPrime", "(", int, ")"
```

- Operand: int.
- Result: int. Returns 2 if $n < 2$.
- A Value Error is raised if $n < 0$.

Examples:

```
f.int.nextPrime(1)      #c# Returns 2
f.int.nextPrime(10)     #c# Returns 11
f.int.nextPrime(17)     #c# Returns 19
```

Previous Prime

Returns the largest prime strictly less than n .

Morphology:

EBNF:

```
"f.int.previousPrime", "(", int, ")"
```

- Operand: int.
- Result: int.
- A Value Error is raised if $n \leq 2$ (no prime less than 2).

Examples:

```
f.int.previousPrime(10)  #c# Returns 7
f.int.previousPrime(17)  #c# Returns 13
```

Nearest Prime

Returns the prime nearest to n . For ties (e.g. $n = 6$, equidistant from 5 and 7), returns the smaller prime.

Morphology:

EBNF:

```
"f.int.nearestPrime", "(", int, ")"
```

- Operand: int.
- Result: int. Returns 2 if $n < 2$.
- A Value Error is raised if $n < 0$.

Examples:

```
f.int.nearestPrime(10)   #c# Returns 11
f.int.nearestPrime(6)    #c# Returns 5 (tie, returns smaller)
f.int.nearestPrime(12)   #c# Returns 11
```

Nth Prime

Returns the n -th prime (1-indexed: $p_1 = 2$, $p_2 = 3$, $p_3 = 5$).

Morphology:

EBNF:

```
"f.int.nthPrime", "(", int, ")"
```

- Operand: `int` (≥ 1).
- Result: `int`.
- A Value Error is raised if $n < 1$.

Examples:

```
f.int.nthPrime(1)      #c# Returns 2
f.int.nthPrime(10)     #c# Returns 29
f.int.nthPrime(100)    #c# Returns 541
```

Prime Pi

Computes $\pi(n)$, the number of primes less than or equal to n .

Morphology:

EBNF:

```
"f.int.primePi", "(", int, ")"
```

- Operand: `int` (≥ 0).
- Result: `int`.
- A Value Error is raised if $n < 0$.

Examples:

```
f.int.primePi(10)      #c# Returns 4  (2, 3, 5, 7)
f.int.primePi(100)     #c# Returns 25
f.int.primePi(1)       #c# Returns 0
```

Prime Factors

Returns the prime factorization of n as a sorted list (with repetition).

Morphology:

EBNF:

```
"f.int.primeFactors", "(", int, ")"
```

- Operand: `int` (≥ 1).
- Result: `list[int]`. Returns `[]` for $n = 1$.
- A Value Error is raised if $n < 0$.

Examples:

```
f.int.primeFactors(12)      #c# Returns [2, 2, 3]
f.int.primeFactors(100)    #c# Returns [2, 2, 5, 5]
f.int.primeFactors(1)      #c# Returns []
f.int.primeFactors(17)     #c# Returns [17]
```

Distinct Prime Factors

Returns the distinct prime factors of n as a sorted list.

Morphology:

EBNF:

```
"f.int.distinctPrimeFactors", "(", int, ")"
```

- Operand: `int` (≥ 1).
- Result: `list[int]`. Returns `[]` for $n = 1$.
- A Value Error is raised if $n < 0$.

Examples:

```
f.int.distinctPrimeFactors(12)    #c# Returns [2, 3]
f.int.distinctPrimeFactors(100)   #c# Returns [2, 5]
f.int.distinctPrimeFactors(1)     #c# Returns []
```

Omega Function

Computes $\omega(n)$, the number of distinct prime factors of n .

Morphology:

EBNF:

```
"f.int.omega", "(", int, ")"
```

- Operand: `int` (≥ 1). $\omega(1) = 0$.
- Result: `int`.
- A Value Error is raised if $n < 0$.

Examples:

```
f.int.omega(1)      #c# Returns 0
f.int.omega(12)     #c# Returns 2  (factors: 2, 3)
f.int.omega(30)     #c# Returns 3  (factors: 2, 3, 5)
```

Big Omega Function

Computes $\Omega(n)$, the total number of prime factors of n (with multiplicity).

Morphology:

EBNF:

```
"f.int.bigOmega", "(", int, ")"
```

- Operand: `int` (≥ 1). $\Omega(1) = 0$.
- Result: `int`.
- A Value Error is raised if $n < 0$.

Examples:

```
f.int.bigOmega(1)      #c# Returns 0
f.int.bigOmega(12)     #c# Returns 3  (2 * 2 * 3)
f.int.bigOmega(30)     #c# Returns 3  (2 * 3 * 5)
```

Euler's Totient Function

Computes $\varphi(n)$, the number of integers in $[1, n]$ that are coprime to n .

Morphology:

EBNF:

```
"f.int.totient", "(", int, ")"
```

- Operand: `int` (≥ 1). $\varphi(1) = 1$.
- Result: `int`.
- A Value Error is raised if $n < 0$.

Examples:

```
f.int.totient(1)      #c# Returns 1
f.int.totient(9)      #c# Returns 6  (1, 2, 4, 5, 7, 8)
f.int.totient(12)     #c# Returns 4  (1, 5, 7, 11)
```

Möbius Function

Computes the Möbius function $\mu(n)$: 0 if n has a squared prime factor, $(-1)^k$ if n is a product of k distinct primes, 1 if $n = 1$.

Morphology:

EBNF:

```
"f.int.mobius", "(", int, ")"
```

- Operand: `int` (≥ 1). $\mu(1) = 1$.
- Result: `int` ($-1, 0$, or 1).
- A Value Error is raised if $n \leq 0$.

Examples:

```
f.int.mobius(1)      #c# Returns 1
f.int.mobius(6)      #c# Returns 1  (2 * 3, two distinct primes)
f.int.mobius(12)     #c# Returns 0  (has squared factor 2^2)
f.int.mobius(30)     #c# Returns -1  (2 * 3 * 5, three distinct primes)
```


Liouville Function

Computes $\lambda(n) = (-1)^{\Omega(n)}$, where $\Omega(n)$ is the total number of prime factors.

Morphology:

EBNF:

```
"f.int.liouville", "(", int, ")"
```

- Operand: `int` (≥ 1). $\lambda(1) = 1$.
- Result: `int` (-1 or 1).
- A Value Error is raised if $n < 0$.

Examples:

```
f.int.liouville(1)      #c# Returns 1
f.int.liouville(6)      #c# Returns 1  (2*3, two factors → (-1)^2)
f.int.liouville(8)      #c# Returns -1  (2^2*2, three factors → (-1)^3)
```

Jacobi Symbol

Computes the Jacobi symbol (a/n) , a generalization of the Legendre symbol to composite moduli.

Morphology:

EBNF:

```
"f.int.jacobiSymbol", "(", int, ",", int, ")"
```

- Operands: a (`int`), n (`int`, odd > 0).
- Result: `int` (-1 , 0 , or 1).
- A Value Error is raised if $n \leq 0$ or n is even.

Examples:

```
f.int.jacobiSymbol(2, 3)  #c# Returns -1
f.int.jacobiSymbol(3, 5)  #c# Returns -1
f.int.jacobiSymbol(2, 7)  #c# Returns 1
```

Legendre Symbol

Computes the Legendre symbol (a/p) : 1 if a is a quadratic residue mod p , -1 if not, 0 if $p \mid a$.

Morphology:

EBNF:

```
"f.int.legendreSymbol", "(", int, ",", int, ")"
```

- Operands: a (`int`), p (`int`, must be an odd prime).
- Result: `int` (-1 , 0 , or 1).

- A Value Error is raised if p is not an odd prime.

Examples:

```
f.int.legendreSymbol(2, 7)    #c# Returns 1  (2 is QR mod 7: 3^2  2)
f.int.legendreSymbol(3, 7)    #c# Returns -1
f.int.legendreSymbol(4, 7)    #c# Returns 1  (4 = 2^2)
```

Kronecker Symbol

Computes the Kronecker symbol (a/n) , a generalization of the Jacobi symbol to all integers n .

Morphology:

EBNF:

```
"f.int.kroneckerSymbol", "(", int, ",", int, ")"
```

- Operands: two int values.
- Result: int (-1 , 0 , or 1).
- A Value Error is raised if both arguments are 0 .

Examples:

```
f.int.kroneckerSymbol(2, 3)    #c# Returns -1
f.int.kroneckerSymbol(5, 1)    #c# Returns 1
f.int.kroneckerSymbol(3, -1)   #c# Returns -1
```

Carmichael Lambda Function

Computes $\lambda(n)$, the smallest positive integer such that $a^{\lambda(n)} \equiv 1 \pmod{n}$ for all a coprime to n .

Morphology:

EBNF:

```
"f.int.carmichaelLambda", "(", int, ")"
```

- Operand: int (≥ 1).
- Result: int.
- A Value Error is raised if $n < 1$.

Examples:

```
f.int.carmichaelLambda(1)     #c# Returns 1
f.int.carmichaelLambda(7)     #c# Returns 6  (= phi(7))
f.int.carmichaelLambda(8)     #c# Returns 2
f.int.carmichaelLambda(12)    #c# Returns 2
```

Continued Fraction

Computes the continued fraction representation of a real number as a list of integers $[a_0; a_1, a_2, \dots]$.

EBNF:

```
"f.int.continuedFraction", "(", real, [",", int], ")"
```

- Operands: `x` (value to expand), optional `maxTerms` (`int`, default 20).
- Result: `list[int]` — continued fraction coefficients $[a_0; a_1, a_2, \dots]$.
- A `Value Error` is raised if `maxTerms` ≤ 0 .

```
f.int.continuedFraction(c.math.pi)          ## Returns [3, 7, 15, 1, 292, ...]
f.int.continuedFraction(3.14159, 5)         ## Returns [3, 7, 15, 1, 292]
f.int.continuedFraction(1.41421)           ## Returns [1, 2, 2, 2, 2, ...] (sqrt(2))
```

Continued Fraction Value

Evaluates a continued fraction from a list of coefficients and returns the resulting rational or real value.

EBNF:

```
"f.int.continuedFractionValue", "(", list[int], ")"
```

- Operands: `list[int]` — continued fraction coefficients $[a_0; a_1, a_2, \dots]$.
- Result: `real` — the value of the continued fraction.
- A `Value Error` is raised if the list is empty.

```
f.int.continuedFractionValue([3, 7, 15, 1]) ## Returns approximately 3.14159
f.int.continuedFractionValue([1, 2, 2, 2, 2]) ## Returns approximately 1.41421
f.int.continuedFractionValue([0, 1, 1, 1, 1, 1]) ## Returns approximately 0.61803 (golden ratio)
```

Is Perfect Power

Checks whether an integer is a perfect power, i.e. $n = m^k$ for some integers $m \geq 1$, $k \geq 2$.

EBNF:

```
"f.int.isPerfect", "(", int, ")"
```

- Operands: `int`.
- Result: `bool` — `True` if n is a perfect power (perfect square, cube, etc.), `False` otherwise.
- Returns `True` for $n = 0$ and $n = 1$.
- A `Type Error` is raised if the argument is not `int`.

```
f.int.isPerfect(4)      ## True   (2^2)
f.int.isPerfect(8)      ## True   (2^3)
f.int.isPerfect(9)      ## True   (3^2)
f.int.isPerfect(10)     ## False
f.int.isPerfect(27)     ## True   (3^3)
f.int.isPerfect(100)    ## True   (10^2)
```

Is Abundant

Checks whether an integer is an abundant number (sum of proper divisors exceeds n).

EBNF:

```
"f.int.isAbundant", "(", int, ")"
```

- Operands: `int` (must be ≥ 1).
- Result: `bool`.
- A `Value Error` is raised if the argument is < 1 .

```
f.int.isAbundant(12)    #c# True   (1 + 2 + 3 + 4 + 6 = 16 > 12)
f.int.isAbundant(6)     #c# False  (1 + 2 + 3 = 6)
f.int.isAbundant(18)    #c# True   (1 + 2 + 3 + 6 + 9 = 21 > 18)
```

Is Deficient

Checks whether an integer is a deficient number (sum of proper divisors is less than n).

EBNF:

```
"f.int.isDeficient", "(", int, ")"
```

- Operands: `int` (must be ≥ 1).
- Result: `bool`.
- A `Value Error` is raised if the argument is < 1 .

```
f.int.isDeficient(8)    #c# True   (1 + 2 + 4 = 7 < 8)
f.int.isDeficient(6)    #c# False  (1 + 2 + 3 = 6, perfect)
f.int.isDeficient(10)   #c# True   (1 + 2 + 5 = 8 < 10)
```

Is Square-Free

Checks whether an integer is square-free (not divisible by any perfect square other than 1).

EBNF:

```
"f.int.isSquareFree", "(", int, ")"
```

- Operands: `int`.
- Result: `bool` — `True` if no prime squared divides n .
- Returns `True` for $n = 0$ and $n = 1$.

```
f.int.isSquareFree(6)    #c# True   (2 * 3)
f.int.isSquareFree(12)   #c# False  (divisible by 4)
f.int.isSquareFree(30)   #c# True   (2 * 3 * 5)
f.int.isSquareFree(1)    #c# True
```

Is Power

Checks whether an integer is a perfect k -th power for a given exponent.

EBNF:

```
"f.int.isPower", "(", int, ",", int, ")"
```

- Operands: n (value), k (exponent, must be ≥ 2).
- Result: `bool` — True if $n = m^k$ for some integer m .
- A Value Error is raised if $k < 2$.

```
f.int.isPower(8, 3)      #c# True   (2^3 = 8)
f.int.isPower(9, 2)      #c# True   (3^2 = 9)
f.int.isPower(10, 2)     #c# False
f.int.isPower(16, 4)     #c# True   (2^4 = 16)
f.int.isPower(1024, 10)  #c# True   (2^10 = 1024)
```

Integer Square Root

Returns the floor of the square root of a non-negative integer. Equivalent to $\lfloor \sqrt{n} \rfloor$ computed exactly using integer arithmetic.

EBNF:

```
"f.int.integerSqrt", "(", int, ")"
```

- Operands: `int` (must be ≥ 0).
- Result: `int`.
- A Value Error is raised if the argument is < 0 .

```
f.int.integerSqrt(0)      #c# Returns 0
f.int.integerSqrt(1)      #c# Returns 1
f.int.integerSqrt(8)      #c# Returns 2   (floor of 2.828)
f.int.integerSqrt(9)      #c# Returns 3
f.int.integerSqrt(100)    #c# Returns 10
f.int.integerSqrt(99)     #c# Returns 9
```

Valuation

Returns the p -adic valuation of n : the exponent of the highest power of p that divides n .

EBNF:

```
"f.int.valuation", "(", int, ",", int, ")"
```

- Operands: n (value), p (prime base, must be > 1).
- Result: `int` — largest k such that $p^k \mid n$.
- Returns 0 if p does not divide n .
- A Value Error is raised if $p \leq 1$.

```

f.int.valuation(8, 2)      ## Returns 3  (2^3 | 8)
f.int.valuation(12, 2)    ## Returns 2  (2^2 | 12, but 2^3 does not)
f.int.valuation(12, 3)    ## Returns 1  (3^1 | 12)
f.int.valuation(100, 10)  ## Returns 2  (10^2 | 100)
f.int.valuation(7, 2)     ## Returns 0  (2 does not divide 7)

```

Chinese Remainder

Solves a system of simultaneous congruences using the Chinese Remainder Theorem. Finds x such that $x \equiv a_i \pmod{m_i}$ for all i .

EBNF:

```
"f.int.chineseRemainder", "(", list[int], ",", list[int], ")"
```

- Operands: `remainders` ($[a_0, a_1, \dots]$), `moduli` ($[m_0, m_1, \dots]$).
- Result: `int` — smallest non-negative solution x .
- Moduli must be pairwise coprime.
- A `Value Error` is raised if lists are empty, have different lengths, or moduli are not pairwise coprime.

```

f.int.chineseRemainder([2, 3, 2], [3, 5, 7])    ## Returns 23  (23 % 3 = 2, 23 % 5 = 3,
f.int.chineseRemainder([1, 2, 3], [3, 4, 5])    ## Returns 58  (58 % 3 = 1, 58 % 4 = 2,

```

Primitive Root

Finds the smallest primitive root modulo n , or returns 0 if none exists. A primitive root g modulo n generates the multiplicative group: $g^1, g^2, \dots, g^{\phi(n)}$ produce all integers coprime to n .

EBNF:

```
"f.int.primitiveRoot", "(", int, ")"
```

- Operands: `n` (modulus, must be > 1).
- Result: `int` — smallest primitive root modulo n , or 0 if none exists.
- A `Value Error` is raised if $n \leq 1$.

```

f.int.primitiveRoot(2)      ## Returns 1
f.int.primitiveRoot(3)      ## Returns 2
f.int.primitiveRoot(5)      ## Returns 2
f.int.primitiveRoot(7)      ## Returns 3
f.int.primitiveRoot(8)      ## Returns 0  (no primitive root mod 8)
f.int.primitiveRoot(11)     ## Returns 2

```

Polygamma Function

Computes the polygamma function $\psi^{(n)}(x) = \frac{d^{n+1}}{dx^{n+1}} \ln \Gamma(x)$. For $n = 0$ this is the digamma function.

Morphology:

EBNF:

```
"f.special.polygamma", "(", int, ",", ( int | real ), ")"
```

- Operands: n (int, order ≥ 0), x (int or real).
- Result: real.
- A **Value Error** is raised if x is a non-positive integer (pole of Γ).
- A **Type Error** is raised if arguments are not int/real.

Examples:

```
f.special.polygamma(0, 1)      #c# Returns -0.5772... (Euler-Mascheroni constant, negative)
f.special.polygamma(0, 5)      #c# Returns 2.0256...
f.special.polygamma(1, 1)      #c# Returns 1.6449... (pi^2 / 6)
f.special.polygamma(2, 1)      #c# Returns -2.4041...
```

IsNaN

Returns **True** if the value is Not-a-Number (NaN).

Morphology:

EBNF:

```
"f.math.isNaN", "(", ( int | real ), ")"
```

- Operand: x (int or real).
- Result: bool.

Examples:

```
f.math.isNaN(0/0)              #c# Returns True   (0/0 = NaN in real division)
f.math.isNaN(1)                #c# Returns False
```

IsInf

Returns **True** if the value is positive or negative infinity.

Morphology:

EBNF:

```
"f.math.isInf", "(", ( int | real ), ")"
```

- Operand: x (int or real).
- Result: bool.

Examples:

```
f.math.isInf(1/0)              #c# Returns True   (1/0 = Inf)
f.math.isInf(1)                #c# Returns False
```

Remap

Linearly maps a value from one range to another.

Morphology:

EBNF:

```
"f.math.remap", "(", ( int | real ), ",", ( int | real ), ",", ( int | real ), ",",  
                    ( int | real ), ",", ( int | real ), ")"
```

- Operands: *value*, *from_low*, *from_high*, *to_low*, *to_high* (all int or real).
- Result: **real**.

Examples:

```
f.math.remap(5, 0, 10, 0, 100)  ## Returns 50.0  
f.math.remap(0, 0, 10, 0, 100)  ## Returns 0.0
```

Covariance

Computes the population covariance of two lists of equal length.

Morphology:

EBNF:

```
"f.stat.covariance", "(", list, ",", list, ")"
```

- Operands: Two `list[int]` or `list[real]` of equal length.
- Result: **real** (population covariance).
- A **Value Error** is raised if lists are empty or unequal length.

Examples:

```
f.stat.covariance([1,2,3], [4,5,6])  ## Returns 0.6666...
```

Correlation

Computes the Pearson correlation coefficient of two lists of equal length.

Morphology:

EBNF:

```
"f.stat.correlation", "(", list, ",", list, ")"
```

- Operands: Two `list[int]` or `list[real]` of equal length.
- Result: **real** (-1 to 1).
- A **Value Error** is raised if lists are empty, unequal length, or have zero variance.

Examples:

```
f.stat.correlation([1,2,3], [2,4,6])  ## Returns 1.0 (perfect positive)  
f.stat.correlation([1,2,3], [6,4,2])  ## Returns -1.0 (perfect negative)
```


Linear Regression

Performs simple linear regression on two lists: $y = mx + b$. Returns [slope, intercept].
Morphology:

EBNF:

```
"f.stat.linearRegression", "(", list, ",", list, ")"
```

- Operands: x (list[int] or list[real]), y (list[int] or list[real]).
- Result: list[real] with [slope, intercept].
- A Value Error is raised if lists are empty or have fewer than 2 points.

Examples:

```
f.stat.linearRegression([1,2,3], [2,4,6])    ## Returns [2.0, 0.0]
f.stat.linearRegression([0,1,2], [1,3,5])    ## Returns [2.0, 1.0]
```

Dawson Function

Computes the Dawson function $F(x) = e^{-x^2} \int_0^x e^{t^2} dt$.
Morphology:

EBNF:

```
"f.special.dawson", "(", ( int | real ), ")"
```

- Operand: int or real.
- Result: real.
- Odd function: $F(-x) = -F(x)$.

Examples:

```
f.special.dawson(0)          ## Returns 0.0
f.special.dawson(1)          ## Returns approximately 0.5381
f.special.dawson(-1)         ## Returns approximately -0.5381
```

Fresnel Sine Integral

Computes the Fresnel sine integral $S(x) = \int_0^x \sin\left(\frac{\pi t^2}{2}\right) dt$.

Morphology:

EBNF:

```
"f.special.fresnelS", "(", ( int | real ), ")"
```

- Operand: int or real.
- Result: real.
- Odd function: $S(-x) = -S(x)$.

Examples:

```
f.special.fresnelS(0)        ## Returns 0.0
f.special.fresnelS(1)        ## Returns approximately 0.4383
```

Fresnel Cosine Integral

Computes the Fresnel cosine integral $C(x) = \int_0^x \cos\left(\frac{\pi t^2}{2}\right) dt$.

Morphology:

EBNF:

```
"f.special.fresnelC", "(", ( int | real ), ")"
```

- Operand: int or real.
- Result: real.
- Odd function: $C(-x) = -C(x)$.

Examples:

```
f.special.fresnelC(0)    #c# Returns 0.0
f.special.fresnelC(1)    #c# Returns approximately 0.7799
```

Exponential Integral

Computes the exponential integral $Ei(x) = -\int_{-x}^{\infty} \frac{e^{-t}}{t} dt$.

Morphology:

EBNF:

```
"f.special.expIntegral", "(", ( int | real ), ")"
```

- Operand: int or real.
- Result: real.
- A Value Error is raised at $x = 0$ (singularity).

Examples:

```
f.special.expIntegral(1)  #c# Returns approximately 1.8951
f.special.expIntegral(-1) #c# Returns approximately -0.2194
```

Logarithmic Integral

Computes the logarithmic integral $li(x) = \int_0^x \frac{dt}{\ln t}$.

Morphology:

EBNF:

```
"f.special.logIntegral", "(", ( int | real ), ")"
```

- Operand: int or real.
- Result: real.
- A Value Error is raised for $x \leq 0$ (domain restriction).

Examples:

```
f.special.logIntegral(2)  #c# Returns approximately 1.0452
f.special.logIntegral(10) #c# Returns approximately 5.1200
```

Cosine Integral

Computes the cosine integral $\text{Ci}(x) = -\int_x^\infty \frac{\cos t}{t} dt$.

Morphology:

EBNF:

```
"f.special.ci", "(", ( int | real ), ")"
```

- Operand: int or real.
- Result: real.
- A Value Error is raised at $x = 0$ (singularity).

Examples:

```
f.special.ci(1)          ## Returns approximately 0.3374
f.special.ci(5)          ## Returns approximately -0.1900
```

Hyperbolic Sine Integral

Computes the hyperbolic sine integral $\text{Shi}(x) = \int_0^x \frac{\sinh t}{t} dt$.

Morphology:

EBNF:

```
"f.special.shi", "(", ( int | real ), ")"
```

- Operand: int or real.
- Result: real.
- Odd function: $\text{Shi}(-x) = -\text{Shi}(x)$.

Examples:

```
f.special.shi(0)          ## Returns 0.0
f.special.shi(1)          ## Returns approximately 1.0573
```

Hyperbolic Cosine Integral

Computes the hyperbolic cosine integral $\text{Chi}(x) = \gamma + \ln x + \int_0^x \frac{\cosh t - 1}{t} dt$.

Morphology:

EBNF:

```
"f.special.chi", "(", ( int | real ), ")"
```

- Operand: int or real.
- Result: real.
- A Value Error is raised at $x = 0$ (singularity).

Examples:

```
f.special.chi(1)          ## Returns approximately 0.8379
f.special.chi(2)          ## Returns approximately 1.3060
```

Hankel Function of the First Kind

Computes the Hankel function of the first kind $H_n^{(1)}(x) = J_n(x) + iY_n(x)$.

Morphology:

EBNF:

```
"f.special.hankelH1", "(", int, ",", ( int | real ), ")"
```

- Operands: order n (int), argument x (int or real).
- Result: complex.

Examples:

```
f.special.hankelH1(0, 1)    #c# Returns approximately 0.7652+0.0883i
```

Hankel Function of the Second Kind

Computes the Hankel function of the second kind $H_n^{(2)}(x) = J_n(x) - iY_n(x)$.

Morphology:

EBNF:

```
"f.special.hankelH2", "(", int, ",", ( int | real ), ")"
```

- Operands: order n (int), argument x (int or real).
- Result: complex.

Examples:

```
f.special.hankelH2(0, 1)    #c# Returns approximately 0.7652-0.0883i
```

Kelvin Function ber

Computes the Kelvin function $\text{ber}_n(x) = \text{Re} [J_n(xe^{i3\pi/4})]$.

Morphology:

EBNF:

```
"f.special.kelvinBer", "(", int, ",", ( int | real ), ")"
```

- Operands: order n (int), argument x (int or real).
- Result: real.

Examples:

```
f.special.kelvinBer(0, 0)    #c# Returns 1.0  
f.special.kelvinBer(0, 1)    #c# Returns approximately 0.9844
```

Kelvin Function bei

Computes the Kelvin function $\text{bei}_n(x) = \text{Im} [J_n(xe^{i3\pi/4})]$.

Morphology:

EBNF:

```
"f.special.kelvinBei", "(", int, ",", "(", int | real ), ")"
```

- Operands: order n (int), argument x (int or real).
- Result: real.

Examples:

```
f.special.kelvinBei(0, 0)  #c# Returns 0.0
f.special.kelvinBei(0, 1)  #c# Returns approximately -0.1196
```

Kelvin Function ker

Computes the Kelvin function $\text{ker}_n(x) = \text{Re} [K_n(xe^{i3\pi/4})]$.

Morphology:

EBNF:

```
"f.special.kelvinKer", "(", int, ",", "(", int | real ), ")"
```

- Operands: order n (int), argument x (int or real).
- Result: real.
- A Value Error is raised for $x \leq 0$ (singularity).

Examples:

```
f.special.kelvinKer(0, 1)  #c# Returns approximately 0.1159
```

Kelvin Function kei

Computes the Kelvin function $\text{kei}_n(x) = \text{Im} [K_n(xe^{i3\pi/4})]$.

Morphology:

EBNF:

```
"f.special.kelvinKei", "(", int, ",", "(", int | real ), ")"
```

- Operands: order n (int), argument x (int or real).
- Result: real.

Examples:

```
f.special.kelvinKei(0, 1)  #c# Returns approximately -0.7895
```

Struve Function H

Computes the Struve function $\mathbf{H}_n(x)$.

Morphology:

EBNF:

```
"f.special.struveH", "(", int, ",", ( int | real ), ")"
```

- Operands: order n (int), argument x (int or real).
- Result: real.

Examples:

```
f.special.struveH(0, 0)    #c# Returns 0.0
f.special.struveH(0, 1)    #c# Returns approximately 0.7102
```

Struve Function L

Computes the modified Struve function $\mathbf{L}_n(x)$.

Morphology:

EBNF:

```
"f.special.struveL", "(", int, ",", ( int | real ), ")"
```

- Operands: order n (int), argument x (int or real).
- Result: real.

Examples:

```
f.special.struveL(0, 0)    #c# Returns 0.0
f.special.struveL(0, 1)    #c# Returns approximately 0.7102
```

Riemann Zeta Function

Computes the Riemann zeta function $\zeta(s) = \sum_{n=1}^{\infty} n^{-s}$.

Morphology:

EBNF:

```
"f.special.riemannZeta", "(", ( int | real ), ")"
```

- Operand: int or real.
- Result: real.
- A Value Error is raised at $s = 1$ (simple pole).
- $\zeta(0) = -0.5$, $\zeta(2) = \pi^2/6$, $\zeta(4) = \pi^4/90$.

Examples:

```
f.special.riemannZeta(2)    #c# Returns approximately 1.6449 (pi^2/6)
f.special.riemannZeta(4)    #c# Returns approximately 1.0823 (pi^4/90)
f.special.riemannZeta(0)    #c# Returns -0.5
```

Hurwitz Zeta Function

Computes the Hurwitz zeta function $\zeta(s, q) = \sum_{n=0}^{\infty} (n+q)^{-s}$. For $q = 1$ this is the Riemann zeta function.

Morphology:

EBNF:

```
"f.special.hurwitzZeta", "(", ( int | real ), ",", ( int | real ), ")"
```

- Operands: s (int or real), q (int or real).
- Result: real.
- A Value Error is raised if $s \leq 1$ or $q \leq 0$.

Examples:

```
f.special.hurwitzZeta(2, 1)    #c# Returns approximately 1.6449 (pi^2/6)
f.special.hurwitzZeta(2, 0.5) #c# Returns approximately 4.9348 (pi^2/2)
```

Dirichlet Eta Function

Computes the Dirichlet eta function $\eta(s) = (1 - 2^{1-s})\zeta(s)$. Also known as the alternating zeta function.

Morphology:

EBNF:

```
"f.special.dirichletEta", "(", ( int | real ), ")"
```

- Operand: int or real.
- Result: real.
- $\eta(0) = 0.5$, $\eta(1) = \ln 2$.

Examples:

```
f.special.dirichletEta(0)    #c# Returns 0.5
f.special.dirichletEta(1)    #c# Returns approximately 0.6931 (ln 2)
f.special.dirichletEta(2)    #c# Returns approximately 0.8225 (pi^2/12)
```

Lerch Transcendent

Computes the Lerch transcendent $\Phi(z, s, q) = \sum_{n=0}^{\infty} \frac{z^n}{(n+q)^s}$.

Morphology:

EBNF:

```
"f.special.lerchPhi", "(", ( int | real | complex ), ",", ( int | real ), ",", ( int | r
```

- Operands: z (int, real, or complex), s (int or real), q (int or real).
- Result: complex.
- A Value Error is raised if $q \leq 0$ or if $|z| \geq 1$ and $s \leq 1$.

Examples:

```
f.special.lerchPhi(0.5, 2, 1)    #c# Returns approximately 1.1447
```

Confluent Hypergeometric Function 0F1

Computes ${}_0F_1(; b; z) = \sum_{n=0}^{\infty} \frac{z^n}{(b)_n n!}$.

Morphology:

EBNF:

```
"f.special.hyp0F1", "(", ( int | real ), ",", ( int | real | complex ), ")"
```

- Operands: b (int or real), z (int, real, or complex).
- Result: complex.
- A Value Error is raised if b is a non-positive integer.

Examples:

```
f.special.hyp0F1(1.5, -0.25)    #c# Returns approximately 0.9048
```

Kummer's Confluent Hypergeometric Function 1F1

Computes ${}_1F_1(a; b; z) = \sum_{n=0}^{\infty} \frac{(a)_n z^n}{(b)_n n!}$. Also known as Kummer's function $M(a, b, z)$.

Morphology:

EBNF:

```
"f.special.hyp1F1", "(", ( int | real ), ",", ( int | real ), ",", ( int | real | complex ), ")"
```

- Operands: a (int or real), b (int or real), z (int, real, or complex).
- Result: complex.
- A Value Error is raised if b is a non-positive integer.

Examples:

```
f.special.hyp1F1(1, 1, 1)    #c# Returns approximately 2.7183 (e)
```

Gauss Hypergeometric Function 2F1

Computes ${}_2F_1(a, b; c; z) = \sum_{n=0}^{\infty} \frac{(a)_n (b)_n z^n}{(c)_n n!}$.

Morphology:

EBNF:

```
"f.special.hyp2F1", "(", ( int | real ), ",", ( int | real ), ",", ( int | real ), ",", ( int | real | complex ), ")"
```

- Operands: a, b, c (int or real), z (int, real, or complex).
- Result: complex.
- A Value Error is raised if c is a non-positive integer.
- Converges for $|z| < 1$; analytic continuation for $|z| \geq 1$.

Examples:

```
f.special.hyp2F1(1, 1, 2, 0.5)    #c# Returns approximately 1.3863
```


Generalized Hypergeometric Function ${}_pF_q$

Computes ${}_pF_q(a_1, \dots, a_p; b_1, \dots, b_q; z)$.

Morphology:

EBNF:

```
"f.special.hypPFQ", "(", list, ",", list, ",", ( int | real | complex ), ")"
```

- Operands: numerator parameters (`list`), denominator parameters (`list`), argument (`int`, `real`, or `complex`).
- Result: `complex`.
- A `Type Error` is raised if the lists do not contain numeric values.
- A `Value Error` is raised if any denominator parameter is a non-positive integer.

Examples:

```
f.special.hypPFQ([1], [2], 1)          ## Returns approximately 1.7183 (e-1)
f.special.hypPFQ([1, 1], [2], 0.5)     ## Returns approximately 1.2730
```

Meijer G-Function

Computes the Meijer G-function $G_{p,q}^{m,n}(z \mid \begin{smallmatrix} a_1, \dots, a_p \\ b_1, \dots, b_q \end{smallmatrix})$. A very general function that includes most common special functions as special cases.

Morphology:

EBNF:

```
"f.special.meijerG", "(", list, ",", list, ",", ( int | real | complex ), ")"
```

- Operands: upper parameters (`list`), lower parameters (`list`), argument (`int`, `real`, or `complex`).
- Result: `complex`.
- A `Type Error` is raised if the lists do not contain numeric values.

Examples:

```
f.special.meijerG([0], [0], 1)  ## Returns 1.0 (delta function case)
```

Fox H-Function

Computes the Fox H-function, a generalization of the Meijer G-function allowing non-integer parameters.

Morphology:

EBNF:

```
"f.special.foxH", "(", list, ",", list, ",", ( int | real | complex ), ")"
```

- Operands: numerator parameters (`list`), denominator parameters (`list`), argument (`int`, `real`, or `complex`).

- Result: complex.
- A Type Error is raised if the lists do not contain numeric values.

Examples:

```
f.special.foxH([[1,1]], [[1]], 1)    ## Returns approximately 1.0
```

Jacobi Theta Function

Computes the Jacobi theta function $\vartheta_n(\tau, z)$ for $n \in \{1, 2, 3, 4\}$.

Morphology:

EBNF:

```
"f.special.jacobiTheta", "(", int, ",", "(", int | real ), ",", "(", int | real ), ")"
```

- Operands: index n (int, must be 1–4), τ (int or real), z (int or real).
- Result: complex.
- A Value Error is raised if $n \notin \{1, 2, 3, 4\}$.

Examples:

```
f.special.jacobiTheta(1, 0, 0)    ## Returns 0.0
f.special.jacobiTheta(3, 0, 0)    ## Returns approximately 1.0
```

Complete Elliptic Integral of the First Kind

Computes $K(m) = \int_0^{\pi/2} \frac{d\theta}{\sqrt{1-m \sin^2 \theta}}$.

Morphology:

EBNF:

```
"f.special.ellipticK", "(", ( int | real ), ")"
```

- Operand: parameter m (int or real).
- Result: real.
- A Value Error is raised for $m \geq 1$ (singularity).

Examples:

```
f.special.ellipticK(0)           ## Returns approximately 1.5708 (pi/2)
f.special.ellipticK(0.5)         ## Returns approximately 1.8541
```

Complete Elliptic Integral of the Second Kind

Computes $E(m) = \int_0^{\pi/2} \sqrt{1-m \sin^2 \theta} d\theta$.

Morphology:

EBNF:

```
"f.special.ellipticE", "(", ( int | real ), ")"
```

- Operand: parameter m (int or real).
- Result: real.

Examples:

```
f.special.ellipticE(0)      ## Returns approximately 1.5708 (pi/2)
f.special.ellipticE(0.5)    ## Returns approximately 1.3506
```

Complete Elliptic Integral of the Third Kind

Computes $\Pi(n, m) = \int_0^{\pi/2} \frac{d\theta}{(1-n \sin^2 \theta) \sqrt{1-m \sin^2 \theta}}$.

Morphology:

EBNF:

```
"f.special.ellipticPi", "(", ( int | real ), ",", ( int | real ), ")"
```

- Operands: characteristic n (int or real), parameter m (int or real).
- Result: real.
- A Value Error is raised for $m \geq 1$ or $n \geq 1$ (singularities).

Examples:

```
f.special.ellipticPi(0, 0.5)  ## Returns approximately 1.8541 (= K(0.5))
f.special.ellipticPi(0.5, 0.5) ## Returns approximately 2.1565
```

Jacobi Elliptic Functions

Computes the Jacobi elliptic functions $\text{sn}(u, m)$, $\text{cn}(u, m)$, $\text{dn}(u, m)$.

Morphology:

EBNF:

```
"f.special.jacobiSN", "(", ( int | real ), ",", ( int | real ), ")"
"f.special.jacobiCN", "(", ( int | real ), ",", ( int | real ), ")"
"f.special.jacobiDN", "(", ( int | real ), ",", ( int | real ), ")"
```

- Operands: argument u (int or real), parameter m (int or real).
- Result: real.
- Identity: $\text{sn}^2 + \text{cn}^2 = 1$, $m \text{sn}^2 + \text{dn}^2 = 1$.

Examples:

```
f.special.jacobiSN(0, 0.5)  ## Returns 0.0
f.special.jacobiCN(0, 0.5)  ## Returns 1.0
f.special.jacobiDN(0, 0.5)  ## Returns 1.0
f.special.jacobiSN(1, 0.5)  ## Returns approximately 0.7997
```

Weierstrass Elliptic Function

Computes the Weierstrass elliptic function $\wp(z; g_2, g_3)$, the inverse of the elliptic integral.
Morphology:

EBNF:

```
"f.special.weierstrassP", "(", ( int | real | complex ), ",", ( int | real | complex ),
```

- Operands: z (int, real, or complex), invariants g_2 and g_3 (int, real, or complex).
- Result: complex.

Examples:

```
f.special.weierstrassP(0, 1, 0)    #c# Returns infinity (pole at z=0)
f.special.weierstrassP(1, 1, 0)    #c# Returns approximately 1.1674
```

Lower Incomplete Gamma Function

Computes $\gamma(s, x) = \int_0^x t^{s-1} e^{-t} dt$.

Morphology:

EBNF:

```
"f.special.incompleteGamma", "(", ( int | real ), ",", ( int | real ), ")"
```

- Operands: s (int or real), x (int or real).
- Result: real.
- $\gamma(s, 0) = 0$, $\gamma(s, \infty) = \Gamma(s)$.
- A Value Error is raised if $s \leq 0$ and $x = 0$.

Examples:

```
f.special.incompleteGamma(1, 0)    #c# Returns 0.0
f.special.incompleteGamma(1, 1)    #c# Returns approximately 0.6321 (1 - 1/e)
```

Upper Incomplete Beta Function

Computes $B(a, b; x) = \int_x^1 t^{a-1} (1-t)^{b-1} dt$, the complement of the regularized incomplete beta function.

Morphology:

EBNF:

```
"f.special.incompleteBeta", "(", ( int | real ), ",", ( int | real ), ",", ( int | real
```

- Operands: a (int or real), b (int or real), x (int or real).
- Result: real.
- $B(a, b; 0) = B(a, b)$, $B(a, b; 1) = 0$.
- A Value Error is raised if a or b is a non-positive integer.

Examples:

```
f.special.incompleteBeta(1, 1, 0)    #c# Returns 1.0 (= B(1,1) = 1)
f.special.incompleteBeta(1, 1, 1)    #c# Returns 0.0
f.special.incompleteBeta(2, 3, 0.5) #c# Returns approximately 0.3125
```

Inverse Tangent Integral (Ti)

Computes the inverse tangent integral $Ti_2(x) = \int_0^x \frac{\arctan(t)}{t} dt$.

Morphology:

EBNF:

```
"f.special.ti2", "(", ( int | real | complex ), ")"
```

- Operands: `int`, `real`, or `complex`.
- Result: `real` for real input, `complex` for complex input.
- Odd function: $Ti_2(-x) = -Ti_2(x)$.
- Special values: $Ti_2(0) = 0$, $Ti_2(1) = G$ (Catalan's constant ≈ 0.915965594).
- Related to the dilogarithm: $Ti_2(x) = \frac{1}{2i}[\text{Li}_2(ix) - \text{Li}_2(-ix)]$.
- A **Type Error** is raised for non-numeric arguments.

Examples:

```
f.special.ti2(0)           ## Returns 0.0
f.special.ti2(1)           ## Returns approximately 0.9160  (= Catalan's constant)
f.special.ti2(0.5)         ## Returns approximately 0.4872
```

Rogers Dilogarithm

Computes the Rogers dilogarithm $L(x) = \text{Li}_2(x) + \frac{1}{2} \ln(x) \ln(1-x)$, a normalized variant satisfying Abel's five-term identity.

Morphology:

EBNF:

```
"f.special.rogersL", "(", ( int | real ), ")"
```

- Operands: `int` or `real`, $0 \leq x \leq 1$.
- Result: `real`.
- Special values: $L(0) = 0$, $L(1) = \frac{\pi^2}{6}$, $L\left(\frac{\sqrt{5}-1}{2}\right) = \frac{\pi^2}{10}$.
- The normalized form $\frac{6}{\pi^2}L(x)$ satisfies Abel's five-term identity.
- A **Value Error** is raised for $x < 0$ or $x > 1$ (domain restriction).
- A **Type Error** is raised for non-numeric arguments.

Examples:

```
f.special.rogersL(0)           ## Returns 0.0
f.special.rogersL(1)           ## Returns approximately 1.6449  (= pi^2/6)
f.special.rogersL(0.5)         ## Returns approximately 0.5822
```

Bloch-Wigner Dilogarithm

Computes the Bloch-Wigner dilogarithm $D(z) = \text{Im}(\text{Li}_2(z)) + \arg(1 - z) \log |z|$, a single-valued modification of the complex dilogarithm used to compute volumes of ideal hyperbolic tetrahedra.

Morphology:

EBNF:

```
"f.special.blochWigner", "(", ( int | real | complex ), ")"
```

- Operands: `int`, `real`, or `complex`.
- Result: `real` (always real-valued, even for complex input).
- $D(z) = 0$ for all real z .
- Special values: $D(i) = G$ (Catalan's constant), $D(e^{\pi i/3}) = \frac{\sqrt{3}}{4}L(\chi_{-3}, 2)$.
- Computes the hyperbolic volume of the ideal tetrahedron with vertices $0, 1, \infty, z$.
- A `Type Error` is raised for non-numeric arguments.

Examples:

```
f.special.blochWigner(0)      #c# Returns 0.0
f.special.blochWigner(1)      #c# Returns 0.0
f.special.blochWigner(0+1i)    #c# Returns approximately 0.9160 (= Catalan's constant)
```

Legendre Chi Function

Computes the Legendre chi function $\chi_\nu(z) = \sum_{k=0}^{\infty} \frac{z^{2k+1}}{(2k+1)^\nu} = \frac{1}{2}[\text{Li}_\nu(z) - \text{Li}_\nu(-z)]$.

Morphology:

EBNF:

```
"f.special.legendreChi", "(", ( int | real ), ",", ( int | real | complex ), ")"
```

- Operands: order ν (`int` or `real`), argument z (`int`, `real`, or `complex`).
- Result: `real` for real input, `complex` for complex input.
- Special cases: $\chi_1(x) = \text{arctanh}(x)$, $\chi_2(x) = \int_0^x \frac{\text{arctanh}(t)}{t} dt$.
- Special values: $\chi_2(1) = \frac{\pi^2}{8}$, $\chi_2(i) = iG$ (where G is Catalan's constant).
- Derivative: $\frac{d}{dx}\chi_\nu(x) = \frac{\chi_{\nu-1}(x)}{x}$.
- A `Type Error` is raised for non-numeric arguments.

Examples:

```
f.special.legendreChi(2, 1)      #c# Returns approximately 1.2337 (= pi^2/8)
f.special.legendreChi(2, 0.5)    #c# Returns approximately 0.5203
f.special.legendreChi(1, 0.5)    #c# Returns approximately 0.5493 (= arctanh(0.5))
```

11.2 User-defined Functions

User-defined functions allow creation of reusable code blocks with local scoping.

11.2.1 Function Definition

Functions are defined using the `function` keyword with the `f.u.` prefix:

EBNF:

```
function_definition = "function", "f.u.", id,  
    "(", [ { id, "," }, id ], ")", ":", "{",  
    f_statement, {f_statement}, "}"
```

```
f_statement = statement | "return", [data]
```

Example:

```
function f.u.add( $a, $b )::{  
    $retval [int]= $a + $b  
}
```

Function parameters are local variables scoped to the function body. Parameters may optionally include type annotations using the `[type]` syntax, consistent with lambda parameters. To return a value, set the `$retval` variable.

11.2.2 Function Calls

User-defined functions are called using the `f.u.` prefix:

EBNF:

```
function_call_ud = "f.u.", id, "(",  
    [{ data, "," }, data ], ")"
```

Example:

```
$result [int]= f.u.add( 3, 5 )    #c# $result = 8
```

11.2.3 Return Values

Functions return values by setting the `$retval` variable. A bare `return` without a value, or a function that never sets `$retval`, returns `None`.

```
function f.u.square( $x )::{  
    $retval [int]= $x ^ 2  
}
```

```
function f.u.greet( $name )::{  
    f.util.print( f.str.str( "Hello, ", $name, "!" ) )  
    #c# No return value  
}
```

```
$sq [int]= f.u.square( 4 )      #c# $sq = 16
f.u.greet( "World" )           #c# Prints: Hello, World!
```

11.2.4 Local Scoping

Functions have local scoping. Variables declared inside a function are only accessible within that function. Parameters are also local variables.

Scope Rules

- Parameters (\$a, \$b, etc.) are local to the function body
- Variables declared inside the function with `$var [type]= val` are local
- Local variables shadow global variables of the same name
- The `$retval` variable is always local to the function
- Functions cannot access local variables from other function invocations

Return Values Functions can return values using two equivalent mechanisms:

- Assign to `$retval` and let execution fall through the closing `}`
- Use the `return` keyword (see Chapter 3, `f_statement` rule), which is syntactic sugar that assigns its argument to `$retval` and immediately exits the function

Both `return $value` and `$retval [type]= $value` followed by natural function exit are valid and equivalent. If both are used, `return` takes precedence.

Examples

```
$x [int]= 10      #c# Global variable

function f.u.doublen()::{
    $x [int]= $x * 2  #c# Local $x shadows global
    f.util.print($x)      #c# Prints: 20
}

f.u.doublen()
f.util.print($x)          #c# Prints: 10 (global unchanged)

function f.u.addLocal( $a, $b )::{
    $sum [int]= $a + $b  #c# $sum is local
    $retval [int]= $sum
}

f.u.addLocal( 3, 4 )      #c# Returns 7
#c# $sum is not accessible here
```


Recursion Local scoping enables recursion since each call has its own copy of parameters:

```
function f.u.factorial( $n )::{  
  if $n <= 1 ::{  
    $retval [int]= 1  
  } else ::{  
    $retval [int]= $n *  
      f.u.factorial( $n - 1 )  
  }  
}  
  
$result [int]= f.u.factorial( 5 )  #c# = 120
```

Part II

Formal Semantics

Chapter 12

Evaluation Model

Expressions in EDADSL are evaluated according to standard mathematical precedence rules. The evaluation model is eager (strict), meaning all sub-expressions are evaluated before the operator is applied, with the following exceptions:

- `&&` and `||`: Short-circuit evaluation. `a && b` evaluates `b` only if `a` is `True`. `a || b` evaluates `b` only if `a` is `False`.
- `?->`: Conditional expressions only evaluate the taken branch.
- `*`: If either operand evaluates to 0 (int or real), the result is 0 and the other operand is not evaluated.

12.1 Evaluation Order

Within an expression, operands are evaluated left-to-right. Function arguments are evaluated left-to-right before the function is called.

12.2 Operator Precedence

The complete operator precedence is defined in Section 8.

12.3 Type Coercion

Type coercion follows the hierarchy: `int` < `real` < `complex`.

- `int` coerces to `real`: 5 becomes 5.0
- `real` coerces to `complex`: 3.14 becomes 3.14 + 0i
- `int` coerces to `complex`: 5 becomes 5 + 0i
- `bool` coerces to `int`: `True` becomes 1
- `bool` coerces to `real`: `True` becomes 1.0
- `bool` coerces to `complex`: `True` becomes 1 + 0i

- `bool` coerces to `string`: `True` becomes `"true"`, `False` becomes `"false"`
- Mixed numeric operations promote to the wider type: `int + real → real`, `real + complex → complex`
- Coercion is never lossy in the upward direction
- Downward coercion (`complex → real`, `real → int`) raises a `Type Error` unless done explicitly via a casting function (e.g., `f.int()`, `f.real()`)

Chapter 13

Execution Order

13.1 Sequential Execution

Program statements are executed sequentially from the entry point (**start**). Electrical statements (**elec.part**, **elec.connect**, **elec.net**) execute immediately and register components to the electrical graph. Physical constraints (**phys.***) execute immediately but their resolution is deferred until after program execution completes.

13.2 Expression Evaluation

Within a single expression, operands are evaluated left-to-right. The pipeline operator **|=>** evaluates the left operand first, then passes the result to the function reference on the right. Function arguments are evaluated left-to-right before the function is called. Short-circuit operators (**&&**, **||**) may skip evaluation of the right operand when the result is determined, unless the unevaluated operand contains a function call that modifies global scope, in which case both operands are always evaluated.

13.3 Assertions

The **assert** statement evaluates its boolean condition at runtime. If the condition is **True**, execution continues. If **False**, the specified error is raised immediately with the provided message. The function form **f.util.assert()** behaves identically. Assertions are not optimized away at compile time.

13.4 Error Handling

The **try/except** construct provides structured error handling. When an error occurs inside a **try** block, the runtime searches the **otherwise** clauses in order for a matching error type. If found, the error is caught and the corresponding block executes. If not found, the error propagates after the **except** block (if present). The **except** block always executes, whether the **try** succeeded, an error was caught, or an unhandled error propagated.

13.5 Function Value Calling

A function value stored in a variable is called using the syntax `$var(args)`. The evaluation proceeds as follows:

1. The variable is evaluated to obtain the function value
2. If the variable holds `null`, a `Runtime Error` is raised
3. If the variable does not hold a `func` value, a `Type Error` is raised
4. All arguments are evaluated left-to-right
5. The function is called with the evaluated arguments
6. If the function does not accept the provided argument count or types, a `Type Error` is raised

13.6 Safe Function Calling

The `?()` and `??()` operators provide null-safe function invocation. Both evaluate all arguments **before** checking the function reference. Errors in argument evaluation are never suppressed.

13.6.1 Safe Call (`?()`)

1. All arguments are evaluated left-to-right
2. The variable is evaluated to obtain the function value
3. If the variable holds `null`: returns `null` (no error)
4. If the variable does not hold a `func` value: raises a `Type Error`
5. The function is called with the evaluated arguments
6. Errors raised within the function **propagate** normally

13.6.2 Null-Safe Call (`??()`)

1. All arguments are evaluated left-to-right
2. The variable is evaluated to obtain the function value
3. If the variable holds `null`: returns `null` (no error)
4. If the variable does not hold a `func` value: returns `null` (no error)
5. The function is called with the evaluated arguments
6. If the function call raises **any** error (`Type`, `Value`, `Division by Zero`, `Runtime`, `Assertion`): returns `null` (error suppressed)

The `??()` operator is equivalent to wrapping the call in `try { fn(args) } otherwise { null }`, but without introducing a new code block.

13.7 Function Composition

The composition operator `<o` creates a new function value at evaluation time. The right operand is evaluated first, then the left operand. Both must evaluate to **func** values. The result is a new anonymous function that, when called, evaluates `$g(x)` first, then passes the result to `$f`. The composed function captures both function values by reference at composition time.

13.8 Declaration Order

Variables must be declared before they are accessed. Reading a variable before its declaration statement has been executed raises a **Runtime Error**. Declarations are processed in order during execution (there is no separate declaration phase). Variable types may change on redeclaration.

Chapter 14

Constraint Resolution

Physical constraints are resolved by the solver according to the priority system defined in Section 22.

14.1 Conflict Resolution

When constraints conflict, the solver applies the higher-priority constraint and optimizes the lower-priority one. See Section 23 for the complete resolution rules.

Chapter 15

Type System Rules

EDADSL uses a dynamic type system with distinct types for electrical and non-electrical objects.

15.1 Type Categories

- Generic types: Boolean, Integer, Real, Complex, String, List, Set, Function
- Electrical types: Part, Pin, Connection, Net

15.2 Type Safety

EDADSL performs type checking at both compile time and runtime:

- **Compile-time:** Type mismatches in variable declarations (e.g., `$x [int]= "hello"`) are detected when the script is parsed.
- **Runtime:** Type mismatches in function arguments and operator usage are detected when the code is executed. A **Type Error** is raised for incompatible types.
- **Sets:** Sets can contain any data type. Mixed-type sets are allowed.
- **Coercion:** Numeric types coerce upward: `int` $\rightarrow$ `real` $\rightarrow$ `complex`. Other types do not coerce. Function values do not coerce to or from any other type.

15.3 Error Types

EDADSL defines seven error types. Each error type has a full name and a short alias for use in `assert` statements and `f.util.assert` calls.

Full Name	Alias	Typical Cause
Type Error	<code>terr</code>	Incompatible types
Value Error	<code>verr</code>	Invalid value (e.g., out of domain)
Division by Zero Error	<code>odiv</code>	Division by zero
Runtime Error	<code>rerr</code>	Invalid runtime state
Assertion Error	<code>aerr</code>	Failed assertion
Permission Error	<code>perm</code>	Filesystem sandbox violation

All errors halt program execution immediately. There is no try/catch mechanism.

Part III

Core Formal Model

Chapter 16

Structure of the Program

16.1 Program Entry Point

Every EDADSL program must begin with the **start** keyword. The program executes sequentially from the entry point.

16.2 Global Scope

Variables and constants declared at the top level of the program have global scope and are visible from their declaration onward. Variables declared inside code blocks (**if**, **for**, **while**, **do-while**) are local to that block. Variables declared inside functions are local to that function (see Section 11.2.4).

16.3 Execution Model

EDADSL uses a single-pass execution model. There is no separate declaration phase — declarations are processed in order during execution (see Appendix for execution-phase diagram). Variables must be declared before they are accessed.

Electrical statements (**elec.part**, **elec.connect**, **elec.net**) are executed immediately and register components and nets to the electrical graph. Physical constraints (**phys.***) are executed immediately but their resolution is deferred until after program execution completes.

The solver interacts with program execution as follows:

1. Program executes sequentially, registering all components, nets, and constraints
2. After program execution completes, the constraint solver runs
3. The solver uses CSP propagation and simulated annealing to determine optimal placement and routing
4. Output files are generated from the solved state

Chapter 17

Structure of the Electrical System

17.1 Electrical Graph

The electrical system is modeled as an undirected graph where:

- Vertices represent Nets
- Edges represent Connections
- Parts attach to the graph via their Pins

17.2 Object Lifecycle

Electrical objects (parts, connections, pins, nets) are committed to the graph immediately when their declaration statement is executed. The graph is modified incrementally as the program runs.

- `elec.part` creates a Part node and attaches Pins
- `elec.net` creates a Net node
- `elec.connect` creates Connection edges between Pins and Nets
- `elec.tag` / `elec.untag` modify Net metadata

There are no undo/redo semantics. Once an object is committed to the graph, it cannot be removed. The graph state at the end of program execution is the final state passed to the constraint solver.

Chapter 18

Structure of the Solver

18.1 Constraint Representation

Physical constraints are represented as optimization objectives with priorities.

18.2 Resolution Algorithm

The constraint solver uses a two-phase approach:

1. **CSP Propagation:** Hard constraints (type matching, connectivity) are enforced first. This reduces the search space by eliminating invalid configurations.
2. **Simulated Annealing:** Soft constraints (proximity, alignment, length matching) are optimized using simulated annealing. Constraints are weighted by priority level:
 - `-d` (default) — weight 1.0
 - `-nth` — weight 0.5
 - `-imp` — weight 2.0
 - `-vimp` — weight 4.0
 - `-oeo` — weight 3.0 (override exact order)
 - `-eo` — weight 1.5 (exact order)
 - `-ne` — weight 0.3 (non-exact)

Convergence criteria: The solver terminates when (a) all hard constraints are satisfied and no soft constraint has improved by more than 0.1% over the last 1000 iterations, or (b) the maximum iteration count (default 10000) is reached.

Part IV

Hardware Model

Chapter 19

Circuit Model

The circuit model represents the electrical system as a graph of interconnected objects.

19.1 Graph Structure

The circuit is represented as a graph where:

- Nodes are Nets (electrical connection points)
- Edges are Connections (physical wires/traces between pins)
- Parts are components that connect to the graph via Pins

19.2 Hierarchy

The model supports hierarchical grouping:

- Parts contain Pins
- Pins connect to Nets via Connections
- Nets can be grouped by Tags

Chapter 20

Electrical Objects

The electrical system consists of five primary object types.

20.1 Nets

A Net is a named electrical node that connects multiple Pins and Connections. Nets are the fundamental grouping mechanism for shared electrical connections.

20.1.1 Creation

Nets are created using the `elec.net` statement:

```
elec.net <name> [<tag1> ... <tag n>]
```

Example:

```
elec.net "Gnd"
```

```
elec.net "hvbus" "hv" "high_i" "noisy"
```

If two `elec.net` statements use the same name, they are merged into a single net. Tags from both declarations are combined, and a warning is issued.

20.1.2 Properties

- Name (unique identifier)
- Connected objects (Pins and Connections)
- Tags (user-defined labels)

20.2 Parts

A Part represents a physical electronic component in the circuit.

20.2.1 Creation

Parts are created using the `elec.part` statement:

```
elec.part <reference>: <model>(<args>);  
    des="<description>"; fp="<footprint>"
```

Example:

```
elec.part "R3": R(R=220, P=0.25, type="carbon");  
    des="220 ohm resistor";  
    fp="Resistor_THT:R_Axial_DIN0309"
```

20.2.2 PCB Text Visibility Flags

Per-component PCB text visibility can be overridden in the property list:

```
elec.part "R3": R(R=220); des="220R"; fp="R_Axial_DIN0309";  
    show_ref=True; show_value=False; show_tol=True; show_mpn=False
```

Available flags: `show_ref`, `show_value`, `show_tol`, `show_mpn`. These override the global `syst.flag.show_pcb_part_*` flags (see Chapter 24).

20.2.3 Properties

- Reference designator (e.g., R3, Q1, U1)
- Model and parameters
- Description
- Footprint
- Pins (derived from model/footprint)

20.3 Pins

A Pin is a connection point on a Part. Pins are accessed using the bracket notation.

20.3.1 Creation

Pins are created implicitly when Parts are instantiated.

20.3.2 Reference Syntax

```
<part_ref>[<pin_ref>]
```

Examples:

```
R1[1]    #c# Pin 1 of resistor R1  
Q1[d]    #c# Drain pin of MOSFET Q1  
U1[v+]   #c# Non-inverting input of op-amp U1
```

20.4 Connections

A Connection is a binary electrical connection between exactly two Pins.

20.4.1 Creation

Connections can be created using the pipe syntax or `elec.connect`:

```
#c# Pipe syntax (creates a Connection object):  
( R2[1] | J4[7] )
```

```
#c# Connect statement (joins pins to a Net):  
elec.connect <net> <pin1> <pin2> ... <pin n>
```

Example:

```
elec.connect Gnd Q2[s] C6[-] R3[2]
```

20.5 Models

Models define the electrical characteristics and available pins of a Part type.

20.5.1 Model Declaration Syntax

Custom models are defined using `elec.makemodel`:

EBNF:

```
model_declaration = "elec.makemodel", model_name, ":",  
                    "{", pin_entries, [metadata], "}"
```

```
model_name        = id, ["/", id]
```

```
pin_entry         = pin_name, "->", pin_id, ";",  
                    "T", "=", pin_type
```

```
pin_name          = id
```

```
pin_id            = integer
```

```
pin_type          = "input" | "power" | "output"
```

```
metadata          = des_entry | dts_entry | fp_entry
```

```
des_entry         = "des", "=", string_type
```

```
dts_entry         = "dts", "=", string_type
```

```
fp_entry          = "fp", "=", string_type
```

The `model_name` consists of a category and variant separated by / (e.g., `mosfet/C3M0016120K1`).

Pin types classify the electrical role:

- `input` – Signal input (gate, base, control)

- **power** – Power connection (source, drain, emitter, collector)
- **output** – Signal output (drain for sense, kelvin connections)

20.5.2 Built-in Models

- **R** – Resistor. Params: **R** (resistance), **P** (power rating), **tol** (tolerance), **type** (carbon, metal film, etc.)
- **C** – Capacitor. Params: **C** (capacitance), **V** (voltage rating), **type** (ceramic, electrolytic, etc.)
- **L** – Inductor. Params: **L** (inductance), **core** (core material)
- **opamp** – Operational Amplifier. Params: **type** (generic, LM741, etc.), **GBP** (gain-bandwidth product), **slew** (slew rate), **Vos** (offset voltage)
- **MOSFET** – MOSFET transistor. Params: **type** (N/P), **Vds** (drain-source voltage), **Id** (drain current), **Rds** (on-resistance), **Vgs** (gate threshold)
- **LED** – Light Emitting Diode. Params: **color** (red, green, blue, etc.), **Vf** (forward voltage), **If** (forward current)
- **D** – Diode. Params: **type** (signal, rectifier, zener), **Vf** (forward voltage), **If** (forward current), **Vr** (reverse voltage)
- **Q** – BJT Transistor. Params: **type** (NPN/PNP), **Vce** (collector-emitter voltage), **Ic** (collector current), **hfe** (DC gain)
- **X** – Crystal/Oscillator. Params: **freq** (frequency), **load** (load capacitance), **ESR** (series resistance)
- **J** – Connector. Params: **pins** (pin count), **type** (header, barrel, etc.), **pitch** (pin spacing)
- **F** – Fuse. Params: **I** (current rating), **V** (voltage rating), **type** (glass, blade, etc.)
- **Y** – Transformer. Params: **ratio** (turns ratio), **Vpri** (primary voltage), **Vsec** (secondary voltage), **power** (VA rating)

20.5.3 Custom Model Examples

MOSFET Model

```
elec.makemodel MOSFET/IRLZ44N:{
    g -> 1; T = input
    S -> 2; T = power
    d -> 3; T = power
}
```

Non-Polarized Capacitor

```
elec.makemodel nonpol_C:{  
    1 -> 1; T = power  
    2 -> 2; T = power  
}
```

SiC MOSFET with Metadata

```
elec.makemodel mosfet/C3M0016120K1:{  
pins:  
    g      -> 4; T = input  
    s      -> 2; T = power  
    d      -> 1; T = power  
    kelvin_s -> 3; T = output  
  
des = "1200 V, 125A, 16 mOhm, TO-247-4"  
  
dts = "https://assets.wolfspeed.com/..."  
  
fp  = "Package_TO_SOT_THT/TO-247-4_Vert"  
}
```

The `des` (description), `dts` (datasheet URL), and `fp` (footprint path) are optional metadata entries.

20.6 Electrical Modules

Electrical modules encapsulate reusable subcircuits with configurable parameters. Modules can contain parts, connections, nets, physical constraints, and nested modules.

20.6.1 Definition

Modules are defined using `elec.makemodule`:

```
elec.makemodule <name>(<$param1>, <$param2>, ...) {  
    pin <name> <direction>  
  
    <full program syntax>  
}
```

- Parameters are declared after the module name as dollar-prefix variables.
- Exposed pins are declared with `pin <name> <direction>` (input, output, power).
- The body uses full program syntax: `elec.part`, `elec.connect`, `elec.net`, `phys.*`, etc.
- Parameters are substituted into the body at instantiation time.

20.6.2 Instantiation

Modules are instantiated via `elec.part` using the module name as the model:

```
elec.part "<reference>": <module_name>(<params>)
```

Only the module name and parameter values are provided. Internal parts, connections, and constraints are defined inside the module.

20.6.3 Example

Definition:

```
elec.makemodule LowPassFilter($R, $C) {  
    pin in input  
    pin out output  
  
    elec.part "R1": R(R=$R); des="Resistor"  
    elec.part "C1": C(C=$C); des="Capacitor"  
  
    elec.connect in R1[1]  
    elec.connect R1[2] C1[1] out  
    elec.connect C1[2] Gnd  
}
```

Usage:

```
elec.part "F1": LowPassFilter(R=10k, C=100n)  
elec.part "F2": LowPassFilter(R=4.7k, C=1u)
```

20.7 Relationship between the Objects

- Parts contain Pins
- Pins are connected to Nets via Connections
- Nets can have Tags for grouping
- The global set `*e` contains all objects in creation order

Chapter 21

Querying

EDADSL provides a powerful querying system for selecting electrical objects based on their relationships.

21.1 Output and Input Quantifiers

The quantifier syntax selects objects connected to a net:

```
<quantifier>@<net_name>
```

21.2 Net based Querying

Quantifiers filter by object type:

```
P@<net>    #c# Parts connected to net
C@<net>    #c# Connections on net
T@<net>    #c# Pins connected to net
I@<net>    #c# Connections and Pins on net
H@<net>    #c# Holes on net
E@<net>    #c# Everything on net (or just @<net>)
```

Example:

```
$input [set]= I@Vin
```

21.3 Tagged Net based Querying

Query by tag instead of net name:

```
<quantifier>@^<tag>
```

This expands to the union of the quantifier across all nets with the given tag.

Example:

```
elec.tag Vin "sensitive"
elec.tag Vfb "sensitive"
I@^sensitive #c# All connections/pins on
              #c# nets tagged "sensitive"
```

21.4 Affiliation-based Querying

Affiliation-based querying selects objects affiliated with a specific base object, regardless of net:

`<primary>@<secondary>~<object>`

- `<primary>` – What to return (same quantifiers as net-based: C, P, T, N, I, H, E)
- `<secondary>` – Relationship type: p (part), c (connection), n (net), t (pin), or e (no type filter)
- `<object>` – The base object to query from

The result is a set of objects of type `<primary>` affiliated with `<object>` through the relationship classified by `<secondary>`.

Examples:

```
C@p~U2      #c# All connections affiliated with part U2
T@p~U2      #c# All pins of part U2
N@p~R1      #c# All nets connected to R1
P@c~N1      #c# All parts connected via net N1
E@p~U2      #c# Everything affiliated with U2
E@e~U2      #c# Same as above (no type filter)
```

21.5 General Querying

General querying provides two powerful forms for filtering and transforming sets: set builder notation and set query notation. Both support nesting.

21.5.1 Set Builder Notation

Set builder notation iterates over a set or list, filtering and optionally transforming elements.

EBNF:

```
set_builder = "{", variable, "E", ( set | list ),
              ":", filter_expr, [ "~", transform_expr ], "}"
```

```
filter_expr  = expression    #c# any expression evaluating to bool
transform_expr = function_call #c# applied to filtered elements
```

The filter function receives each element and returns `True` to keep it. The transform function is applied **per-element** to each item that passes the filter; the result set contains the transformed values. If transform is omitted, elements are returned as-is.

Examples:

```
#c# Filter: only active parts
{ $x E *p : f.u.is_active($x) ~ noop($x) }

#c# Filter and transform: get names of large caps
```



```

{ $x E *p : f.u.is_capacitor($x) &&
    f.u.value($x) > 100u ~
    f.u.name($x) }

## Nested: find all nets connected to parts
## in a set
{ $x E *p : f.u.is_mosfet($x) ~
    { $y E *n : f.u.connected($y, $x) ~
        noop($y) } }

```

21.5.2 Set Query Notation

Set query notation applies a filter and optional transform to an existing set, with optional output assignment.

EBNF:

```

set_query = "{", set_expr, "@~", "[",
    filter_expr, "]",
    [ "~[", transform_expr, "]" ],
    "~>", [ variable ], "}"

```

The @~ operator queries the set. The trailing ~> optionally writes the result to a variable; if omitted, the result is returned directly.

Examples:

```

## Query with filter only
{ *p @~[f.u.is_active] ~> }

## Query with filter and transform
{ *p @~[f.u.is_mosfet]
    ~[f.u.name] ~> }

## Store result in variable
{ *p @~[f.u.is_capacitor]
    ~[f.u.value] ~> $cap_values }

## Query connections on a net
{ C@Vin @~[f.u.is_high_current] ~> }

```

21.5.3 Nesting

Both forms can be nested to any depth. Each nested query creates a new scope for its loop variable.

Example:

```

## For each MOSFET, find all connected nets,
## then filter to high-voltage nets
{ $q E *p : f.u.is_mosfet($q) ~
    { C@~power @~[f.u.connected_to($q)]

```

```
      ~[f.u.voltage_rating] ~>  
$ratings } }
```

Part V

Physical System

Chapter 22

Physical Constraints

Physical constraints are specified using the `phys.` prefix and control the physical layout and manufacturing requirements of the PCB.

22.1 Units and SI Prefixes

Warning: All physical values use base SI units (meters, ohms, farads, etc.). A value like `min=15` means 15 **meters**, not 15 millimeters!

Always use SI prefixes for typical PCB dimensions:

- `min=15m` – 15 millimeters (15×10^{-3} m)
- `min=15u` – 15 micrometers (15×10^{-6} m)
- `d=0.25m` – 0.25 millimeters (250 μ m)

SI prefixes commonly used in PCB design: `n` (10^{-9}), `u` (10^{-6}), `m` (10^{-3}), `k` (10^3), `M` (10^6), `G` (10^9). All SI prefixes (quecto through quetta) are supported — see Section 5.1 for the full table.

22.2 Priority System

All physical constraints accept a priority parameter that determines how conflicts are resolved:

Priorities:

<code>(none)</code> or <code>-d</code>	Default
<code>-nth</code>	Nice to have
<code>-imp</code>	Important
<code>-vimp</code>	Very important
<code>-oeo</code>	Only explicit overrides
<code>-eo</code>	Explicit override
<code>-ne</code> = <code>-nonneg</code>	No exceptions

Priority Chain:

`-d` < `-nth` < `-imp` < `-vimp` < `-oeo` < `-eo` < `-ne` = `-nonneg`

22.3 Proximity Constraint

Controls the distance between sets of objects:

```
phys.prox <set_a> <set_b>  
    min=<min_distance> max=<max_distance>  
    <priority>  
phys.prox <set> minim <priority>
```

- **min** – Minimum distance between edges of set a and set b
- **max** – Maximum distance anything in set a can be from anything in set b
- **minim** – Tries to minimize distance between all objects in the set

Example:

```
phys.prox $hvsys $lvsys min=15m -ne  
phys.prox $noisy_loop minim -vimp
```

22.4 Creepage Constraint

Controls the surface distance between sets (affected by slots, vias, traces, and conductive material):

```
phys.creep <set_a> <set_b>  
    d=<min_creepage_distance> <priority>
```

Note: Unlike **phys.prox** (Euclidean distance), **phys.creep** measures surface distance. Conductive material contributes 0 distance.

Example:

```
phys.creep $hvsys $lvsys d=30m -nonneg
```

22.5 Layer Constraint

Assigns objects to specific board layers:

```
phys.layer <set> <layer_number> <priority>
```

Layer 0 is the top layer; the bottom layer is layer n (where n is the total number of layers).

Example:

```
phys.layer *p 0 -oeo
```

22.6 Board Definition

Defines the PCB stackup characteristics. Can only be invoked once; a **Runtime Error** is raised if called a second time.

```
phys.board N=<layers> T=[<thicknesses>]  
          W=[<weights>] M=[<materials>]
```

- N – Number of board layers
- T – List of thicknesses between layers in meters (length $n - 1$)
- W – List of copper weights in oz/ft² (length n)
- M – List: [dielectric, conductor, surface-plating]

Default: `phys.board N=2 T=[0.0016] W=[1, 1] M=["FR4", "copper", "HASL"]`

Example:

```
phys.board N=2 T=[0.0016] W=[2, 2]
          M=["FR4", "copper", "ENIG"]
```

22.7 Footprint Unification

Merges multiple parts into a single physical unit on the PCB. The parts share one combined footprint and reference designator, but retain independent electrical pins. In the schematic, united parts are grouped together.

```
phys.unite { <part>, <part>, ... }
```

- The argument is a set of part references.
- United parts share a single footprint on the PCB.
- Reference designators are combined (e.g., R1.R2.R3).
- Pins remain independently accessible for net connections.

Example:

```
phys.unite { R1, R2, R3 }
#c# R1, R2, R3 share one footprint on the PCB
#c# Schematic groups them together
#c# R1[1], R2[1], R3[1] are still individually accessible
```

22.8 Net Shrink

Minimizes the physical extent of a net:

```
phys.shrinknet <net> <priority>
```

A **Value Error** is raised if the referenced net does not exist.

Example:

```
elec.net "Vsw"
phys.shrinknet Vsw -vimp
```

22.9 Plane Creation

Creates a copper plane on a layer connected to a net:

```
phys.mkplane <layer_number> <net>
```

A `Value Error` is raised if the referenced net or layer does not exist.

Example:

```
elec.net "Gnd"  
phys.mkplane 1 Gnd
```

22.10 Text Annotation

Places text on the board as a physical object:

```
phys.mktext txt = <text>  
    font = <font_name>  
    size = <point_size>  
    material = <material>  
    layer = <layer>
```

- `txt` – Text string to display
- `font` – Font name (e.g., "Ubuntu Mono")
- `size` – Font size in points
- `material` – Material ("cu" for copper silkscreen)
- `layer` – Target layer (e.g., T for top silkscreen). Note: In `phys.mktext`, T refers to the top silkscreen layer, not the top copper layer as in `phys.layer`.

Example:

```
phys.mktext txt = "HV" font = "Ubuntu Mono"  
    size = 9 material = "cu" layer = T
```

22.11 Via Fill

Fills a pad with vias for thermal management:

```
phys.viafill <pad> v=<via_hole_diameter>  
    d=<via_spacing> [p=<pattern>] [l=<layers>]
```

- `v` – Via hole diameter (use m for mm)
- `d` – Minimum spacing between via centers
- `p` – Fill pattern (default: `grid`)
- `l` – Layer span (default: `all`). Can be `all` or a pair like 1-2

Edge Cases

- A `Value Error` is raised if `v` or `d` is ≤ 0 .
- A `Value Error` is raised if `d < v` (vias would overlap).
- A `Value Error` is raised if an invalid pattern is specified.
- A `Value Error` is raised if `l` references layers outside the board stackup.
- A `Type Error` is raised if the `pad` argument is not a valid pin reference.

22.11.1 Pattern Types

All patterns tile the pad area with vias at spacing `d`. Each via is placed at the vertices of the respective tiling.

grid Rectangular lattice. Vias at $(i \cdot d, j \cdot d)$ for integers i, j .

```
phys.viafill Q1[d] v=0.25m d=0.5m p=grid
```

hex Densest circle packing arrangement. Each via has 6 nearest neighbors at distance `d`. Approximately 15% denser than grid, best thermal performance due to maximized copper coverage.

```
phys.viafill Q1[d] v=0.25m d=0.5m p=hex
```

stagger Offset rows forming the same spacing as hex but with alternating row offsets. Equivalent to hex for thermal purposes but creates straighter routing channels between rows.

```
phys.viafill Q1[d] v=0.25m d=0.5m p=stagger
```

radial Concentric rings emanating from pad center. Ring radii at $r_n = d \cdot n$ for integer $n \geq 1$. Optimal for round thermal pads under ICs.

```
phys.viafill U1[pin5] v=0.3m d=0.8m p=radial
```

random Poisson-disk distributed vias with minimum spacing `d`. Avoids periodic structures that can cause resonance or EMI.

```
phys.viafill Q1[d] v=0.25m d=0.5m p=random
```

22.11.2 Examples

```
#c# Standard grid fill for MOSFET drain
phys.viafill Q1[d] v=0.25m d=0.5m p=grid
```

```
#c# Dense hex fill for high-current pad
phys.viafill Q1[d] v=0.2m d=0.4m p=hex
```



```
#c# Radial fill for IC thermal pad
phys.viafill U1[pin5] v=0.3m d=0.8m p=radial
```

```
#c# Random fill to avoid EMI resonance
phys.viafill Q1[d] v=0.25m d=0.5m p=random
```

22.12 Connection Current

Sets the minimum RMS current a connection must carry:

```
phys.connectioncurrent <connection | set>
    <rms_current> <priority>
```

Example:

```
phys.connectioncurrent ( J1[2] | F1[1] ) 30 -oeo
```

22.13 Alignment

Aligns parts along an axis with specified spacing:

```
phys.align <list_of_parts>
    axis=<x|y> d=<edge_spacing> <priority>
```

Example:

```
phys.align [ Q1, Q2 ] axis=x d=5m -vimp
```

22.14 Connection Matching

Minimizes variance of a property across a set of connections:

```
phys.connmatch <property>
    <set_of_connections> <priority>
```

The set of connections is a **set** of connection objects (e.g., connections between parts and nets), or a variable holding such a set.

22.14.1 Properties

- **l** – Physical trace length
- **res** – DC resistance
- **L** – Self-inductance
- **cap** – Capacitance to reference plane
- **imp** – Characteristic impedance
- **t** – Signal propagation delay

22.14.2 Value Calculation Methods

Calculation Mode Selection

The calculation method can be selected using the `syst.cm.calc` flag:

`syst.cm.calc <mode>`

Available modes:

- **approx** – (Default) Uses simplified closed-form formulas for fast computation. Suitable for most designs.
- **calculus** – Uses numerical integration and differential equations for higher accuracy. Computationally more expensive.
- **hybrid** – Uses closed-form formulas for initial estimates, then refines with calculus-based methods for critical nets.

Example:

```
syst.cm.calc calculus
phys.connmatch imp $high_speed_lines -vimp
```

Length (len)

The physical length of the trace path from source pin to destination pin, measured in millimeters. Calculated as the sum of all trace segments.

Resistance (res)

DC resistance is calculated using:

$$R = \frac{\rho \cdot l}{W \cdot T} \quad (22.1)$$

where:

- ρ – Resistivity of the conductor (copper: $1.72 \times 10^{-8} \Omega \cdot \text{m}$)
- l – Trace length (mm)
- W – Trace width (mm)
- T – Trace thickness (mm, derived from copper weight in oz/ft²)

Calculus Mode When `syst.cm.calc calculus` is active, skin effect is considered:

$$R_{ac} = R_{dc} \cdot \left(1 + \frac{1}{12} \cdot \left(\frac{\delta}{T} \right)^4 \right) \quad (22.2)$$

where $\delta = \sqrt{\frac{2\rho}{\omega\mu}}$ is the skin depth at frequency ω .

Inductance (ind)

Self-inductance is approximated using the loop inductance formula for a microstrip:

$$L \approx \frac{\mu_0 \cdot l_{trace}}{2\pi} \cdot \ln \left(\frac{2h}{w+t} \right) \quad (22.3)$$

where:

- μ_0 – Permeability of free space ($4\pi \times 10^{-7}$ H/m)
- l_{trace} – Trace length
- h – Height above reference plane
- w – Trace width
- t – Trace thickness

Calculus Mode When `syst.cm.calc calculus` is active, mutual inductance and frequency-dependent effects are computed using numerical integration:

$$L_{total} = \int_0^{l_{trace}} \frac{\mu_0}{2\pi} \cdot \ln \left(\frac{2h(x)}{w(x)+t} \right) dx \quad (22.4)$$

This accounts for trace width variations and non-uniform height above the reference plane.

Capacitance (cap)

Capacitance to the reference plane is calculated using the microstrip capacitance formula:

$$C \approx \frac{\varepsilon_0 \cdot \varepsilon_r \cdot w \cdot l_{trace}}{h} \quad (22.5)$$

where:

- ε_0 – Permittivity of free space (8.854×10^{-12} F/m)
- ε_r – Relative permittivity of the dielectric (FR4: ≈ 4.5)
- w – Trace width
- l_{trace} – Trace length
- h – Height above reference plane

Calculus Mode When `syst.cm.calc calculus` is active, fringing effects and dielectric variations are included:

$$C = \int_0^{l_{trace}} \frac{\varepsilon_0 \cdot \varepsilon_r(x) \cdot w_{eff}(x)}{h(x)} dx \quad (22.6)$$

where w_{eff} includes fringing capacitance contributions and $\varepsilon_r(x)$ accounts for material variations along the trace.

Impedance (imp)

Characteristic impedance for a microstrip is calculated as:

$$Z_0 \approx \frac{87}{\sqrt{\varepsilon_r + 1.41}} \cdot \ln \left(\frac{5.98h}{0.8w + t} \right) \quad (22.7)$$

For stripline:

$$Z_0 \approx \frac{60}{\sqrt{\varepsilon_r}} \cdot \ln \left(\frac{4b}{0.67(0.8w + t)} \right) \quad (22.8)$$

where b is the total dielectric thickness for stripline configurations.

Calculus Mode When `syst.cm.calc calculus` is active, the full telegrapher's equations are solved numerically:

$$Z_0 = \sqrt{\frac{R + j\omega L}{G + j\omega C}} \quad (22.9)$$

where R , L , G , and C are per-unit-length parameters computed from the trace geometry and material properties. This accounts for frequency-dependent losses and dispersion.

Propagation Time (t)

Signal propagation delay is calculated as:

$$t = \frac{l_{trace}}{v_p} = \frac{l_{trace} \cdot \sqrt{\varepsilon_{eff}}}{c} \quad (22.10)$$

where:

- v_p – Propagation velocity
- c – Speed of light in vacuum (3×10^8 m/s)
- ε_{eff} – Effective permittivity (depends on geometry and dielectric)

Calculus Mode When `syst.cm.calc calculus` is active, propagation delay is computed by integrating along the trace path:

$$t = \int_0^{l_{trace}} \frac{\sqrt{\varepsilon_{eff}(x)}}{c} dx \quad (22.11)$$

This accounts for varying dielectric thickness, material transitions, and impedance discontinuities along the trace.

22.14.3 Matching Objective

The solver minimizes the scaled variance of the selected property across all connections in the set:

$$\text{minimize} \quad \frac{\text{var}(P)}{||S||} \quad (22.12)$$

where P is the property value for each connection and $||S||$ is the cardinality of the connection set.

22.14.4 Example

```
phys.connmatch t $CPU_to_RAM_lines -vimp
```

This ensures all CPU-to-RAM signal lines have matched propagation delays, critical for parallel data buses.

Chapter 23

Solver Model

23.1 Overview

The EDADSL solver transforms declarative physical constraints into an optimized PCB layout. The process involves collecting constraints, selecting an optimization algorithm, and generating the final layout through placement and routing phases.

23.2 Algorithm Selection

The solver algorithm can be selected using the `syst.alg` flag:

```
syst.alg <algorithm_name>
```

23.2.1 Available Algorithms

Simulated Annealing (sa)

A probabilistic optimization technique inspired by the annealing process in metallurgy. The algorithm explores the solution space by accepting worse solutions with a probability that decreases over time.

Objective Function Let $\mathbf{x} \in \mathcal{X}$ represent the complete layout state (component positions, trace routes, via placements). The objective function $f : \mathcal{X} \rightarrow \mathbb{R}$ quantifies the total constraint violation cost:

$$f(\mathbf{x}) = \sum_{i=1}^n w_i \cdot v_i(\mathbf{x}) \quad (23.1)$$

where w_i is the priority weight of constraint i and $v_i(\mathbf{x})$ is the violation measure (0 if satisfied, positive otherwise).

Neighbourhood Structure A neighbour state $\mathbf{x}'$ is generated by applying a random perturbation δ to the current state:

$$\mathbf{x}' = \mathbf{x} + \delta, \quad \delta \sim \mathcal{U}(-\epsilon, \epsilon) \quad (23.2)$$

where ϵ is the maximum perturbation magnitude, adaptively scaled based on acceptance history.

Acceptance Probability The Metropolis acceptance criterion determines whether to move to a neighbour state:

$$P(\text{accept}) = \begin{cases} 1 & \text{if } f(\mathbf{x}') \leq f(\mathbf{x}) \\ \exp\left(-\frac{f(\mathbf{x}') - f(\mathbf{x})}{T}\right) & \text{if } f(\mathbf{x}') > f(\mathbf{x}) \end{cases} \quad (23.3)$$

where $T > 0$ is the current temperature parameter.

Cooling Schedule The temperature decreases according to a geometric cooling schedule:

$$T_{k+1} = \alpha \cdot T_k, \quad \alpha \in (0, 1) \quad (23.4)$$

with typical values $\alpha \in [0.95, 0.999]$. The initial temperature T_0 is set such that the initial acceptance probability for a worst-case perturbation is approximately 0.8:

$$T_0 = -\frac{\Delta f_{\max}}{\ln(0.8)} \quad (23.5)$$

Convergence Criterion The algorithm terminates when one of the following conditions is met:

- Temperature threshold: $T_k < T_{\min}$ (typically 10^{-6})
- No improvement: $f(\mathbf{x}_{\text{best}})$ has not improved for N_{stale} iterations
- Maximum iterations: $k > k_{\max}$

Algorithm

1. Initialize $\mathbf{x}_0$ randomly, set T_0 , $k = 0$
2. Generate neighbour $\mathbf{x}'$ from $\mathbf{x}_k$
3. Compute $\Delta f = f(\mathbf{x}') - f(\mathbf{x}_k)$
4. If $\Delta f \leq 0$ or $\mathcal{U}(0, 1) < \exp(-\Delta f/T_k)$, set $\mathbf{x}_{k+1} = \mathbf{x}'$
5. Otherwise, set $\mathbf{x}_{k+1} = \mathbf{x}_k$
6. Update $T_{k+1} = \alpha \cdot T_k$
7. If convergence not met, $k \leftarrow k + 1$, goto 2

Properties

- Time complexity: $O(k_{\max} \cdot |\mathcal{N}(\mathbf{x})|)$ where $|\mathcal{N}(\mathbf{x})|$ is the neighbourhood evaluation cost
- Probability of finding global optimum approaches 1 as $k_{\max} \rightarrow \infty$ with sufficiently slow cooling
- Space complexity: $O(|\mathbf{x}|)$ for storing the current and best states

Genetic Algorithm (ga)

An evolutionary optimization method that maintains a population of candidate solutions and evolves them through biologically-inspired operators: selection, crossover, and mutation.

Population Representation The population at generation t is a set of N chromosomes:

$$P(t) = \{\mathbf{x}_1^{(t)}, \mathbf{x}_2^{(t)}, \dots, \mathbf{x}_N^{(t)}\} \quad (23.6)$$

Each chromosome $\mathbf{x}_i^{(t)} \in \mathcal{X}$ encodes a complete layout state as a gene vector:

$$\mathbf{x}_i^{(t)} = (g_1, g_2, \dots, g_m) \quad (23.7)$$

where each gene g_j represents a design variable (position, route choice, etc.).

Fitness Function The fitness of chromosome $\mathbf{x}_i$ is derived from the objective function:

$$\text{fit}(\mathbf{x}_i) = \frac{1}{1 + f(\mathbf{x}_i)} \quad (23.8)$$

where $f(\mathbf{x}_i)$ is the constraint violation cost. Higher fitness indicates better solutions.

Selection Parent selection uses tournament selection of size τ :

1. Sample τ individuals uniformly from $P(t)$
2. Select the individual with highest fitness as parent

This is repeated to select N parents for mating.

Crossover Single-point crossover with probability p_c :

$$\text{crossover}(\mathbf{x}_i, \mathbf{x}_j) = \begin{cases} (g_1^{(i)}, \dots, g_c^{(i)}, g_{c+1}^{(j)}, \dots, g_m^{(j)}) & \text{if } \mathcal{U}(0, 1) < p_c \\ \mathbf{x}_i & \text{otherwise} \end{cases} \quad (23.9)$$

where $c \sim \mathcal{U}\{1, m-1\}$ is the crossover point.

Mutation Per-gene mutation with probability p_m :

$$g'_j = \begin{cases} g_j + \mathcal{N}(0, \sigma^2) & \text{if } \mathcal{U}(0, 1) < p_m \\ g_j & \text{otherwise} \end{cases} \quad (23.10)$$

where σ is the mutation strength, often adapted over generations.

Elitism The best $\lfloor e \cdot N \rfloor$ individuals are copied unchanged to the next generation, where $e \in [0, 0.2]$ is the elitism rate.

Convergence Criterion The algorithm terminates when:

- Maximum generations: $t > t_{\max}$
- Fitness stagnation: Best fitness has not improved for t_{stale} generations
- Target fitness: $\text{fit}(\mathbf{x}_{\text{best}}) > 1 - \epsilon_{\text{fit}}$

Algorithm

1. Initialize population $P(0)$ randomly
2. Evaluate fitness for all $\mathbf{x}_i^{(t)} \in P(t)$
3. Apply elitism: copy top $e \cdot N$ individuals to $P(t+1)$
4. Select parents via tournament selection
5. Apply crossover with probability p_c
6. Apply mutation with probability p_m
7. Replace population: $P(t) \leftarrow P(t+1)$
8. If convergence not met, $t \leftarrow t+1$, goto 2

Properties

- Time complexity: $O(t_{\max} \cdot N \cdot m)$ for N individuals of length m
- Maintains population diversity, reducing local optima entrapment
- Space complexity: $O(N \cdot |\mathbf{x}|)$ for storing the population
- Naturally parallelizable across individuals

Constraint Programming (cp)

A declarative optimization approach that models the problem as a set of variables, domains, and constraints, using propagation and backtracking to find solutions.

Problem Formulation A constraint satisfaction problem (CSP) is defined as a tuple:

$$\mathcal{P} = (\mathcal{X}, \mathcal{D}, \mathcal{C}) \tag{23.11}$$

where:

- $\mathcal{X} = \{x_1, x_2, \dots, x_n\}$ is the set of decision variables
- $\mathcal{D} = \{D(x_1), D(x_2), \dots, D(x_n)\}$ is the set of variable domains
- $\mathcal{C} = \{c_1, c_2, \dots, c_m\}$ is the set of constraints

Domains Each variable x_i has an initial domain $D(x_i) \subseteq \mathbb{R}$ (or discrete set). For PCB layout:

- Position variables: $D(x_i) = [x_{\min}, x_{\max}] \times [y_{\min}, y_{\max}]$
- Route variables: $D(x_i) = \{\text{path}_1, \text{path}_2, \dots\}$

Constraint Types Constraints are classified as:

- **Unary:** $c(x_i)$ — restricts a single variable
- **Binary:** $c(x_i, x_j)$ — relates two variables
- **Global:** $c(x_{i_1}, \dots, x_{i_k})$ — involves multiple variables (e.g., `alldiff`)

Arc Consistency (AC-3) For each binary constraint $c(x_i, x_j)$, enforce arc consistency:

$$\forall v_i \in D(x_i), \exists v_j \in D(x_j) \text{ such that } c(v_i, v_j) = \text{true} \quad (23.12)$$

If no such v_j exists, remove v_i from $D(x_i)$. Repeat until no more domains can be reduced.

Propagation When a domain is reduced, propagate effects to all connected constraints:

1. Maintain a worklist W of affected variables
2. For each $x_i \in W$, re-evaluate all constraints involving x_i
3. If any domain $D(x_j)$ is reduced, add x_j to W
4. Continue until W is empty (fixpoint reached) or a domain becomes empty (failure)

Backtracking Search The solver explores the search tree depth-first:

1. Select unassigned variable x_i (branching heuristic: smallest domain first)
2. For each value $v \in D(x_i)$ (best-first ordering):
 - (a) Assign $x_i \leftarrow v$
 - (b) Propagate constraints
 - (c) If propagation succeeds (no empty domains), recurse
 - (d) If propagation fails, backtrack

Optimization Extension For optimization (minimizing $f(\mathbf{x})$), add a bound constraint:

$$f(\mathbf{x}) < f(\mathbf{x}_{\text{best}}) \quad (23.13)$$

This is tightened each time a better solution is found (branch-and-bound).

Convergence Criterion

- Solution found: All variables assigned, all constraints satisfied
- No solution: Search tree exhausted (problem infeasible)
- Optimality: All nodes pruned, current best is optimal

Properties

- Time complexity: Exponential in worst case, but propagation prunes large portions of the search tree
- Guarantees constraint satisfaction (complete when solution exists)
- Space complexity: $O(n \cdot |D_{\max}| + m)$ for domain storage and constraint graph
- Deterministic: Same problem instance yields same solution

Gradient Descent (gd)

A first-order iterative optimization algorithm that finds local minima by moving in the direction of steepest descent of the objective function.

Objective Function Assume the objective function $f : \mathbb{R}^n \rightarrow \mathbb{R}$ is differentiable, where n is the number of design variables (component positions, trace parameters, etc.).

Gradient Computation The gradient of f at point $\mathbf{x}_k$ is:

$$\nabla f(\mathbf{x}_k) = \left(\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \dots, \frac{\partial f}{\partial x_n} \right)^\top \quad (23.14)$$

Computed via automatic differentiation or finite differences:

$$\frac{\partial f}{\partial x_i} \approx \frac{f(\mathbf{x}_k + h\mathbf{e}_i) - f(\mathbf{x}_k - h\mathbf{e}_i)}{2h} \quad (23.15)$$

where h is a small perturbation and $\mathbf{e}_i$ is the i -th unit vector.

Update Rule The basic gradient descent update is:

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \eta_k \nabla f(\mathbf{x}_k) \quad (23.16)$$

where $\eta_k > 0$ is the learning rate at iteration k .

Learning Rate Schedules

- **Constant:** $\eta_k = \eta$
- **Decay:** $\eta_k = \eta_0 / (1 + \beta k)$
- **Exponential:** $\eta_k = \eta_0 \cdot \gamma^k$
- **Adaptive:** $\eta_{k,i} = \eta_0 / \sqrt{s_i + \epsilon}$ (per-parameter, AdaGrad-style)

where η_0 is the initial learning rate, β, γ are decay parameters, s_i accumulates past squared gradients, and ϵ prevents division by zero.

Momentum Enhancement To accelerate convergence and dampen oscillations:

$$\mathbf{v}_{k+1} = \mu \mathbf{v}_k + \eta_k \nabla f(\mathbf{x}_k) \quad (23.17)$$

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \mathbf{v}_{k+1} \quad (23.18)$$

where $\mu \in [0, 1]$ is the momentum coefficient (typical: 0.9).

Constraint Handling For constraints $g_j(\mathbf{x}) \leq 0$, use a penalty method:

$$\tilde{f}(\mathbf{x}) = f(\mathbf{x}) + \lambda \sum_{j=1}^m \max(0, g_j(\mathbf{x}))^2 \quad (23.19)$$

where λ is a penalty weight, increased iteratively:

$$\lambda_{k+1} = \min(\lambda_k \cdot \rho, \lambda_{\max}), \quad \rho > 1 \quad (23.20)$$

Convergence Criterion

- Gradient norm: $\|\nabla f(\mathbf{x}_k)\| < \epsilon_{\text{grad}}$
- Parameter change: $\|\mathbf{x}_{k+1} - \mathbf{x}_k\| < \epsilon_{\text{step}}$
- Function change: $|f(\mathbf{x}_{k+1}) - f(\mathbf{x}_k)| < \epsilon_f$
- Maximum iterations: $k > k_{\max}$

Algorithm

1. Initialize $\mathbf{x}_0, \eta_0, \lambda_0, k = 0$
2. Compute $\nabla f(\mathbf{x}_k)$ and $\nabla g_j(\mathbf{x}_k)$
3. Update penalty: $\lambda_{k+1} = \min(\lambda_k \cdot \rho, \lambda_{\max})$
4. Compute search direction: $\mathbf{d}_k = -\nabla \tilde{f}(\mathbf{x}_k)$
5. Line search: find η_k satisfying Armijo condition
6. Update: $\mathbf{x}_{k+1} = \mathbf{x}_k + \eta_k \mathbf{d}_k$
7. If convergence not met, $k \leftarrow k + 1$, goto 2

Properties

- Time complexity: $O(k_{\max} \cdot n)$ per iteration for gradient computation
- Converges to local minima; requires convexity or good initialization for global optimality
- Space complexity: $O(n)$ for storing the current point and gradient
- Fast per-iteration, but may require many iterations for convergence

Hybrid (hybrid)

A multi-strategy approach that combines the strengths of different algorithms in a sequential pipeline, adapting the optimization strategy to the problem characteristics at each phase.

Architecture The hybrid solver operates in three sequential phases:

$$\mathbf{x}_{\text{final}} = \text{Phase}_3(\text{Phase}_2(\text{Phase}_1(\mathbf{x}_0))) \quad (23.21)$$

Phase 1: Global Exploration (Simulated Annealing)

- Algorithm: Simulated annealing with reduced iterations ($k_{\text{max}}^{(1)} = k_{\text{max}}/4$)
- Purpose: Explore the solution space broadly, identify promising regions
- Output: $\mathbf{x}_1$ with $f(\mathbf{x}_1) < f_{\text{threshold}}$

The temperature schedule uses faster cooling ($\alpha = 0.95$) to quickly identify basins of attraction.

Phase 2: Local Refinement (Gradient Descent)

- Algorithm: Gradient descent with momentum
- Purpose: Refine the solution from Phase 1 to a local minimum
- Input: $\mathbf{x}_1$ from Phase 1
- Output: $\mathbf{x}_2$ with $\|\nabla f(\mathbf{x}_2)\| < \epsilon_{\text{grad}}$

The penalty weight is initialized from Phase 1's final value and continues to increase.

Phase 3: Constraint Satisfaction (Constraint Programming)

- Algorithm: Constraint propagation and backtracking
- Purpose: Verify and enforce all hard constraints
- Input: $\mathbf{x}_2$ from Phase 2
- Output: $\mathbf{x}_{\text{final}}$ feasible or infeasible report

If $\mathbf{x}_2$ violates hard constraints, CP attempts repair. If repair fails, the solver reports the conflict set.

Adaptive Phase Selection Based on problem characteristics, the hybrid solver may skip phases:

$$\text{skip}(\text{Phase}_i) = \begin{cases} \text{true} & \text{if } n < n_{\text{simple}} \text{ and } i = 1 \\ \text{true} & \text{if all constraints linear and } i = 1 \\ \text{false} & \text{otherwise} \end{cases} \quad (23.22)$$

Phase Transition Criteria Transition from Phase i to Phase $i + 1$ occurs when:

- Improvement rate falls below threshold: $\Delta f / \Delta k < \epsilon_{\text{rate}}$
- Maximum phase iterations reached: $k^{(i)} > k_{\text{max}}^{(i)}$
- Solution quality threshold met: $f(\mathbf{x}) < f_{\text{threshold}}^{(i)}$

Algorithm

1. Initialize $\mathbf{x}_0$, determine phase parameters from problem size n
2. **Phase 1:** Run SA with $k_{\text{max}}^{(1)}$, obtain $\mathbf{x}_1$
3. **Phase 2:** Run GD from $\mathbf{x}_1$ until convergence, obtain $\mathbf{x}_2$
4. **Phase 3:** Run CP from $\mathbf{x}_2$, verify/repair constraints
5. Return $\mathbf{x}_{\text{final}}$ or infeasibility report

Properties

- Time complexity: Sum of phase complexities, typically $O(k_{\text{max}}^{(1)} \cdot |\mathcal{N}| + k_{\text{max}}^{(2)} \cdot n + \text{CP}_{\text{search}})$
- Combines global exploration with local exploitation and constraint guarantees
- Adapts to problem complexity via phase skipping
- Space complexity: Maximum of individual phase requirements

23.2.2 Default Algorithm

If no algorithm is specified, the solver uses simulated annealing (**sa**) as the default.

23.3 Constraint Resolution

The solver resolves physical constraints according to the following rules:

23.3.1 No Conflicts

If all constraints can be satisfied without conflicts, the solution is complete.

23.3.2 Conflicts Between Different Priorities

When constraints with different priorities conflict:

- The higher-priority constraint is satisfied
- The lower-priority constraint is optimized as much as possible without impairing the higher rule
- A warning is printed to the console

23.3.3 Conflicts Between Same Priorities

When constraints with the same priority conflict, behavior depends on the priority level:

Lower Priorities (`-d`, `-nth`, `-imp`, `-vimp`)

- An error message is printed to the console
- The program continues execution

Higher Priorities (`-oeo`, `-eo`, `-ne`, `-nonneg`)

- An error message is printed to the console
- The program does NOT continue execution (halts)

23.4 Layout Generation Pipeline

The final layout is generated through a multi-phase pipeline:

23.4.1 Phase 1: Constraint Collection

All physical constraints are collected from the program and organized by type and priority.

23.4.2 Phase 2: Board Setup

The board stackup is defined (layers, thicknesses, materials) and the design area is initialized.

23.4.3 Phase 3: Component Placement

Parts are placed on the board according to:

1. Alignment constraints (`phys.align`)
2. Proximity constraints (`phys.prox`)
3. Layer constraints (`phys.layer`)
4. Creepage constraints (`phys.creep`)

The solver optimizes placement to satisfy all constraints simultaneously, respecting the priority hierarchy.

23.4.4 Phase 4: Trace Routing

Connections are routed as traces on the board:

1. Current capacity constraints (`phys.connectioncurrent`) determine minimum trace width
2. Connection matching constraints (`phys.connmatch`) ensure length/impedance matching
3. Net shrinkage (`phys.shrinknet`) minimizes trace length

23.4.5 Phase 5: Via Placement

Vias are placed to connect traces across layers:

1. Via fill constraints (`phys.viafill`) populate thermal vias
2. System flags (`syst.flag.vias`, etc.) control via availability

23.4.6 Phase 6: Plane Generation

Copper planes are generated on specified layers:

1. Plane constraints (`phys.mkplane`) create ground and power planes
2. Clearances are maintained around non-connected objects

23.4.7 Phase 7: Design Rule Check

The final layout is verified against all constraints:

1. All physical constraints are checked
2. Violations are reported with severity based on priority
3. Critical violations (`-ne`, `-nonneg`) halt generation

23.4.8 Phase 8: File Export

The verified layout is exported to KiCad format:

1. `make.schematic` generates the schematic file (`.kicad_sch`)
2. `make.pcb` generates the PCB file (`.kicad_pcb`)
3. `halt` triggers final export and log file generation

23.5 Ordering of Global Set Constant `*e`

Inside the `*e` pseudo-constant (and when converting any set to a list via `f.list.fromSet()`), element order is determined by lexicographic sorting of each element's **canonical string representation**.

23.5.1 Canonical String Rules

Each data type produces a deterministic canonical string:

- **int** – decimal string (e.g., `42` → `"42"`)
- **real** – decimal string (e.g., `3.14` → `"3.14"`)
- **bool** – literal (e.g., `True` → `"True"`)
- **string** – quoted (e.g., `"a"` → `'"a"'`)

- **list** – bracket-enclosed, preserving element order (e.g., [3, 1, 2] → "[3, 1, 2]")
- **set** – brace-enclosed, elements sorted first (e.g., {Q1, R5, Q2} → "{Q1, Q2, R5}")
- **part** – reference designator (e.g., R1 → "R1")
- **pin** – parent ref and pad id (e.g., U1[3] → "U1[3]")
- **connection** – sorted pin pair in parentheses (e.g., (R1[1], U1[+]) → "(R1[1],U1[+])")
- **net** – name (e.g., Gnd → "Gnd")

Elements are then sorted lexicographically by their canonical strings.

23.5.2 Example

```
elec.part "U1": opamp(type="LM741"); des="";
    fp="Package_DIP:DIP-8_W7.62mm"
elec.net "V-"
elec.connect V- U1[v-]
```

```
#c# Canonical strings:
#c# (U1[V-],U1[v-]) -- connection
#c# U1                -- part
#c# U1[+]             -- pin
#c# U1[-]             -- pin
#c# U1[out]           -- pin
#c# U1[v+]            -- pin
#c# U1[v-]            -- pin
#c# V-                -- net
#c#
#c# *e = { (U1[V-],U1[v-]), U1, U1[+], U1[-],
#c#   U1[out], U1[v+], U1[v-], V- }
```

Chapter 24

Flags

System flags control PCB manufacturing capabilities using the `syst.flag.` prefix:

`syst.flag.<flag_name> <value>`

24.1 Available Flags

- `exec_mode` – Execution mode: `lenient` (default) or `strict`. In strict mode, all warnings become errors and halt execution.
- `phy_mode` – Physical constraint enforcement mode (default: `phylenient`):
 - `phystRICT` – All physical constraints must be met exactly. Violations halt execution immediately.
 - `phylenient` – Physical constraints are enforced best-effort. Violations produce warnings but execution continues.
 - `phyrelaxed` – The solver may relax or ignore physical constraints to find any valid layout.
- `vias` – Controls whether vias are allowed
- `slots` – Controls whether slots / PCB cuts are allowed
- `jumpers` – Controls whether jumper wires are allowed
- `solder_reinforced_tracks` – Controls whether solder-reinforced tracks are allowed
- `buried_vias` – Controls whether buried vias are allowed
- `approx_eps` – Additive tolerance for approximate equality operator `~` (default: 1)
- `approx_eps_mult` – Multiplicative tolerance for approximate equality operator `~*` (default: 1)
- `max_loop_iter` – Maximum iterations per while/do-while loop before raising a `Value Error` (default: 100000)
- `show_pcb_part_ref` – Show reference designator on PCB (default: `true`)
- `show_pcb_part_value` – Show component value on PCB (default: `true`)

- `show_pcb_part_tol` – Show tolerance/rating on PCB (default: `true`)
- `show_pcb_part_mpn` – Show MPN/manufacturer on PCB (default: `true`)
- `debug_mode` – Enable debug functions (`dbg.*`). When `false`, all debug functions are no-ops (default: `false`)
- `debug_output` – Default output destination for `dbg.print`: `"console"`, `"log"`, or `"both"` (default: `"console"`)
- `default_hash` – Default hash algorithm for `f.hash.hash`: `"md5"`, `"sha1"`, `"sha224"`, `"sha256"`, `"sha384"`, `"sha512"` (default), `"sha3_256"`, `"sha3_512"`, `"blake2b"`, `"blake2s"`, `"blake3"`

24.2 PCB Part Text Visibility

Global flags control which part text fields are visible on the PCB. Each field can be toggled independently:

```
syst.flag.show_pcb_part_ref False
syst.flag.show_pcb_part_value True
```

Per-component overrides can be set in the `elec.part` property list (see Section 20.2.1). If a per-component flag is set, it overrides the global flag for that component:

```
elec.part "R3": R(R=220); des="220R"; fp="R_Axial_DIN0309";
    show_ref=True; show_value=False
```

Available per-component flags: `show_ref`, `show_value`, `show_tol`, `show_mpn`.

Part VI

Toolchain

Chapter 25

Compiler and Execution Pipeline

The EDADSL toolchain compiles source code into PCB and schematic files.

25.1 Compilation Phases

1. Lexical Analysis – Tokenization of source code
2. Parsing – Syntax analysis and AST construction
3. Semantic Analysis – Type checking and validation
4. Execution – Program execution with electrical system construction
5. Constraint Solving – Physical layout optimization
6. Code Generation – Output file generation

25.2 Entry Point

Every program must begin with the **start** keyword. The program halts when **halt** is reached or an unrecoverable error occurs.

Chapter 26

Code Generation

26.1 PCB Generation

The `make.pcb` command generates a PCB file based on the circuit and physical constraints:

```
make.pcb
```

This command must be placed after all circuit definitions and physical constraints.

26.2 Schematic Generation

The `make.schematic` command generates a schematic file:

```
make.schematic
```

26.3 Export

The `halt` command terminates execution and exports all created files:

```
halt
```

Chapter 27

Standard Library

The standard library provides built-in component models, footprint libraries, and material definitions.

27.1 Component Models

Built-in models include: **R** (Resistor), **C** (Capacitor), **L** (Inductor), **opamp** (Operational Amplifier), **MOSFET** (MOSFET transistor), **LED** (Light Emitting Diode), **D** (Diode), **Q** (BJT Transistor), **X** (Crystal/Oscillator), **J** (Connector), **F** (Fuse), **Y** (Transformer).

27.2 Footprint Libraries

Footprints are organized by package type: **Package_SMD**, **Package_TH**, **Package_BGA**, **Package_Power**, **Package_Connector**. Each library contains standard footprints following IPC naming conventions.

27.3 Material Library

Materials define physical properties for PCB stackup: **FR4** (fiberglass-epoxy), **copper** (conductor), **ENIG** (surface finish), **soldermask** (insulator).

Chapter 28

External Library

External libraries extend EDADSL with additional component models and footprints.

28.1 Importing Libraries

TODO: Document how external libraries are integrated with EDADSL.

28.2 Supported Formats

EDADSL supports integration with:

- KiCad symbol libraries (`.kicad_sym`)
- KiCad footprint libraries (`.kicad_mod`)
- SPICE models (`.lib`, `.sub`)
- IPC-2581 component databases

Chapter 29

Debugging and Logging

29.1 Debug Mode

Debug functions (`dbg.*`) are enabled by the `debug_mode` flag. When disabled (default), all debug functions are no-ops:

```
syst.flag.debug_mode True
```

The `debug_output` flag controls the default destination for `dbg.print`:

```
syst.flag.debug_output "console"    ## default
syst.flag.debug_output "log"
syst.flag.debug_output "both"
```

29.2 Quick Reference

The following output and debugging tools are available:

Tool	Purpose
<code>echo \$var</code>	Console output (simple)
<code>f.util.print(...)</code>	Console output (expressions)
<code>writetolog 'msg'</code>	Append to log file
<code>dbg.print(\$var, mode)</code>	Debug output with type info
<code>dbg.breakpoint(cond)</code>	Pause execution
<code>dbg.graph()</code>	Dump electrical graph
<code>dbg.resources()</code>	Memory, CPU, object counts
<code>dbg.trace(msg)</code>	Execution trace log
<code>dbg.profile(fn, mode)</code>	Function timing
<code>dbg.memory()</code>	Detailed memory report
<code>dbg.cpu()</code>	Instantaneous CPU usage
<code>dbg.watch(\$var)</code>	Variable change log
<code>dbg.stack()</code>	Call stack dump
<code>dbg.inspect(\$var)</code>	Detailed variable info

For complete documentation (EBNF, type overloading, examples, error conditions) of all `dbg.*` functions, see Section 11.1.10 in the Built-in Functions chapter.

29.3 Workflow

A typical debugging session:

1. Enable debug mode: `syst.flag.debug_mode True`
2. Set output destination: `syst.flag.debug_output ‘both’`
3. Add `dbg.watch()` for variables of interest
4. Add `dbg.breakpoint()` at critical points
5. Add `dbg.trace()` calls for execution flow
6. Review the debug log file after execution

Part VII

Reference

Chapter 30

Exceptions

30.1 Type Errors

Raised when an operation is applied to a value of the wrong type.

30.2 Value Errors

Raised when a function receives an argument outside its domain (e.g., division by zero, invalid list index). Out-of-bounds list or set indexing (e.g., accessing index 10 on a 3-element list) raises a Value Error and halts execution.

30.3 Reference Errors

Raised when referencing an undefined variable, part, or function.

30.4 Constraint Errors

Raised when physical constraints cannot be resolved (see Section 23).

Chapter 31

Error and Warning Model

31.1 Errors

Errors halt program execution. The error message includes the error type and location.

31.2 Warnings

Warnings are printed for constraint conflicts between different priorities. The program continues execution after warnings.

Chapter 32

Future Improvements

Planned features for future versions:

- Differential pair routing constraints
- Impedance-controlled routing
- 3D component placement visualization
- Signal integrity analysis integration
- Power integrity analysis
- Automated thermal relief generation

Chapter 33

Implementations

33.1 Reference Implementation

A reference implementation is planned and will be developed in Rust for performance and safety. It will cover the full language specification described in this document, including lexical analysis, parsing, type checking, evaluation, constraint solving, and layout generation.

33.2 Build and Installation

Build and installation instructions will be provided once the reference implementation is publicly available.

Chapter 34

References

34.1 Language Specification

This document serves as the authoritative specification for the EDADSL language.

34.2 Related Standards

- IPC-2581 — Generic Standard for Printed Board Assembly Products
- IPC-7351 — Land Pattern Naming Conventions for Surface Mount Designs
- KiCad File Formats — Schematic and PCB file format specifications
- SPICE — Simulation Program with Integrated Circuit Emphasis

Appendix A

Standard Libraries

A.1 Material Libraries

A.1.1 PCB Substrate Material Library

Common substrate materials:

- FR4 – Standard glass-reinforced epoxy laminate
- Rogers – High-frequency laminates
- Metal-core – Aluminum or copper core substrates

A.1.2 PCB Conductor Material Library

- Copper – Standard conductor (specified in oz/ft²)

A.1.3 PCB Surface-finish Material Library

- HASL – Hot Air Solder Leveling
- ENIG – Electroless Nickel Immersion Gold
- OSP – Organic Solderability Preservative

A.2 Basic Model Libraries

A.2.1 Passive Component Model Library

- R – Resistor (parameters: R, P, type)
- C – Capacitor
- L – Inductor

A.2.2 Active Component Model Library

- MOSFET – MOSFET transistor
- Diode – Standard diode
- BJT – Bipolar junction transistor

A.2.3 Integrated Circuit Component Model Library

- opamp – Operational amplifier
- Additional models in external libraries

A.2.4 Board Hardware Component Model Library

- Connectors
- Mounting holes

A.3 Additional Language Libraries

EDADSL can be extended with custom libraries containing component models, footprints, and constraint templates. Libraries follow the standard library organization.

A.4 Footprint Libraries

Footprints follow KiCad library naming conventions:

`<Library>:<Footprint>`

Examples:

```
Resistor_THT:R_Axial_DIN0309_L9.0mm_D3.2mm_P12.70mm_Horizontal
Package_T0_SOT_THT:T0-247-3_Horizontal_TabUp
Package_DIP:DIP-8_W7.62mm
```

Appendix B

Examples

B.1 LED Circuit

B.1.1 Intent

A simple LED current-limiting circuit.

B.1.2 EDADSL Code

```
start

elec.part "R1": R(R=330, P=0.25);
    des="330 ohm resistor";
    fp="Resistor_THT:R_Axial_DIN0309"
elec.part "D1": LED(color="red");
    des="Red LED";
    fp="LED_THT:LED_D3.0mm"

elec.net "Vcc"
elec.net "Gnd"

elec.connect Vcc R1[1]
elec.connect R1[2] D1[a]
elec.connect D1[c] Gnd

halt
```

B.1.3 Explanation

The circuit connects a 330Ω resistor in series with an LED between Vcc and Gnd. The resistor limits current to approximately 10mA (assuming 5V supply and 2V LED forward voltage).

B.2 Arithmetic and Type Coercion

B.2.1 Intent

Demonstrate basic arithmetic, variable declarations, and implicit type promotion from int to real to complex.

B.2.2 EDADSL Code

```
start

$a [int]= 10
$b [real]= 3.14
$c [complex]= 2 + 3i

$sum t= $a + $b          #c# 13.14 (int + real -> real)
$prod t= $a * $c         #c# 20 + 30i (int * complex -> complex)
$hyp t= f.log.sqrt($a^2 + $b^2) #c# ~10.49 (real)
$ang t= f.cplx.angle($c)   #c# ~0.9828 rad (real)

f.util.print(f.str.str("a + b = ", $sum))
f.util.print(f.str.str("a * c = ", $prod))
f.util.print(f.str.str("hypotenuse = ", $hyp))
f.util.print(f.str.str("angle = ", $ang))

halt
```

B.2.3 Explanation

Variables are declared with explicit types. Arithmetic operations implicitly promote to the wider type: int + real yields real, int + complex yields complex. The `f.cplx.angle` function returns the argument (phase) of a complex number in radians.

B.3 Conditionals and Classification

B.3.1 Intent

Use nested if/else to classify a resistor value into standard E-series ranges.

B.3.2 EDADSL Code

```
start

$value [real]= 4.7e3

$series [string]= ""
if $value < 1e3 ::{
    $series = "E12 or below"
```

```

} else ::{
  if $value < 100e3 ::{
    $series = "E24 common"
  } else ::{
    $series = "High value"
  }
}

$tolerance [string]= ""
if $value == 1.0e3 ::{
  $tolerance = "tight"
} else ::{
  if $value == 4.7e3 ::{
    $tolerance = "standard"
  } else ::{
    $tolerance = "check datasheet"
  }
}

f.util.print(f.str.str($value, " ohms: ", $series, ", ", ", $tolerance))

halt

```

B.3.3 Explanation

The if/otherwise/else chain classifies the resistor into ranges. Use `otherwise` for else-if conditions. For cleaner multi-branch logic, use the `cases` statement. The expected output is 4700.0 ohms: E24 common, standard.

B.4 Lambdas and Higher-Order Functions

B.4.1 Intent

Use lambda expressions with `f.util.map`, `f.util.filter`, and `f.util.reduce` to process a list of resistor values.

B.4.2 EDADSL Code

```

start

$resistors [list]= [100, 220, 330,
  470, 1e3, 2.2e3, 4.7e3, 10e3]

# Compute power dissipation at 5V
$powers [list]= f.util.map($resistors,
  L::($r [real]) -> { (5.0^2) / $r })
f.util.print(f.str.str("Powers: ", $powers))

```

```

# Filter to only resistors under 1k
$small [list]= f.util.filter($resistors,
    L::($r [real]) -> { $r < 1000 })
f.util.print(f.str.str("Under 1k: ", $small))

# Sum all power dissipation
$total_power [real]= f.util.reduce($powers,
    L::($acc [real], $p [real]) -> { $acc + $p },
    0.0)
f.util.print(f.str.str("Total power: ", $total_power, " W"))

# Sort resistors ascending
$sorted [list]= f.util.sort($resistors,
    L::($a [real], $b [real]) -> { $a - $b })
f.util.print(f.str.str("Sorted: ", $sorted))

halt

```

B.4.3 Explanation

Lambdas are declared with `L::` prefix, typed parameters, and `-> { }` body. `f.util.map` applies a function to each element, `f.util.filter` selects elements satisfying a predicate, and `f.util.reduce` folds the list with an accumulator. The sort comparator returns negative if the first argument should come first.

B.5 Complex Arithmetic and FFT

B.5.1 Intent

Perform complex arithmetic, generate a signal, and compute its frequency spectrum using FFT.

B.5.2 EDADSL Code

```

start

# Complex arithmetic
$z1 [complex]= 3 + 4i
$z2 [complex]= 1 - 2i

$sum      t= $z1 + $z2      # 4 + 2i
$product  t= $z1 * $z2      # 11 - 2i
$mag      t= f.math.abs($z1) # 5.0
$conj_z   t= f.cplx.conj($z2) # 1 + 2i

f.util.print(f.str.str("|z1| = ", $mag))

```

```

f.util.print(f.str.str("z1 * z2 = ", $product))
f.util.print(f.str.str("conj(z2) = ", $conj_z))

# Generate a 256-point signal: 50Hz + 120Hz
$N [int]= 256
$fs [real]= 1000.0
$t [list]= [0:1:$N - 1]
$signal [list]= f.util.map($t,
  L::($n [int]) -> { f.trig.sin(2 * c.math.pi * 50
    * $n / $fs)
    + 0.5 * f.trig.sin(2 * c.math.pi * 120
    * $n / $fs) })

# Compute FFT and extract magnitudes
$spectrum [list]= f.ee.fft($signal)
$mags [list]= f.util.map($spectrum,
  L::($z [complex]) -> { f.math.abs($z) })

# Find the dominant frequency
$max_mag [real]= f.math.max($mags)
$peak_index [int]= f.stat.argmax($mags)
$freq [real]= ($peak_index * $fs) / $N

f.util.print(f.str.str("Dominant frequency: ", $freq, " Hz"))
f.util.print(f.str.str("Magnitude: ", $max_mag))

halt

```

B.5.3 Explanation

Complex numbers support all arithmetic operators. The `f.ee.fft` function returns a list of complex values; `f.util.map` with `f.math.abs` extracts magnitudes. The dominant frequency is recovered by finding the index of the peak magnitude and scaling by f_s/N . The expected dominant frequency is 50 Hz.

B.6 Narray Matrix Operations

B.6.1 Intent

Use tensor syntax and matrix operators to solve a system of linear equations.

B.6.2 EDADSL Code

```

start

# Solve A * x = b where:
#   A = [[2, 1], [5, 7]]

```

```

#   b = [11, 13]
$A [tensor]= |[[2, 1], [5, 7]]
$b [tensor]= |[11, 13]

# Compute inverse and solve
$A_inv [tensor]= f.linalg.inv($A)
$x [tensor]= $A_inv .* $b

f.util.print(f.str.str("Solution x = ", $x))

# Verify: A .* x should equal b
$check [tensor]= $A .* $x
f.util.print(f.str.str("Verification A .* x = ", $check))

# Matrix properties
f.util.print(f.str.str("det(A) = ", f.linalg.det($A)))
f.util.print(f.str.str("rank(A) = ", f.linalg.rank($A)))

halt

```

B.6.3 Explanation

Tensors are explicitly typed with `[tensor]` or pipe literal syntax. The `.*` operator performs matrix multiplication, and `f.linalg.inv` computes the matrix inverse. Functions `f.linalg.det` and `f.linalg.rank` return the determinant and rank respectively. The expected solution is $x = [6.2, -1.4]$.

B.7 Closures and Lambdas

B.7.1 Intent

Demonstrate lambdas capturing outer variables and lambda composition for filtering and mapping.

B.7.2 EDADSL Code

```

start

$resistors [list]= [100, 330, 1e3, 2.2e3, 4.7e3, 10e3]

# Filter: keep resistors above 1k (captures $threshold)
$threshold [real]= 1000
$high [list]= f.util.filter($resistors,
  L::($v [real]) -> { $v > $threshold })
f.util.print(f.str.str("Above 1k: ", $high))

# Filter: keep resistors above 4.7k

```



```

$threshold = 4700
$vhhigh [list]= f.util.filter($resistors,
    L::($v [real]) -> { $v > $threshold })
f.util.print(f.str.str("Above 4.7k: ", $vhhigh))

# Element-wise add scalar to each list element
$offset [int]= 10
$shifted [list]= f.util.map([10, 20, 30],
    L::($i [int]) -> { $i + $offset })
f.util.print(f.str.str("Shifted: ", $shifted))

halt

```

B.7.3 Explanation

Lambdas capture variables from the enclosing scope. Here `$threshold` is captured by reference: reassigning it before calling `f.util.filter` changes which values pass the filter. The `f.util.map` lambda captures `$offset` to add a constant to each element.

B.8 Design-Rule-Driven Component Placement

B.8.1 Intent

Declare components with parametric constraints and let the solver handle placement.

B.8.2 EDADSL Code

```

start

elec.part "U1": opamp(type="generic", GBP=1e6, slew=5)
    ; des="Generic op-amp"
    ; fp="Package_DIP:DIP-8_W7.62mm"

elec.part "Rf": R(R=10e3, P=0.125, tol=1)
    ; des="Feedback resistor"
    ; fp="Resistor_THT:R_Axial_DIN0309"

elec.part "Rin": R(R=1e3, P=0.125, tol=1)
    ; des="Input resistor"
    ; fp="Resistor_THT:R_Axial_DIN0309"

elec.part "C1": C(C=100e-12, V=50)
    ; des="Compensation capacitor"
    ; fp="Capacitor_THT:C_Axial_DIN0207"

elec.net "Input"
elec.net "Output"

```

```

elec.net "Vcc"
elec.net "Gnd"

elec.connect Input Rin[1]
elec.connect Rin[2] U1[2]      # inverting input
elec.connect Rf[1] U1[2]
elec.connect Rf[2] U1[1]      # output
elec.connect C1[1] U1[2]
elec.connect C1[2] U1[1]
elec.connect Output U1[1]
elec.connect U1[4] Vcc
elec.connect U1[7] Gnd

# Constraint: keep feedback path short
phys.prox {Rf} {U1} max=15m -imp

halt

```

B.8.3 Explanation

Components are declared with parametric models and footprint assignments using `;` separators. The `phys.prox` keyword specifies a physical layout constraint that the placement solver must satisfy. This example inverts the op-amp with gain $G = -R_f/R_{in} = -10$ and includes compensation.

B.9 Signal Processing Pipeline

B.9.1 Intent

Build a complete signal processing pipeline: generate, filter, transform, and analyze.

B.9.2 EDADSL Code

```

start

# Step 1: Generate a noisy 100Hz signal
$N [int]= 512
$fs [real]= 1000.0
$t [list]= [0:1:$N - 1]
$clean [list]= f.util.map($t,
    L::($n [int]) -> { f.trig.sin(2 * c.math.pi
        * 100 * $n / $fs) })

# Step 2: Add noise (simple LCG pseudo-random)
$noise [list]= f.util.map($t,
    L::($n [int]) -> { 0.3 * f.trig.sin(2 * c.math.pi
        * 7 * $n) })

```

```

$noisy [list]= f.util.map($t,
    L::($i [int]) -> { $clean[$i] + $noise[$i] })

# Step 3: Compute FFT of noisy signal
$spectrum [list]= f.ee.fft($noisy)
$mags [list]= f.util.map($spectrum,
    L::($z [complex]) -> { f.math.abs($z) })

# Step 4: Find peak frequency
$peak_idx [int]= f.stat.argmax($mags)
$peak_freq [real]= ($peak_idx * $fs) / $N
f.util.print(f.str.str("Detected frequency: ", $peak_freq, " Hz"))

# Step 5: Compute signal statistics
$sig_rms [real]= f.stat.rms($noisy)
$sig_peak [real]= f.stat.peak($noisy)
f.util.print(f.str.str("RMS: ", $sig_rms, ", Peak: ", $sig_peak))

# Step 6: Build magnitude spectrum pairs
$bin_width [real]= $fs / $N
$spec_pairs [list]= []
for $i E [0:1:$N / 2 - 1] ::{
    $freq_val [real]= $i * $bin_width
    $mag_val [real]= $mags[$i]
    $spec_pairs = $spec_pairs
        + [[ $freq_val, $mag_val ]]
}

f.util.print(f.str.str("Spectrum bins (first 5): ",
    $spec_pairs[0:5]))

halt

```

B.9.3 Explanation

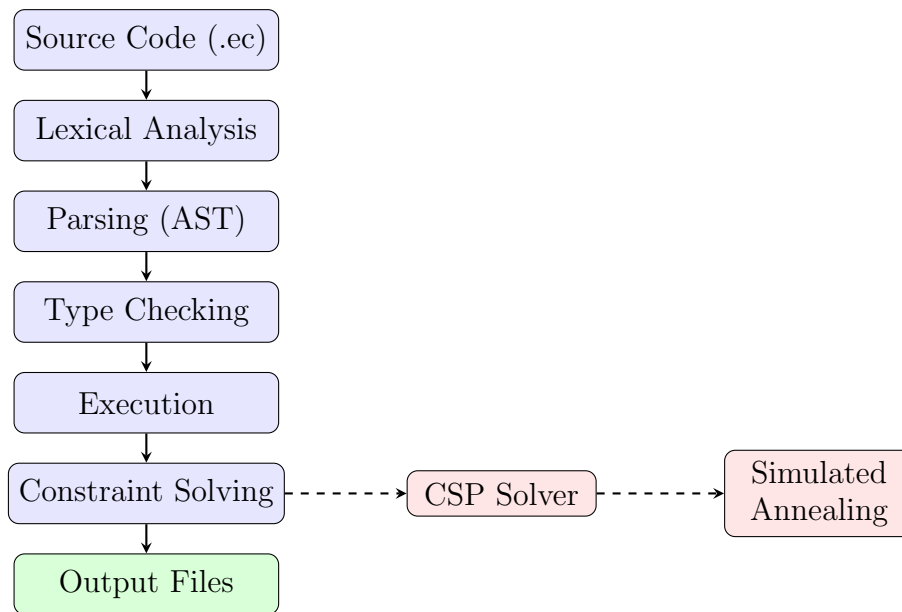
This example chains together multiple language features: `f.util.map` for list comprehensions, complex FFT, higher-order functions for magnitude extraction, signal statistics (`f.stat.rms`, `f.stat.peak`), and list slicing. The pipeline demonstrates a realistic signal analysis workflow from raw data through frequency-domain representation.

Appendix C

Diagrams

C.1 Execution-phase Diagram

The following diagram shows the execution phases of an EDADSL program, from source code through constraint solving to final output.



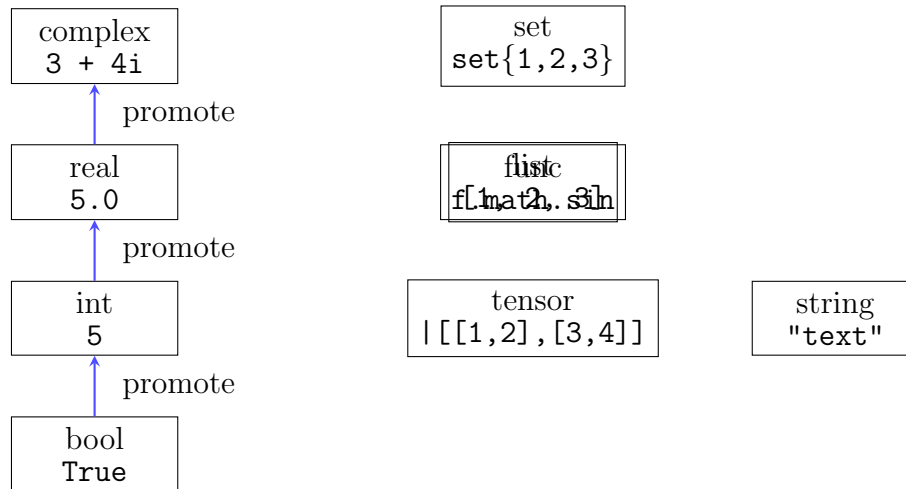
Phase Descriptions:

- **Lexical Analysis** – Tokenizes the source file into keywords, identifiers, literals, and operators.
- **Parsing** – Builds an Abstract Syntax Tree (AST) from the token stream.
- **Type Checking** – Validates types, performs implicit promotions, and resolves overloads.
- **Execution** – Evaluates expressions, executes control flow, and evaluates functions. Components and nets are registered.

- **Constraint Solving** – Physical constraints are solved using CSP propagation and simulated annealing to determine optimal placement and routing.
- **Output Files** – Generates manufacturing files (Gerbers, drill files, BOM, netlists).

C.2 Type Hierarchy Diagram

The following diagram shows the EDADSL type hierarchy, including numeric promotion, collection types, and their relationships.

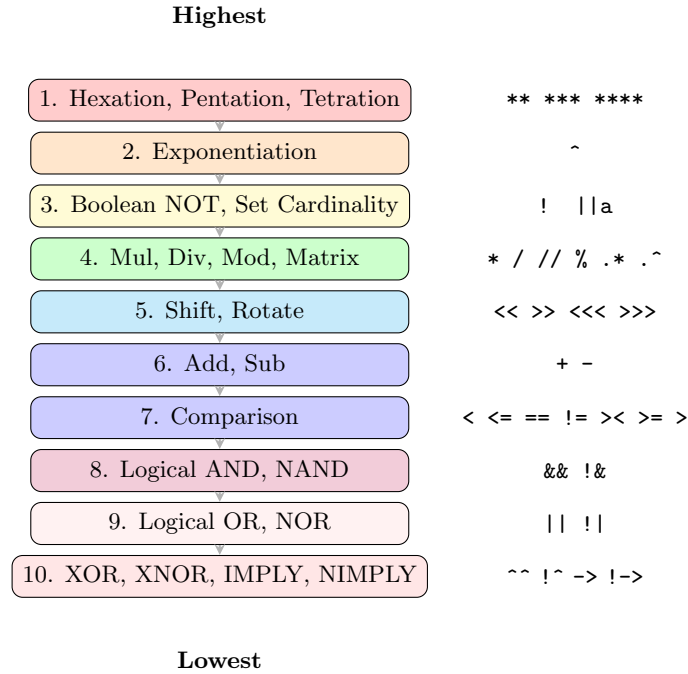


Key Relationships:

- Numeric types promote automatically: `bool` → `int` → `real` → `complex`
- A `tensor` is a `list` that is rectangular and all-numeric; explicitly typed with `[tensor]` or pipe literal `| [...]`
- `list` element types are polymorphic; `tensor` elements are restricted to `bool`, `int`, `real`, or `complex`
- `set` is unordered and unique; `list` is ordered and allows duplicates
- `func` values are callable; they can be built-in references, user-defined references, or lambdas

C.3 Operator Precedence Diagram

The following diagram visualizes operator precedence from highest (top) to lowest (bottom).



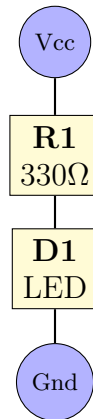
Precedence Table:

1. Hexation (****), Pentation (***), Tetration (**) – right-associative
2. Exponentiation (^) – right-associative
3. Boolean negation (!), Set cardinality (||a)
4. Multiplication (*), Division (/), Floor Division (//), Modulo (%), Matrix (.* .^)
5. Shift (<< >>), Rotate (<<<[w] >>>[w])
6. Addition (+), Subtraction (-)
7. Comparison (< <= == != >< >= >)
8. Logical AND (&&), NAND (!&)
9. Logical OR (||), NOR (!|)
10. XOR (^^), XNOR (!^), IMPLY (->), NIMPLY (!->)

C.4 Example Circuit: LED Current-Limiting

The following diagram shows the LED circuit from Appendix B alongside its EDADSL code.

Schematic



EDADSL Code:

```
elec.part "R1": R(R=330)
    des="330 ohm"
    fp="Resistor_THT:R_Axial"
elec.part "D1": LED(color="red")
    des="Red LED"
    fp="LED_THT:LED_D3.0mm"

elec.net "Vcc"
elec.net "Gnd"

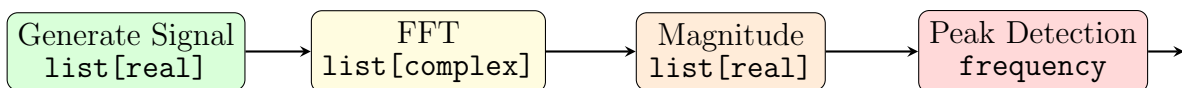
elec.connect Vcc R1[1]
elec.connect R1[2] D1[a]
elec.connect D1[c] Gnd
```

Circuit Description: A 330Ω resistor (R1) limits current through a red LED (D1) between Vcc (5V) and Gnd. Expected current: $I = (5V - 2V)/330\Omega \approx 9.1\text{mA}$.

C.5 FFT Pipeline Diagram

The following diagram shows the signal processing pipeline for FFT analysis.

FFT Pipeline



EDADSL Code:

```

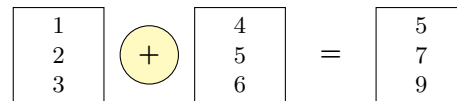
$signal [list]= f.util.map([0:1:$N - 1],
    L::($n [int]) -> { f.trig.sin(2 * c.math.pi
        * 100 * $n / $fs) })
$spectrum [list]= f.ee.fft($signal)
    # list[complex]
$mags      [list]= f.util.map($spectrum,
    L::($z [complex]) -> { f.math.abs($z) })
    # list[real]
$peak_idx [int]= f.stat.argmax($mags)
$freq     [real]= ($peak_idx * $fs) / $N    # = 100 Hz

```

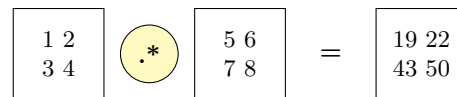
C.6 Narray Operations

The following diagram visualizes key tensor operations: element-wise operations, matrix multiplication, and broadcasting.

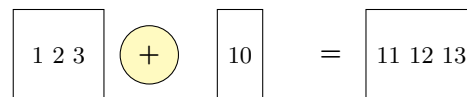
Element-wise +



Matrix Multiply



Broadcasting



EDADSL Code:

```

$a [tensor]= |[1, 2, 3]
$b [tensor]= |[4, 5, 6]
$a + $b      #c# = |[5, 7, 9]
$m1 [tensor]= |[ [1,2], [3,4] ]
$m2 [tensor]= |[ [5,6], [7,8] ]
$m1 .* $m2   #c# = |[ [19,22], [43,50] ]
$a + 10      #c# = |[11, 12, 13]

```

Operations:

- **Element-wise** – Standard operators (+ - * / ^) applied element-by-element
- **Matrix Multiply** – .* computes matrix product (rows × columns)
- **Broadcasting** – Scalars are broadcast to match the tensor shape
- **Transpose** – .T flips rows and columns; .T in-place

C.7 Custom Model Definition and Usage

C.7.1 Intent

Define a custom SiC MOSFET model with a kelvin source pin, then use it in a half-bridge circuit.

C.7.2 EDADSL Code

```
# Define custom SiC MOSFET model
elec.makemodel mosfet/C3M0016120K1:{
pins:
    g      -> 4; T = input
    s      -> 2; T = power
    d      -> 1; T = power
    kelvin_s -> 3; T = output

des = "1200 V, 125A, 16 mOhm, TO-247-4"
dts = "https://assets.wolfspeed.com/..."
fp  = "Package_TO_SOT_THT/TO-247-4_Vert"
}

# Use custom model in a half-bridge
elec.part "Q1": mosfet/C3M0016120K1();
    des="High-side SiC MOSFET";
    fp="Package_TO_SOT_THT/TO-247-4_Vert";
    show_ref=True; show_value=True

elec.part "Q2": mosfet/C3M0016120K1();
    des="Low-side SiC MOSFET";
    fp="Package_TO_SOT_THT/TO-247-4_Vert";
    show_ref=True; show_value=True

# Define nets
elec.net "Vbus" "hv"
elec.net "Vphase"
elec.net "Vgate_high"
elec.net "Vgate_low"
elec.net "Gnd"

# Gate drive
elec.connect Vgate_high Q1[g]
elec.connect Vgate_low  Q2[g]

# Half-bridge output
elec.connect Vphase Q1[d] Q2[d]
```

```

# Power rails
elec.connect Vbus Q1[s]
elec.connect Gnd  Q2[s]

# Kelvin connections for gate drive return
elec.connect Vgate_low Q1[kelvin_s]
elec.connect Gnd      Q2[kelvin_s]

# Unite into single physical package
phys.unite { Q1, Q2 }

```

C.7.3 Explanation

The `elec.makemodel` keyword defines a new component model with named pins, pin types, and optional metadata. Pin types (`input`, `power`, `output`) classify the electrical role of each pin. The `des`, `dts`, and `fp` fields attach documentation and footprint information to the model.

When used in `elec.part`, the custom model is referenced by its full name (`mosfet/C3M0016120K1`). Per-component PCB text flags (`show_ref`, `show_value`) override the global defaults. The `phys.unite` statement merges Q1 and Q2 into a single physical package on the PCB.